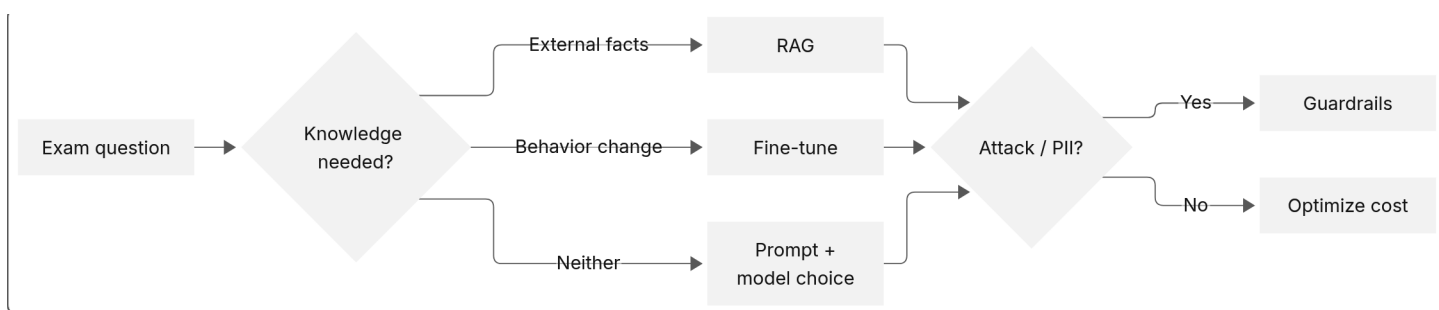


# AIP-C01 Crash Course

This book contains the complete exam information. It shows every domain. It shows what you must remember under time pressure. It shows decision rules, numbers, trap patterns, and quick practice questions. Read the book one time. Then use the question sets for each domain.

This helps you practice what you do not know. If you can answer every quick question here, you can answer every exam question.

## The exam in 30 seconds



Crash decision ladder: external facts mean RAG, behavior change means fine-tune, neither means prompt plus model choice; then add guardrails for attacks or PII, and optimize cost

*Crash decision ladder: external facts mean RAG, behavior change means fine-tune, neither means prompt plus model choice; then add guardrails for attacks or PII, and optimize cost* Source: original figure for the AWS AIP-C01 Study Guide.

| Fact                                                              | Value                                                                                                                |
|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Questions                                                         | 75 total questions (65 scored, 10 not scored). You cannot know which questions count. Treat all questions as scored. |
| Time                                                              | 180 minutes. This is about 2.4 minutes for each question.                                                            |
| Pass                                                              | 750 out of 1000 (scaled score)                                                                                       |
| Cost                                                              | \$300                                                                                                                |
| Delivery                                                          | Pearson VUE. Testing center or online proctored.                                                                     |
| Formats                                                           | Multiple choice (one answer) and multiple response (select 2 or 3 answers). There is no partial credit.              |
| D1 Foundation Model Integration, Data Management, and Compliance  | 31% (about 20 scored questions)                                                                                      |
| D2 Implementation and Integration                                 | 26% (about 17 scored questions)                                                                                      |
| D3 AI Safety, Security, and Governance                            | 20% (about 13 scored questions)                                                                                      |
| D4 Operational Efficiency and Optimization for GenAI Applications | 12% (about 8 scored questions)                                                                                       |
| D5 Testing, Validation, and Troubleshooting                       | 11% (about 7 scored questions)                                                                                       |

## HOW THE EXAM ASKS

**The pattern:** There is a company scenario with many paragraphs. There are two to four hard requirements. There are four long, possible options. Three options are good ideas in the wrong situation. One option meets every requirement. First, read the scenario for the main requirement.

Then compare the remaining options.

The adjectives in the question decide the answer. "least operational overhead" means the fewest self-managed components. "least custom development effort" means managed features, not your own code. "most cost-effective" means token math. Premium extras always lose if there is no stated need.

### The three adjectives that pick the winner

The stem names the winner before you read the options.

#### "least operational overhead"

Fewest self-managed components. Example: Aurora Serverless over a self-managed OpenSearch cluster.

#### "least custom development effort"

Managed features over your own code. Example: Knowledge Bases over a hand-built ingestion pipeline.

#### "most cost-effective"

Token math. Example: The smallest model that clears the quality bar; caching; batch; cascade.

The scenario does not change; the winner does. Read the adjective first.

Three adjectives decide many answers. Least operational overhead means fewest self-managed components. Least custom development effort means managed features. Most cost-effective means token math.

Source: original figure for the AWS AIP-C01 Study Guide.

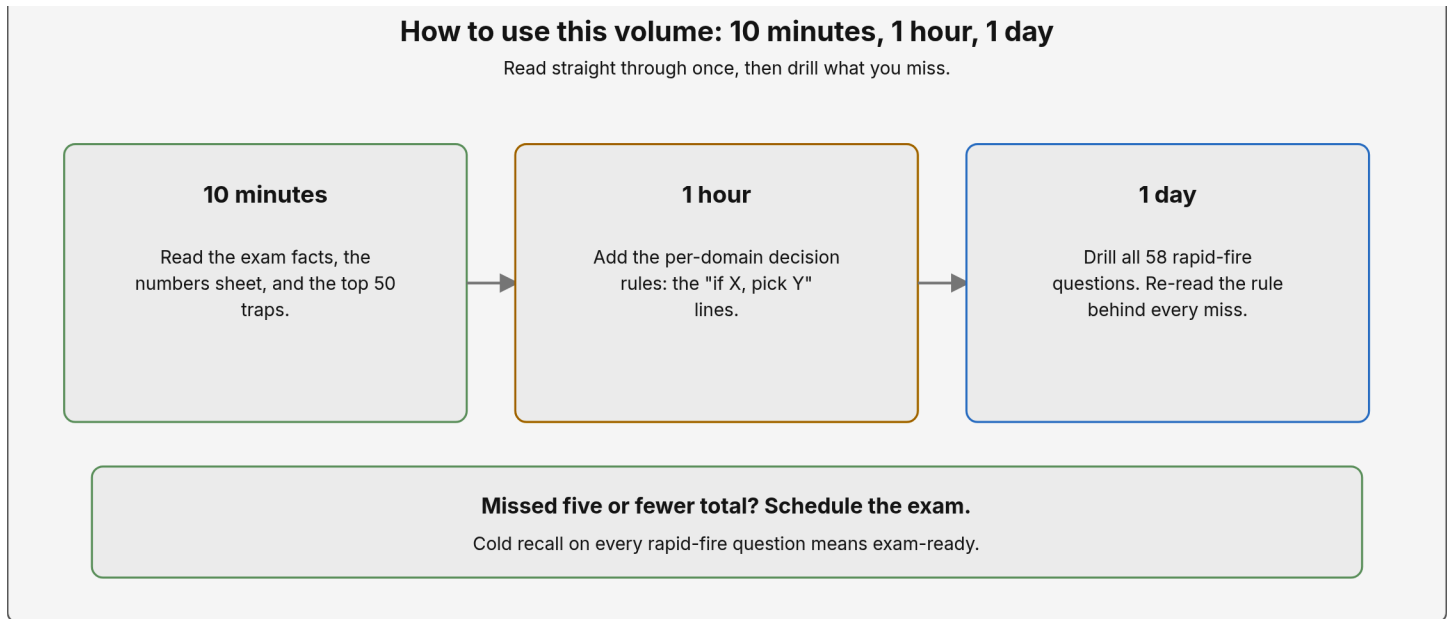
How to read this diagram

1. Row one: least operational overhead. Count self-managed components.
2. Row two: least custom development effort. Pick managed features over your own code.
3. Row three: most cost-effective. Do the token math.

## How to use this book

- **10 minutes:** Read the exam facts above. Read the numbers sheet. Read the top 50 traps.
- **1 hour:** Add the decision rules for each domain to your memory. These are the "if X, pick Y" lines.

- **1 day:** Practice all 58 quick questions. Then read the rules again for every question you answered wrong. Did you answer five or fewer questions wrong? Schedule the exam.



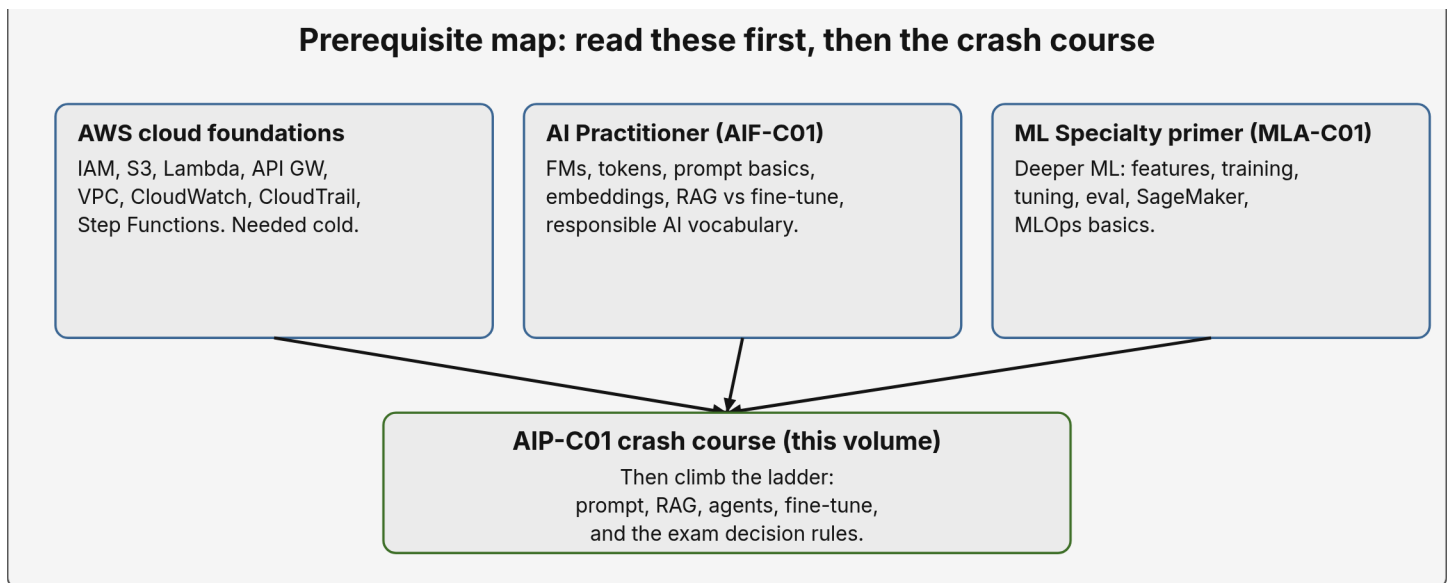
Study in three passes. Ten minutes on facts and traps. One hour on decision rules. One day drilling rapid-fire questions.

Source: original figure for the AWS AIP-C01 Study Guide.

#### How to read this diagram

1. Pass one: facts and traps. Memorize the numbers sheet.
2. Pass two: decision rules. Learn the if-X-pick-Y lines.
3. Pass three: rapid-fire questions. Drill until every answer is fast.

**This document is a prerequisite map. It shows the three documents that come before this document.**



Prerequisite map: AWS cloud foundations, AI Practitioner, and ML Specialty primer feed into the AIP-C01 crash course

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Start at the three top boxes: each names the volume and what it must give you. Follow the arrows down: all three feed this crash course. Takeaway: if a domain volume reads as jargon, the missing layer is one of the three boxes.

AIP-C01 needs two types of background knowledge. It needs general AWS knowledge and basic AI knowledge. If you do not have this knowledge, read the correct prerequisite document first. This short course does not teach this knowledge again.

- **AWS cloud foundations:** This document describes the AWS core that AIP-C01 uses. It includes IAM (users, roles, policies, least privilege), S3 (object storage, versioning, event notifications), Lambda (serverless functions), API Gateway (REST and HTTP APIs), VPC (subnets, endpoints, PrivateLink), CloudWatch (metrics, logs, alarms), CloudTrail (API audit), EventBridge, SQS, SNS, and basic Step Functions. You must know these topics very well. The exam uses these services in its answers. It expects you to know their limits. For example, Lambda has a 15-minute limit. The exam does not tell you this limit.
- **AI Practitioner (AIF-C01):** This document describes the AI knowledge layer. It covers what a foundation model is. It covers tokens and context windows. It covers basic prompt engineering. It covers embeddings as meaning-vectors. It covers fine-tuning versus RAG at a conceptual level. It covers responsible AI vocabulary: bias, fairness, hallucination, toxicity. It covers the AWS AI service map: Bedrock, SageMaker, Comprehend, Rekognition, Transcribe, Textract. This is the vocabulary that the exam uses. Without this vocabulary, the domain documents will seem like jargon.
- **ML Specialty primer (MLA-C01):** This document describes the deeper ML layer. It is for questions that go beyond GenAI. It includes data preparation and feature engineering, training and tuning concepts, model evaluation metrics (accuracy, precision, recall, F1, ROC), SageMaker training and deployment parts, and ML

operations (pipelines, monitoring, drift). AIP-C01 talks about these topics when GenAI meets classic ML. This is mostly in evaluation and data pipeline questions.

Rule: If you can explain IAM least privilege, S3 event notifications, and what an embedding is without stopping, skip the prerequisite documents. Start at D1 below.

### Prerequisite certifications: the hub checklist

The study hub this crash course now covers assumes four certifications before AIP-C01. Map them to this repo.

| Certification                                     | Maps to                                                                  |
|---------------------------------------------------|--------------------------------------------------------------------------|
| AWS AI Practitioner (AIF-C01)                     | The AI Practitioner primer: vocabulary, FM basics, Bedrock concepts      |
| AWS Cloud Practitioner (CLF-C02)                  | The AWS cloud foundations primer: IAM, S3, Lambda, core services         |
| AWS Solutions Architect Associate (SAA-C03)       | The AWS cloud foundations primer, deeper: VPC, storage, compute choices  |
| AWS Machine Learning Engineer Associate (MLA-C01) | The ML Specialty primer: training, tuning, evaluation metrics, SageMaker |

AIP-C01 builds on all four. It does not reteach them. If a domain section reads as jargon, the gap is in one of these four, not in this crash course. Exam facts stay as published: 75 questions, 180 minutes, 750 to pass, \$300, Pearson VUE.

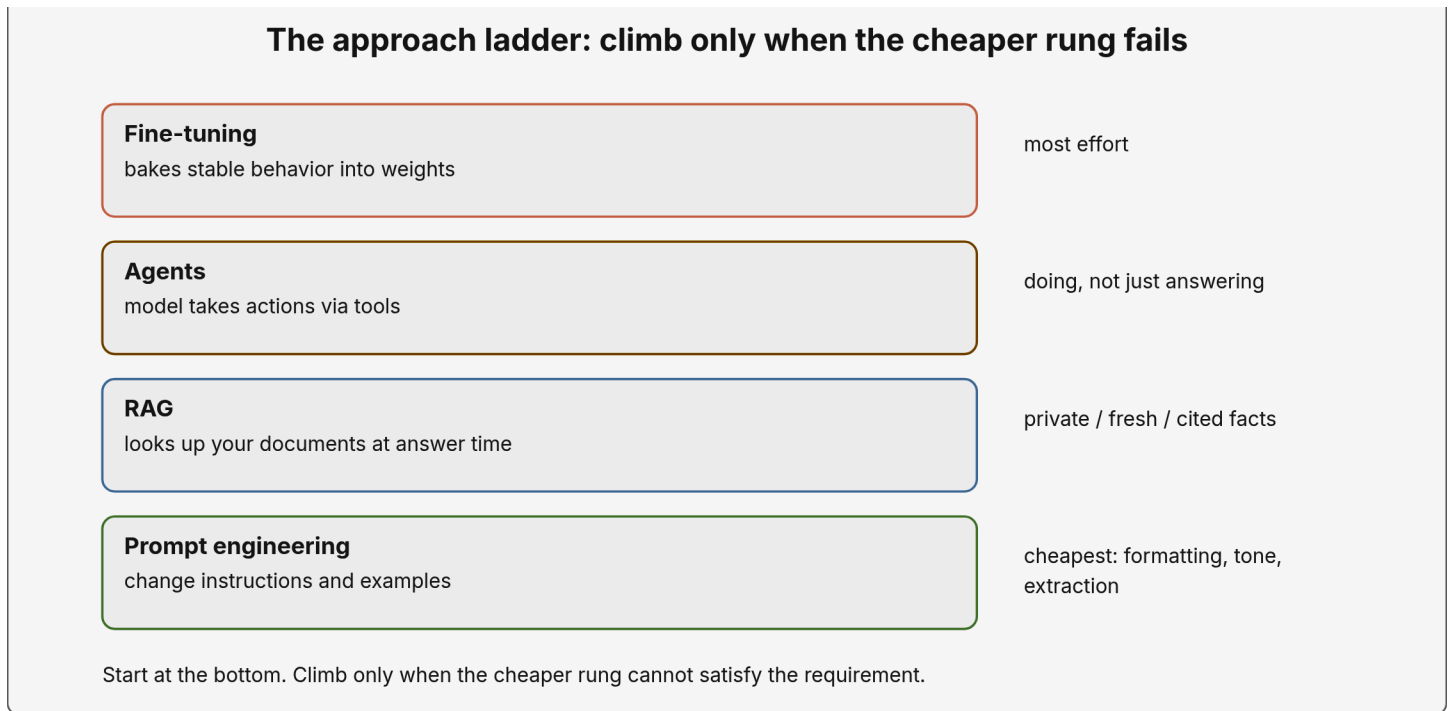
## D1: Foundation Model Integration, Data Management, and Compliance (31%)

---

This is the biggest domain. It sets the vocabulary for all later domains. D1 tests three things. It tests which GenAI approach fits the scenario. It tests how to call and feed Bedrock models correctly. It tests how to keep data compliant while you do this.

Almost every question uses the approach ladder plus one API or service detail.

## The approach ladder: Climb only when the cheaper step does not work.



The approach ladder: prompt engineering at the bottom, RAG above it, agents above that, fine-tuning at the top

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Read bottom to top: prompt engineering, RAG, agents, fine-tuning. Each rung lists what it solves: formatting at the bottom, private facts one rung up, doing things with agents, stable behavior at the top. Takeaway: the exam rewards the cheapest rung that works, never the fanciest.

A **foundation model** is a large model. It is pre-trained on much data. You adapt it instead of training it from the start. **Amazon Bedrock** is the managed AWS service. It provides foundation models from many providers. These providers include Anthropic Claude, Meta Llama, Mistral, Cohere, Amazon Titan and Nova, AI21, and Stability.

It uses one set of APIs. So, you never manage GPUs. The ladder shows the order you try solutions. Start with the cheapest solution first.

**Prompt engineering** is the bottom step. You change instructions and examples in the prompt. This does not add new knowledge. It does not need training. It does not add new cost. It solves formatting, tone, extraction structure, and simple classification.

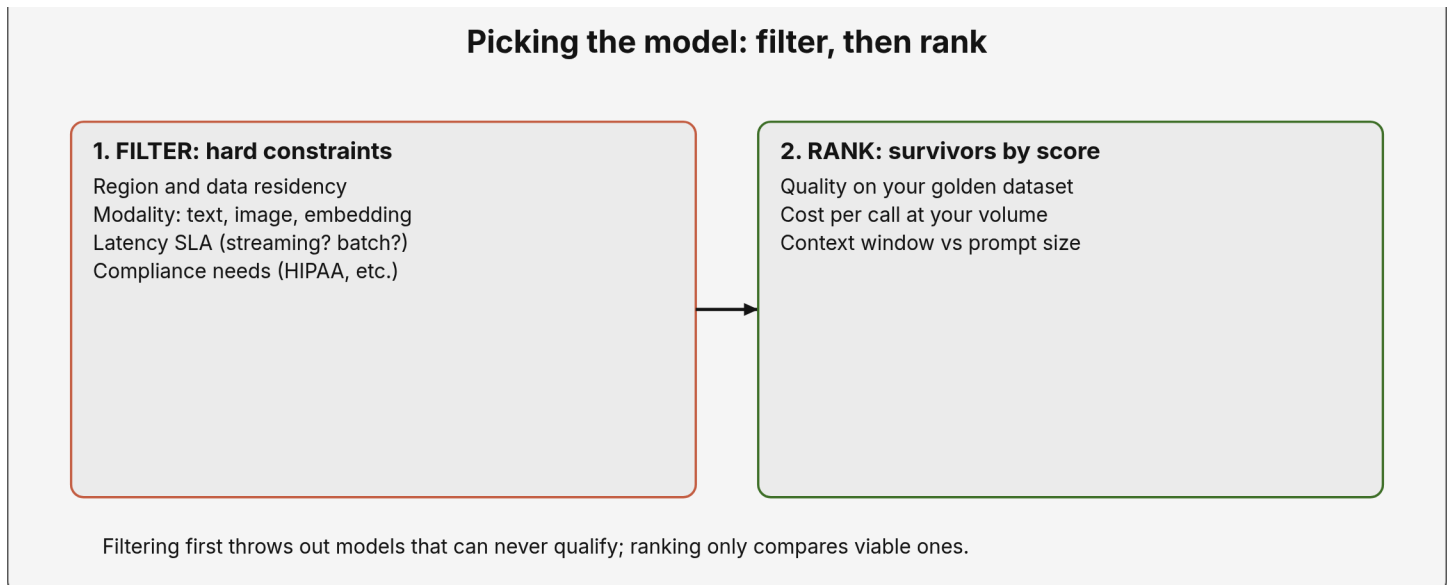
**RAG** (Retrieval-Augmented Generation) is the next step. The model finds facts from your documents when it gives an answer. It cites these facts. It solves private, changing, or cited knowledge. The model was never trained on this knowledge. **Agents** are next.

The model takes actions through tools. It calls APIs, queries databases, and runs multi-step plans. This step solves anything that needs action, not just an answer. **Fine-tuning** is the top step. You retrain parts of the model. You use labeled examples.

A new behavior becomes part of the model's weights. It solves stable behavior, tone, and format needs. Prompts cannot reliably keep these.

Rule: Start at the bottom. Climb only when the cheaper step cannot meet a stated need. "Documents change" makes fine-tuning useless. (Weights become old.) "Must cite sources" makes fine-tuning useless. (Weights cannot cite pages.) "Takes actions" makes RAG useless. (Knowledge bases do not call APIs.) "New tone" makes RAG useless. (Retrieval does not change behavior.) Read the need. Find which step it names. Pick the cheapest step that meets all needs.

## Picking the model: filter, then rank



Model selection funnel: hard constraints filter the field first, then rank survivors by quality and cost

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** First box: filter by region, modality, latency SLA, and compliance. Arrow into the second box: rank only the survivors on golden-dataset quality and cost per call. Takeaway: filter before you rank; never compare models that can never qualify.

Model selection is a filtering process. It is not a popularity contest. First, filter by **capability floor**. Run candidate models on a golden dataset. (This is a small set of prompts with good answers.) Remove any model below your quality level. Provider benchmarks never meet this level for you.

They measure the provider's prompts, not yours. Second, filter by **regional availability**. Bedrock publishes a list of supported models per region. A model in us-east-1 may not exist in eu-west-1. Model access is a one-time console action. Do this for each model and each region.

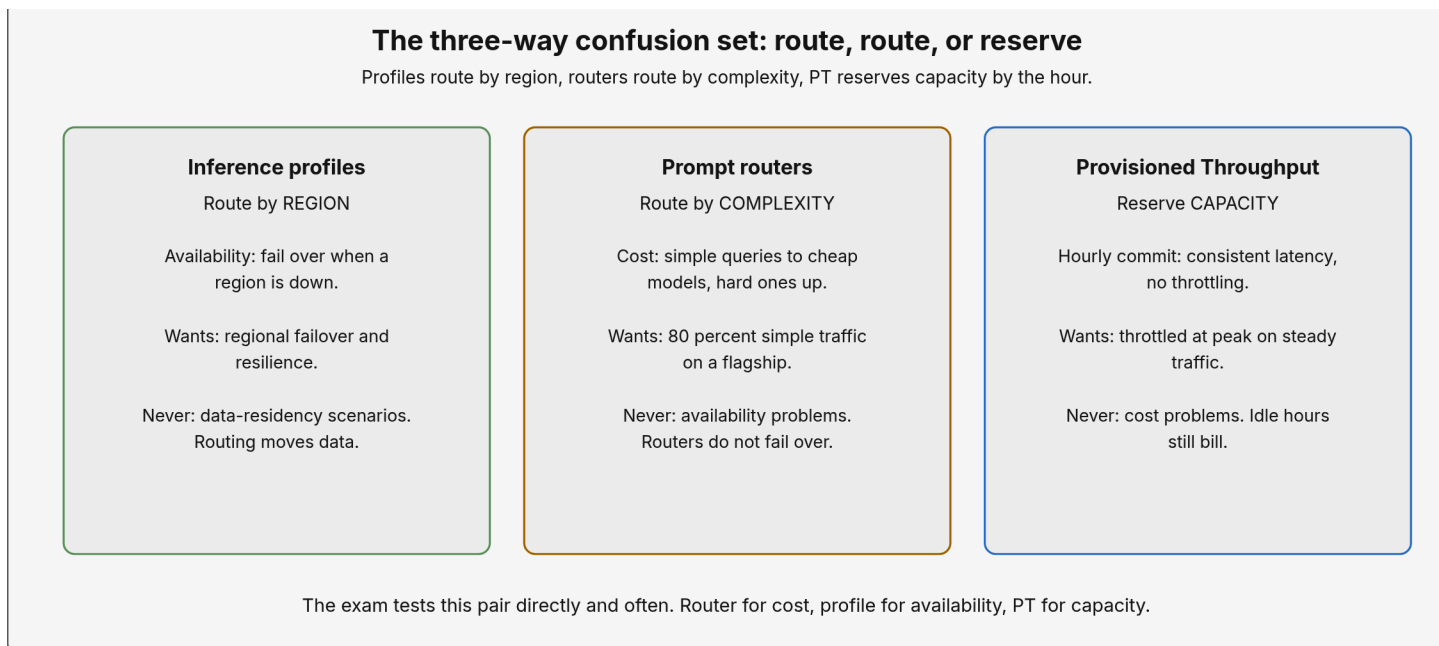
It costs nothing to enable. Third, rank the remaining models by **cost and latency**. Measure tokens per request. Measure time to first token on realistic inputs. Fourth, check the **context window**. (This is the maximum tokens a model accepts in one call.

It is specific to the model. Large models handle hundreds of thousands. Small models handle tens of thousands.) Check it against your longest realistic input plus the expected output. If the input does not fit, you need chunking, summarization, or a bigger window. The model with the lowest cost that meets all criteria wins. Check again every three months.

Prices and capabilities change.

There are two availability tools. Never swap them. **Cross-region inference profiles** send requests across regions for resilience. If one region throttles or goes down, traffic moves to another. They are the answer to "model X is only in two regions and we need resilience." They do not work with data-residency rules. This is because routing across regions moves data across regions.

**Prompt routers** (also called model routers) send requests by predicted prompt complexity for cost. Simple queries go to a cheap, small model. Hard queries go to the main model. They are the answer to "80% of our traffic is simple FAQ on an expensive model." Use a router for cost. Use a profile for availability. The exam tests this pair directly and often.



Three routing ideas look alike. Inference profiles route for availability. Prompt routers route for cost. Provisioned Throughput reserves capacity.

Source: original figure for the AWS AIP-C01 Study Guide.

How to read this diagram

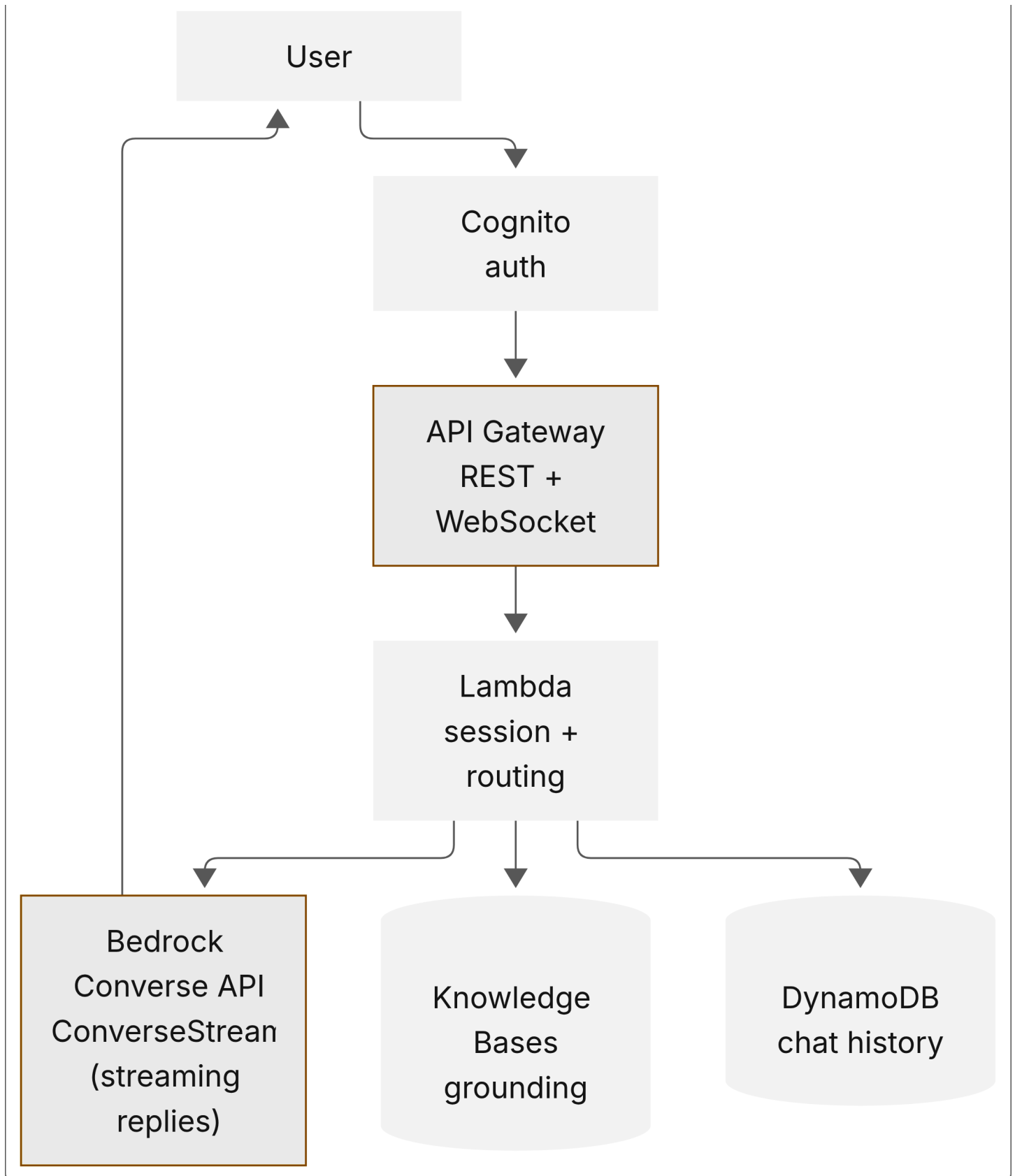
1. Left: inference profiles. They move traffic between regions for availability.
2. Middle: prompt routers. They send simple prompts to cheap models for cost.
3. Right: Provisioned Throughput. It reserves capacity. It does not route.

## Calling the model: four APIs, one decision

**Converse** is the unified Bedrock runtime API. It has one request shape. It includes `modelId`, `messages` (user and assistant roles), `system`, `inferenceConfig` (`maxTokens`, `temperature`, `topP`, `stopSequences`), optional `toolConfig` (function definitions), optional `guardrailConfig` (attaches a Bedrock Guardrail, the content filters and policy checks taught fully in D3, by guardrail identifier plus version, with a trace flag), optional `cachePoint` (prompt caching marker), and `additionalModelRequestFields` (the escape hatch for provider-specific extras). It works across providers. It is the default for text generation, tool use, and inline guardrails.

**InvokeModel** is the original API. You send the provider's raw JSON body. You get the provider's raw response. For Claude, you must include `anthropic_version: "bedrock-2023-05-31"` and `max_tokens`. If you do not, the call fails with a validation error. InvokeModel is the ONLY way for embeddings.

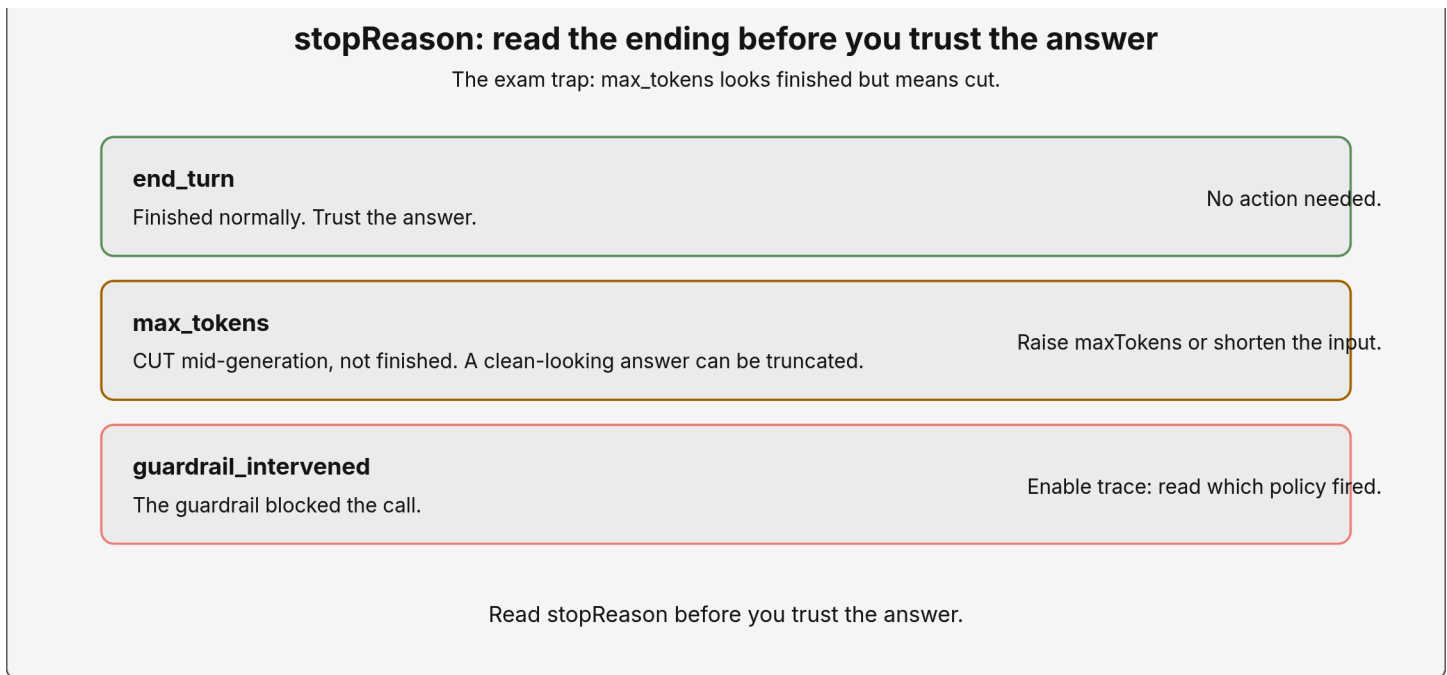
This is because Converse is only for text generation. The exam offers Converse as a wrong answer for embedding tasks. It is always wrong.



Converse is the default call shape: one contract across providers, with tool use, caching, and guardrails built in.

Source: original figure for the AWS AIP-C01 Study Guide.

Reading a Converse response: `output.message.content` is a list of content blocks. The `stopReason` field tells you why generation ended. `end_turn` means the run finished. `max_tokens` means the output was cut during generation. It did not finish. A good-looking answer with this stop reason is cut short. Increase `maxTokens` or make the input shorter. `guardrail_intervened` means the guardrail blocked the call.



`stopReason` tells you why generation ended. `end_turn` means finished. `max_tokens` means cut mid-generation. `guardrail_intervened` means the guardrail blocked the call.

Source: original figure for the AWS AIP-C01 Study Guide.

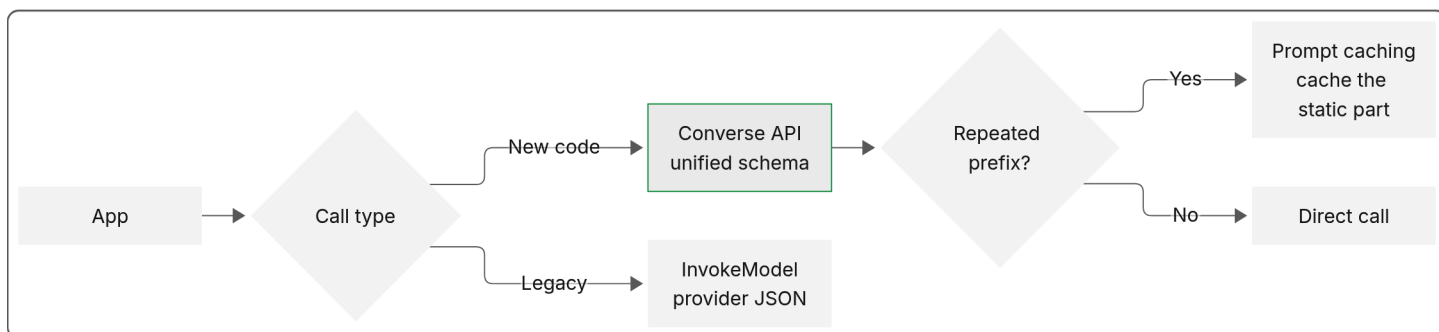
### How to read this diagram

1. Top row: `end_turn`. The run finished. Trust the answer.
2. Middle row: `max_tokens`. The output was cut. Raise `maxTokens` or shorten the input.
3. Bottom row: `guardrail_intervened`. The guardrail blocked the call. Check which policy fired.

Latency and cost have their own call shapes. **Streaming** (`ConverseStream`, `InvokeModelWithResponseStream`) returns tokens as the model generates them. It changes how fast it feels, not the total time. A chat that starts answering in less than one second feels fast. This is true even if the full answer takes ten seconds. Streaming is the answer to "users report slow responses" on interactive traffic.

**Batch inference** is the offline shape. It uses a JSONL file (one JSON object per line) into S3. Results go out to S3. It has hours of latency. It is about 50 percent cheaper per token than on-demand. It is the answer to "cheapest for nightly bulk work with no SLA." Never use it for interactive SLAs.

**Async invocation** (`start_async_invoke`) is for a single slow request. One prompt that no one waits for. The result goes to S3. Batch is many prompts from S3. Async is one slow prompt. Streaming sends tokens to a waiting user.



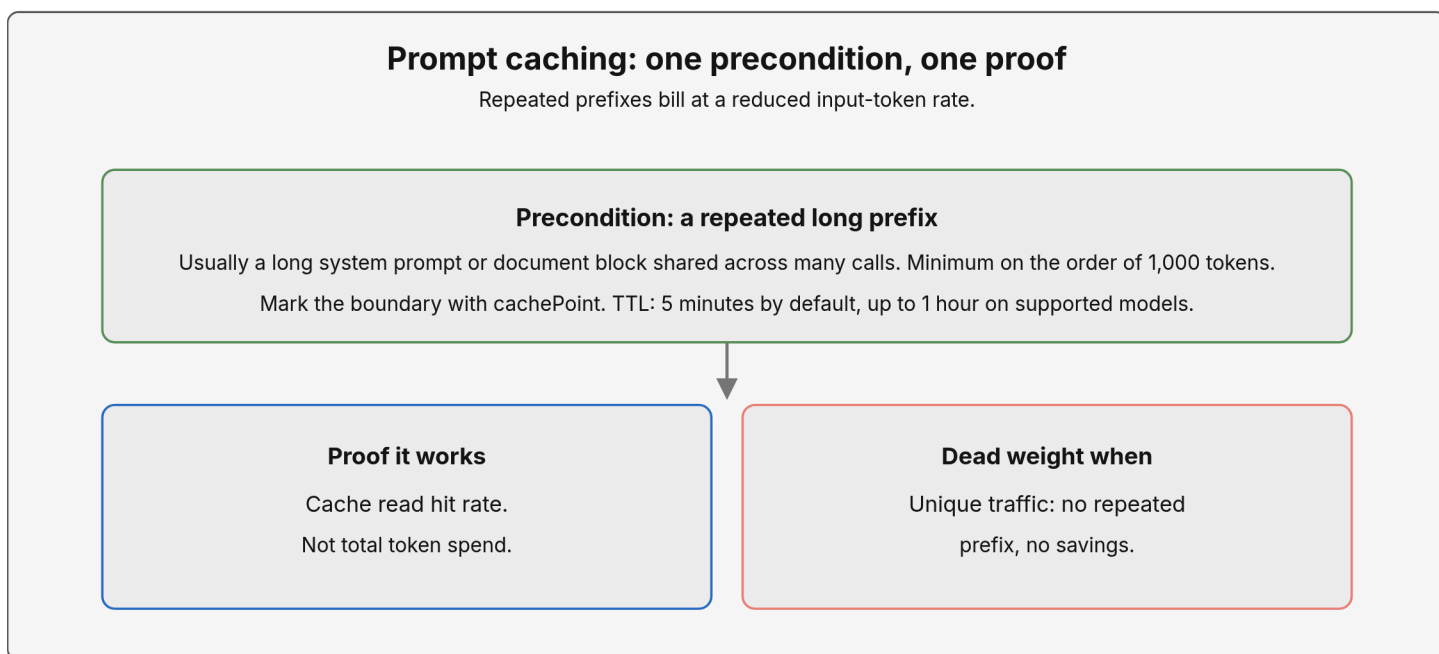
Prompt caching: repeated long prefixes bill at a reduced rate. Its only precondition is a repeated prefix.

Source: original figure for the AWS AIP-C01 Study Guide.

**Prompt caching** needs one condition. It needs a repeated long prefix. This is usually a long system prompt or document block. Many calls share it. Mark the boundary with `cachePoint`. Cache hits bill at a lower input-token rate.

The minimum prefix length depends on the model. It is about one thousand tokens. The cache TTL is 5 minutes by default. It can be up to 1 hour on supported models. Unique traffic makes caching useless. No repeated prefix means no savings.

The metric that shows caching works is cache read hit rate. It is not total token spend.



Prompt caching needs a repeated long prefix. Mark it with `cachePoint`. The proof is cache read hit rate.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

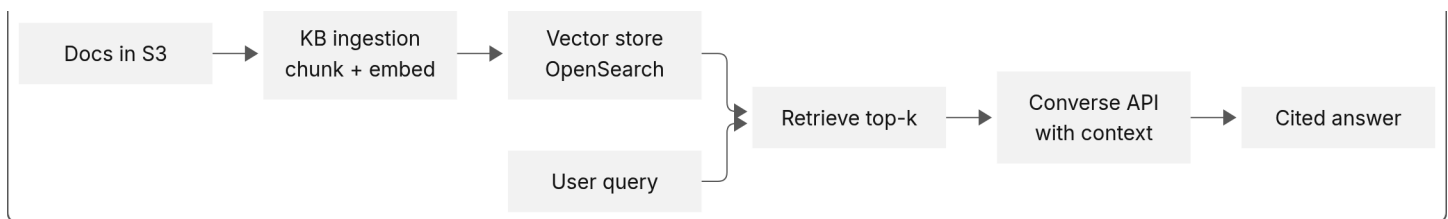
1. Left: the precondition. A long prefix repeats across calls.
2. Middle: the mark. `cachePoint` tells Bedrock what to cache.
3. Right: the proof. Watch cache read hit rate, not total spend.

## RAG in depth: This is the most tested pattern in the domain.

RAG uses your documents to ground the model. At ingestion, the system chunks documents into small pieces. Each piece fits the context window. Each piece keeps a tight focus to match precisely. The system converts each chunk to an **embedding**.

An embedding is a vector of numbers. Similar meanings are close together in this vector. Vectors, text, and metadata go into a **vector store**. At query time, the system embeds the question with the same model. The store returns the nearest chunks. The model makes an answer and cites the chunks.

An **Amazon Bedrock Knowledge Base** is the managed version. Point it at data sources. Data sources include S3, web crawlers, Confluence, and SharePoint. Data sources also include Salesforce. It handles chunking, embedding, storage, and sync. `RetrieveAndGenerate` does the full loop in one call.



The RAG pipeline: chunk, embed, store once; embed the query, retrieve, generate at answer time.

Source: original figure for the AWS AIP-C01 Study Guide.

Embeddings have two choices for the exam. **Amazon Titan Text Embeddings v2** is the low-cost default. It has configurable dimensions: 1024, 512, or 256. Changing from 1024 to 256 reduces vector storage by 75 percent. It also makes search faster. This change has an accuracy trade-off.

You measure this trade-off on a golden dataset. **Cohere Embed** is the multilingual choice. It supports more than 100 languages. It takes an `input_type` parameter. This parameter is `search_query` or `search_document`. It embeds queries and documents with different prefixes.

These prefixes are good for each role. Embedding calls use `InvokeModel`. They do not use `Converse`.

The rule for retrieval is important. Use the **same embedding model and dimensions at ingestion and query time**. If you do not, retrieval breaks silently. This is because the vector spaces do not align. If answers are confident but wrong after a re-sync, the re-sync changed the embedding model version. Or the re-sync changed the chunking configuration.

Find the change and fix it.

Vector store choice depends on data size. Small datasets, under about 1 million records, use **Aurora Serverless with pgvector**. This has the fewest self-managed components. It is the answer for "least operational overhead." Large datasets use **OpenSearch Serverless**. It uses approximate search. This includes HNSW or IVFFlat indexes.

## Two embedding choices, one iron rule

Embeddings go through InvokeModel. Converse is text generation only.

### Titan Text Embeddings v2

The cheap default.

Dimensions: 1024, 512, or 256.

256 cuts storage to a quarter  
and speeds search.

Measure the accuracy trade-off  
on a golden dataset.

### Cohere Embed

The multilingual pick: 100+ languages.

input\_type: search\_query  
vs search\_document.

Different prefixes for queries  
and documents.

### THE IRON RULE

**Same embedding model and dimensions at ingest and query time.**

Mismatch breaks retrieval silently: the vector spaces do not align.

Match the embedding model at ingest and query time. Titan Text Embeddings v2 gives configurable dimensions. Cohere Embed covers many languages.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left: Titan Text Embeddings v2. Pick the dimension count for your cost and quality target.
2. Right: Cohere Embed. Use it for multilingual text.
3. Bottom rule: the query must use the same model and dimensions as ingest.

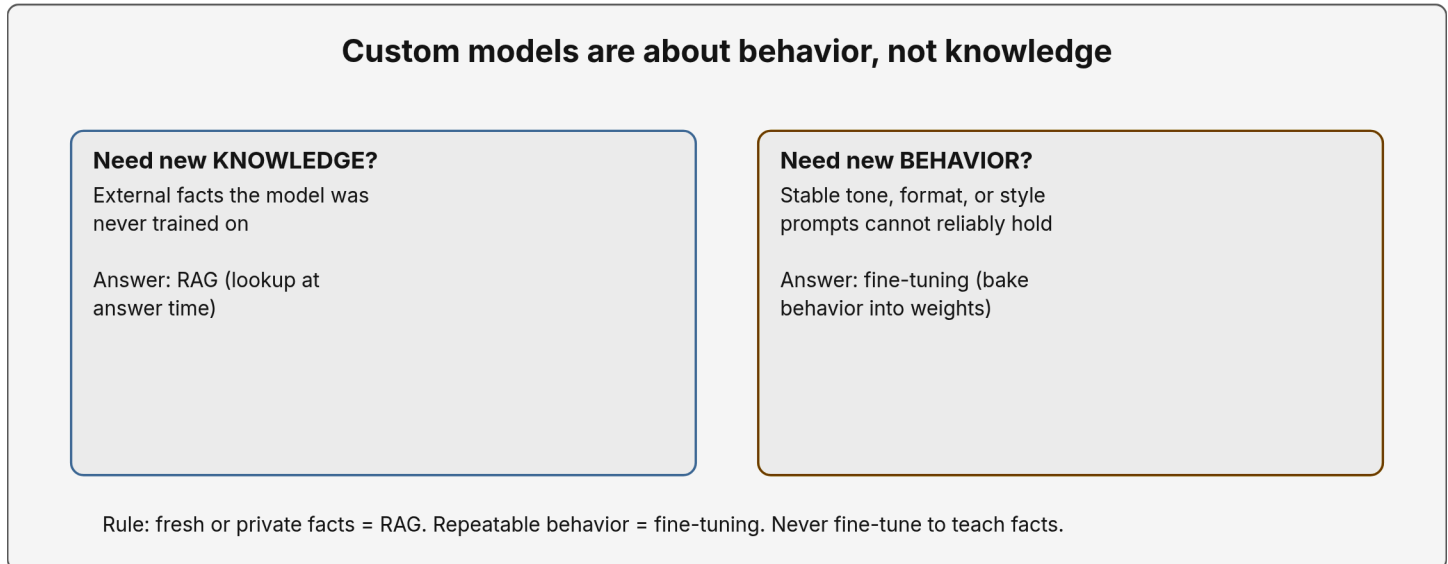
These are approximate index types. They trade a small loss in recall for large speedups. "100% recall required on small data" means **exact flat search**. Approximate indexes trade recall for speed by design. Small data makes the exact scan affordable. Do not embed relational tables into a vector store.

Structured data with SQL-shaped questions uses **text-to-SQL** against RDS or Aurora. It does not use chunks. Unstructured documents, like PDF, HTML, and Word, belong in the knowledge base.

The exam names two quality upgrades. **Hybrid search** combines keyword matching with vector similarity. BM25 is the classic keyword-ranking formula. Exact terms and meaning both count. A **reranker** re-scores the top retrieved chunks. Cohere Rerank on Bedrock is an example.

It uses a stronger model and keeps the best chunks. Together, they answer "retrieval quality is too low and we did the basics." The checklist for silent retrieval problems has three items. First, embedding model mismatch between ingest and query. Second, chunk boundaries that split key facts across chunks. Third, a knowledge base sync that is not in-sync.

## Custom models: behavior, not knowledge



Behavior versus knowledge: new knowledge means RAG, new behavior means fine-tuning

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Two columns answer one question. Left: need new knowledge (facts the model never trained on) so the answer is RAG. Right: need new behavior (tone or format prompts cannot hold) so the answer is fine-tuning. Takeaway: never fine-tune to teach facts.

Bedrock offers four ways to adapt a model. The exam tests which gap each one fills. **Fine-tuning** is supervised. It trains on labeled prompt-and-response pairs in S3. It teaches behavior. This includes support tone, classification, and response format.

Useful runs start with hundreds to a few thousand high-quality labeled examples. Quality is more important than quantity. **Continued pre-training** trains further on unlabeled domain text. It teaches vocabulary and domain knowledge. **Distillation** trains a small, low-cost student model. It imitates a large teacher model on one narrow task.

It gives near-teacher quality at student cost. This is the answer to "same narrow task millions of times, cost is the problem." **Imported models** bring weights you customized elsewhere into Bedrock. They serve through Bedrock APIs. On the SageMaker side, the same idea appears as **LoRA**. LoRA means Low-Rank Adaptation. Train only small weight deltas, called adapters.

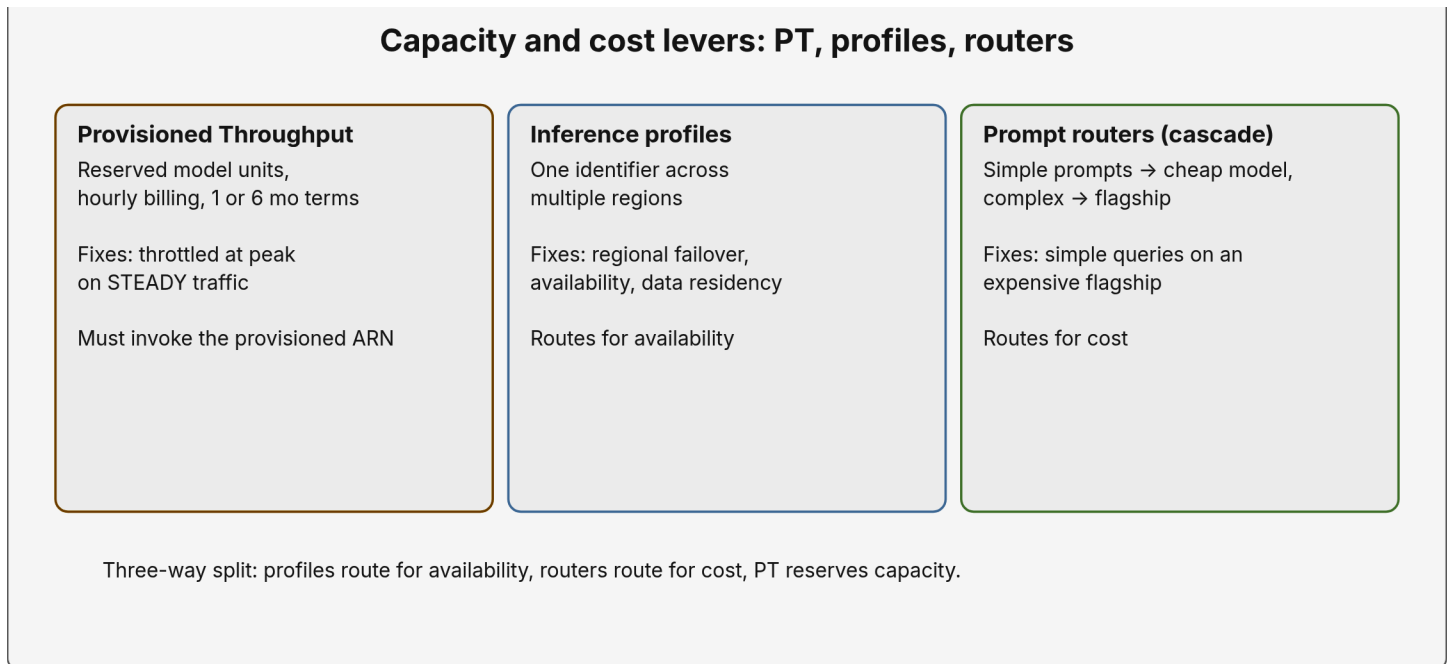
Do not train all weights. Then version them in the **SageMaker Model Registry** with pipelines and rollback.

A fine-tuning job reads training and validation files from S3. It runs for hours. It makes a custom model with its own ARN. ARN means Amazon Resource Name. It is the unique ID of any AWS resource. Serving it usually needs **Provisioned Throughput**.

This is reserved inference capacity. The next section explains it. An exception is some models, like Nova custom models. They support on-demand. "The customization job succeeded but calls fail" means the custom model has no throughput provisioned. It does not mean bad training data.

The main trap: fine-tuning for **knowledge** instead of **behavior**. For quarterly document updates, citations, and fresh facts, use RAG. For tone, format, and stable behavior, use fine-tuning. The exam will make a knowledge problem look like a behavior problem. Read the requirement, not the appearance.

### Capacity and cost levers: PT, profiles, routers



Capacity and cost levers: Provisioned Throughput, inference profiles, and prompt routers

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Three boxes, three different jobs. Left: PT reserves hourly capacity for steady traffic. Middle: profiles route across regions for availability. Right: routers cascade simple prompts to cheap models for cost. Takeaway: match the lever to the symptom, they are not interchangeable.

**Provisioned Throughput (PT)** is reserved inference capacity. You buy **model units**. A model unit is a fixed amount of processing capacity for one model. You buy it for a commitment term. This is commonly 1 month or 6 months. You pay an hourly rate per unit.

You pay whether you use it or not. In return, requests to your provisioned model ARN get dedicated capacity. This means consistent latency. It also means no throttling from shared on-demand quotas. Create it with

`CreateProvisionedModelThroughput`. Invocations must target the returned **provisioned model ARN**.

They must not target the base model ID. "Bought PT but still throttles" means the code passes the base model ID. It does not pass the provisioned ARN.

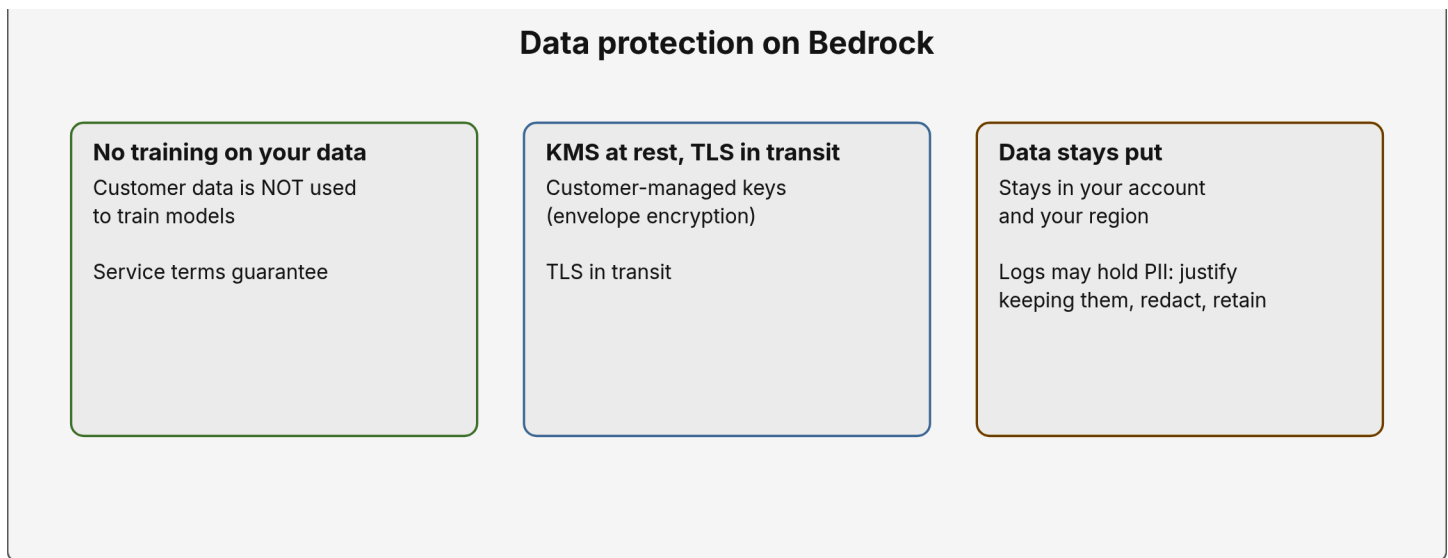
PT guarantees latency and capacity for steady production baselines. It is not a discount for spiky traffic. Idle hours still bill. So, bursty workloads stay on on-demand.

There are three confusing items. The exam tests them often. **Inference profiles** route for availability across regions. **Prompt routers** route for cost by prompt complexity. **Provisioned Throughput** reserves capacity by the hour. Regional failover needs a profile.

Simple queries on an expensive flagship model need a router. Throttling at peak on steady traffic needs PT. The exam also tests the cascade pattern for routers. A small classifier sends simple requests to a low-cost, lightweight model. Simple requests include classification, extraction, and short answers. It sends only complex requests to the flagship model.

Right-sizing means choosing the smallest model that meets the quality standard. This is the same idea stated in another way.

## Data protection and compliance



Bedrock data protection: no training on your data, KMS at rest, TLS in transit, data stays in your account

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Three guarantees side by side. Left: your data never trains models. Middle: encryption at rest and in transit. Right: data stays in your account and region. Takeaway: the privacy answer is almost always already in one of these three boxes.

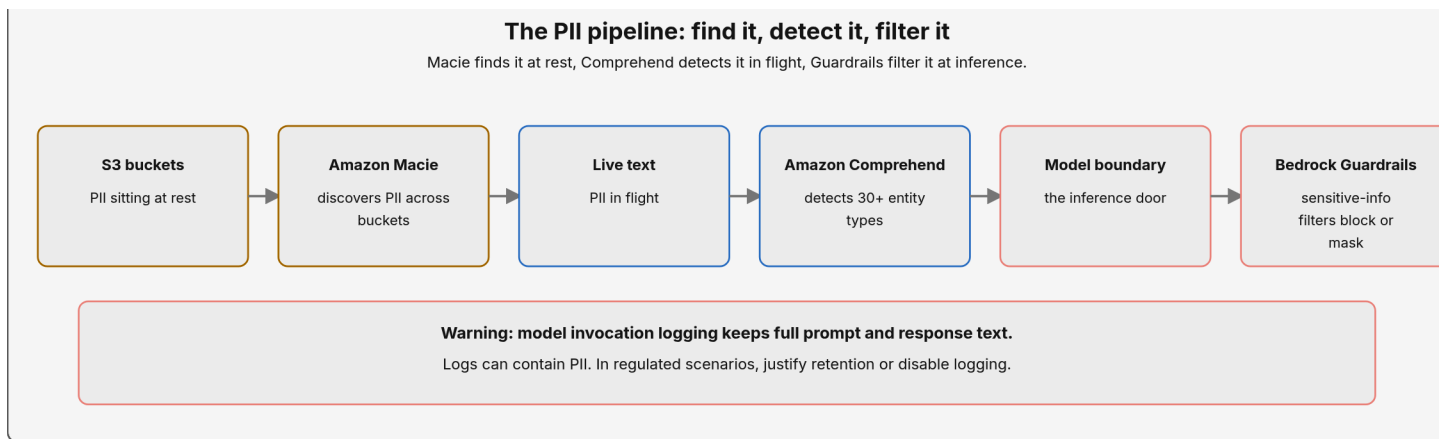
Bedrock traffic stays private. It uses **VPC interface endpoints**. This is PrivateLink. It is for the Bedrock, Bedrock runtime, agent, and agent-runtime APIs. There is no NAT gateway. There is no public internet.

**IAM** policies define which identities can invoke which model. They also define which knowledge base and agent ARNs. **KMS** encrypts data at rest. This is envelope encryption with customer-managed keys. **TLS** encrypts data in transit. Customer data sent to Bedrock is not used to train models.

This is under the service terms. Data you choose to log stays in your account.

The PII pipeline has three services in order. **Amazon Macie** finds PII at scale in S3. It scans buckets and flags sensitive data before ingestion. **Amazon Comprehend** detects PII entities in live text in real time. It finds more than 30 entity types. These include names, SSNs, and account numbers.

**Bedrock Guardrails** sensitive-information filters block or mask PII. This happens at the model boundary. It happens on inputs and outputs. Macie finds it at rest. Comprehend detects it in flight. Guardrails filter it at inference.



Each service finds PII in one place. Macie scans S3 at rest. Comprehend detects PII in live text. Guardrails filter PII at inference.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left box: Macie. It finds PII stored in S3 buckets.
2. Middle box: Comprehend. It detects PII entities in text you send it.
3. Right box: Guardrails. They filter PII at the model boundary.

In scenarios with high compliance, consider if **model invocation logging** should exist. This is the opt-in record of full prompt and response text. You can route it to S3 or CloudWatch. You can encrypt it with KMS. Logs can contain PII. So, retention policies and redaction are part of the design.

### D1 decision rules: if the stem says X, pick Y

- If the stem says "Facts change over time" or "must cite sources," pick RAG with Knowledge Bases. Never use fine-tuning.
- If the stem says "New tone, format, style, behavior," pick fine-tuning or prompt engineering. Never use RAG.
- If the stem says "Take actions," "call APIs," or "multi-step with tools," pick Bedrock Agents. Knowledge bases do not act.
- If the stem says "Unstructured docs (PDF, HTML)," pick Knowledge Bases. If it says "Relational data, SQL questions," pick text-to-SQL on RDS. Never embed tables into a vector store.
- If the stem says "Small vector dataset" (under about 1 million records), pick Aurora Serverless with pgvector. If it says "Large," pick OpenSearch Serverless. If it says "100% recall on small data," pick exact flat search.
- If the stem says "Throttled at peak, steady traffic," pick Provisioned Throughput. Pass the provisioned ARN as modelId. If it says "Spiky or low traffic," pick on-demand.
- Use cross-region inference profiles for regional failover and resilience. This is not possible with data-residency constraints.
- Use a prompt router or cascade for simple queries on an expensive model. Never use Provisioned Throughput for a cost problem.
- Use prompt caching for a repeated long system prompt. Check the cachePoint and the cache read hit rate. Use streaming or a smaller model for unique interactions.

- If users report slow responses, use streaming (ConverseStream) for perceived latency. Provisioned Throughput fixes capacity. It does not fix first-token speed.
- Use batch inference for nightly bulk work with no SLA. Put JSONL into S3. Put results into S3. This saves about 50% of the cost.
- To swap models without redeploying, use a model ID in AppConfig. AppConfig is AWS's runtime configuration service. Read the model ID at runtime. CloudFormation changes are deployments.
- Use InvokeModel for embeddings. Never use Converse. Use the same model and dimensions at ingest and query.
- If answers are confident but wrong after a re-sync, check the embedding model version. Check for chunking drift. Debug the change.
- Use Macie for PII across S3 buckets. Use Comprehend for PII in live text. Use Guardrails to redact at call time.
- Use VPC interface endpoints for no public internet. A NAT gateway does not meet this constraint.

### Custom models: the full playbook

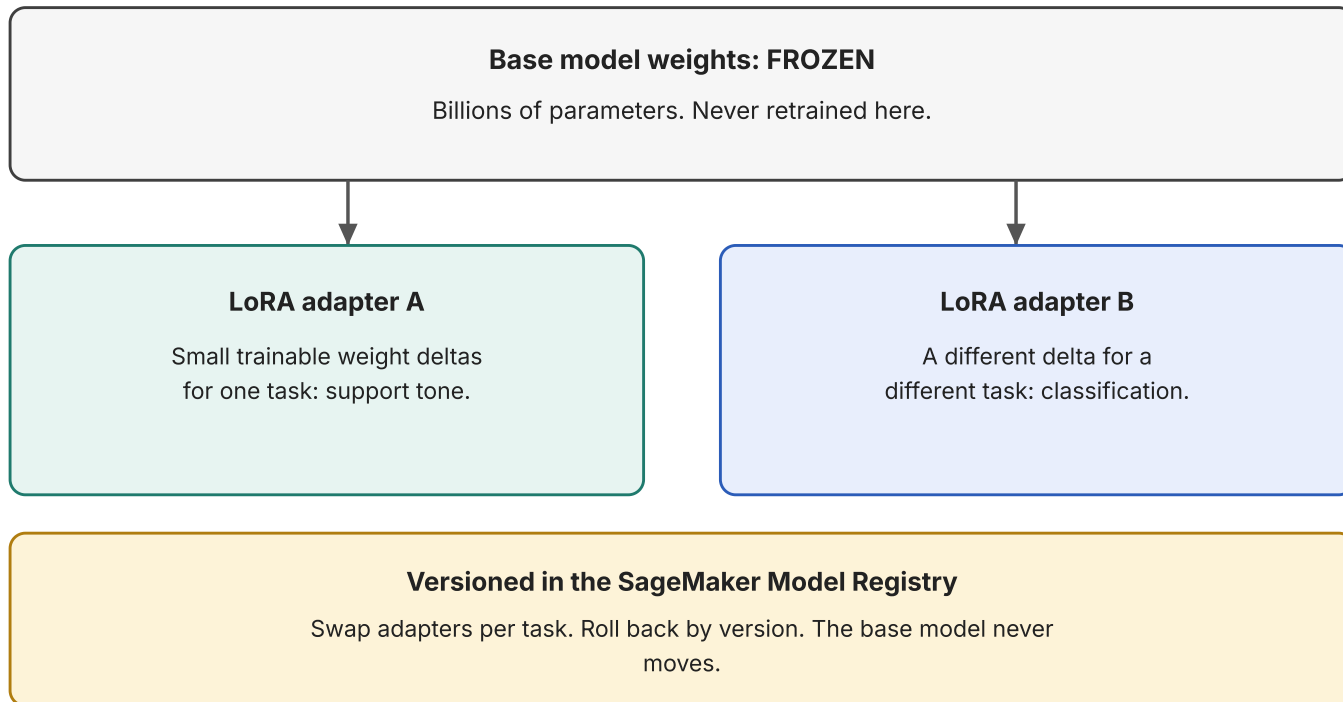
Bedrock offers six ways to customize a model. Each one answers a different need. The exam names the need. You name the method.

| Need                                                             | Method                    | Data                                     | Cost and effort                                                                                        |
|------------------------------------------------------------------|---------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------|
| New behavior from labeled examples: tone, format, classification | Fine-tuning (supervised)  | Labeled prompt and response pairs        | Medium. This is the default customization answer.                                                      |
| Domain vocabulary from unlabeled text                            | Continued pre-training    | Large unlabeled corpus in the domain     | High. It teaches the model a new dialect.                                                              |
| Small cheap model that matches a big one on one narrow task      | Distillation              | Teacher model outputs on the narrow task | Medium. Near-big quality at small-model cost.                                                          |
| Tune with feedback or reward signals                             | Reinforcement fine-tuning | Preference or reward data                | High. It aligns the model to human preference.                                                         |
| Serve a model customized outside Bedrock                         | Imported model            | Weights built elsewhere                  | Low on Bedrock. The training happened elsewhere.                                                       |
| Parameter-efficient training per task                            | LoRA adapters             | Small task dataset                       | Low. Base weights stay frozen. Adapters version in the SageMaker Model Registry and roll back cleanly. |

Read the stem for the data it gives you. Labeled pairs point to fine-tuning. Unlabeled domain text points to continued pre-training. A cost problem on one narrow task points to distillation.

**LORA: TRAIN THE DELTA, FREEZE THE BASE**

Low-Rank Adaptation trains small adapters instead of all weights.



LoRA keeps the base model frozen and trains small adapters per task. Each adapter versions separately, so a bad task update rolls back without touching the base.

Source: original figure for the AWS AIP-C01 Study Guide.

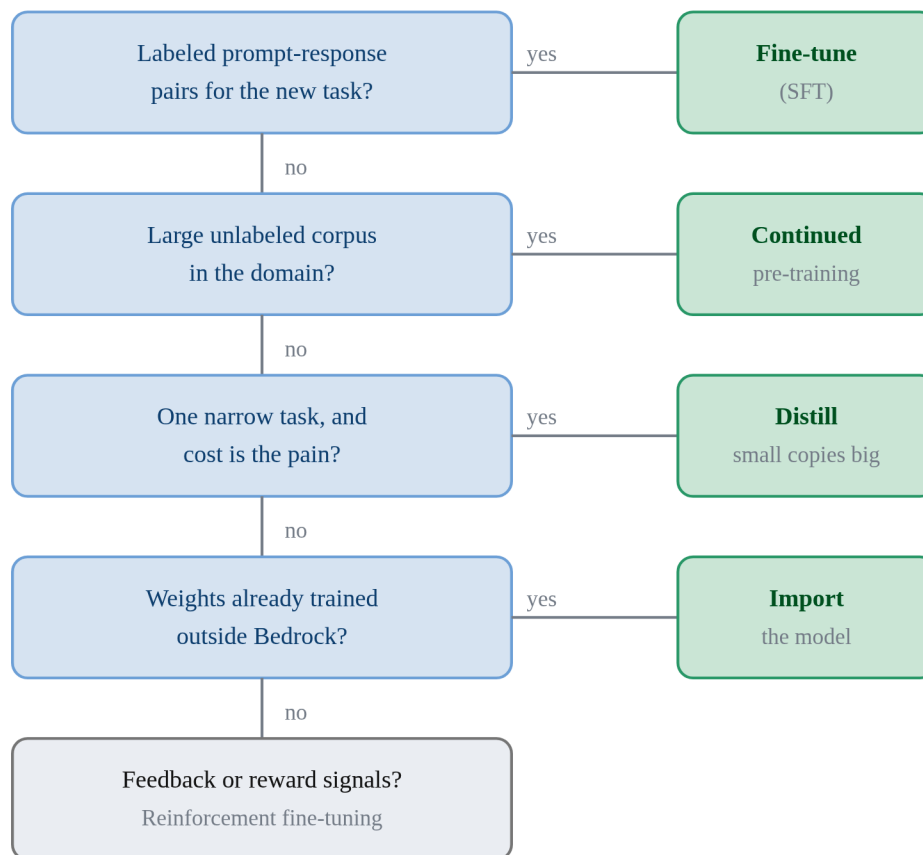
**How to read this diagram**

1. Left: the base model. Its weights never change.
2. Middle: one small adapter per task. Only the adapter trains.
3. Right: the registry. Adapters version and roll back like code.

**Exam relevance.** D1 is 31% of the exam. Customization method questions are pure decision rules. Match the need to the row.

## Which customization method?

One claim: the data you hold names the method.



Read the stem for the data it gives you. The data names the method.

Shell: decision flowchart. Source: original figure for the AWS AIP-C01 Study Guide.

Follow the data. Labeled pairs mean fine-tuning. Unlabeled domain text means continued pre-training. One narrow task with a cost problem means distillation.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Start at the top. Answer each question with yes or no.
2. Yes leads right to the method. No leads down to the next question.
3. The data you hold names the method. Nothing else matters.

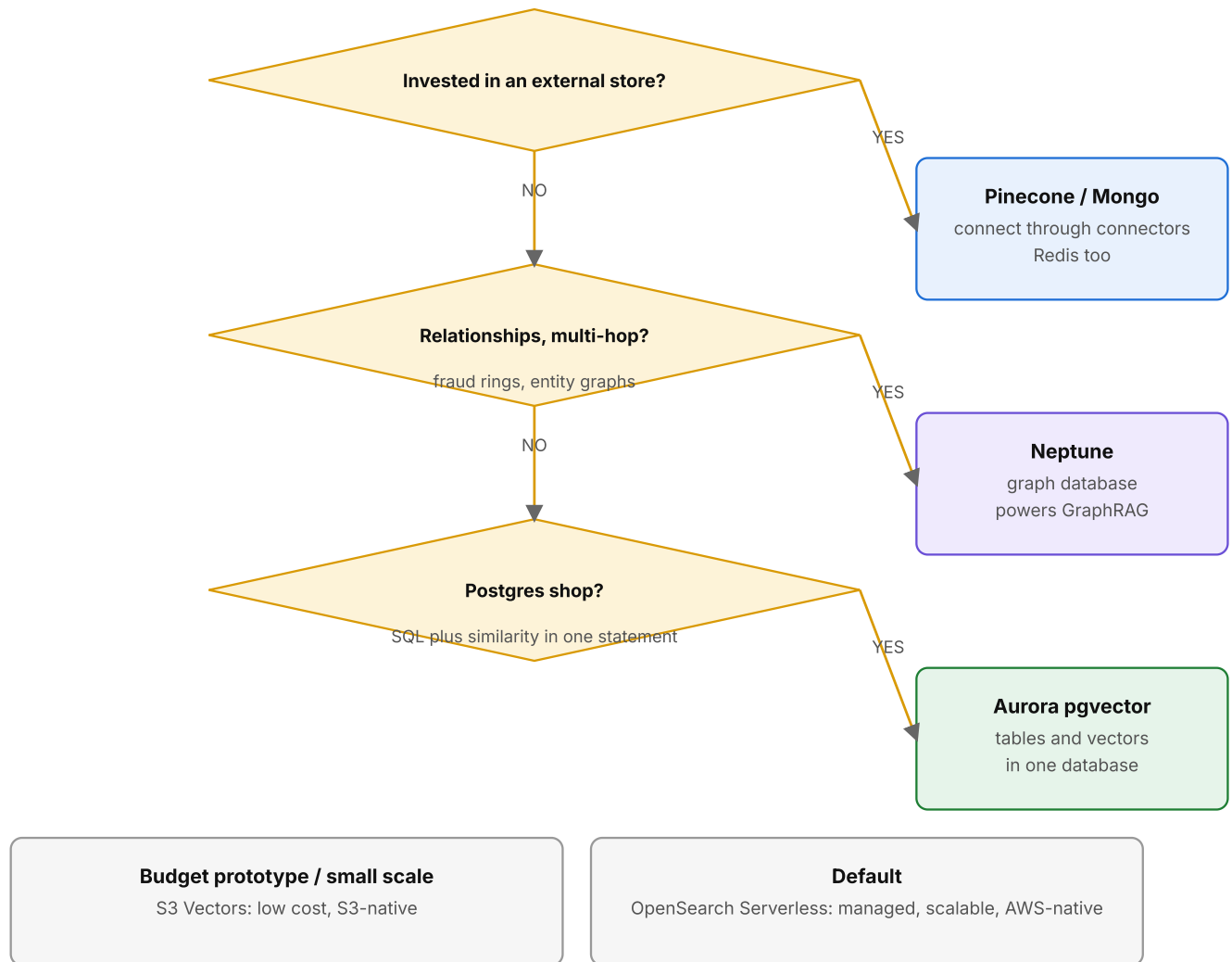
## Embeddings and vector stores

Two decisions make or break RAG. Pick the embedding model. Pick the vector store. Both have exam-favorite answers.

| Embedding model          | Use when                        | Key fact                                                                |
|--------------------------|---------------------------------|-------------------------------------------------------------------------|
| Titan Text Embeddings v2 | Default choice for English text | Configurable dimensions. It is the cheap default.                       |
| Cohere Embed             | Multilingual content            | Supports more than 100 languages. It is the exam's multilingual answer. |

One rule governs both: use the same model and the same dimensions at ingest and at query time. A mismatch breaks retrieval. The vector spaces do not align.

| Vector store                    | Use when                                                                                  |
|---------------------------------|-------------------------------------------------------------------------------------------|
| Aurora Serverless with pgvector | Small dataset, under about 1 million records                                              |
| OpenSearch Serverless           | Large dataset, full search features                                                       |
| S3 Vectors                      | Cheap S3-native storage for vectors                                                       |
| DynamoDB                        | Key-value access patterns with vectors attached                                           |
| Pinecone or MongoDB Atlas       | External SaaS vector database                                                             |
| Neptune                         | GraphRAG: relationships between entities matter. Never use it as a plain document store.  |
| Exact flat search               | 100% recall required on small data. Approximate indexes trade recall for speed by design. |

**WHICH VECTOR STORE? FOLLOW THE SCENARIO VERB**

Match the verb in the scenario to the branch. When none of the special cases apply, OpenSearch Serverless is the default.

Match the store to the scenario verb. "Small dataset" points to Aurora. "Graph relationships" points to Neptune. "Exact recall" points to flat search.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Read the scenario verb first: small, large, graph, exact.
2. Follow it to one store. Each verb has one home.
3. Check the trap column: the wrong stores are real services doing the wrong job.

**Never confuse:** Neptune with a document vector store. Neptune is a graph store. It serves GraphRAG relationships. It does not serve plain document chunks. Kendra with a RAG vector store. Kendra is enterprise search. It is not the RAG path.

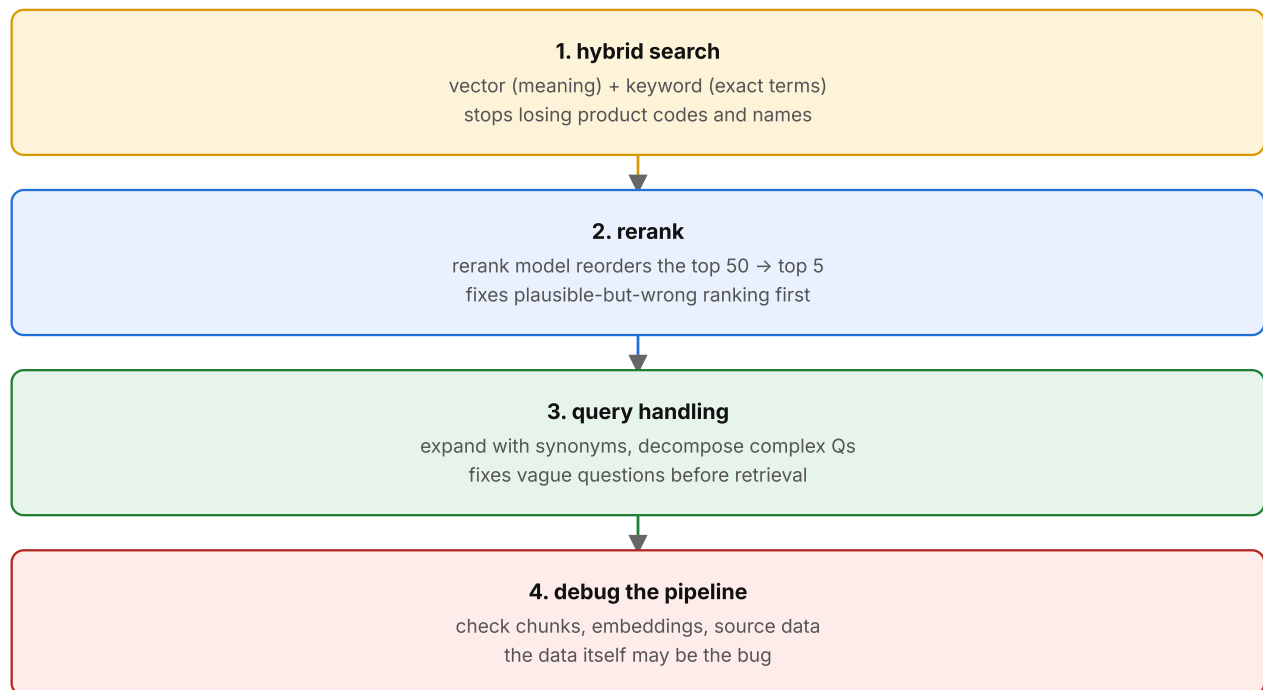
**Exam relevance.** Vector store questions test one verb. Find the verb in the stem. Pick the matching row. The distractors are real stores with the wrong verb.

## Retrieval quality: fix in this order

Bad RAG answers have a fix order. The exam asks what to change first. Memorize the ladder.

1. **Chunking strategy.** Fixed, hierarchical, or semantic. About 300 tokens per chunk with overlap is the typical shape.
2. **Embedding model.** A better model or better dimensions for the content type.
3. **Retrieval tuning.** Top-k size, hybrid search, reranking. Too few chunks starve the answer. Too many add noise.
4. **Prompt and generation.** Only after retrieval is solid.

### RETRIEVAL IS WRONG? TRY THE LEVERS IN THIS ORDER



Rule: hybrid first, rerank second, query handling third, pipeline debugging last. The exam expects this order.

Fix retrieval top to bottom. Chunking first, then the embedding model, then retrieval tuning, then the prompt. Never start at the bottom.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Top lever: chunking. Wrong cuts poison everything downstream.
2. Second lever: the embedding model. It decides what "similar" means.
3. Third lever: retrieval tuning. Top-k and reranking shape the candidate pool.
4. Bottom lever: the prompt. It cannot fix bad chunks.

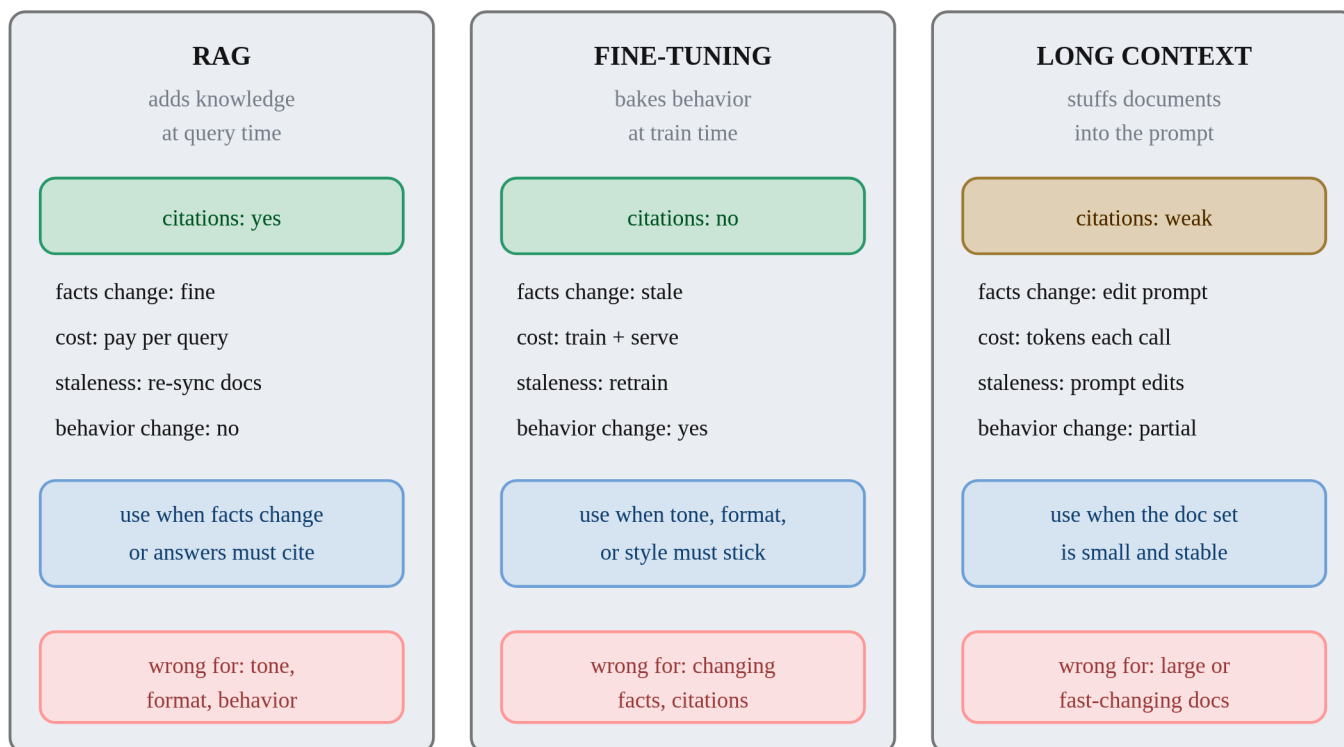
Bedrock Knowledge Bases give two query APIs. **RetrieveAndGenerate** is one managed call. Bedrock retrieves, builds the prompt, generates, and returns the answer with citations. **Retrieve** returns only the chunks. You build the prompt yourself.

| Need                                                            | Pick                                                                       |
|-----------------------------------------------------------------|----------------------------------------------------------------------------|
| Grounded answers with citations, least effort                   | RetrieveAndGenerate                                                        |
| Custom reranking, custom citation format, custom prompt control | Retrieve, then Converse                                                    |
| Chunks for a downstream system, no generation                   | Retrieve                                                                   |
| Similarity vs relevance                                         | Vector search finds similar chunks. The reranker picks relevant ones.      |
| Strands vs Squad                                                | Strands loops tools inside one agent. Squad routes across many agents.     |
| TTFT vs latency                                                 | TTFT is request to first token, streaming only. Latency is the whole call. |
| CloudTrail vs invocation logs                                   | CloudTrail proves the call happened. Invocation logs show the prompt.      |

**Exam relevance.** "What do you fix first when RAG answers are wrong" is a ladder question. The answer is always chunking before the model. "Citations with least effort" is always RetrieveAndGenerate.

## RAG vs fine-tuning vs long context

One claim: match the change you need to the column that makes it.



Knowledge questions take the left column. Behavior questions take the middle column.

Shell: comparison plate. Source: original figure for the AWS AIP-C01 Study Guide.

RAG adds knowledge at query time. Fine-tuning bakes behavior at train time. Long context stuffs documents into the prompt. Match the change you need to the column that makes it.

Source: original figure for the AWS AIP-C01 Study Guide.

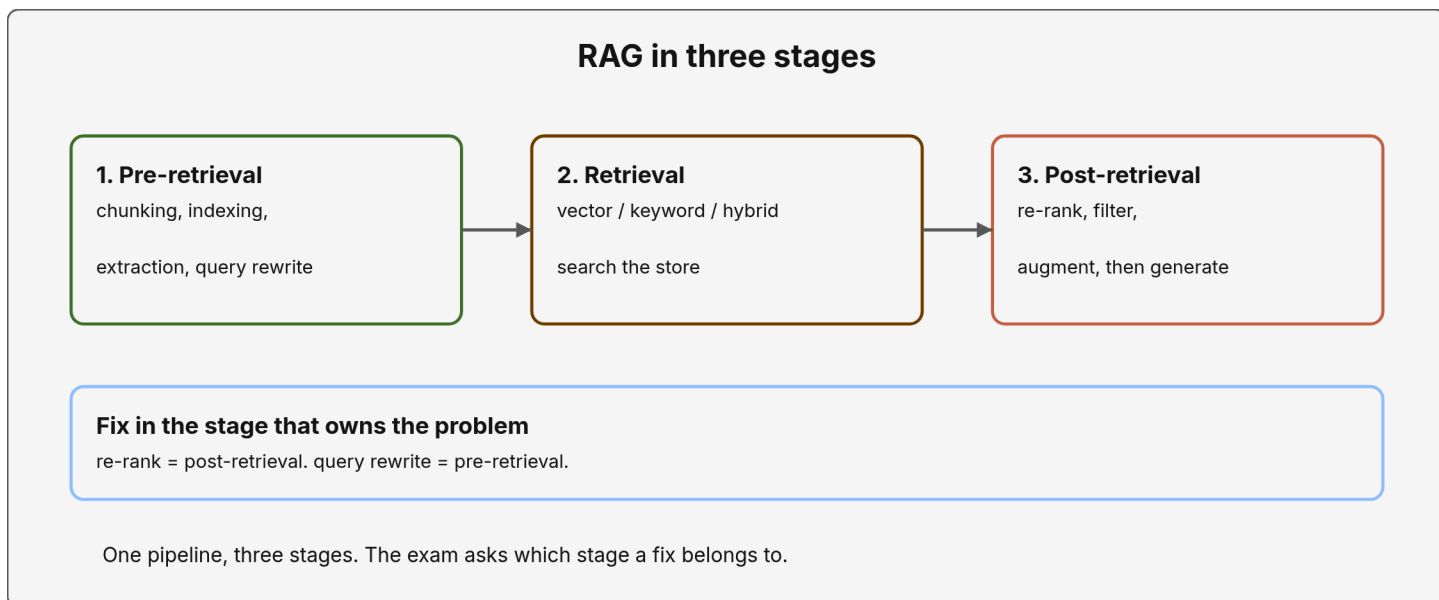
### How to read this diagram

1. Three columns. Each answers a different change: knowledge, behavior, or window.
2. Read the green rows for when to use it. Read the red rows for when it fails.
3. Knowledge questions take the left column. Behavior questions take the middle.

### The RAG pipeline has three stages

Retrieval is not one step. It has three stages. The exam asks which stage a fix belongs to. Learn the stages and the levers inside each one.

1. **Pre-retrieval.** Indexing choices, chunking granularity, and data extraction. Query manipulation lives here too. Rewrite the query so it retrieves better.
2. **Retrieval.** The actual search against the vector store. Vector search, keyword search, or hybrid.
3. **Post-retrieval.** Re-rank the results. Filter the results. Augment the results before generation.



Pre-retrieval shapes the index and the query. Retrieval searches. Post-retrieval re-ranks and filters. Fixes land in the stage that owns the problem.

*Pre-retrieval shapes the index and the query. Retrieval searches. Post-retrieval re-ranks and filters. Fixes land in the stage that owns the problem.* Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left box: pre-retrieval. Chunking, indexing, extraction, query rewriting.
2. Middle box: retrieval. Vector, keyword, or hybrid search.
3. Right box: post-retrieval. Re-ranking, filtering, augmentation.

**Exam relevance.** "Re-rank the retrieved chunks" is a post-retrieval fix. "Rewrite the user query" is a pre-retrieval fix. The exam separates them on purpose.

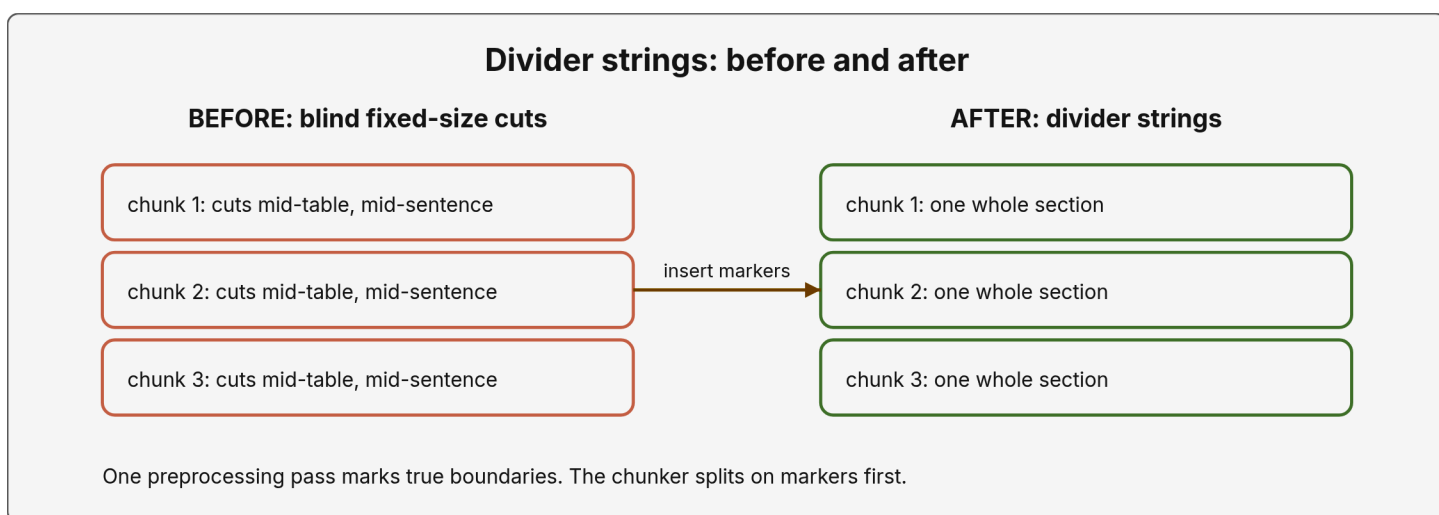
**Failure modes per stage.** Pre-retrieval fails silently: bad chunking poisons every later stage and no amount of reranking recovers it. Retrieval fails loudly: empty results or wrong-language chunks point at the index or the query. Post-retrieval fails subtly: an over-aggressive filter drops the one chunk that held the answer, and the model hallucinates on the survivors. When answers go wrong, debug in stage order. Never tune the prompt before the stages are clean.

### Chunking types: pick the cut that fits the corpus

Chunking is the first lever in the retrieval fix order. Bedrock Knowledge Bases offer four chunking choices. The default is fixed-size at about 300 tokens per chunk with overlap.

| Type         | How it cuts                                                                                                             | When it wins                                                                                   |
|--------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Fixed-size   | Token count per chunk plus overlap percentage. Default is about 300 tokens.                                             | General documents. The safe default.                                                           |
| No chunking  | You chunked the documents yourself before ingestion.                                                                    | You already control the cuts upstream.                                                         |
| Hierarchical | Nested parent and child chunks. Small children give precise retrieval. Swap in the parent to restore context.           | Long documents where precision and context both matter.                                        |
| Semantic     | An FM cuts on meaning and topic shifts. Parameters: max tokens per chunk, buffer size, breakpoint percentile threshold. | Documents where topic boundaries beat token counts. Costs more because an FM does the cutting. |

**Semantic chunking parameters.** Buffer size is the number of surrounding sentences the FM considers per sentence. A buffer size of 1 means three sentences: the target plus one before and one after. A higher breakpoint percentile means chunks are more distinct but bigger and fewer.



Blind cuts split tables mid-row. Divider strings keep one whole section per chunk. One preprocessing pass fixes the boundaries.  
*Blind cuts split tables mid-row. Divider strings keep one whole section per chunk. One preprocessing pass fixes the boundaries.* Source: original figure for the AWS AIP-C01 Study Guide.

How to read this diagram

1. Left: blind fixed-size cuts. Chunks break mid-table and mid-sentence.
2. Right: divider strings first. Each chunk holds one whole section.
3. The arrow is the preprocessing pass: insert markers, then chunk.

**The size tradeoff.** Smaller chunks give more precise matches but less context to verify the match. Larger chunks give more context but fuzzier matches. Sentence boundaries usually survive the cut because full sentences carry full meaning.

**What breaks.** Fixed-size chunking splits tables mid-row and procedures mid-step. The chunks look fine in the store. Retrieval returns fragments that confuse the model. Hierarchical chunking breaks when parent and child drift out of sync after document edits. Semantic chunking breaks on cost: an FM cutting a million documents is a line

item. The exam twist: "retrieval got worse after we improved chunking" usually means the new chunks are smaller and more numerous. Then top-k covers less of the document. Raise top-k or fix the cut.

**Exam relevance.** "300 tokens" is the fixed-size default the exam quotes. "Child chunks for precision, parent for context" is the hierarchical pattern. "An FM cuts the documents" means semantic chunking and higher cost.

## Metadata: the second retrieval axis

Add a **metadata.json** file to your vector store. It carries facts about each chunk that are not in the text: document ID, category, access control, data lineage, creation date, source citation.

Metadata is not chunked like text. It rides alongside the vectors. Use it for hybrid retrieval: filter on metadata first, then run vector search on the survivors. A year filter can drop 90 percent of the corpus before the expensive search starts.

**Exam relevance.** "Filter documents by year before vector search" is a metadata play. "Relevance scoring against metadata" is the hybrid approach. The exam contrasts metadata filtering with pure vector similarity.

## Embeddings: dimensions, density, freshness

**Dimension tradeoff.** Smaller vectors cost less to store and search. Smaller vectors retrieve worse. Amazon Titan defaults to 1024 or more dimensions. Ask whether your domain needs that many. Fewer dimensions can be enough for narrow corpora.

**Sparse versus dense.** Sparse vectors are long and mostly empty, like one-hot encodings. They compute similarity fast and suit recommender systems. Dense vectors are short and packed with meaning. They use less memory and suit semantic search.

**Keep the KB fresh.** Data source changes should trigger a sync. A Lambda can fire on the change event. If the source changes constantly, batch the syncs instead of syncing every change. Use EventBridge to schedule the sync or AWS Batch to run it. Syncing every single change wastes money and compute.

**Exam relevance.** "Reduce vector storage cost" points at smaller dimensions or quantization. "Source changes often" points at scheduled batch syncs, not per-change syncs.

## Evaluating RAG: three relevances plus the job list

RAG evaluation is not exact science. Three concepts still show up on the exam.

1. **Context relevance.** Is the retrieved context relevant to the original query?
2. **Groundedness.** Is the generated response grounded in the retrieved context? If not, it is a hallucination.
3. **Answer relevance.** Does the generated response answer the original query?

Bedrock runs RAG evaluation jobs. They score ten dimensions. Five measure quality: Correctness, Completeness, Helpfulness, Logical coherence, and Faithfulness. Faithfulness means the response aligns with the retrieved text. Four measure safety: Citation precision and coverage, Harmfulness, Stereotyping and Biases, and Refusal. Refusal means whether the model dodges the question.

**Ground truth kills subjectivity.** Provide an evaluation dataset in JSON. It states what you consider a good response. It is also called a prompt dataset. It holds prompts with instructions on how to judge. LLM-as-a-judge takes your system input and output, applies the prompt dataset with benchmarks and judging instructions, and returns scores. Different FMs can serve as the judges.

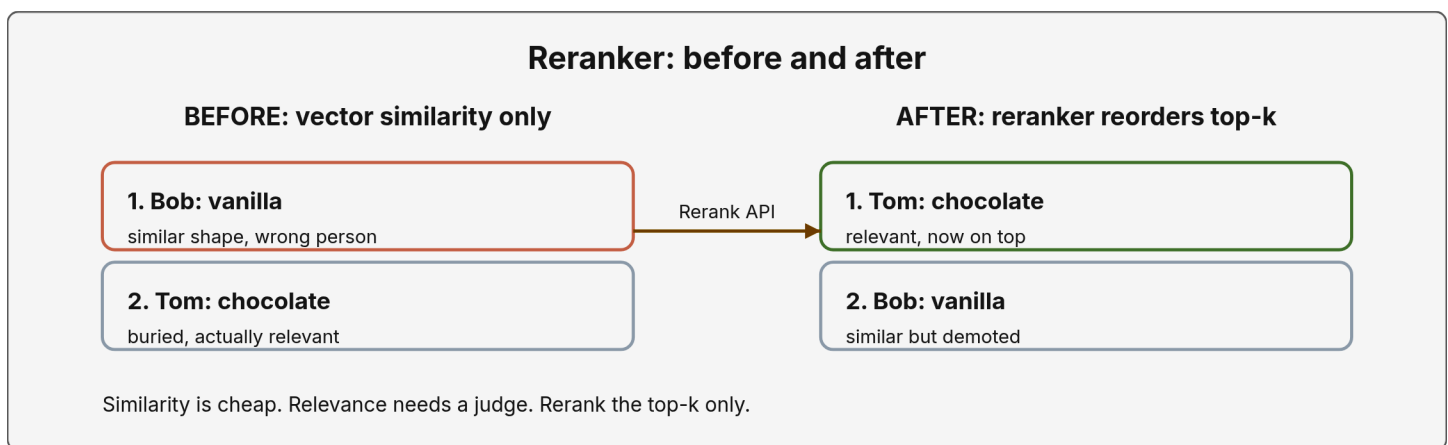
**Exam relevance.** "Context relevance versus groundedness versus answer relevance" is a vocabulary question. "Provide a ground truth dataset" is the answer to "how do you make RAG evaluation less subjective."

## Rerankers: similarity is not relevance

Vector search is cheap and fast. But a chunk can be similar to the query without being relevant. The classic example uses an ice cream query about Tom. The top vector hit talks about Bob and vanilla. The sentence shape matches, so similarity ranks it first. A lower chunk holds the real answer about Tom. Similarity ranked the shape. Relevance needed the reranker.

**The pattern.** Vector search retrieves the top-k candidates fast. A reranker scores each candidate for true relevance to the query. Reranking is heavier than vector search, so it only runs on the top-k, never on the whole corpus.

**In Bedrock.** Use the Rerank API operation. Or set a reranker model on the Knowledge Base so reranking applies automatically. Bedrock offers rerankers from Amazon and from Cohere.



Vector similarity ranks the wrong chunk first. The reranker scores true relevance and promotes the right chunk. Rerank the top-k only.

*Vector similarity ranks the wrong chunk first. The reranker scores true relevance and promotes the right chunk. Rerank the top-k only.* Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left: similarity ranking. "Bob's vanilla" wins on sentence shape.
2. Right: after the Rerank API. "Tom's chocolate" wins on relevance.
3. The arrow names the operation. It runs on the top-k, never the corpus.

**Trap: "rerank the whole corpus."** Wrong. Rerankers are expensive per candidate. The exam offers "run the reranker over all vectors" as a distractor. The correct shape is vector search for top-k, then rerank the k.

**Under the hood.** A reranker reads the query and the chunk together through a cross-encoder. Vector search never does this. It compares two precomputed vectors. That is why similarity is cheap and relevance costs more. The exam twist: a reranker cannot fix a bad candidate pool. If the top-k holds no relevant chunk, the reranker promotes the least wrong one.

**Exam relevance.** "Most similar is not most relevant" always means add a reranker. "Only on the top results" is the cost guardrail the exam checks.

## Vector stores: OpenSearch, S3 Vectors, and the rest

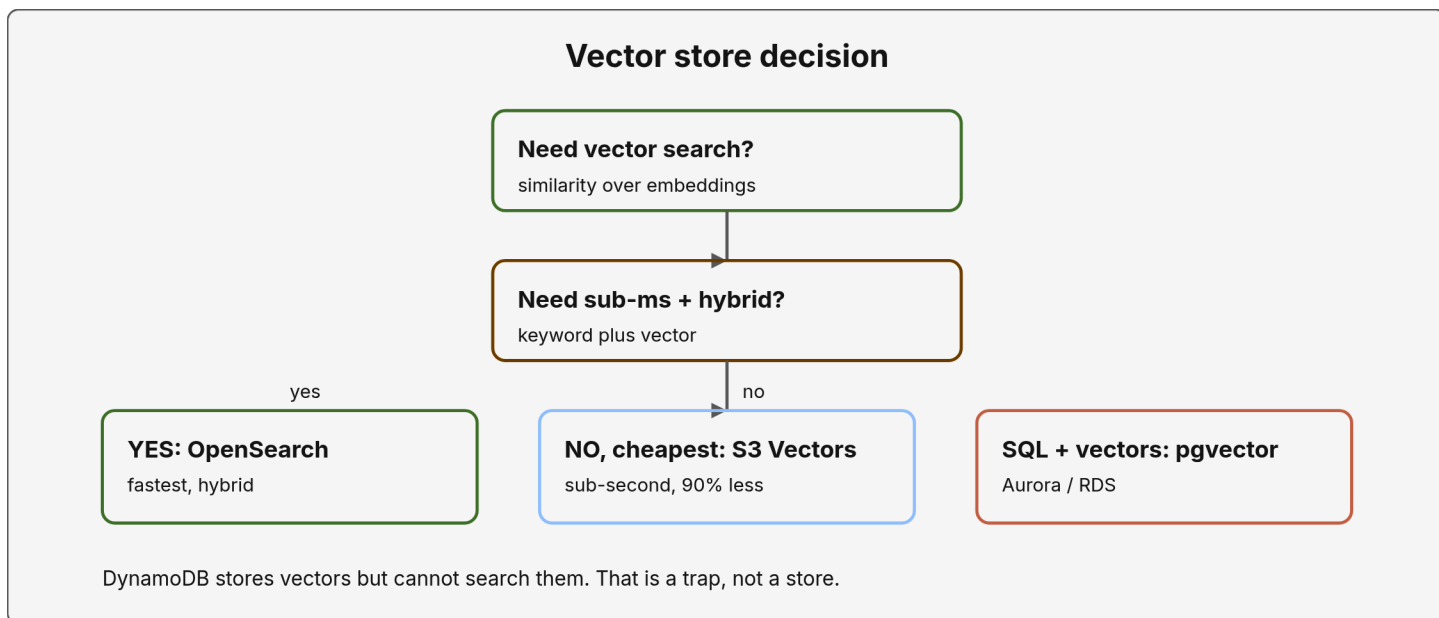
**Amazon OpenSearch Service.** A managed search and analytics engine built on Apache Lucene. Two modes. Managed clusters: you size the nodes, scaling is manual but online. Serverless: on-demand autoscaling, capacity in OpenSearch Compute Units, collections instead of domains.

**OpenSearch as a vector store.** Serverless is the default backing for Bedrock Knowledge Bases. Deleting the KB does not delete the serverless collection. Delete both or you keep paying. Vector search engines: FAISS, NMSLib, Apache Lucene. The default algorithm is HNSW, an approximate kNN that is fast and high quality but memory hungry. Use larger shards for semantic search (30 to 50 GB) and smaller ones for hybrid (10 to 30 GB). The neural plugin lets OpenSearch call your Bedrock embedding model during ingest or search.

**S3 Vectors.** Store embeddings directly in S3 vector buckets. It is simpler and far cheaper than OpenSearch (up to 90 percent less) but slower: sub-second latency versus sub-millisecond. Operations are `put_vectors` and `query_vectors`. Data is strongly consistent. A bucket holds up to 10,000 indices with 2 billion vectors per index. Bedrock Knowledge Bases can use S3 Vectors as the store.

| Need                                                              | Pick                                                                                                             |
|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Fastest vector search, hybrid keyword plus vector                 | OpenSearch Service                                                                                               |
| Cheapest vector storage at scale, latency tolerant                | S3 Vectors                                                                                                       |
| Small to medium RAG with mostly structured data and SQL filtering | Aurora with pgvector                                                                                             |
| Vectors plus complex SQL filter logic in one database             | RDS with pgvector                                                                                                |
| Chat history, session state, near-real-time context for the FM    | DynamoDB. It stores vectors but cannot search them. Zero-ETL can pipe DDB into OpenSearch for the actual search. |

**Trap: "DynamoDB as the vector store."** Wrong. DynamoDB can hold vector values but cannot run vector search. The exam pairs it with session state and agent memory, not with similarity search.



Need vector search first. Then pick on latency, cost, or SQL. DynamoDB is not a vector store.

*Need vector search first. Then pick on latency, cost, or SQL. DynamoDB is not a vector store.* Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

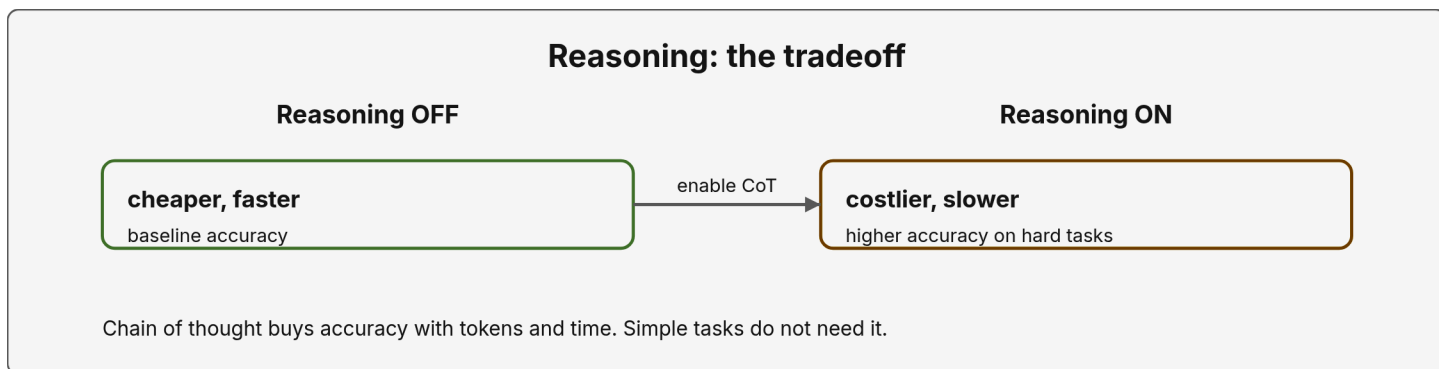
1. Top question: do you need vector search at all.
2. Second question: sub-millisecond plus hybrid, or cheapest, or SQL plus vectors.
3. Three answers: OpenSearch, S3 Vectors, pgvector. The footer names the trap.

**Under the hood.** HNSW builds a layered graph of vectors. Search walks the graph from coarse to fine layers. It is fast and accurate but memory hungry, which is why FP16 and binary quantization matter at scale. Multi-vector embeddings (one vector per token, matched with late interaction) trade more storage for finer ranking. The exam rarely goes this deep. The exam twist: "vector search slowed as data grew" points at shard count and quantization, not at the embedding model. Fewer, larger shards cut cross-shard coordination.

**Exam relevance.** "Sub-millisecond vector search" means OpenSearch. "Cheapest at scale" means S3 Vectors. "Structured plus vector in one SQL database" means pgvector.

### Reasoning models: pay for thought

Enabling reasoning means the FM applies chain of thought. It breaks a complex task into steps before it answers.



Reasoning off is cheaper and faster. Reasoning on costs tokens and time but wins on hard tasks. Match the toggle to the task.

*Reasoning off is cheaper and faster. Reasoning on costs tokens and time but wins on hard tasks. Match the toggle to the task.* Source: original figure for the AWS AIP-C01 Study Guide.

|          | Reasoning off | Reasoning on                    |
|----------|---------------|---------------------------------|
| Cost     | Lower         | Higher. More tokens per answer. |
| Latency  | Faster        | Slower. More thinking time.     |
| Accuracy | Baseline      | Higher on complex tasks.        |

**Exam relevance.** "Complex reasoning, cost is secondary" points at a flagship reasoning model. "Simple classification at scale" points at a small fast model with reasoning off. The exam tests the tradeoff, not the toggle name.

## Multimodal models and pipelines

Multimodal models need a specialized encoder per media type: text, image, audio, video, documents. Each encoder turns its media into vectors that live in one shared space. Multimodal embedding models convert different media into compatible vectors, so one query can search across media by vector similarity.

**Titan Multimodal Embeddings G1.** The request body is JSON. It carries **inputText** and **inputImage** as a base64-encoded string. An optional **embeddingConfig** sets `outputEmbeddingLength` to 256, 384, or 1024. The default is 1024.

**The pipeline does the encoding.** Raw media is not model-ready. A data pipeline encodes it first: base64 for images, transcripts for audio, extracted text for documents. Build the pipeline with Glue, SageMaker, or Bedrock Data Automation. The model expects prepared data. Your app owns the preparation.

**Under the hood.** Each media type gets its own encoder. The encoders train to place matching text and images near each other in one shared space. That shared space is what makes "find photos with a sentence" work. The exam twist: multimodal search still needs the right chunking and metadata on the text side. Vectors do not replace the pipeline.

**Exam relevance.** "Image data must be base64-encoded in JSON" is the Titan multimodal input shape. "Search images with a text query" is the multimodal embedding use case.

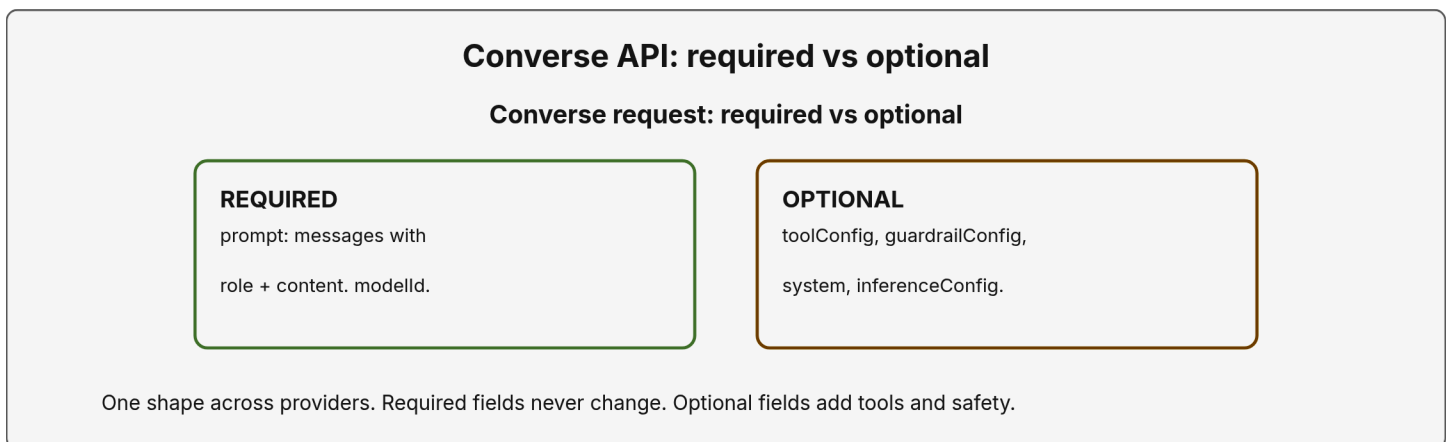
## Data into the model: shape it first

Bedrock FMs and SageMaker LLMs almost always expect JSON-structured input. Non-LLM SageMaker models may want CSV or Protobuf instead. It is your responsibility to send the right shape. Your app or endpoint owns this.

**Unstructured text loses structure.** OCR or PDF ingestion drops headings, tables, and metadata into a sea of text. Fixes: format the source as HTML (models read HTML well, and tables survive), or extract structure with Amazon Comprehend or Amazon Textract. Automate the formatting in a Glue ETL pipeline or with Bedrock Data Automation.

**Divider strings control chunking.** A divider string is a unique delimiter you insert between logical sections during preprocessing. A Lambda preprocessor can convert HTML headings into markers like `<SECTION_BREAK:Title>`. The chunker splits on the marker first, then applies size limits inside each section. Sections stay whole. Tables stop splitting mid-row. This is application code, not an AWS feature.

**Converse API message shape.** The Converse API takes JSON with a role (user or assistant) and content (the messages). Every call carries this structure no matter which model sits behind it.



Prompt and modelId are required. Tool config, guardrails, system, and inference config are optional. One shape for every model.

*Prompt and modelId are required. Tool config, guardrails, system, and inference config are optional. One shape for every model. Source: original figure for the AWS AIP-C01 Study Guide.*

**Exam relevance.** "Tables break during chunking" points at divider strings or HTML formatting. "Who formats the input" is always your application, never Bedrock.

## Fighting FM bias: three fixes, three moments

Bias comes from training data. Groups with few examples in the data get poor or unfair results. An image model trained mostly on one group draws that group for every prompt.

Three fixes, each at a different moment.

1. **Disambiguation in the prompt.** Name the group in the prompt. "Draw a doctor" becomes "Draw a female doctor of South Asian descent." This is the fastest fix. It needs no retraining.
2. **Fix the training data.** Add the missing examples before training. Balance the classes. This is the deepest fix. It costs a retrain.
3. **Counterfactual data augmentation.** Edit examples after the fact. Swap an attribute (name, gender, age) and keep the label. The model learns the label does not depend on the attribute.

**Exam relevance.** "The model favors one group" points at the data, not the prompt template. Guardrails filter biased outputs at the door. These three fixes stop the bias at the source: prompt, data, or augmented data.

## Structured output: prompt or tool use

Sometimes the system must answer in an exact response format, like a fixed JSON schema. Two ways to enforce it.

1. **Enforce it in the prompt.** State the format clearly and simply. Number the steps that build the output. Show one example. This needs some trust that the FM follows instructions.
2. **Enforce it with tool use.** Define the schema as a tool in the Converse API toolConfig. The model fills the tool inputs, which must match the schema. Modern FMs handle this well.

**Exam relevance.** "Must return exact JSON" with no tools mentioned points at prompt enforcement with numbered steps and an example. "Tool definitions available" points at toolConfig schemas.

## Amazon FMs: Titan and Nova

**Titan.** Amazon Titan covers text, image generation, and embeddings. Titan Text Embeddings is the workhorse for RAG vectors. Titan Multimodal Embeddings G1 handles text plus image in one vector.

**Nova.** Nova is the newer Amazon family. It spans fast cheap models (Nova Micro and Nova Lite class) to flagships (Nova Pro class) for complex reasoning. Nova also has multimodal variants. Nova Reels targets video generation. Note: Nova Reels is not covered in the repo volumes. Treat it as a name to recognize, not a feature to reason about.

**Exam relevance.** "Amazon embedding model" means Titan. "Flagship Amazon model for complex reasoning" means Nova Pro class. "Cheapest Amazon model for classification at scale" means Nova Micro or Lite class.

## More D1 decision rules

These rules extend the D1 decision list above. Same format. Same exam.

- If the stem says "same narrow task millions of times" and "cost is the problem," pick distillation. Never pick continued pre-training.
- If the stem says "domain vocabulary" or "unlabeled text," pick continued pre-training. Never pick fine-tuning.
- If the stem says "preference data" or "reward signals," pick reinforcement fine-tuning.
- If the stem says "weights trained outside Bedrock," pick an imported model.
- If the stem says "many tasks, one base model," pick LoRA adapters. Version them in the Model Registry.
- If the stem says "multilingual content," pick Cohere Embed. Never pick Titan for 100+ languages.
- If the stem says "custom reranking" or "custom citation format," pick Retrieve, then build the prompt with Converse. RetrieveAndGenerate cannot do custom reranking.
- If the stem says "graph relationships between entities," pick Neptune for GraphRAG. Never use Neptune as a plain document store.
- If the stem says "provider-specific parameter not in Converse," pick InvokeModel with the native body. Converse cannot carry every exotic parameter.
- If the stem says "structured output" or "JSON schema," pick Converse with toolConfig. Define the schema for the tool inputs.
- If the stem says "answers must cite sources," pick RAG with RetrieveAndGenerate. Fine-tuning cannot cite.

- If the stem says "model choice must be evidence-based," pick a golden-dataset evaluation. Never trust provider benchmarks.

### More D1 traps, with why the wrong answer fails

- Distillation for new knowledge. Distillation copies behavior on a narrow task. It does not add facts. Knowledge questions need RAG.
- Continued pre-training for a tone change. Continued pre-training teaches vocabulary from unlabeled text. It does not teach tone. Tone needs fine-tuning or prompts.
- Fine-tuning for citations. Fine-tuned weights hold no source links. Only retrieval can cite. Pick RAG.
- Converse for embeddings. Converse generates text. It cannot embed. Embeddings need InvokeModel with an embedding model.
- Mixing embedding models between ingest and query. The vector spaces do not align. Retrieval breaks silently. Keep model and dimensions identical.
- Neptune for document chunks. Neptune is a graph store. Document chunks need a vector store. Neptune serves GraphRAG only.
- Kendra for RAG. Kendra is enterprise search. The RAG path is Knowledge Bases. Do not mix them.
- ElastiCache as a vector store. ElastiCache is ephemeral. Vectors need durable storage. Use a real vector store.
- Provider benchmark scores for model choice. Benchmarks do not measure your data or your prompts. Only your golden dataset counts.
- Exact flat search on large data. Exact search scans everything. It is affordable only on small data. Large data needs approximate indexes.

### D1 self-test: more questions with answers

D1 · MEDIUM · ONE ANSWER CC51. A bank needs a support model that speaks in a strict legal tone. The tone must hold across millions of calls. Documents rarely change. Which approach do you use first? A) Build RAG with a Knowledge Base. RAG adds knowledge. The need here is behavior, not knowledge. B) Fine-tune a model on labeled examples of the legal tone. C) Use a long system prompt with tone examples. Prompts work, but millions of calls need baked-in behavior. D) Continue pre-training on legal text. Unlabeled text teaches vocabulary. It does not teach tone. Reveal answer Why B is correct: "Strict tone across millions of calls" is a behavior need. Fine-tuning bakes behavior into weights. RAG cannot change tone. Prompts cannot hold at this scale.

D1 · MEDIUM · ONE ANSWER CC52. A team serves a big model for one narrow classification task. The bill is too high. Quality must stay the same. What do you do? A) Fine-tune the big model. This does not cut serving cost. B) Distill a small model from the big model on the narrow task. C) Switch to long-context prompting. This raises token cost per call. D) Use continued pre-training. This teaches vocabulary. It does not cut cost. Reveal answer Why B is correct: "Same narrow task, cost is the problem" is the distillation sentence. A small model copies the big model on one task. Serving cost drops. Quality holds.

D1 · EASY · ONE ANSWER CC53. A RAG system retrieves well in English but fails in French and German. What is the fix? A) Raise top-k. More chunks do not fix language mismatch. B) Switch the embedding model to Cohere Embed. C) Change the chunk size. Chunk size does not fix language coverage. D) Add a reranker. Reranking cannot

fix bad embeddings. Reveal answer Why B is correct: Multilingual content needs a multilingual embedding model. Cohere Embed supports more than 100 languages. It is the exam's multilingual answer.

D1 · MEDIUM · ONE ANSWER CC54. After a re-sync, answers are confident but wrong. Nothing else changed. What do you check first? A) The prompt template. The prompt did not change. B) The embedding model version and chunking settings. C) The model temperature. Temperature does not cause confident wrong answers after a sync. D) The guardrail policies. Guardrails block. They do not rewrite facts. Reveal answer Why B is correct: "Confident but wrong after a re-sync" points at the ingest pipeline. A changed embedding model or changed chunking breaks the vector space. Debug the delta.

D1 · MEDIUM · ONE ANSWER CC55. A team needs custom citation formats and a custom reranker in their RAG pipeline. Which query API do they use? A) RetrieveAndGenerate. This is a managed call. It cannot do custom reranking. B) Retrieve, then Converse with a custom prompt. C) InvokeModel directly on the vector store. Vector stores do not generate text. D) Batch inference. Batch is for offline bulk work. It is not a query API. Reveal answer Why B is correct: Custom control needs the two-step path. Retrieve returns the chunks. Converse builds the custom prompt with the custom format.

D1 · EASY · ONE ANSWER CC56. A startup has one base model and twelve tasks. Each task needs its own behavior. Retraining the base twelve times is too expensive. What do you use? A) Twelve full fine-tunes. This is the expensive path the stem rejects. B) LoRA adapters, one per task, versioned in the Model Registry. C) Twelve separate base models. This multiplies cost. D) Prompt engineering for all twelve. Prompts may work but the stem wants trained behavior per task. Reveal answer Why B is correct: LoRA keeps base weights frozen and trains small adapters per task. Adapters version and roll back independently. This is the parameter-efficient answer.

## D1 trap patterns

- Use fine-tuning for knowledge. This includes quarterly documents, citations, and new facts. Always use RAG.
- Converse is for text generation only. Do not use Converse for an embeddings task.
- The standalone ApplyGuardrail API covers any text. This includes external models. Guardrails do not work only with Converse.
- Only your own golden-dataset evaluation counts. Do not use provider benchmarks to decide the model choice.
- If you have AccessDenied on InvokeModel, debug it as IAM. First, check model access and regional availability.
- If you miss `anthropic_version` or `max_tokens` in a Claude InvokeModel body, you will debug IAM for one hour. The error is the request shape.
- Do not use a cross-region inference profile under a data-residency constraint. Profiles move data across regions.
- Provisioned Throughput is not a cost fix for bursty traffic. PT bills hourly. It bills whether you use it or not.
- Do not use prompt caching on unique traffic. There is no repeated prefix. There are no savings.
- ElastiCache is ephemeral caching. Do not use ElastiCache as a vector store. Durable vectors live in a vector store.
- Kendra is enterprise search. It is not the RAG path. Do not use Kendra for RAG.
- The stopReason `max_tokens` means the output was cut. It does not mean "finished."
- Set temperature and topP. They interact. Pick one.

## D1 rapid-fire questions

### D1 · EASY · ONE ANSWER

**CC01.** A retailer wants a chatbot. The chatbot must answer questions from its 2,000-page returns policy. The policy changes each quarter. Each answer must show the exact section. Which approach do you use first?

- A) Fine-tune a model on the policy PDFs each quarter.  
Weights become old each quarter. The model cannot show sections. This is the main knowledge-versus-behavior problem.
- B) Build RAG with a Bedrock Knowledge Base. Use the policy documents.**
- C) Put the full policy in the system prompt.  
2,000 pages will not fit reliably. Quarterly edits mean prompt changes. There is no way to show citations.
- D) Train a custom model from the start.  
This takes months of work. Managed RAG solves this in one afternoon.

**Why B is correct:** Changing private knowledge and citations is a RAG requirement. Knowledge Bases manage ingestion, chunking, and sync. They do not need training.

**The trap:** "fine-tune" when documents change. Changing knowledge is always RAG.

### D1 · EASY · ONE ANSWER

**CC02.** 90% of prompts use a 4,000-token system prompt. Costs rise. Which lever do you use?

- A) Prompt caching with a cachePoint after the shared prefix**
- B) Provisioned Throughput.  
PT reserves capacity. It does not reduce the per-token cost of a repeated prefix.
- C) Batch inference.  
Batch is for offline bulk work. It is not for interactive prompts.

**Why A is correct:** A repeated long prefix is the only condition for caching. This question gives you this condition. Proof it works: cache read hit rate.

**D1 · MEDIUM · ONE ANSWER**

**CC03. A team needs embeddings for a knowledge base. Which API do they use?**

A) Converse.

Converse is only for text generation. This is the exam's favorite API distractor.

**B) InvokeModel with the embedding model's provider body**

C) ApplyGuardrail.

ApplyGuardrail applies safety filters. It does not generate embeddings.

---

**Why B is correct:** Embeddings are a provider-specific feature. So, they go through InvokeModel. Use Titan Text Embeddings v2 (1024/512/256 dimensions) or Cohere Embed (100+ languages, input\_type search\_query/search\_document).

**D1 · MEDIUM · ONE ANSWER**

**CC04. A knowledge base re-syncs. After the re-sync, answers are confident but wrong. Is this a generation or retrieval problem?**

A) Retrieval: check the embedding model version. Check the chunking configuration. The re-sync changed these.

B) Generation: rewrite the system prompt.

The re-sync is the only change. Debug the difference, not the prompt.

C) Generation: raise temperature.

Temperature changes randomness. It does not change the correctness of retrieved facts.

---

**Why A is correct:** Do change-point debugging. The re-sync changed ingestion. Verify the same embedding model and dimensions at ingest and query. Verify chunk boundaries did not split key facts. Verify the KB status is in-sync.

**D1 · MEDIUM · ONE ANSWER**

**CC05. A model is available in two regions. The application needs resilience. Data must stay in one region. What is the cross-region inference profile?**

A) Yes, profiles are the resilience answer

Profiles route across regions. This moves data across regions. This breaks the residency constraint.

**B) No: reject it. The residency constraint stops cross-region routing.**

C) Yes, with Provisioned Throughput

PT is capacity. It is not a residency mechanism.

**Why B is correct:** Profiles give availability. But every answer must satisfy every constraint. Data residency is a hard constraint. Check the region first.

**D1 · EASY · ONE ANSWER**

**CC06. A vector dataset has 200k records. It needs the least operational overhead. Which store is best?**

**A) Aurora Serverless with pgvector**

B) Self-managed OpenSearch cluster

This needs more scale than 200k records. Self-managed means operational overhead. The adjective decides the answer.

C) ElastiCache

This is ephemeral caching. It is not a durable vector store.

**Why A is correct:** Small data plus "least operational overhead" means the fully managed serverless option. Large data means OpenSearch. 100% recall on small data means exact flat search.

**D1 · MEDIUM · ONE ANSWER**

**CC07. A company runs the same narrow classification task millions of times daily. It uses a flagship model. The token bill is the problem. What is the best fix?**

**A) Fine-tune the flagship on more data**

Fine-tuning the expensive model keeps the expensive per-call cost.

**B) Distill the task into a small student model served on Provisioned Throughput**

**C) Prompt caching**

Caching helps repeated prefixes. It does not solve a per-call cost problem on a huge model.

---

**Why B is correct:** A narrow repeated task at massive scale is the distillation shape. Bake the behavior into a small, cheap model. Serve the predictable baseline on PT. PT has hourly commit, 1 to 6 month terms.

**D1 · MEDIUM · ONE ANSWER**

**CC08. The team bought PT, but on-demand calls still throttle. The code passes the base model ID. How do you fix this?**

**A) Pass the provisioned model ARN as modelId. This routes traffic to the reserved capacity.**

**B) Buy more model units**

Idle capacity means the code never routes to what was bought. Route first, then get capacity.

**C) Switch to batch inference**

Batch changes latency characteristics. It does not use reserved capacity.

---

**Why A is correct:** PT capacity is only used when invocations target the provisioned ARN. The base model ID keeps hitting shared on-demand quotas.

**D1 · EASY · ONE ANSWER**

**CC09. A fine-tuned model's customization job succeeded. But every invoke call fails. What do you check first?**

**A) Did you provision Provisioned Throughput for the custom model?**

**B) Retrain with more examples.**

The job succeeded. Training is not the problem. Custom models need PT to work.

**C) Rewrite the prompt.**

Prompt quality does not cause invoke failures.

---

**Why A is correct:** This is the serving problem. Most custom models need Provisioned Throughput. Then you can invoke them. The base model ID will not serve them.

**D1 · MEDIUM · ONE ANSWER**

**CC10. Compliance wants to find PII in many S3 buckets. This must happen before KB ingestion. Which service do you use?**

**A) Comprehend**

Comprehend finds PII in text you send it. It does not find PII in buckets at rest.

**B) Macie**

**C) Guardrails**

Guardrails filter at the model boundary. They do not filter across S3.

---

**Why B is correct:** This is the PII pipeline order. Macie finds PII in S3 at rest. Comprehend finds PII in live text. Guardrails filter PII at inference time.

**D1 · MEDIUM · ONE ANSWER**

**CC11. A support team answers questions. The questions come from a relational orders database. An example question is "which orders shipped late last week?". Do you embed the tables into a vector store?**

A) Yes, chunk the tables into the knowledge base.

Structured data with SQL-shaped questions does not belong in a vector store. This is a common RAG over-application on the exam.

B) No: use text-to-SQL with the database. Keep documents in the knowledge base.

**Why B is correct:** Unstructured documents go to the knowledge base. Relational data uses text-to-SQL. Match the retrieval method to the data shape.

**D1 · MEDIUM · ONE ANSWER**

**CC12. Basic RAG works. Retrieval quality is too low. Which two upgrades does the exam name?**

A) Hybrid search (keyword plus vector) and a reranker such as Cohere Rerank.

B) Bigger chunks and higher temperature.

Bigger chunks make matching less precise. Temperature changes generation randomness. It does not change retrieval quality.

C) Fine-tune the embedding model on the documents.

Use the managed retrieval upgrades before exotic fixes.

**Why A is correct:** Hybrid search finds exact terms that pure vectors miss. The reranker re-scores top chunks with a stronger model. Bedrock manages both features. They are the "least custom effort" answer.

**D1 · EASY · RECALL**

**CC13. A Converse response shows stopReason `max_tokens`. The answer looks clean. What happened?**

**Answer:** The output was cut during generation. It did not finish. Increase `maxTokens` or make the input shorter. The other values are: `end_turn` (done), `guardrail_intervened` (blocked).

**D1 · EASY · RECALL**

**CC14. You set Temperature to 0.7 and topP to 0.9 on a Converse call. Is this correct?**

**Answer:** No. They interact. Sampling behavior becomes unpredictable. Set temperature or topP. Do not set both. Temperature 0 is the deterministic setting.

**Sampling: pick one knob, not two**

Temperature reshapes the curve. topP trims the tail. Together they fight.

**temperature**

0 = deterministic  
higher = more random

Shapes the whole  
probability curve.

**topP (nucleus)**

0.9 = top 90% of  
probability mass

Cuts the long tail of  
unlikely tokens.

**Set temperature or topP. Never both.**

They interact, and the sampling behavior turns unpredictable.

Set temperature or topP. Never set both. Temperature 0 is deterministic.

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this diagram**

1. Left: temperature. It controls randomness. Zero means deterministic.
2. Right: topP. It controls the token pool. Use one control only.
3. Bottom: the trap. Setting both makes sampling unpredictable.

**Next:** D2 takes those model calls. It turns them into a working application. This application has agents that act. It has workflows that orchestrate. It has APIs that face users.

**D1 · EASY · ONE ANSWER**

**QH01.** A RAG system retrieves a top hit about Bob and vanilla. The query asked about Tom. A lower-ranked chunk holds the correct answer about Tom. What do you add?

- A) Larger chunks.  
Bigger chunks add context but do not fix relevance ranking.
- B) A reranker model on the top-k results.**
- C) More embedding dimensions.  
Dimensions change similarity, not relevance judgment.
- D) Rerank the entire corpus.  
Too expensive. Rerank only the top-k.

**Why B is correct:** Vector search finds similar chunks. A reranker judges true relevance and reorders the top-k. Amazon and Cohere rerankers work through the Bedrock Rerank API.

**The trap:** "similar" versus "relevant". The exam separates them on purpose.

**D1 · MEDIUM · ONE ANSWER**

**QH02.** PDF tables keep splitting mid-row during chunking. Retrieval quality drops. What is the cheapest fix?

- A) Use smaller fixed-size chunks.  
Smaller blind chunks make more fragments, not fewer.
- B) Insert divider strings at section boundaries during preprocessing, then split on them.**
- C) Switch to a larger embedding model.  
The embedding model cannot fix broken chunk boundaries.
- D) Increase top-k.  
More broken chunks is not the fix.

**Why B is correct:** Divider strings mark true section boundaries. The chunker splits on the marker first, then applies size limits. Tables stay whole.

**The trap:** "smaller chunks" as the fix for boundary violations. Boundary problems need boundary-aware cuts.

**D1 · EASY · ONE ANSWER**

**QH03.** You must build image search where a text query finds matching product photos. Which embedding input shape do you use?

**B) Titan Multimodal Embeddings G1 with inputText plus base64-encoded inputImage.**

A) Titan Text Embeddings with the image file path as text.  
A file path carries no image content.

C) Converse API with the image attached.  
Converse generates text. It does not produce search vectors.

D) Two separate text embedding calls.  
Text-only vectors cannot represent image content.

**Why B is correct:** Multimodal embeddings put text and images in one shared vector space. The G1 request takes inputText and base64 inputImage, with output length 256, 384, or 1024.

**The trap:** text-only embeddings for image content. Cross-media search needs multimodal vectors.

**D1 · MEDIUM · ONE ANSWER**

**QH04.** A team must extract the same five fields from 50,000 vendor invoices. Which Bedrock Data Automation setup is correct?

A) Standard output on each file.  
Standard output guesses the structure. It will not return the same five fields every time.

**B) A custom blueprint naming the five fields, invoked with InvokeDataAutomationAsync.**

C) Textract with no blueprint.  
Textract extracts text. It does not enforce a field schema.

D) A synchronous call per invoice from the web app.  
50,000 files need async processing with S3 output, not blocking calls.

**Why B is correct:** Custom blueprints define exact fields. InvokeDataAutomationAsync processes files asynchronously and writes structured output to S3. Check completion with GetDataAutomationStatus.

**The trap:** standard output for fixed-schema extraction. Fixed fields always mean a blueprint.

## D2: Implementation and Integration (26%)

---

D1 teaches you how to choose an approach. D1 teaches you how to call a model. D2 answers the next question. How do these calls become a working application?

D2 is the build domain. This domain includes agents that act. It includes workflows that orchestrate. It includes APIs that face users. It includes prompts that you treat as code. The exam often shows a failure mode here.

This is when you pick a dynamic tool for a fixed sequence. Or you pick a fixed sequence when the job needs reasoning. Learn the difference between an agent and a flow. Learn the timeout math. Then most of D2 is clear.

### Bedrock Agents: when the model decides the steps

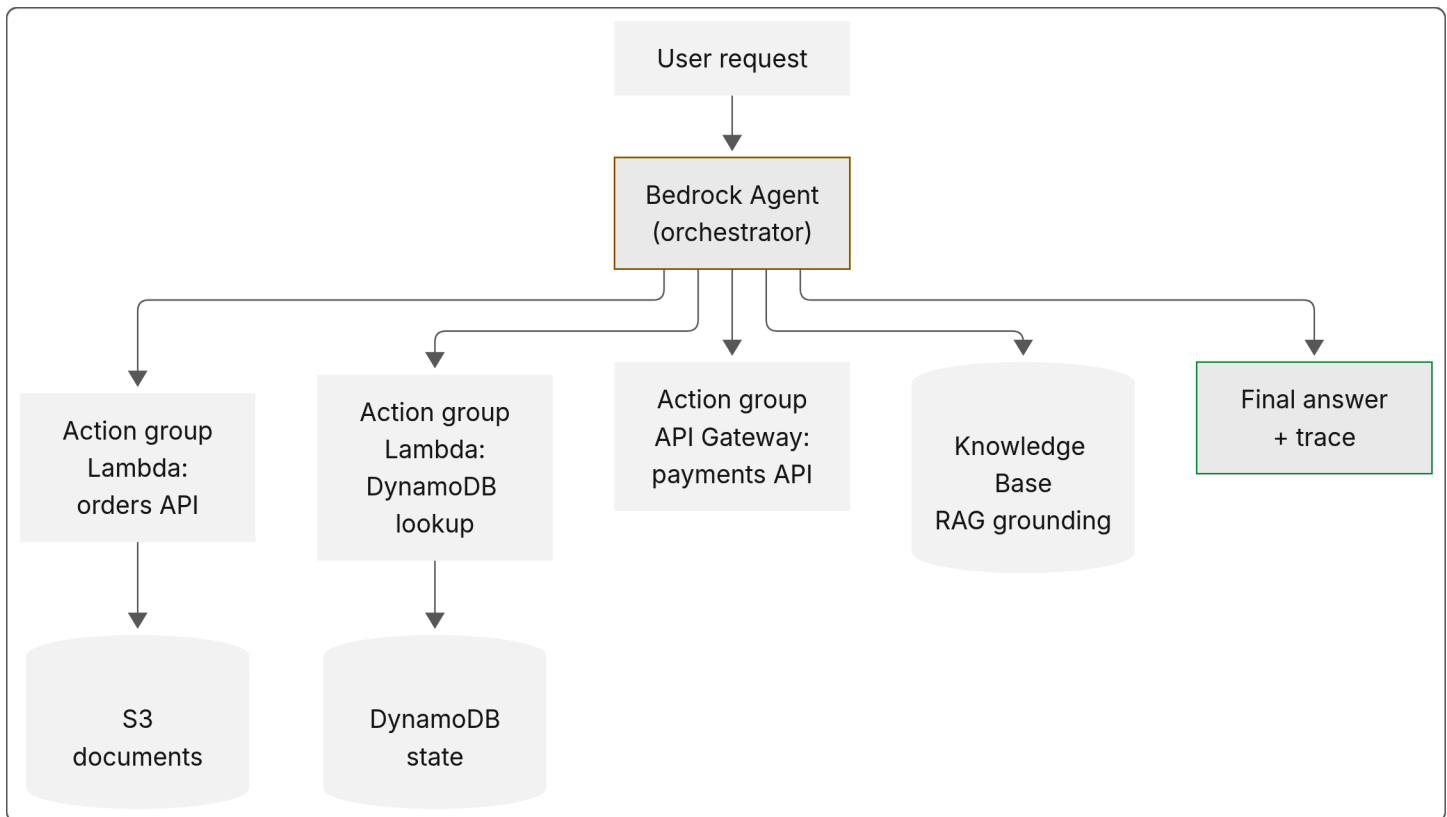
A **Bedrock Agent** is a foundation model. It has instructions. It has **action groups**. Action groups are tool definitions. Lambda functions or API schemas support these definitions. The agent reasons about a goal.

It decides which tools to call. It decides the order of the calls. It continues until the task is complete. Use agents when the scenario says the model reasons. Use agents when it says the model decides. Use agents when it says the model chooses tools.

Use agents for multi-step tasks with tools. This is the agent loop: Your application calls `invoke_agent`. It calls this on the bedrock-agent-runtime API. It uses `agentId`.

It uses `agentAliasId`. It uses `sessionId`. It uses `inputText`. The agent adds its instruction. It adds the conversation history for that session. It asks the model for the next step.

It executes tool calls. It repeats this process.



Agent production pattern: the agent reasons, Lambda action groups execute, knowledge bases ground, and session state persists per user.

Source: original figure for the AWS AIP-C01 Study Guide.

The exam tests four details. First, **aliases**: `agentAliasId` must point to the PROD alias for production. The agent ID alone routes to DRAFT. This is a common bug. It causes "production uses the wrong behavior." Second, **PrepareAgent**: You must run prepare-agent after any configuration change. This includes a new action group or a new instruction.

This compiles DRAFT into a prepared version. If you skip this, the agent silently runs the old configuration. "Agent ignores the new action group" always means you skipped prepare. Third, **sessionId** is the conversation memory key. Use one session ID per user session. Reusing one ID across users shares conversation history between them.

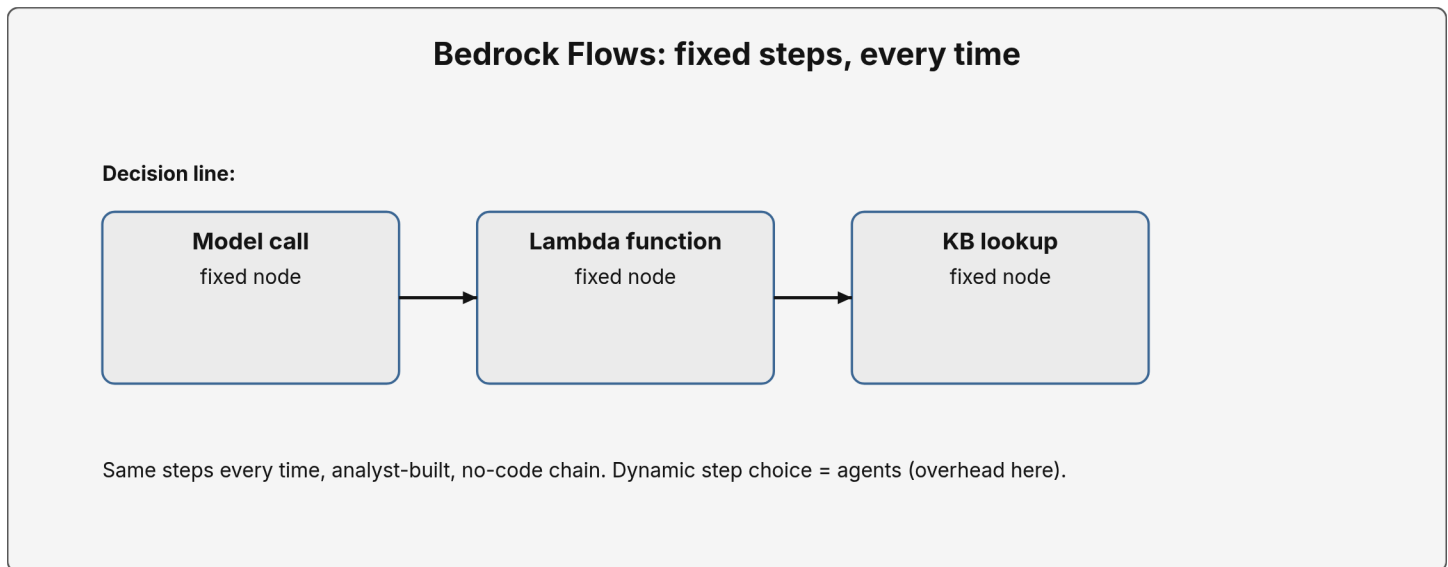
Fourth, **enableTrace** on `invoke_agent` returns the reasoning trace. This trace shows which tools the agent chose. It shows why the agent chose them. It is the exam's transparency answer. "Show why the agent did that" is always `enableTrace`.

The **Lambda action function** has a fixed contract. Both sides must follow this contract. Parameters arrive as a LIST of `{"name": ..., "value": ...}` dictionaries. They do not arrive as a plain dictionary. If you treat it as a dictionary, you lose every parameter. The response must echo the exact nested shape.

It must include `messageVersion`. It must include `actionGroup`. It must include `function`. The result string must be inside `responseBody.TEXT.body`. If the shape is wrong, the agent cannot parse the tool result. For multi-tenant agents, the **supervisor pattern** routes to collaborator agents.

IAM filtering for each knowledge base ensures this. Tenants only reach their own data.

## Bedrock Flows: when the steps are fixed



Bedrock Flows: model call, Lambda, and knowledge base lookup wired as fixed steps that run the same way every time

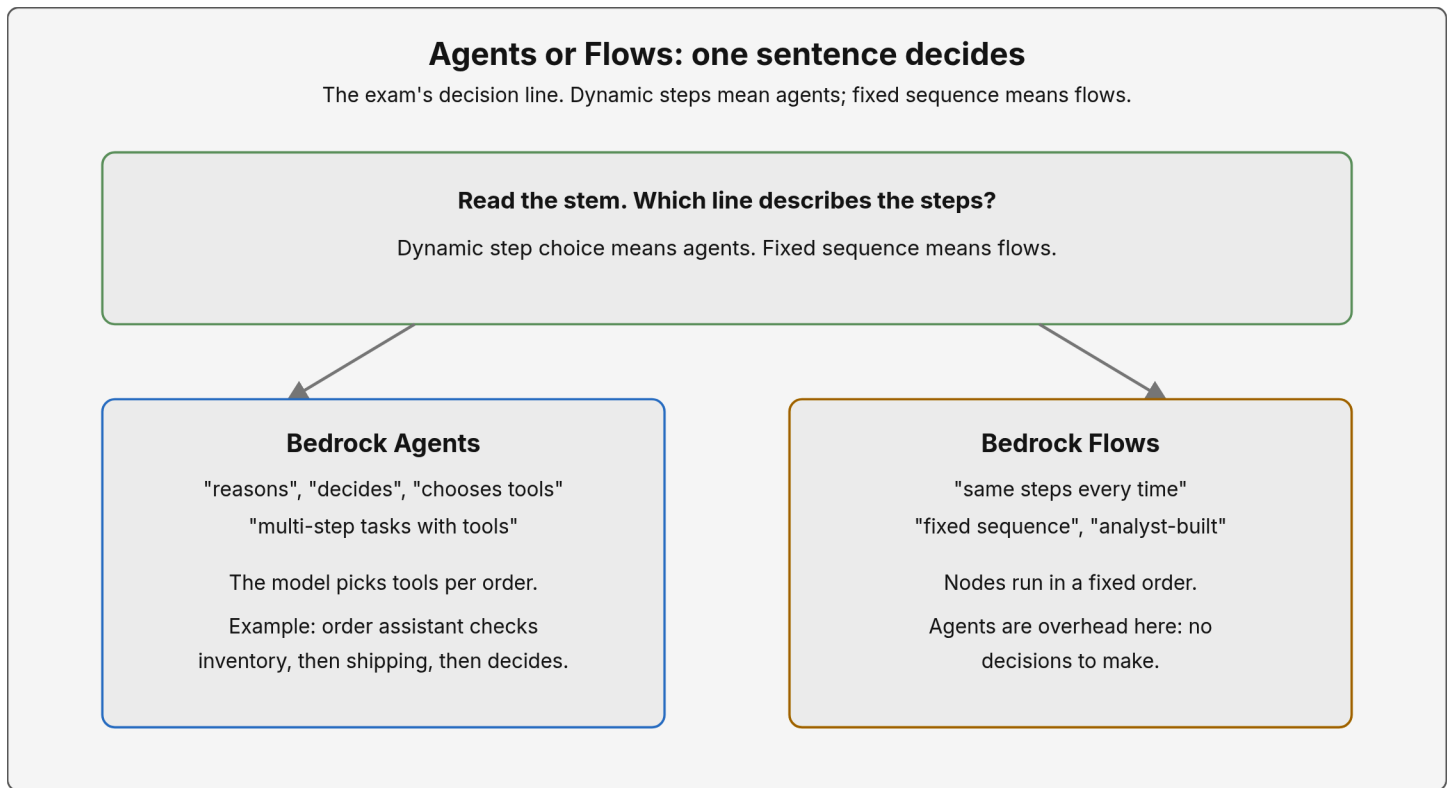
Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Read left to right: fixed nodes in a fixed order. The decision line above says when to use it: same steps every time. Takeaway: dynamic step choice means agents; a fixed sequence means flows, and agents are overhead here.

A **Bedrock Flow** is a deterministic prompt chain. It is a fixed sequence of model calls. It includes Lambda functions. It includes knowledge base lookups. The flow connects them as nodes. It runs the same steps every time.

Use flows when the scenario says same steps every time. Use flows when it says fixed sequence. Use flows when it says analyst-built. Use flows when it says no-code chain. Agents are extra work here. You pay a reasoning model to decide steps.

These steps never change. The exam's decision line is one sentence. Dynamic step choice means agents. Fixed sequence means flows.



### How to read this diagram

1. Left path: the model chooses the next step each time. Use Bedrock Agents.
2. Right path: the steps never change. Use Bedrock Flows.

Watch for invented features in options. Flows do not have an S3 copy node. Flows do not have invented action nodes. Check node lists before you choose.

### Orchestration and the timeout math

GenAI work is rarely one call. A pipeline classifies data. It extracts data. It validates data. It waits for human review. Then it publishes data.

## Timeout math: 30 seconds beats 29 seconds

Do this math in your head. The ceiling sits at API Gateway, not Lambda.

**3,000 tokens at 100 tokens per second = 30 seconds of streaming**

The REST integration timeout at API Gateway is 29 seconds.

**30 seconds is longer than 29 seconds.**

The call dies at the gateway, even though Lambda could run 900 seconds.

Raising the Lambda timeout cannot fix a gateway timeout.

### Fix 1: WebSocket API

stream tokens off the  
request-response path

### Fix 2: async with polling

return a job ID, deliver  
the result later

Slow answers break fast timeouts. A 3000-token answer at 100 tokens per second takes 30 seconds. API Gateway allows 29 seconds.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

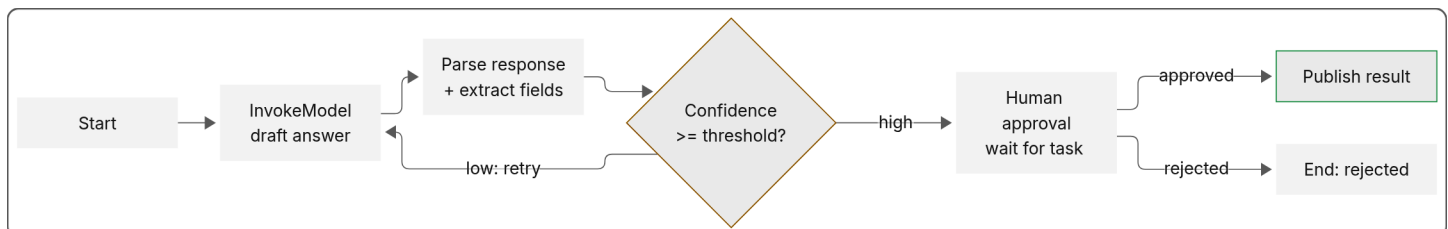
1. Top: the math. Tokens divided by tokens per second gives seconds.
2. Middle: the limit. API Gateway allows 29 seconds per call.
3. Bottom: the fix. Stream the answer, or move the slow work to an async path.

A Lambda trying to do that hits its **15-minute (900-second) ceiling**. It has no durable memory of its state if it crashes. **Step Functions** keeps execution state durably. It retries with backoff. It can pause. There are two types.

Never swap them. **Standard** runs up to 1 year. It costs about \$0.025 for 1,000 state transitions. It supports **wait-for-callback**. A task token lets the workflow pause indefinitely. This is for human approval.

Then it resumes exactly where it stopped. The wait itself is free. **Express** is for short, high-volume flows. It has a 5-minute maximum duration. It has no callbacks. "Approval takes hours" is always Standard with wait-for-callback.

Express cannot pause.



The approval pattern: model call, confidence check, task-token wait for the human, then publish. Express cannot hold this wait.

Source: original figure for the AWS AIP-C01 Study Guide.

For parallel bulk work, **Distributed Map** sends work to up to 10,000 parallel child executions. These executions work on S3 objects. This is the answer to "process 10k objects in a tight window." Sequential processing fails on time. Plain Lambda fan-out fails on orchestration overhead. You must do timeout math in your head. A REST API Gateway in front of a Lambda has a **29-second integration timeout**.

A model streams 100 tokens per second. A 3,000-token answer takes 30 seconds. So a synchronous call through the gateway fails. This happens even if Lambda itself can run for 15 minutes. The fix is asynchronous. Return a job ID.

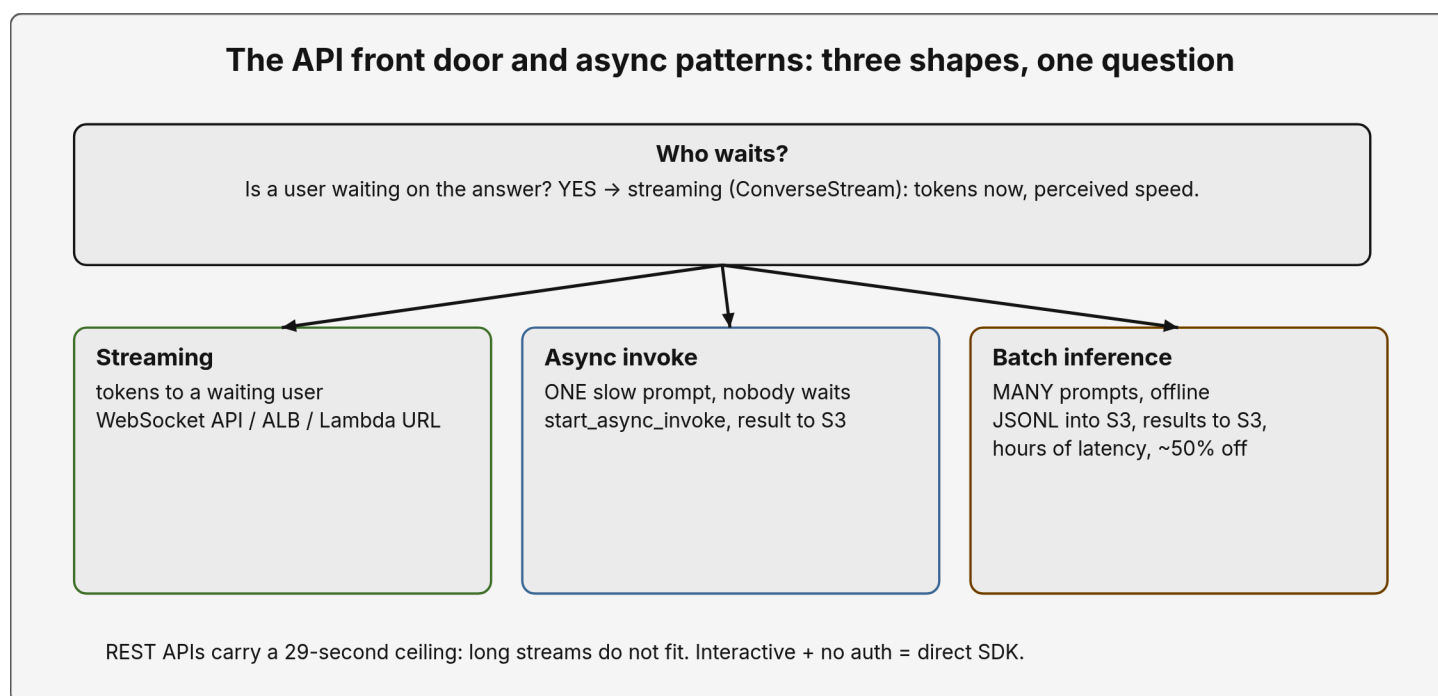
Poll or push the result. Or use WebSocket streaming.

– **Worked example: the timeout math**

Streaming speed: 100 tokens per second. Answer size: 3,000 tokens. Time =  $3,000 / 100 = 30$  seconds. API Gateway REST integration timeout: 29 seconds. 30 is greater than 29. So the call fails at the gateway.

This happens even though the Lambda behind it can run for 900 seconds. The fix moves off the request-response path. Use WebSockets. Or use asynchronous with polling.

## The API front door and async patterns



Three call shapes from one question: who waits? Streaming for a waiting user, async for one slow prompt, batch for many offline prompts

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** The top box asks the one question: who waits. Arrows lead to the three shapes with their mechanisms. Takeaway: batch is many prompts from S3, async is one slow prompt nobody waits on, streaming is tokens to a waiting user.

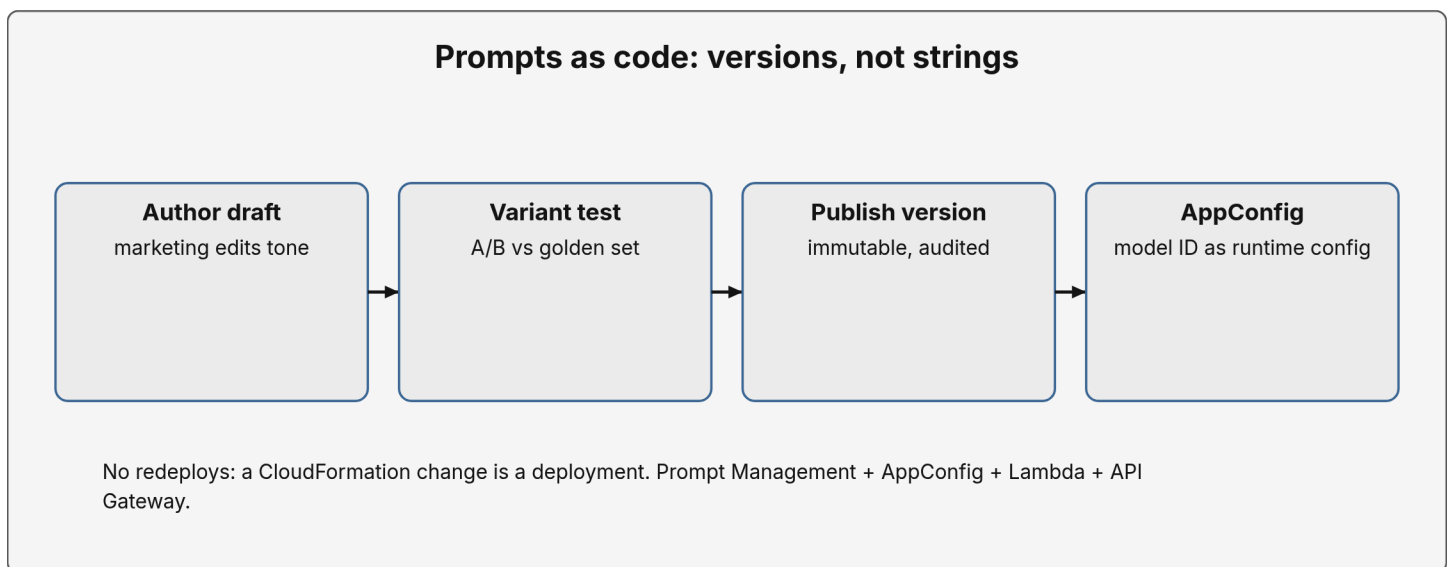
**API Gateway** is the stable endpoint in front of Bedrock. It provides one URL. It provides authentication. It provides throttling. It provides usage plans. The model behind it can change.

Direct SDK calls win only when latency is critical. They win when no authentication is necessary. For streaming, use **WebSocket APIs**. Use an Application Load Balancer. This load balancer spreads incoming HTTP traffic across targets. Or use Lambda function URLs.

REST endpoints have the 29-second limit. They do not fit long streams. The asynchronous single-request pattern is `start_async_invoke`. The call returns an invocation ARN. The result lands in S3. The exam's three-way split: Batch is many prompts from S3.

Asynchronous is one slow prompt. No one waits for this prompt. Streaming sends tokens to a waiting user. Spiky traffic gets **SQS buffering**. It also gets reserved concurrency for the critical function.

## Prompts as code: Prompt Management and AppConfig



Prompts as code: author a draft, test variants, publish a version, serve the model ID from AppConfig

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Left to right: the prompt moves from draft to tested variant to published immutable version, while AppConfig carries the model ID as runtime config. Takeaway: no redeloys ever; a CloudFormation change is a deployment.

Prompts are code. They get versions. **Bedrock Prompt Management** stores prompt templates. These templates carry versions. They have **variants**. You can test two versions together.

They have **variables**. You fill these placeholders when the system runs. This answers the question: "15 prompts, marketing changes tone each week, no redeloys." Marketing edits a draft variant. Marketing tests it with a golden set. Marketing publishes a new version. There are zero engineering redeloys.

There is a full audit trail. "Versioned prompt templates reused across flows" is always Prompt Management. It is not DynamoDB. It is not environment variables. For runtime model switching without redeloys, use three tools. Use **AppConfig**.

It provides the model ID as runtime configuration. Use Lambda for routing. Use API Gateway for a stable URL. A CloudFormation change is a deployment. It always breaks the "no redeploy" rule.

## The API matrix: read the need, pick the row

Bedrock has many ways to call a model. Each answers a different need. The exam tests the mapping.

| Need                                                             | API                           | Key fact                                                                                                                                             |
|------------------------------------------------------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| One request shape across providers, with tool use and guardrails | Converse                      | Unified API. Same shape for Claude, Llama, Titan, Nova. Fields: modelId, messages, system, inferenceConfig, toolConfig, guardrailConfig, cachePoint. |
| Tokens to a waiting user                                         | ConverseStream                | Streaming twin of Converse. It fixes perceived latency.                                                                                              |
| Provider-specific parameters or embeddings                       | InvokeModel                   | Native provider body. The only path for embeddings. Use it when Converse lacks the parameter.                                                        |
| Stream with a native body                                        | InvokeModelWithResponseStream | Streaming for provider-specific calls.                                                                                                               |
| One slow request nobody waits for                                | Async invocation              | S3 in, S3 out. Use it for single requests that take minutes.                                                                                         |
| Many prompts, no SLA, lowest price                               | Batch inference               | JSONL in S3, results in S3, hours of latency, about 50% off on-demand.                                                                               |
| Grounded answer with citations, least effort                     | RetrieveAndGenerate           | Managed RAG call against a Knowledge Base.                                                                                                           |
| Chunks only, custom prompt control                               | Retrieve                      | Returns chunks. You build the prompt.                                                                                                                |
| Safety check on any text, any model                              | ApplyGuardrail                | Standalone API. It covers external models too. Guardrails do not work only with Converse.                                                            |

## The API matrix: read the need, pick the row

Four ways to call a model, each with a different latency, payload, and cost profile.

|                                                                                    |                                                       |
|------------------------------------------------------------------------------------|-------------------------------------------------------|
| <b>Same code for Claude, Llama, Titan</b><br>with tool use and guardrails          | <b>Converse / ConverseStream</b>                      |
| <b>Embeddings</b><br>Converse cannot do vectors                                    | <b>InvokeModel</b>                                    |
| <b>Provider-specific feature</b><br>additionalModelRequestFields: the escape hatch | <b>InvokeModel, or Converse with</b>                  |
| <b>Real-time token streaming</b><br>tokens as they generate                        | <b>ConverseStream / InvokeModelWithResponseStream</b> |
| <b>Async, hours OK, cheapest</b><br>S3 in, S3 out                                  | <b>Batch inference</b>                                |
| <b>Steady baseline + latency SLA</b><br>invoke the provisioned model ARN           | <b>Provisioned Throughput</b>                         |
| <b>Model missing in my region</b><br>us., eu., apac., global. prefixes             | <b>Cross-region inference profile</b>                 |

The exam tests the decision, not the syntax. Read the need, pick the row.

The API matrix maps the need to the call. Portable code takes Converse. Embeddings take InvokeModel. Bulk offline work takes batch.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Read the need in the left column: portable, native, bulk, steady.
2. Follow it to one API row. Each need has one home.
3. Check the trap: the wrong APIs are real calls doing the wrong job.

**Never confuse:** Converse with InvokeModel. Converse is the unified text API. InvokeModel is the native body API and the only embeddings path. Async invocation with batch inference. Async is one slow request. Batch is many requests.

**Exam relevance.** D2 is 26% of the exam. API-choice questions are the most tested mapping in the domain. Read the need. Pick the row.

## Agents: the full anatomy

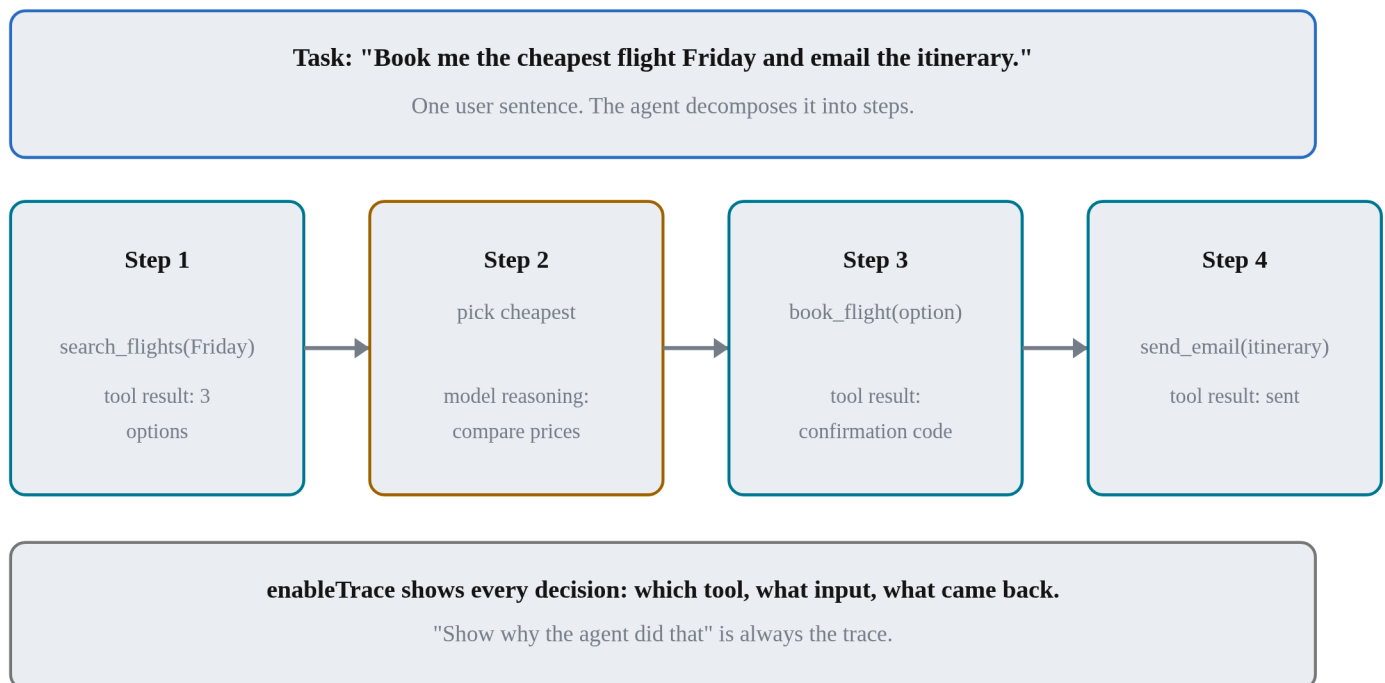
A Bedrock agent wraps the model loop with tools, memory, and a plan. Five parts matter for the exam.

| Part                     | What it does                   | Exam fact                                                                                |
|--------------------------|--------------------------------|------------------------------------------------------------------------------------------|
| Instructions             | Sets the agent's role and task | Part of the agent's prompt anatomy, with conversation history and action groups.         |
| Action groups            | Tools the agent can call       | Each action group maps to Lambda-backed APIs. The agent reasons, then invokes the group. |
| Knowledge Bases          | Grounding for the agent        | The agent pulls grounding when it needs facts. Grounding is separate from tools.         |
| Memory and session state | Remembers the conversation     | Session state persists per user across turns. Memory keeps multi-turn context.           |
| Guardrails               | Safety at the boundary         | The agent returns a final response with a trace and Guardrails applied.                  |

The agent runs a ReAct loop. It reasons about the request. It acts by calling a tool. It observes the tool result. It repeats until the task is done.

### Multi-step decomposition: one task becomes four tool calls

The model plans, the tools execute. The trace records it all.



One request becomes many traced steps. The agent picks the next step each time. That dynamic choice is what makes it an agent, not a flow.

Source: original figure for the AWS AIP-C01 Study Guide.

## How to read this diagram

1. Left: one user request. It is too big for one call.
2. Middle: the agent splits it into steps. Each step is traced.
3. Right: tools execute the steps. The agent observes each result.

Multi-agent patterns use a supervisor. The supervisor agent splits work across specialist agents. Each specialist owns its tools. The supervisor owns the plan.

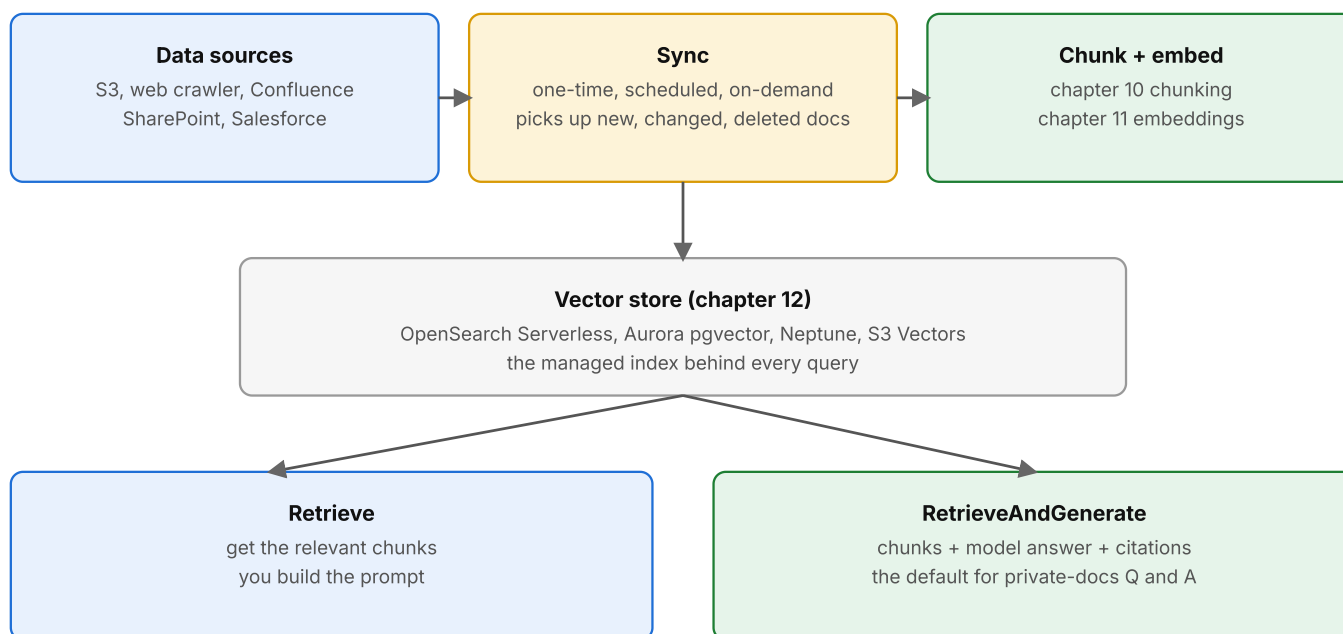
**Agents versus Flows, one line:** the model chooses the next step each time means agents. The steps never change means Flows. The exam tests this line directly.

**Exam relevance.** Agent questions test the dynamic-step line and the anatomy parts. If the stem names tools, memory, or a trace, it is an agent question.

## Knowledge Bases in production

A Knowledge Base is managed RAG. Documents go in. Answers come out. Three operations matter.

| Operation             | What happens                         | Exam fact                                                                                                                |
|-----------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Connect a data source | S3 is the default source             | New files in S3 become searchable with no code. This is the exam's default answer for "ground answers in our documents." |
| Sync on a schedule    | Ingestion runs when documents change | Chunk, embed, and store run offline. The sync schedule decides how fresh the answers are.                                |
| Query                 | RetrieveAndGenerate or Retrieve      | Managed call with citations, or chunks only for custom prompts.                                                          |

**KNOWLEDGE BASES: DOCUMENTS IN, GROUNDED ANSWERS OUT**

**Exam default: any private documents plus citations scenario picks Knowledge Bases.**

Hand-rolled RAG is a distractor unless the scenario needs what the managed service cannot do, like a custom retrieval algorithm.

Attach the same knowledge base to a Bedrock Agent to ground the agent too.

The sync lifecycle: documents land in S3, the sync chunks and embeds them, queries retrieve top-k chunks, the model generates cited answers.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left: documents in S3. The sync schedule watches them.
2. Middle: chunk, embed, store. This runs offline.
3. Right: query time. Retrieve, then generate with citations.

Chunking choices on a Knowledge Base: fixed, hierarchical, or semantic. The ingest pipeline is: documents, then chunks of about 300 tokens with overlap, then embeddings, then the vector store with vector, text, and source metadata.

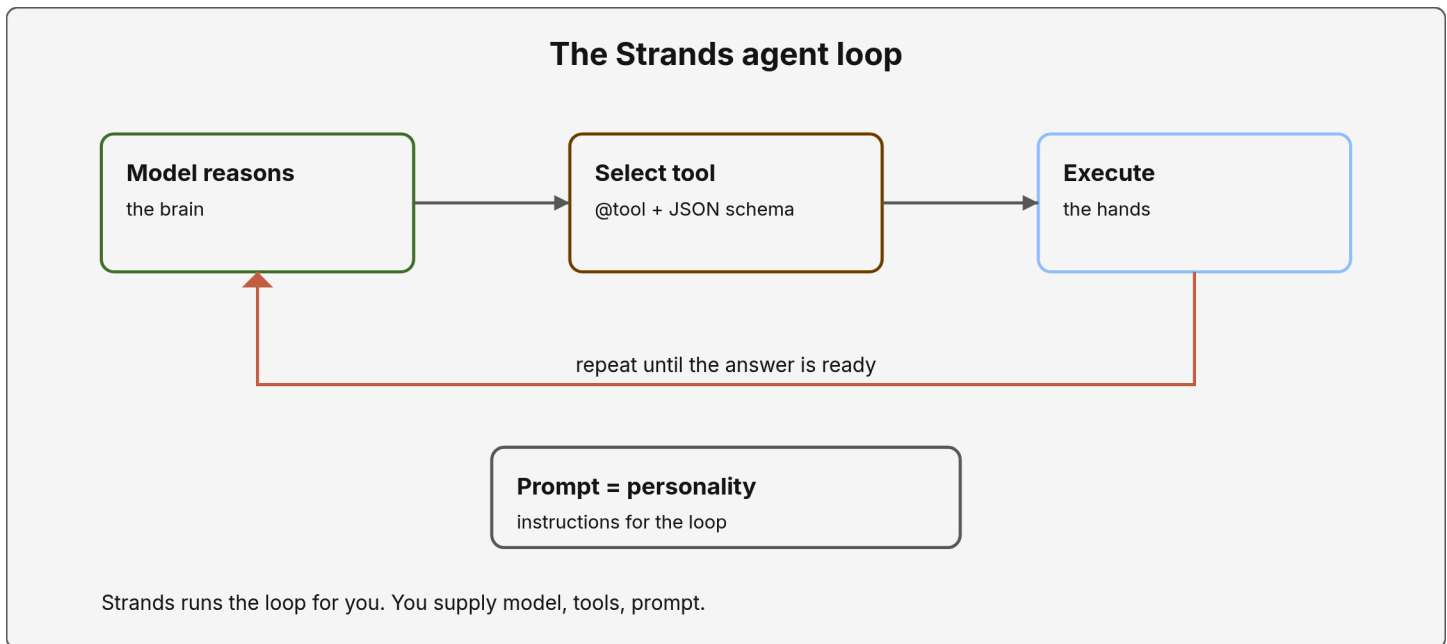
**Exam relevance.** "Support answers must cite the current policy library. The library updates weekly." The answer is a Knowledge Base with an S3 data source, a sync schedule, and RetrieveAndGenerate. Weekly fine-tuning is the classic wrong answer: slow, expensive, and stale between runs.

### Strands Agents: the code-first loop

Strands Agents is an open-source Python SDK from Amazon for building agents. An agent has three parts: a model (the brain), tools (the hands), and a prompt (the personality and instructions). You create an Agent, hand it a list of tools, and call it like a function. The SDK runs the agent loop for you.

**The loop.** The model reasons, selects a tool, executes it, reads the result, and repeats until the answer is ready. Each tool is a Python function decorated with `@tool`. The decorator registers the name, the docstring, and a JSON schema built from the type hints. The Strands Bedrock model provider calls through the Converse API.

**Why it exists.** It is the code-first alternative to the Bedrock Agents console. Run the same agent on a laptop, in a container, on Lambda, or on AgentCore Runtime. It supports MCP servers, non-AWS models, and Boto3 integrations with Bedrock and Lambda. Built-in tools cover web search over HTTP, shell commands, speech through Amazon Polly, and agent memory.



The model reasons, selects a tool, executes it, and repeats. The prompt sets the personality for the whole loop.

*The model reasons, selects a tool, executes it, and repeats. The prompt sets the personality for the whole loop.* Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Three boxes left to right: reason, select tool, execute.
2. The red arrow loops back. The loop runs until the answer is ready.
3. The bottom box is the prompt. It shapes every loop iteration.

**Trap: "Strands replaces Bedrock Agents."** Wrong. They serve different needs. Bedrock Agents is the managed console service with action groups and aliases. Strands is the SDK you host yourself. Pick Strands when the scenario stresses code-first deployment on your own infrastructure.

**Exam relevance.** "Python agent, minimal boilerplate, full code control" names Strands Agents. "Managed, no code" names Bedrock Agents.

### Agent Squad: routing, not loops

AWS Agent Squad is an open-source framework for multi-agent workflows built around intent classification. The pip package is **agent-squad**. Register several specialist agents with one orchestrator. Each request passes through a classifier, usually a small fast model at temperature 0. The classifier picks the best specialist. The orchestrator hands over the request and returns the answer with routing metadata.

**Why specialists beat generalists.** A billing agent, a support agent, and a general assistant use different tools, prompts, and knowledge. Adding a fourth specialty means registering one more agent, not rewriting a router. The framework also supports a ChainAgent for fixed pipelines and a SupervisorAgent for coordination. Chat history uses a ChatStorage layer: in-memory by default, DynamoDB or SQL in production.

**Exam relevance.** "Route each request to the right specialist" names Agent Squad. "One agent using many tools in a loop" names Strands. The exam separates routing from tool use.

### Strands vs Squad vs Bedrock Agents: pick the row

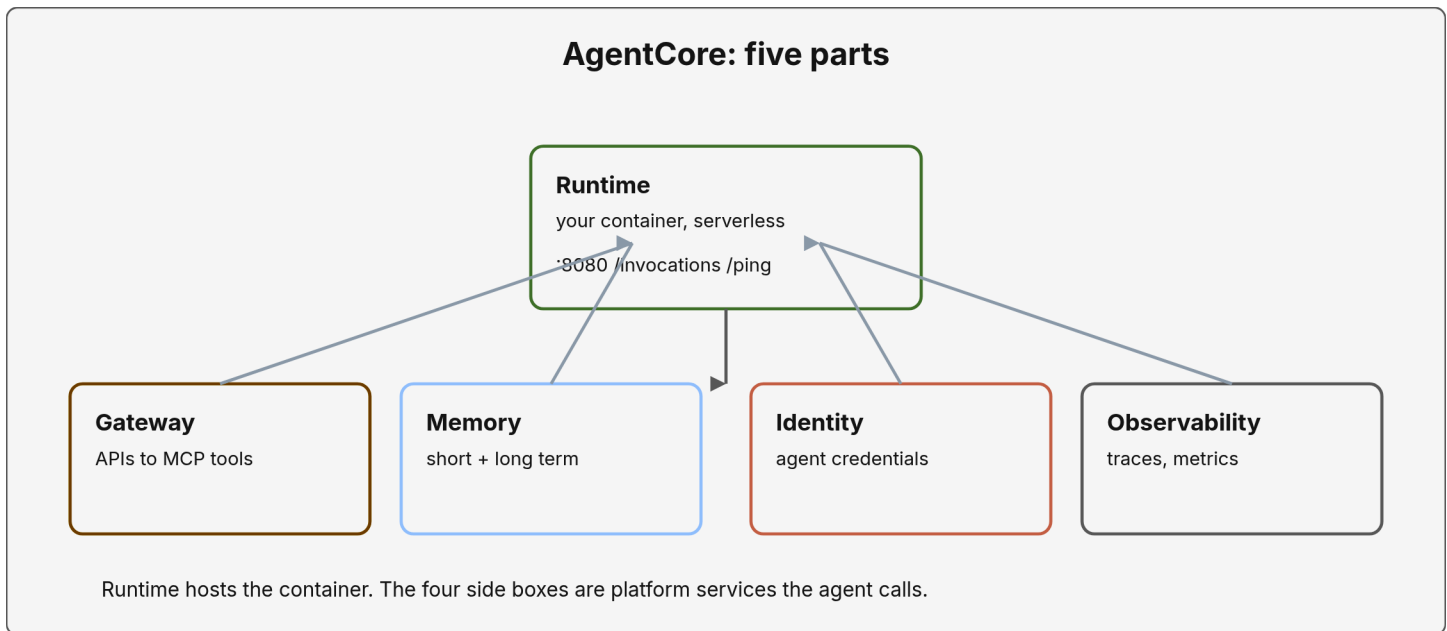
| Need                                                          | Pick                      |
|---------------------------------------------------------------|---------------------------|
| Code-first agent with a tool-use loop, hosted anywhere        | Strands Agents            |
| Many specialist agents behind one entry point, intent routing | Agent Squad               |
| Fully managed agent, console setup, action groups, no code    | Bedrock Agents            |
| Fixed steps with conditions, auditable, no model decisions    | Bedrock Flows             |
| Any-framework agent deployed serverless at scale              | Bedrock AgentCore Runtime |

**The one-line split.** Strands is the tool-use loop inside one agent. Squad is the router across many agents. Bedrock Agents is the managed service. AgentCore is the serverless platform that can host any of them.

### Bedrock AgentCore: five parts, one platform

You built an agent with an SDK. Now you must deploy it. AgentCore is the managed platform that runs agents in production. It has five parts you must name.

1. **Runtime.** Serverless hosting for your agent container. Any framework works: Strands, LangGraph, CrewAI, OpenAI Agents SDK, Google ADK. The container contract is fixed: listen on port 8080, answer POST /invocations with agent logic, answer GET /ping as a health check. A starter toolkit scaffolds the deploy.
2. **Gateway.** Converts your existing APIs and Lambda functions into MCP-compatible tools. Agents call them through Gateway endpoints.
3. **Memory.** Managed short-term and long-term agent context that outlives any single container.
4. **Identity.** Agent authentication and credential handling for third-party applications.
5. **Observability.** Tracing and monitoring with OpenTelemetry, surfaced in CloudWatch.



Runtime hosts any-framework container. Gateway turns APIs into MCP tools. Memory, Identity, and Observability are platform services the agent calls.

*Runtime hosts any-framework container. Gateway turns APIs into MCP tools. Memory, Identity, and Observability are platform services the agent calls. Source: original figure for the AWS AIP-C01 Study Guide.*

### How to read this diagram

1. Center box: Runtime. Your container. POST /invocations, GET /ping, port 8080.
2. Four side boxes: Gateway, Memory, Identity, Observability. Platform services.
3. Arrows: the agent calls the four services. The platform hosts the container.

**The migration story.** Move from Bedrock Agents to AgentCore when you need framework freedom without giving up managed hosting. The exam tests this exact tradeoff.

**Not in the repo volumes.** Some AgentCore details live in the repo volumes, not in this crash course. Four areas. Built-in tools: Browser Tool (managed headless browser) and Code Interpreter (sandboxed code execution) run in isolated sessions. AgentCore Identity covers OAuth 2.0 flows and Bedrock API key auth. AgentCore Policy covers Cedar language policies, deny by default, and log-only versus enforce mode. AgentCore Evaluators covers built-in quality evaluations.

**Failure modes.** The runtime contract is strict. A container that ignores GET /ping fails health checks and never serves traffic. A container that reads the wrong JSON shape from POST /invocations drops every request. Sessions add another trap: state kept in container memory vanishes when the container recycles. That is what AgentCore Memory is for. The exam twist: "the agent worked locally but fails on AgentCore" points at the contract, not the model.

**Exam relevance.** "Deploy a LangGraph agent serverless" names AgentCore Runtime. "Expose REST APIs as agent tools" names AgentCore Gateway. "Remember user preferences across weeks" names AgentCore Memory.

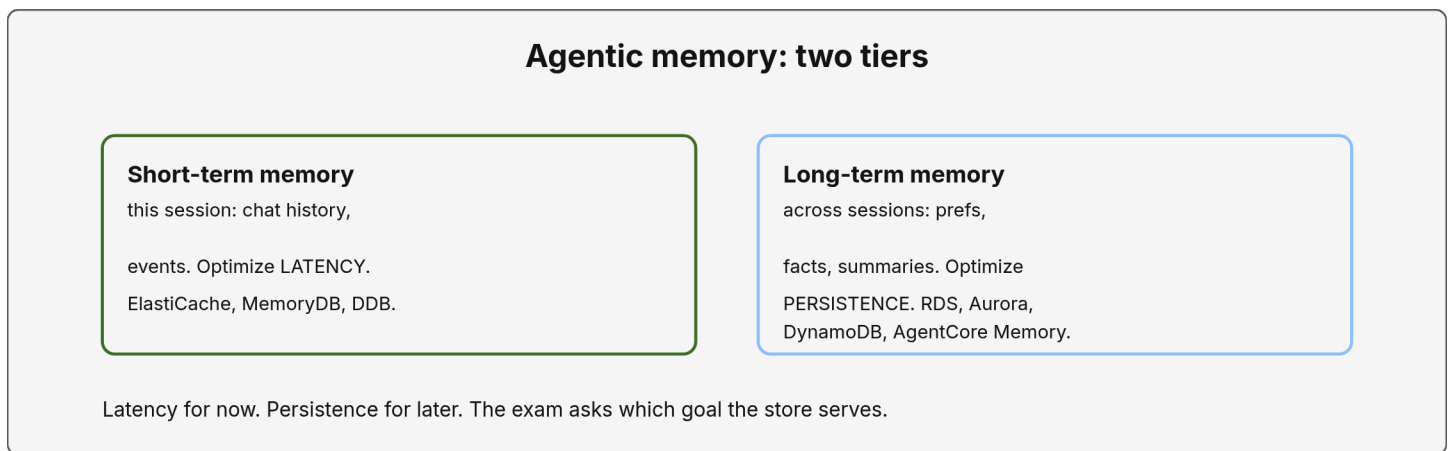
### MCP: the USB-C port for AI

The Model Context Protocol is an open standard for connecting AI applications to external systems. It standardizes how apps give context to LLMs, so every integration follows the same pattern instead of a custom one.

**The layers.** The data layer is JSON-RPC 2.0. It defines the message structure. The transport layer is stdio for local servers or Streamable HTTP for remote servers. Source: Model Context Protocol documentation, architecture page.

**Servers expose three primitives.** Tools (functions the model can call), resources (data the model can read), and prompts (reusable prompt templates). Popular MCP servers exist for Slack, PostgreSQL, GitHub, GitLab, and most SaaS tools. Any agent SDK worth using can call external MCP servers.

| MCP server shape                               | Host it on                   |
|------------------------------------------------|------------------------------|
| Stateless, lightweight tools                   | Lambda functions             |
| Complex tools needing long runs or local state | Containers on ECS or Fargate |
| Agent tooling at scale with managed identity   | Bedrock AgentCore            |
| Public HTTPS endpoint for the server           | API Gateway in front         |



Short-term memory optimizes latency for this session. Long-term memory optimizes persistence across sessions. Pick the store by the goal.

*Short-term memory optimizes latency for this session. Long-term memory optimizes persistence across sessions. Pick the store by the goal.*

Source: original figure for the AWS AIP-C01 Study Guide.

**Failure modes.** MCP servers fail like any network service. A down MCP server breaks every tool call behind it. Timeouts on the transport layer look like model failures in the trace. Debug the server before the prompt. The exam twist: "the agent stopped calling tools" after a deploy points at the MCP server or the Gateway, not at the agent loop.

**Exam relevance.** "Standard way to connect an agent to external tools" names MCP. "JSON-RPC 2.0" is the data layer the exam may quote. "Stateless lightweight MCP server" points at Lambda.

## OpenAPI: the contract between model and tool

OpenAPI is the standardized specification for web service interfaces. It was first called Swagger. It defines function inputs, outputs, and error conditions in one machine-readable document.

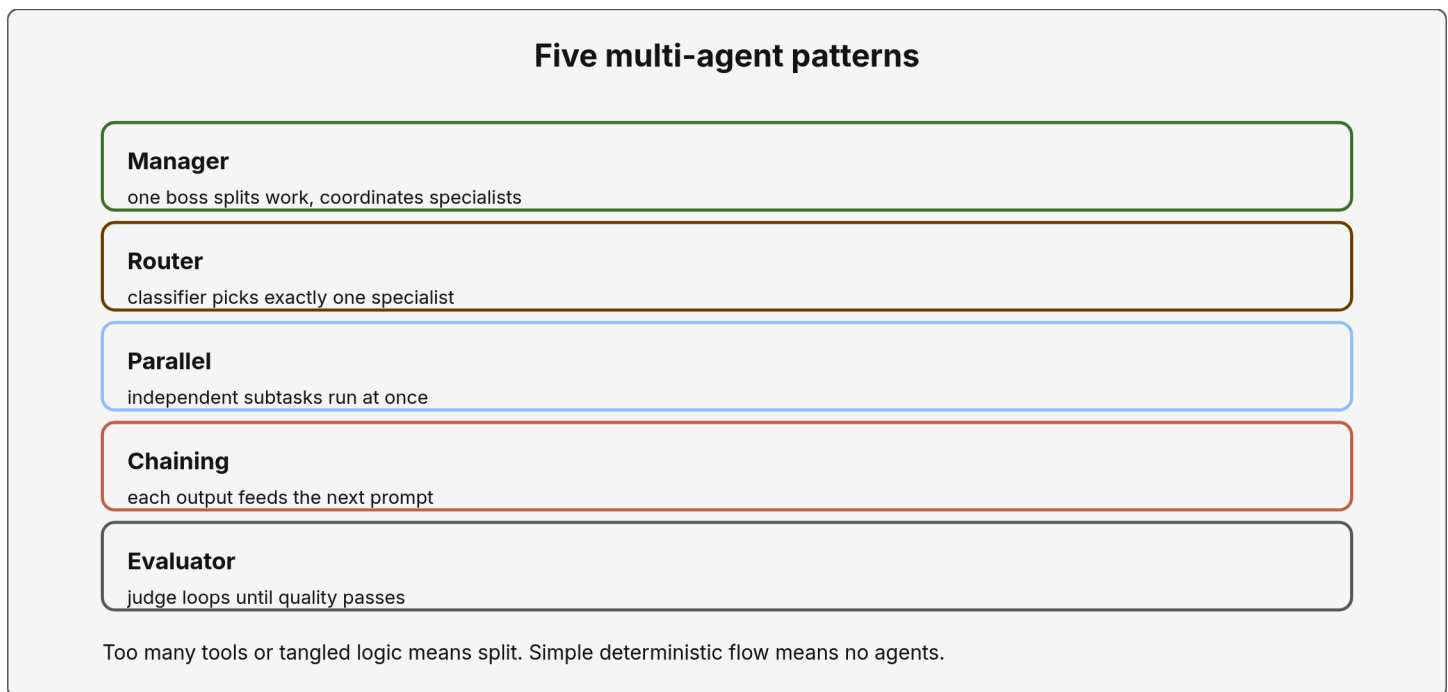
**Why agents care.** Bedrock Agent action groups use OpenAPI schemas to define AI tools. You upload the schema to S3 or paste it in the console. The model reads the schema and knows exactly how to call the tool. Standardized definitions make tool calls more reliable than free-text descriptions.

**Exam relevance.** "Define the interface between the FM and an external web service" points at an OpenAPI schema. "Action group needs a tool definition" is the same answer.

## Multi-agent: one agent or many

**When many agents win.** Use multiple specialized agents when one agent carries too many tools and picks the wrong one. Or when the prompt logic turns into a maze of conditionals. Specialists stay sharp.

**When one is enough.** If the workflow is deterministic and simple, keep it simple. Multi-agent systems cost more and fail in more ways. The exam rewards the simplest design that meets the constraints.



Five patterns. Manager splits work across specialists. Router picks one specialist. Parallel runs independent subtasks. Chaining feeds outputs forward. Evaluator loops until quality passes.

*Five patterns. Manager splits work across specialists. Router picks one specialist. Parallel runs independent subtasks. Chaining feeds outputs forward. Evaluator loops until quality passes.* Source: original figure for the AWS AIP-C01 Study Guide.

## How to read this diagram

1. Manager (orchestrator): one boss agent decomposes the task and coordinates specialists.
2. Router: one classifier picks exactly one specialist for the task.
3. Parallelization: independent subtasks run at the same time. No complex orchestration needed.
4. Prompt chaining: each prompt output feeds the next prompt. Gates can validate or break the flow.
5. Evaluator and optimizer: one LLM produces output, another judges it, and they iterate until it is good enough.

**Exam relevance.** "Too many tools for one agent" means split into specialists. "Independent subtasks" means parallelization, not a manager. "Simple deterministic flow" means no agents at all.

## Agentic memory: latency or persistence

Agents need two kinds of memory. Optimize each for a different goal.

|              | Short-term memory                                        | Long-term memory                                                                             |
|--------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Holds        | Current session context: chat history, in-session events | Cross-session facts: user preferences, past interactions, session summaries, knowledge bases |
| Optimize for | Latency                                                  | Persistence                                                                                  |
| Store in     | ElastiCache, MemoryDB, DynamoDB                          | RDS, Aurora, DynamoDB, AgentCore Memory                                                      |

**Exam relevance.** "Remember chat history in this session" points at ElastiCache or MemoryDB. "Remember customer tier across sessions for weeks, query by customer ID in milliseconds" points at DynamoDB with server-side encryption. "Managed memory for agents" points at AgentCore Memory. Note: Mem0 is a third-party agent memory option named in the study hub. It is not covered in the repo volumes.

### Humans in the loop: three patterns

Most AI systems need human supervision, especially early. Three patterns show up on the exam.

1. **Human augmentation.** The AI drafts, humans refine. Customer support emails and recruiting emails follow this shape. Low tech, high value.
2. **Escalation to humans.** The agent handles routine cases. Complex or uncertain cases route to human experts based on confidence scores or risk. The routing must be correct or escalation never fires.
3. **Human feedback for improvement.** Collect frontend feedback through API Gateway, store it with a Lambda in DynamoDB, and analyze which model or variant users prefer. RLHF is the automated form: the system learns from human feedback directly.

**Exam relevance.** "AI drafts, humans refine" names augmentation. "Route uncertain cases to experts" names escalation. "Collect ratings to improve the model" names the feedback loop.

### Bedrock Data Automation: shred any file

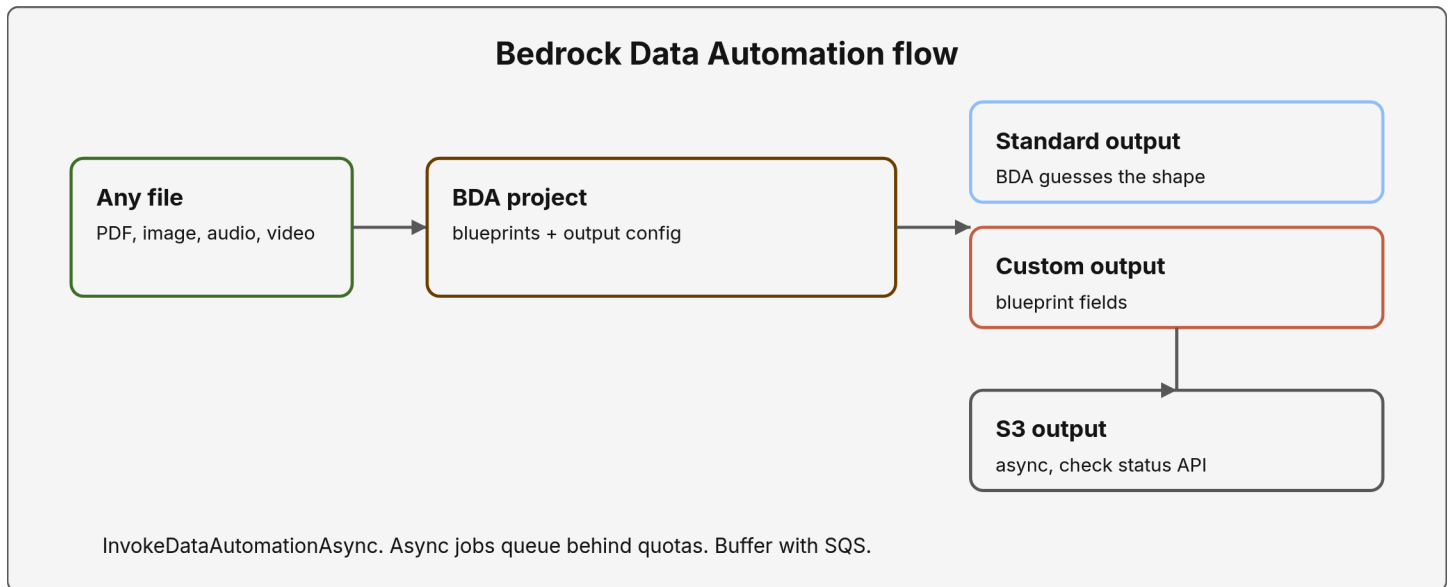
Bedrock Data Automation extracts structured data from documents, images, audio, and video. It is multimodal. Use it to prepare data for a vector store, to run intelligent document processing, or to analyze media at scale with scripts.

| Output          | What you get                                                                                                       |
|-----------------|--------------------------------------------------------------------------------------------------------------------|
| Standard output | BDA guesses the best structure. Documents become JSON, audio becomes a transcript.                                 |
| Custom output   | You define exactly what to extract with a Blueprint: fields, tables, field groups, compound fields like addresses. |

**Blueprints.** A Blueprint names the fields to extract. Prebuilt blueprints cover passports and US driver licenses. You can build one from a prompt in plain language. Blueprints do four extra jobs. Classification matches documents to a blueprint. Normalization gives one name and one format per field. Transformation splits and restructures fields. Validation checks ranges and sizes.

**Projects and invocation.** A project stores output configurations with one or more blueprints. Trigger processing with the InvokeDataAutomationAsync API against your project. Processing is asynchronous and queues behind service quotas, so buffer bursts with SQS.

**Media coverage.** Documents (PDF, TIFF, JPEG, PNG, DOCX) yield JSON, HTML, or CSV at page, element, or word granularity. Images yield summaries, text in image, logos, and IAB taxonomy. Audio yields transcripts with speaker labels. Video yields summaries, chapters, transcripts, and content moderation flags.



Any file enters a BDA project. Blueprints shape custom output. Results land in S3. Async jobs queue behind quotas.

*Any file enters a BDA project. Blueprints shape custom output. Results land in S3. Async jobs queue behind quotas.* Source: original figure for the AWS AIP-C01 Study Guide.

**Failure modes.** Async jobs queue behind service quotas. A morning burst of 10,000 files without SQS buffering throttles and drops jobs. A blueprint that is too strict rejects valid documents. A blueprint that is too loose extracts garbage fields. The exam twist: "BDA worked in testing, fails in production" almost always means quotas or burst shape, not the blueprint.

**Exam relevance.** "Extract these exact fields from invoices" means a custom blueprint. "Just structure this file somehow" means standard output. "10,000 files a day" means async invocation with SQS buffering.

### Cross-region inference: two profile shapes

Cross-region inference routes requests across AWS regions. It survives regional outages, dodges quota limits, and balances load. Two profile shapes control where requests may land.

| Profile    | Routing                                                         | Pricing                                  |
|------------|-----------------------------------------------------------------|------------------------------------------|
| Global     | Bedrock picks the optimal commercial region                     | Up to 10 percent cheaper, max throughput |
| Geographic | Bedrock picks the optimal region inside your geography (US, EU) | Standard pricing                         |

Source for profile shapes: AWS Bedrock documentation on cross-region inference.

## Cross-region inference profiles

### Global profile

Bedrock picks the best

commercial region. Max throughput.  
Up to 10% cheaper.

### Geographic profile

best region inside your

geography (US, EU). Standard price.

### Neither works with Provisioned Throughput

SCPs can block regions too

Data residency picks geographic. Cost and throughput pick global.

Global picks the best commercial region. Geographic stays inside your geography. Neither works with Provisioned Throughput.  
*Global picks the best commercial region. Geographic stays inside your geography. Neither works with Provisioned Throughput.* Source: original figure for the AWS AIP-C01 Study Guide.

**Restrictions.** Cross-region inference does not work with Provisioned Throughput. AWS Organizations service control policies can block regions and break routing. If the account sits in an Organization, confirm the SCPs allow every needed region or leave the region unspecified.

**Exam relevance.** "Data must stay in the EU" means a geographic profile. "Cheapest resilient inference" means a global profile. "Cross-region with Provisioned Throughput" is a contradiction the exam uses as a distractor.

## Lambda for GenAI: the glue jobs

Lambda is the serverless custom code in almost every GenAI architecture. Nine jobs show up on the exam.

1. Connect Bedrock Agents to tools. Calling a tool usually means calling a Lambda.
2. Validate parameters and handle errors before the model call.
3. Invoke FMs on demand with no provisioned capacity. Call the Bedrock API from the function.
4. Receive webhooks from external tools.
5. Handle JSON events from API Gateway, which acts as the frontend.
6. Aggregate across models: weighted averages, voting, fallbacks.
7. Sync raw data to databases on change events.
8. Preprocess documents: convert HTML headings to divider strings before chunking.
9. Screen data with Comprehend before it reaches Bedrock: redact PII, detect language, classify content.

**Exam relevance.** "Serverless custom code" in a GenAI architecture almost always means Lambda. "Screen user uploads before the agent sees them" means Lambda plus Comprehend.

## Step Functions: when workflows beat agents

Agentic SDKs orchestrate plenty. Step Functions still wins in three cases. Auditable state transitions. Explicit approval steps where the flow stops until a human approves. Retry and failure isolation managed in one place.

**Circuit breaker pattern.** Build it with Step Functions, Lambda, and DynamoDB. A Lambda checks whether a service is up and writes the state (circuit open or closed) to DynamoDB. The state machine skips calls to a down service and routes to a fallback. In GenAI this means: if the reasoning model is down, route to a standard model. Degraded but alive beats a hard failure.

**Exam relevance.** "Stop calling the FM that is down, route to a fallback" names the circuit breaker. "Human must approve before the next step" names Step Functions, not an agent.

## AppSync, Outposts, Wavelength, Amplify: the edges

**AppSync.** Serverless GraphQL and pub/sub APIs. Resolvers can call Lambda functions that run FM inference, and VTL mapping templates translate GraphQL requests into model inputs. Pick it when the frontend wants one GraphQL endpoint over many sources including FMs.

**Outposts.** AWS hardware on your premises. Pick it when data privacy law or policy forbids data leaving the building. Size the compute for FM inference and use local caching to cut data movement to external models.

**Wavelength.** AWS infrastructure inside mobile carrier networks. Pick it for 5G edge deployments: light fast work at the edge, heavy FM work in the region.

**Amplify.** Full-stack web and mobile app hosting: auth, storage, APIs, CI/CD. Pick it when the GenAI app needs a frontend fast and the model lives behind API Gateway and Lambda.

**Exam relevance.** "GraphQL over FMs" means AppSync. "Data cannot leave the country" means Outposts. "5G edge" means Wavelength. These are recognition questions, not deep ones.

## SageMaker AI: the exam distractor

SageMaker AI builds and trains ML models end to end. In AIP-C01 it is rarely the answer. Training a model takes time, effort, and operational cost. Bedrock gives you a managed model today. Choosing SageMaker over Bedrock usually means reinventing the wheel. The exam offers SageMaker as a distractor when the scenario wants a managed FM.

**When SageMaker is correct.** Train or tune in SageMaker, then deploy in Bedrock through Bedrock Custom Model Import for serverless inference. SageMaker Inference Components give each model its own scaling policy. Model compression trims deployments: quantization shrinks weights, pruning removes dead weights, distillation trains a small model from a large one.

**Not in the repo volumes.** These live in the repo volumes, not in this crash course. DJL (Deep Java Library for containers). SageMaker ML Lineage Tracking (LineageQuery API, cross-account tracking through AWS RAM). SageMaker Unified Studio (the team web UI). MLFlow on SageMaker.

**Exam relevance.** "Train a custom FM from scratch" with Bedrock available is the classic SageMaker distractor. "Fine-tune then serve serverless" is the SageMaker-to-Bedrock import path.

## Not in the repo volumes: read these before the exam

The study hub covers topics the repo volumes do not. Do not invent answers for them. Read the official docs instead.

- **Amazon Q Developer Rules.** Markdown files in the .amazonq/rules directory that enforce coding standards and workflow preferences for Q Developer. Like instruction files for coding assistants.
- **AgentCore Policy and Evaluators.** Cedar-language tool policies and built-in LLM-as-a-judge evaluations. See the Bedrock AgentCore developer guide.
- **Nova Reels details.** Video generation in the Nova family. Name recognition only.
- **Mem0.** Third-party agent memory service. Named as a long-term memory option.
- **Latency-optimized inference.** Bedrock performanceConfig with latency set to optimized. Improves time to first token and output token rate.

## More D2 decision rules

- If the stem says "same request shape for Claude and Llama," pick Converse. Never pick InvokeModel.
- If the stem says "embeddings," pick InvokeModel with an embedding model. Converse cannot embed.
- If the stem says "provider-specific parameter missing from Converse," pick InvokeModel with the native body.
- If the stem says "one slow request, nobody waits," pick async invocation. S3 in, S3 out.
- If the stem says "many prompts, no SLA, lowest price," pick batch inference. JSONL in S3, about 50% off.
- If the stem says "safety check on text from an external model," pick ApplyGuardrail. It is standalone.
- If the stem says "steps change each time," pick Bedrock Agents. If the stem says "fixed sequence," pick Bedrock Flows.
- If the stem says "agent needs company facts," attach a Knowledge Base. Grounding is separate from action groups.
- If the stem says "human approval takes hours," pick Step Functions Standard with wait-for-callback. Express cannot pause.
- If the stem says "slow answers break a 29-second API Gateway timeout," pick streaming or move the work to an async path.
- If the stem says "swap models without redeploying," put the model ID in AppConfig. Read it at runtime.
- If the stem says "Lambda hits its 15-minute ceiling," move the workflow to Step Functions. Step Functions keeps state durably.

## More D2 traps, with why the wrong answer fails

- Flows for dynamic steps. Flows run a fixed sequence. Steps that change need agent reasoning. Match the line.
- Agents for a fixed sequence. Agents add tool-calling overhead. A fixed sequence needs Flows. Do not pay for reasoning you do not use.
- Converse for embeddings. Converse generates text only. The embeddings path is always InvokeModel.
- Batch inference for one interactive request. Batch has hours of latency. Interactive requests need sync or streaming.

- Async invocation for bulk work. Async is one slow request. Bulk work needs batch inference.
- Step Functions Express for human approval. Express caps at 5 minutes and cannot pause. Approval takes hours. Use Standard with wait-for-callback.
- Lambda for durable multi-hour workflows. Lambda caps at 15 minutes and forgets state on crash. Step Functions keeps state durably.
- Guardrails only with Converse. Wrong. ApplyGuardrail is standalone. It checks any text from any model.
- Invented Flow nodes. Flows do not have an S3 copy node. Check node lists before you choose. Invented features are distractors.
- CloudFormation change for a model swap. A CloudFormation change is a deployment. AppConfig serves the model ID at runtime with no redeploy.

## D2 self-test: more questions with answers

D2 · MEDIUM · ONE ANSWER CC57. A document pipeline has four fixed steps: extract text, validate fields, normalize dates, load to a database. The steps never change. Which service do you use? A) Bedrock Agents. Agents choose steps dynamically. The steps here never change. B) Bedrock Flows. C) Step Functions with Lambda. This works, but Flows is the purpose-built GenAI answer. D) An agent with a supervisor. Supervisor agents split dynamic work. The work here is fixed. Reveal answer Why B is correct: "Fixed steps that run the same way every time" is the Flows sentence. Dynamic step choice means agents. A fixed sequence means Flows.

D2 · MEDIUM · ONE ANSWER CC58. A safety team must screen text from a third-party model before it reaches users. The model does not run on Bedrock. Which API do you use? A) Converse with guardrailConfig. Converse calls Bedrock models. The text comes from outside. B) ApplyGuardrail. C) RetrieveAndGenerate. This is the RAG API. It does not screen arbitrary text. D) InvokeModel. This invokes a Bedrock model. It does not screen text. Reveal answer Why B is correct: ApplyGuardrail is the standalone safety API. It checks any text from any model. Guardrails do not work only with Converse.

D2 · EASY · ONE ANSWER CC59. A nightly job scores 50,000 support tickets with a model. Nobody waits for results. Cost matters. Which call shape do you use? A) ConverseStream. Streaming serves waiting users. Nobody waits here. B) Batch inference with JSONL in S3. C) Async invocation. Async is one slow request, not 50,000. D) Provisioned Throughput. PT is capacity for steady traffic. It is not the bulk answer. Reveal answer Why B is correct: "Many prompts, no SLA, lowest price" is the batch sentence. JSONL goes to S3. Results return to S3. The price is about 50% off on-demand.

D2 · MEDIUM · ONE ANSWER CC60. A loan workflow needs a human underwriter to approve flagged applications. Approval takes up to two days. Which orchestration do you use? A) Step Functions Express. Express caps at 5 minutes. It cannot pause for days. B) Step Functions Standard with wait-for-callback. C) Lambda with a long sleep. Lambda caps at 15 minutes. D) EventBridge Scheduler alone. The scheduler triggers. It does not hold workflow state. Reveal answer Why B is correct: "Approval takes hours or days" is always Standard with wait-for-callback. A task token pauses the workflow indefinitely. The wait itself is free. Express cannot pause.

D2 · EASY · ONE ANSWER CC61. An app must swap the model behind a feature flag without redeploying. Where does the model ID live? A) In the CloudFormation template. A template change is a deployment. B) In AppConfig, read at runtime. C) Hardcoded in the Lambda. A code change is a deployment. D) In an S3 file polled daily. This works but AppConfig is the purpose-built answer. Reveal answer Why B is correct: Prompts as code means no red deploys. AppConfig carries the model ID as runtime config. The app reads it at runtime.

## D2 decision rules: if the stem says X, pick Y

- "Reasons, decides steps, chooses tools" means Bedrock Agents. Use action groups and Lambda.
- "Fixed sequence, same steps every time" means Bedrock Flows. Agents are extra work.
- `invoke_agent` needs `agentId`. It needs `agentAliasId`. This must be PROD, never DRAFT. It needs `sessionId`. Use one per user. It needs `inputText`. It returns a stream. You iterate this stream.
- The agent does not use the new action group. The team skipped `PrepareAgent`.
- Show why the agent did that. Use `enableTrace=true`.
- Lambda action function: parameters arrive as a LIST of name/value dictionaries. The response shows the nested shape. The result is in `responseBody.TEXT.body`.
- The task runs longer than 15 minutes. Use Step Functions, not Lambda.
- Human approval takes hours. Use Step Functions Standard with wait-for-callback. Express has a 5-minute limit. Express cannot pause.
- Process 10,000 S3 objects in a short time. Use Distributed Map. It can run up to 10,000 parallel children.
- Streaming answers time out. Use WebSockets or async with polling. Do not use a bigger Lambda timeout. The 29-second limit is at API Gateway.
- You need a stable URL, authentication, and throttling. Use API Gateway and Lambda. Use direct SDK only if latency is very important and you do not need authentication.
- You need versioned prompt templates. You do not want redeployments. Use Prompt Management. You want to swap models without redeploying. Use AppConfig.
- You need a quality gate before you deploy. Use Bedrock evaluation jobs in the pipeline. Deploy only if the score is above the threshold.

## D2 trap patterns

- Lambda waits, sleeps, or polls for more than 15 minutes. It stops.
- The Agent ID is alone in production. It routes to DRAFT instead of the PROD alias.
- The app shares one `sessionId` across users. This leaks conversation memory.
- Do not use Express for human waits that take hours. It has a 5-minute limit. It has no callbacks.
- Do not raise the Lambda timeout for gateway timeouts. The 29-second limit is at API Gateway.
- Do not use Fabricated Flow nodes (S3 copy node). Check the node and feature lists.
- Do not use CloudFormation for runtime switching. Configuration changes are deployments. AppConfig is for runtime.
- Do not use Agents for fixed sequences. Reasoning overhead has no benefit.
- Do not use multi-tenant agents without per-KB IAM filtering. Tenants must not reach each other's data.

## D2 rapid-fire questions

### D2 · EASY · ONE ANSWER

CC15. An order assistant must check inventory with an API. It must check shipping with another API. Then it must tell the customer. The steps change for each order. Should you use Agents or Flows?

A) Bedrock Agents: the model decides which tools to call for each order

B) Bedrock Flows

Flows run a fixed sequence. Steps that change for each order need agent reasoning.

**Why A is correct:** The steps change. The system must make a decision. The system uses many tools. The dynamic-step line points to agents. Fixed steps always point to Flows.

### D2 · EASY · ONE ANSWER

CC16. The `invoke_agent` command must use the PROD deployment. It must not use the draft. Which parameter must it use?

A) `agentId` only

The Agent ID only routes to DRAFT. This is the common production error.

B) `agentAliasId` set to the PROD alias

C) `sessionId`

The Session ID is the memory key. It is not the deployment selector.

**Why B is correct:** The alias selects the deployment. The full call shape is: `agentId` + `agentAliasId` + `sessionId` + `inputText`. The response is a stream. You iterate the stream.

**D2 · MEDIUM · ONE ANSWER****CC17. An agent does not use a new action group. What is the first check?****A) Did prepare-agent run after the configuration change?****B) The Lambda timeout**

The system does not use the action group before any Lambda runs. Configuration changes need 'prepare' to compile DRAFT into effect.

**C) The embedding model**

Embeddings are for retrieval. They are not related to action groups.

**Why A is correct:** If you skip prepare-agent, the agent uses the old configuration. "The configuration changed but the behavior did not" means you skipped 'prepare'.

**D2 · EASY · ONE ANSWER****CC18. A Lambda action function receives `event["parameters"]`. What is its shape?****A) A simple dictionary of name to value**

If you treat it as a dictionary, you lose every parameter. The exam tests this exact error.

**B) A list of `{"name": ..., "value": ...}` dictionaries**

**Why B is correct:** This is the fixed contract. The response must repeat `messageVersion`, `actionGroup`, and `function`. It must include the result string in `responseBody.TEXT.body`.

**D2 · EASY · ONE ANSWER****CC19. A claim pipeline must stop for manager approval. This takes 6 hours on average. Then it must start again. Which orchestration must you use?****A) Step Functions Express**

Express has a 5-minute limit. It cannot stop for callbacks.

**B) Step Functions Standard with wait-for-callback (task token)****C) Lambda with a sleep loop**

Lambda stops after 15 minutes. This is much less than 6 hours.

**Why B is correct:** You need Standard's task-token callback for waits that are many hours long. It runs for up to 1 year. The wait itself costs nothing. Express is for short, high-volume flows.

**D2 · MEDIUM · ONE ANSWER**

**CC20. A chatbot streams answers. This can take 90 seconds. Users see timeouts behind a REST API. How do you fix this?**

**A) Increase the Lambda timeout to 5 minutes**

The 29-second limit is at API Gateway. It is not at Lambda. The timeout is in the wrong place.

**B) Use API Gateway WebSockets plus Lambda streaming. Or use async with polling.**

**C) Provisioned Throughput**

PT fixes capacity. PT does not fix the gateway timeout.

**Why B is correct:** The timeout math shows this. 90 seconds is more than the 29-second REST integration timeout. The stream must move off the request-response path.

**D2 · MEDIUM · ONE ANSWER**

**CC21. You must process 10,000 S3 objects in 30 minutes. Which pattern do you use?**

**A) Step Functions Distributed Map (up to 10,000 parallel children)**

**B) 10,000 sequential Lambda invocations**

Sequential invocations take too much time. The time limit is the killer constraint.

**C) One Lambda loops over all objects**

One Lambda stops after 15 minutes. The loop is sequential anyway.

**Why A is correct:** The Distributed Map shape handles a tight time limit over many objects. Find the killer constraint (the clock). Then compare the remaining options.

**D2 · MEDIUM · ONE ANSWER**

**CC22. Marketing changes the prompt tone every week. Engineering cannot redeploy the application. Which service do you use?**

A) AppConfig

AppConfig holds runtime configuration like the model ID. Prompt Management handles versioned prompt templates with variants.

**B) Bedrock Prompt Management (versions, variants, variables)**

C) DynamoDB table of prompt strings

This option has no versioning. It has no variants. It has no test harness. It has no audit trail.

---

**Why B is correct:** "versioned prompt templates reused without redeploy" is the Prompt Management sentence. AppConfig answers "swap models without redeploying."

**D2 · EASY · RECALL**

**CC23. A support bot uses one `sessionId` for all users. What breaks?**

---

**Answer:** Conversation memory leaks between users. The session ID is the memory key. Use one ID per user session.

**D2 · EASY · RECALL**

**CC24. Compliance asks "show why the agent called that tool." Which flag do you use?**

---

**Answer:** Use `enableTrace=true` on `invoke_agent`. It returns the reasoning trace. This trace shows which tools the agent chose and why.

**D2 · MEDIUM · ONE ANSWER**

**CC25. One document-generation request takes 4 minutes. Nobody waits for it. Which call shape do you use?**

A) `ConverseStream`

Streaming sends tokens to a waiting user. Nobody waits here.

B) `start_async_invoke`, result to S3

C) Batch inference

Batch inference uses many prompts from S3. This is one slow prompt.

**Why B is correct:** This is the three-way split. Batch inference uses many prompts. Async inference uses one slow prompt that nobody waits for. Streaming sends tokens to a waiting user.

**D2 · MEDIUM · ONE ANSWER**

**CC26. Switch between Claude, Titan, and Llama without client redeploy. Which trio?**

A) `AppConfig` (model ID at runtime) + `Lambda` (routing) + `API Gateway` (stable URL)

B) `CloudFormation` + `Lambda` + `API Gateway`

A `CloudFormation` change is a deployment. This violates "without redeploy."

C) `EventBridge` + `Lambda` + `API Gateway`

`EventBridge` routes events. It does not hold runtime configuration.

**Why A is correct:** The gateway URL stays fixed. The `Lambda` reads the model ID from `AppConfig` at runtime. `CloudFormation` is always the "redeploy" distractor.

**Next:** D3 guards what D2 built. It uses guardrails at the model door. It uses IAM at every account. It uses the audit trail that proves it.

**D2 · MEDIUM · ONE ANSWER**

**QH05. A team writes a Python agent that calls internal functions. They need minimal boilerplate, full code control, and deployment on their own containers. Which do they pick?**

**B) Strands Agents.**

A) Bedrock Agents console.

Managed console, not code-first on own infrastructure.

C) Hand-rolled tool loops with raw InvokeModel calls.

Works but carries high operational burden.

D) Bedrock Flows.

Flows run fixed steps. They are not code-first agent loops.

**Why B is correct:** Strands is the open-source Python SDK for the agent loop: model reasons, selects a tool, executes, repeats. It deploys anywhere, including AgentCore Runtime.

**The trap:** "Strands replaces Bedrock Agents". They serve different needs.

**D2 · MEDIUM · ONE ANSWER**

**QH06. One entry point must route requests to a billing specialist, a support specialist, and a general assistant. Which framework fits?**

A) One Strands agent with all tools.

Too many tools on one agent hurts tool selection.

**B) Agent Squad with an intent classifier.**

C) A single Bedrock Flow.

Flows run fixed steps. They do not classify intent.

D) Three separate API Gateway endpoints.

No routing intelligence. Clients must pick the specialist.

**Why B is correct:** Agent Squad is built for intent classification and routing. A small fast classifier picks the specialist per request. Strands is the tool-use loop. Squad is the router.

**The trap:** one agent with every tool. The hub rule: too many tools means split into specialists.

**D2 · EASY · ONE ANSWER**

**QH07. A LangGraph agent must run serverless with managed sessions, memory, and identity. Which service hosts it?**

- A) Lambda with a 15-minute timeout.  
No managed sessions, memory, or identity for agents.
- B) Bedrock AgentCore Runtime.**
- C) Bedrock Agents.  
Bedrock Agents hosts its own agents, not any-framework containers.
- D) SageMaker endpoint.  
Model hosting, not agent hosting.

**Why B is correct:** AgentCore Runtime hosts any-framework agent containers serverless. The contract is POST /invocations and GET /ping on port 8080. Gateway, Memory, Identity, and Observability come as platform services.

**The trap:** "AgentCore only hosts Bedrock Agents". It hosts any framework.

**D2 · EASY · ONE ANSWER**

**QH08. An agent must call company REST APIs through a standard tool interface. Which combination is correct?**

- B) Expose the APIs through AgentCore Gateway as MCP tools.**
- A) Hardcode HTTP calls in the agent prompt.  
Prompts cannot make HTTP calls.
- C) Describe the APIs in an OpenAPI schema and hope the model guesses.  
OpenAPI schemas define tools for action groups, but Gateway is the managed MCP path.
- D) Copy the API responses into the system prompt.  
Static text, not live tool access.

**Why B is correct:** AgentCore Gateway converts existing APIs and Lambda functions into MCP-compatible tools. MCP is the open standard. JSON-RPC 2.0 is its data layer.

**The trap:** prompt text as a tool interface. Tools need a real interface: MCP or action groups.

## D3: AI Safety, Security, and Governance (20%)

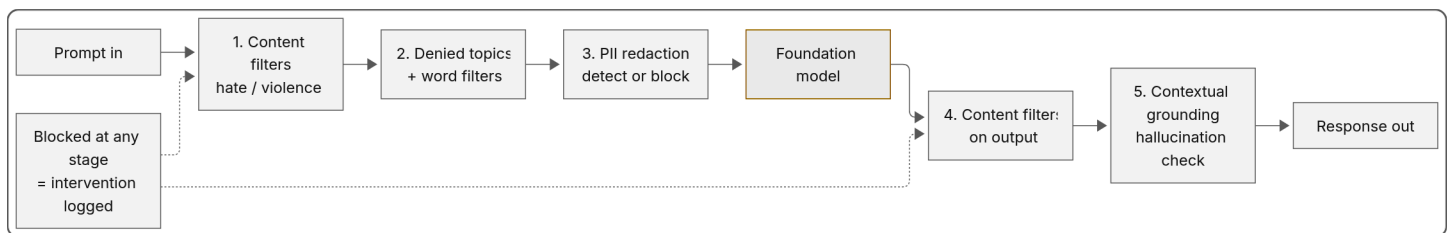
D2 built the application. D3 guards the application. A working application can leak data. It can answer attacks. It cannot prove who did what. This is a failed application.

D3 tests the bouncer, the locks, and the audit trail. Guardrails filter content at the model boundary. IAM and organization controls decide who can touch what. CloudTrail, CloudWatch, and invocation logging record what happened at three different depths. The exam often offers a real service with an invented capability. Learn exactly what each control can and cannot do.

### Guardrails: the bouncer at the model door

A **Bedrock Guardrail** is a named configuration. It holds your safety rules. Every request and every response passes it. There are six policies. You can switch each policy independently. These policies are: **content filters**, **denied topics**, **word filters**, **sensitive information filters** (PII detection and redaction), **contextual grounding checks**, and **Automated Reasoning checks**.

Attach it inline to a Converse call with `guardrailConfig`. This includes the guardrail identifier and version. It also has a trace flag that shows what fired. Or, apply it to any text with the standalone **ApplyGuardrail** API. This is the answer when the scenario says "we call an external model but need AWS guardrails."



Guardrails evaluate inputs and outputs against six independent policies before the response reaches the user.

Source: original figure for the AWS AIP-C01 Study Guide.

Content filters are the toxicity screen. There are six fixed AWS-defined categories. These are hate, insults, sexual, violence, misconduct, and prompt attack. Each category has four sensitivity strengths. This means 24 independent settings per guardrail. **Prompt attack** is the category that catches jailbreaks.

This includes "ignore all previous instructions." It also catches injections smuggled through pasted documents or tool outputs. The guardrail evaluates it on inputs, where attacks arrive. **Denied topics** block named subjects on both inputs and outputs. "Block bullying topics" is a denied topic. "Notify on bullying but never block" is the same filter. It runs in a monitor mode.

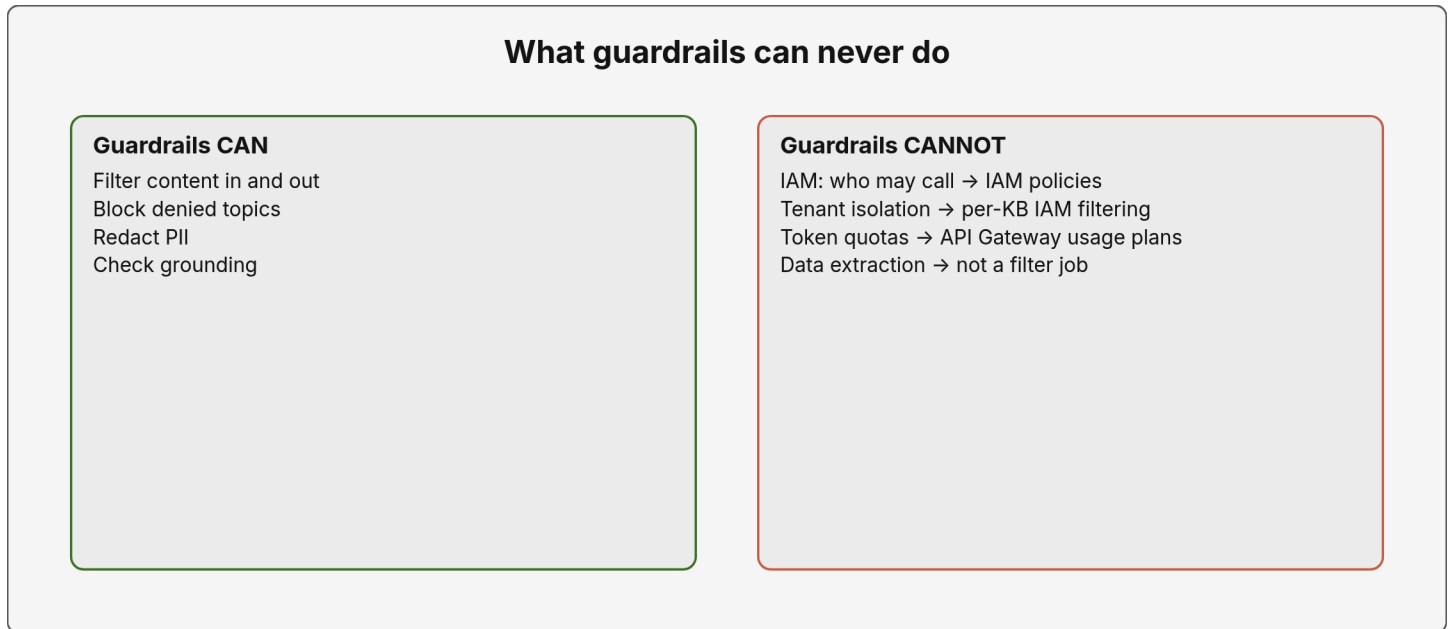
This mode logs intervene events instead of blocking. **Word filters** block exact strings. This includes profanity lists and custom terms. Sensitive information filters detect and redact PII.

The two verifiers catch two different lies. **Contextual grounding checks** ask whether the source text supports the answer. This is the hallucination check for RAG. **Automated Reasoning checks** translate the response and your policy into formal logic. They prove if the response obeys the policy. This is the strongest guarantee in the guardrail set. You build the policy first: upload the policy document as a PDF to the

`CreateAutomatedReasoningPolicy` API. Bedrock extracts the rules and variables from the document.

It is the answer to "most mathematically certain that outputs obey policy." Note its limit: Automated Reasoning checks run in detect mode only. They return findings (valid, invalid, ambiguous, too complex) instead of blocking inline.

## What guardrails can never do



What guardrails can never do: IAM, tenant isolation, quotas, and data extraction each belong to a different service

Source: original figure for the AWS AIP-C01 Study Guide.

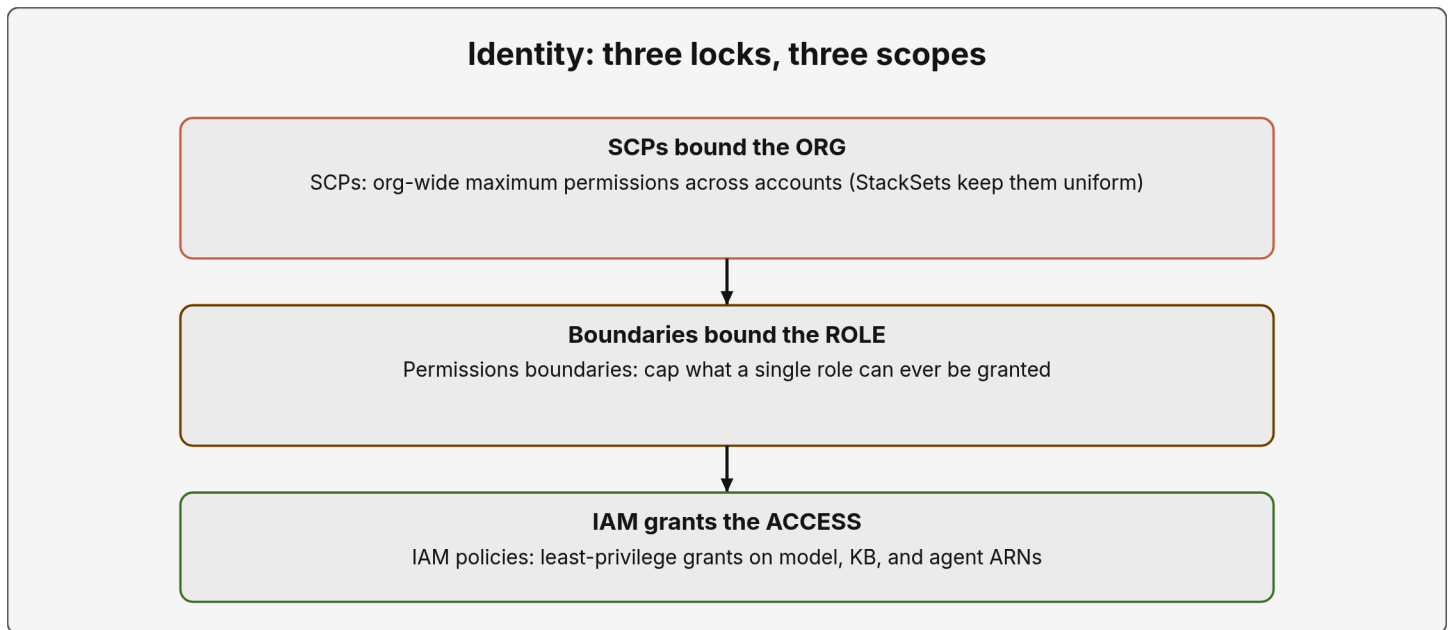
**How to read this.** Left: what guardrails do (filter content). Right: what they cannot do, each paired with the real service that does it. Takeaway: read every guardrail option for scope creep; a real service with an invented capability is a classic distractor.

Guardrails filter content. They cannot do IAM. IAM decides who may call the model. They cannot do tenant isolation. Tenant isolation decides which data a tenant may reach. They cannot do token quotas or billing.

They cannot do data extraction. An option says "use Guardrails to limit each tenant to 1,000 tokens per day." This is a fabricated feature. It is a real service with an invented capability. Tenant isolation is per-knowledge-base IAM filtering. Quotas are API Gateway usage plans. Access control is IAM.

Read every guardrail option for scope creep.

## Identity and access: who can touch the model



Identity in three layers: SCPs bound the org, permissions boundaries bound the role, IAM grants the access

Source: original figure for the AWS AIP-C01 Study Guide.

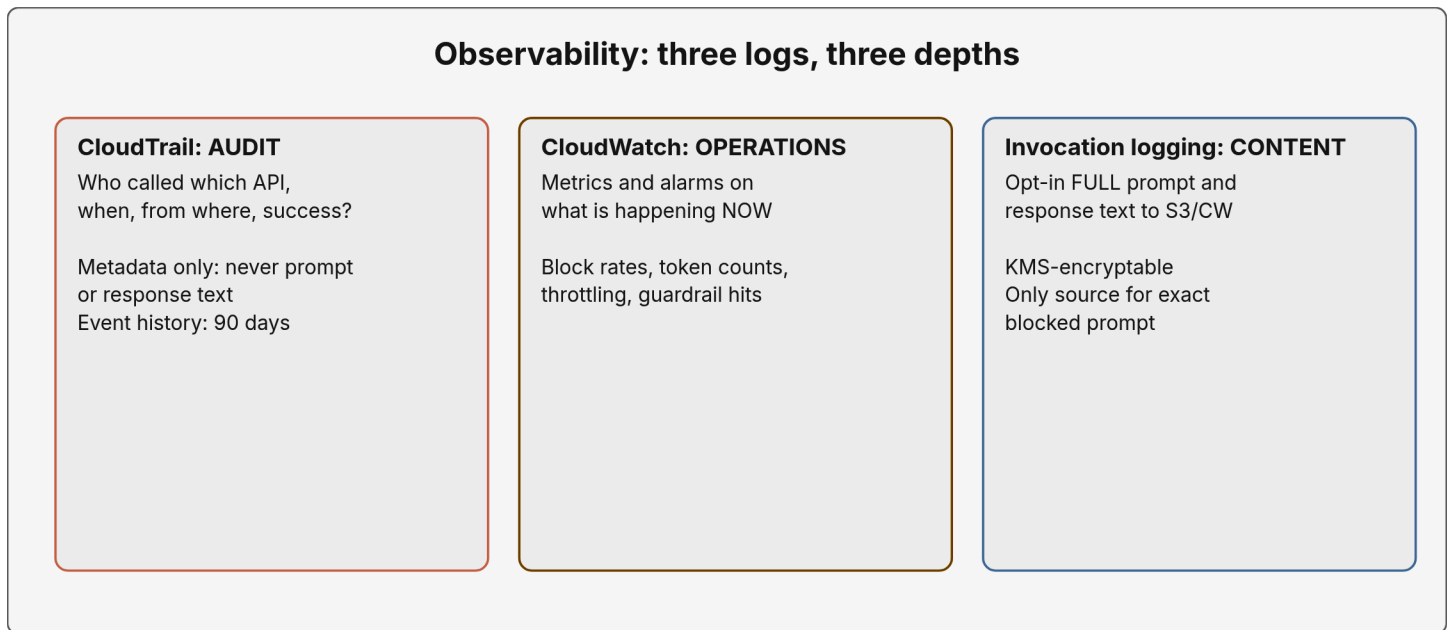
**How to read this.** Top to bottom: the org-wide ceiling first, then the per-role cap, then the actual grants. Takeaway: the exam pairs them exactly this way; mix up the layers and the question is lost.

**IAM** is the first lock. It uses least-privilege policies. These policies scope which identities can invoke which model, knowledge base, and agent ARNs. They also scope which regions. Above IAM are the organization controls. **Service Control Policies (SCPs)** set organization-wide maximum permissions.

These apply across accounts in AWS Organizations. This is the answer to "restrict model access across 40 accounts." Deploy them at scale with StackSets. StackSets are multi-account deployments of CloudFormation templates. This prevents account drift. **Permissions boundaries** cap what a single role can ever receive. This is the per-role answer.

The exam pairs them as: SCPs bound the organization. Boundaries bound the role. IAM grants the actual access inside those bounds. "Built-in alignment is enough" is always wrong. The model provider's safety work never transfers evaluation and governance to you. The deployer owns both.

## Observability: three logs, three depths



Three logs, three depths: CloudTrail for audit, CloudWatch for operations, invocation logging for content

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Three columns. Red: who called what, metadata only, 90 days. Amber: what is happening now, metrics and alarms. Blue: the full prompt and response text, opt-in. Takeaway: match the question's depth to the column; only one column holds content.

**CloudTrail** is the audit plane. It records who called which Bedrock API. It records when, from where, and if it succeeded. It records API metadata only. It never records prompt or response content. Event history keeps 90 days.

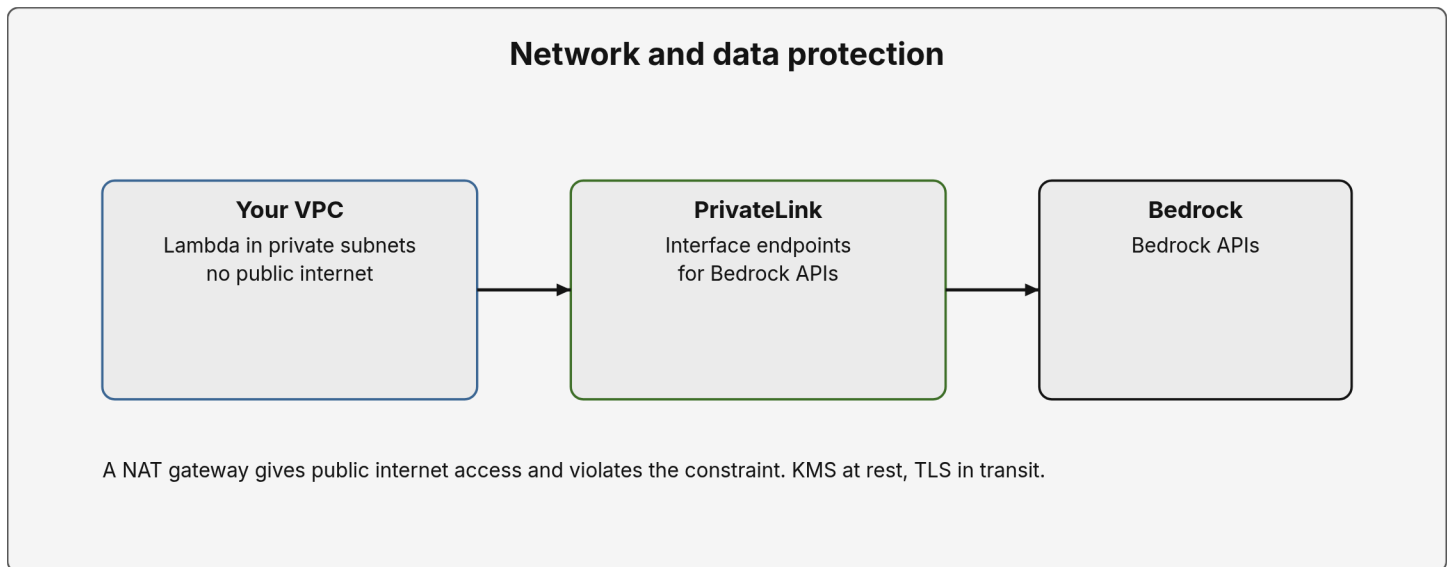
Longer retention means delivering the trail to S3. "Who invoked which model at 2am" is always CloudTrail.

**CloudWatch** is the operations plane. It has metrics and alarms on what happens now. It shows which prompts the guardrail blocked and why. It shows guardrail block rates, token counts, and throttling.

Use CloudWatch metrics and custom alarms. "Proactively alert" kills every reactive option. Count tokens before the call with client-side estimation. The InputTokenCount metric fires after the call runs. **Model invocation logging** is the content plane. It is the opt-in record of full prompt and response text.

It goes to S3 or CloudWatch. It is KMS-encryptable. It is the only source for "show us the exact blocked prompt." In regulated scenarios, you must justify keeping it at all. Use retention policies and redaction. Logs can contain PII.

## Network and data protection



Bedrock with no public internet: your VPC reaches Bedrock APIs through a PrivateLink interface endpoint

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Left to right: your VPC, the PrivateLink endpoint, Bedrock. The note below kills the NAT gateway option. Takeaway: no public internet always means VPC interface endpoints.

"Bedrock with no public internet" is always **VPC interface endpoints** (PrivateLink) for the Bedrock APIs. Use Lambda in private subnets. A NAT gateway gives public internet access. This violates the constraint. At rest is **KMS**. Use customer-managed keys, envelope encryption.

Log every key use. In transit is **TLS**. Customer data sent to Bedrock is not used to train models under the service terms. Any data you log stays in your account. For human oversight of risky model outputs, **A2I** (Augmented AI) puts flagged interactions in front of human reviewers. This is for critical flagged cases only.

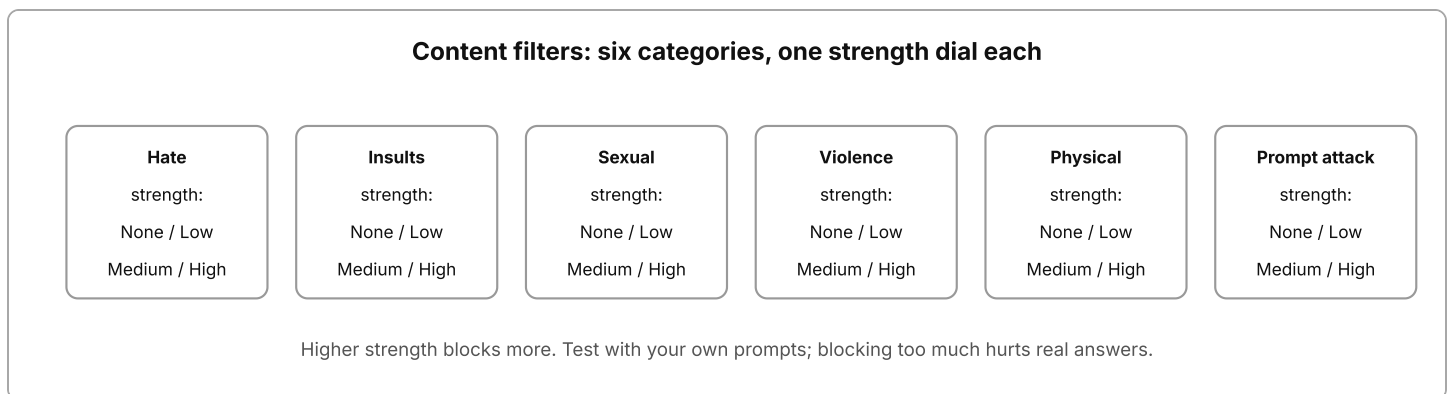
It is never for all traffic.

## The six guardrail policies in detail

One guardrail holds six independent policies. Each answers a different failure. Memorize the mapping: failure on the left, policy on the right.

| Policy                        | Failure it answers           | How it works                                                                                              |
|-------------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------|
| Content filters               | Harmful content              | Screens six toxicity categories. Strength is a fixed dial: NONE, LOW, MEDIUM, HIGH. It is not prose.      |
| Denied topics                 | Forbidden subjects           | Blocks a subject in any wording. A user cannot rephrase around it.                                        |
| Word filters                  | Exact banned words           | Blocks exact words or phrases. Rephrasing dodges it. This is the key difference from denied topics.       |
| Sensitive information filters | PII leaks                    | Detects and redacts PII at the model boundary. It covers both inputs and outputs.                         |
| Contextual grounding checks   | Hallucination                | Asks one question: does the source text support the answer? It scores grounding against provided sources. |
| Automated Reasoning checks    | Policy violations in outputs | Checks outputs against formal policies. It is the strictest verifier in the set.                          |

Prompt attack detection is a built-in guardrail category. It finds known jailbreak and injection patterns in inputs. It is pattern-based and probabilistic. It makes injection less likely. It does not stop all injection. It is one layer of many.



Content filters screen six toxicity categories. Each category gets its own strength dial: NONE, LOW, MEDIUM, or HIGH.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Each row is one toxicity category.
2. The dial sets the strength per category. The levels are fixed words, not prose.
3. Raise the dial on the categories the use case forbids.

### Denied topics vs word filters: the paraphrase test

#### DENIED TOPIC

whole subject blocked  
e.g. "weapons", "finance"  
any wording of it fails

#### WORD FILTER

exact words blocked  
list: "word1", "word2"  
only those spellings fail

Exam tip: a paraphrase of a denied topic still fails; a paraphrase of a filtered word passes.

Denied topics block a subject in any wording. Word filters block exact words only. The exam tests this line.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left: denied topics. Rephrasing does not escape them.
2. Right: word filters. Rephrasing escapes them.
3. The question names the evasion. Pick the matching policy.

**Never confuse:** denied topics with word filters. Topics block meaning. Words block strings. Contextual grounding with prompt attack detection. Grounding checks answers against sources. Attack detection checks inputs for jailbreaks.

**Exam relevance.** D3 is 20% of the exam. Policy-mapping questions are the core. Name the failure. Pick the policy.

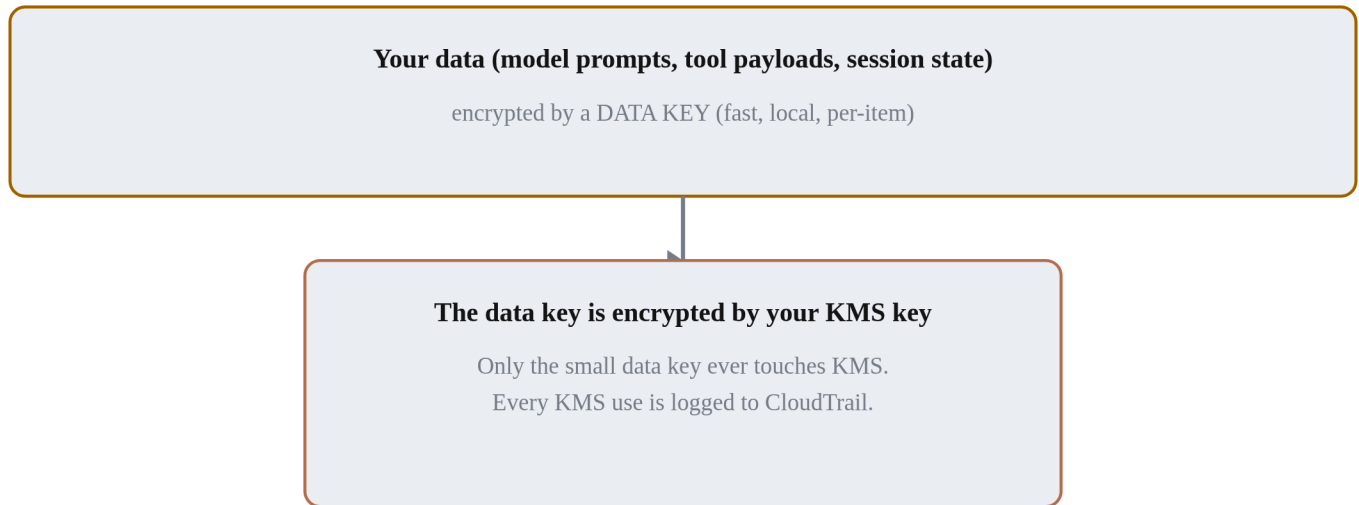
### Data protection: the full stack

Four layers protect data in a Bedrock system. The exam asks which layer owns each job.

| Layer                 | Job                      | Exam fact                                                                                                                |
|-----------------------|--------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Encryption at rest    | KMS encrypts stored data | Envelope encryption: a data key encrypts the data. The KMS key encrypts the data key. Every KMS use is logged.           |
| Encryption in transit | TLS on every hop         | Data moves encrypted between your VPC and Bedrock.                                                                       |
| Network isolation     | No public internet path  | VPC interface endpoints (PrivateLink) reach Bedrock APIs. A NAT gateway does not meet a "no public internet" constraint. |
| PII pipeline          | Find and redact PII      | Macie scans S3 at rest. Comprehend detects PII in live text. Guardrails filter PII at the model boundary.                |

## Envelope encryption: two keys, two jobs

The data key encrypts data. The KMS key encrypts the data key.



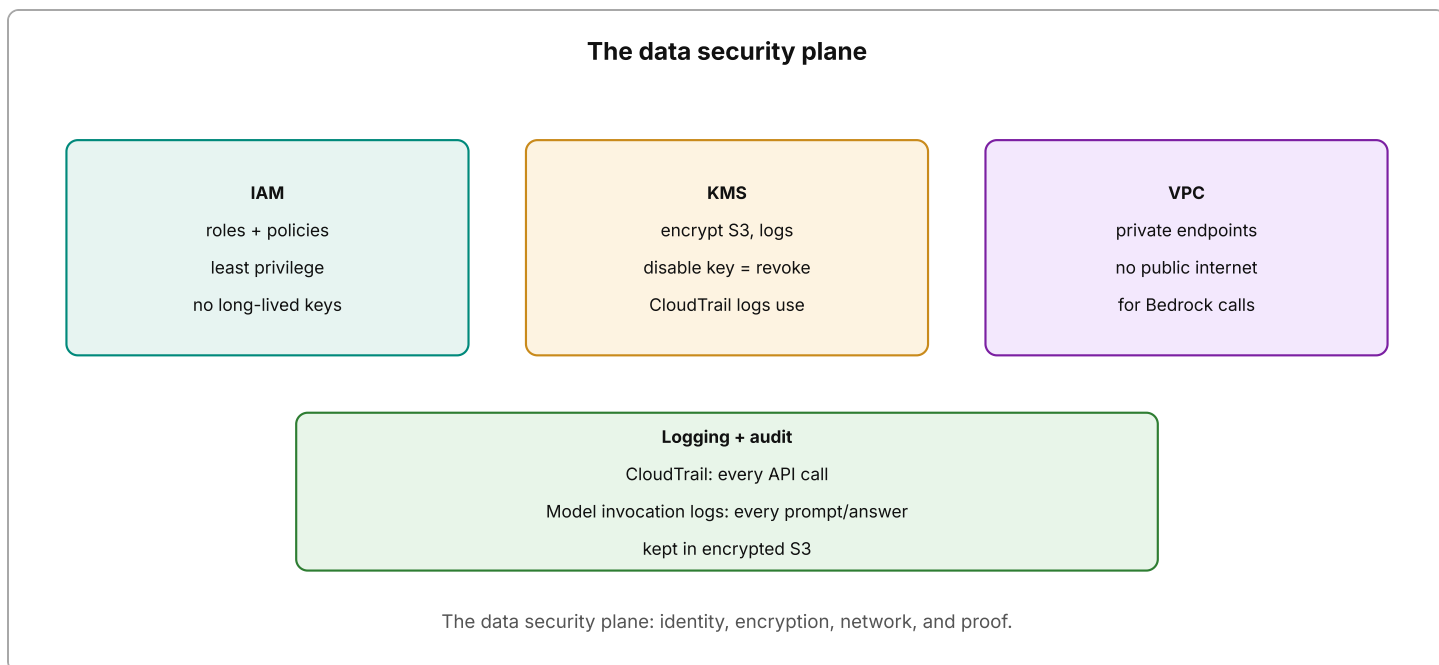
Why two keys: KMS calls are slow and audited. Data keys keep bulk encryption fast and local.

Envelope encryption in three steps. The data key does the bulk work. The KMS key protects the data key. CloudTrail logs every KMS use.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left: the data key encrypts the data locally. It is fast.
2. Middle: the KMS key encrypts the data key. The plaintext key never rests.
3. Right: the log. Every KMS call is audited.



IAM, KMS, VPC, and logging form one plane. Identity decides who. Encryption decides how data rests. The network decides the path. Logs prove it.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Top: IAM. It decides who can act.
2. Middle: KMS and the VPC. They decide how data rests and moves.
3. Bottom: logging. It proves what happened.

Bedrock never trains models on your data. Your prompts and responses stay in your account and region. This is the privacy answer for "does AWS see our data" questions.

**Exam relevance.** "No public internet" always means VPC interface endpoints. "PII in S3" always means Macie. "PII at inference" always means Guardrails. Match the place to the service.

### Bedrock agent tracing: four trace types

When a Bedrock Agent answers, it records a trace. The trace shows the reasoning process, which knowledge bases it hit, and where errors happened. It is the fastest way to debug a misbehaving agent.

**Trace types.** Preprocessing traces cover input handling before the agent acts. Orchestration traces cover the agent loop: model reasoning, tool selection, tool results. Postprocessing traces cover output handling after the loop. The routing classifier trace shows which specialist the request went to.

**Exam relevance.** "Debug why the agent called the wrong tool" means read the orchestration trace. "Debug why the answer format broke" means read the postprocessing trace.

### Token-level redaction: filter the text, not the weights

Tokens here mean input and output text, not model internals. When guardrails are too heavy or still not enough, filter the sensitive tokens directly.

**Three filter points.** Filter training data at ingest. Filter the request in a pre-processing handler before it reaches the FM. Filter the response in a post-processing handler as the last defense before output. Define the handlers as Lambda functions. Use simple logic like regex or banned-word lists, or call Amazon Comprehend for named entity recognition on what to ban.

**Exam relevance.** "Block PII in model output as a last resort" means a post-processing Lambda with token-level redaction. "Guardrails cannot cover this edge case" is the trigger phrase.

### CloudTrail watches calls, CloudWatch watches prompts

CloudTrail records every Bedrock API call. It is mandatory when compliance requires an audit trail. But CloudTrail does not log the actual prompts. Model invocation logging in CloudWatch Logs records prompts and answers. Use CloudWatch Logs to monitor prompt inputs and model outputs, to test prompt regressions, and to track quality, latency, and error rates.

**CloudWatch RUM.** Real User Monitoring measures real sessions: page load times, errors, app launch times. It suits mobile GenAI apps. It integrates with Application Signals, visible in X-Ray.

**Trap: "use CloudTrail to find the bad prompt."** Wrong. CloudTrail shows that InvokeModel ran. It never shows the prompt text. The exam uses this confusion as a distractor.

**Exam relevance.** "Audit which API calls ran" means CloudTrail. "See what the user actually typed" means CloudWatch Logs invocation logging.

### More D3 decision rules

- If the stem says "block a subject in any wording," pick denied topics. Never pick word filters.
- If the stem says "block exact words," pick word filters. Rephrasing dodges them.
- If the stem says "does the source support the answer," pick contextual grounding checks.
- If the stem says "jailbreak or injection attempt," pick prompt attack detection. Set it to HIGH on input.
- If the stem says "notify but never block," pick guardrail monitor mode. Block mode violates the constraint.
- If the stem says "full prompt and response text for audit," pick model invocation logging. CloudTrail holds metadata only.
- If the stem says "who called which API," pick CloudTrail. It keeps 90 days of API history.
- If the stem says "PII across S3 buckets," pick Macie. Never pick Guardrails for data at rest.
- If the stem says "no public internet," pick VPC interface endpoints. A NAT gateway still uses public internet.
- If the stem says "limit what a role can ever receive," pick a permissions boundary. If the stem says "limit the whole org," pick an SCP.
- If the stem says "answers must stay in one region," never pick a cross-region inference profile. Profiles move data across regions.
- If the stem says "redact PII before it reaches the model," pick Guardrails sensitive information filters on input.

### More D3 traps, with why the wrong answer fails

- Guardrails for IAM. Guardrails filter content. They do not grant access. IAM grants access. Wrong plane.
- Guardrails for quotas. Guardrails do not throttle. Quotas and throttling live in the API layer. Wrong plane.

- Guardrails for tenant isolation. Guardrails do not isolate tenants. Isolation is an architecture choice. Wrong plane.
- CloudTrail for prompt text. CloudTrail records API metadata: who, when, which action. It never stores prompt text. Use invocation logging.
- CloudWatch for audit history. CloudWatch holds metrics and alarms for operations. Audit history is CloudTrail.
- NAT gateway for "no public internet." A NAT gateway still routes through public internet. The answer is VPC interface endpoints.
- Word filters for a rephrasing attack. Exact words miss rephrased attacks. Denied topics block the subject in any wording.
- Prompt attack detection as a full fix. Detection is pattern-based and probabilistic. It is one layer. Add human approval and guardrails for high-stakes actions.
- Cross-region profile under data residency. Profiles route across regions by design. Data residency forbids this. Never mix them.
- Block mode when the stem says "notify only." Block mode blocks. Monitor mode notifies. Read the constraint word.

### D3 self-test: more questions with answers

D3 · MEDIUM · ONE ANSWER CC62. A chatbot must never discuss a competitor's products, no matter how the user phrases the question. Which policy do you use? A) Word filters with the competitor name. Rephrasing dodges exact words. B) Denied topics for the competitor subject. C) Content filters on HIGH. Content filters screen toxicity. They do not block a subject. D) Prompt attack detection. This finds jailbreaks. It does not block a subject. Reveal answer Why B is correct: "No matter how the user phrases it" is the denied-topics sentence. Denied topics block a subject in any wording. Word filters block exact strings only.

D3 · MEDIUM · ONE ANSWER CC63. A RAG answer says refunds take 3 to 5 days. The policy document says 5 to 7 days. Which guardrail policy catches this? A) Content filters. The content is not toxic. B) Contextual grounding checks. C) Denied topics. Refunds are not a denied subject. D) Sensitive information filters. No PII is involved. Reveal answer Why B is correct: Contextual grounding asks one question: does the source text support the answer? The policy says 5 to 7 days. The answer says 3 to 5. The source does not support the answer.

D3 · EASY · ONE ANSWER CC64. Compliance needs every prompt and response stored for audit. Which log do you enable? A) CloudTrail. CloudTrail records API metadata only. It never stores prompt text. B) CloudWatch Logs alone. CloudWatch holds operational logs. It does not capture full model text by default. C) Model invocation logging to S3. D) VPC Flow Logs. These record network traffic. They do not record prompt text. Reveal answer Why C is correct: Model invocation logging is the opt-in record of full prompt and response text. Route it to S3 or CloudWatch. Encrypt it with KMS. Only this log holds content.

D3 · MEDIUM · ONE ANSWER CC65. An app runs in a VPC with no public internet. It must call Bedrock. What do you build? A) A NAT gateway. A NAT gateway still uses public internet. The constraint forbids it. B) A VPC interface endpoint (PrivateLink) for Bedrock. C) An internet gateway with tight security groups. This is public internet. The constraint forbids it. D) A proxy EC2 instance. This adds a self-managed hop and still needs a path out. Reveal answer Why B is correct: "No public internet" always means VPC interface endpoints. PrivateLink reaches Bedrock APIs without leaving the AWS network.

D3 · MEDIUM · ONE ANSWER CC66. Developers need broad Bedrock access, but no developer role may ever invoke a restricted model. Which IAM control caps the roles? A) An SCP on the org. An SCP bounds the org, not the role. The need is per-role. B) A permissions boundary on each developer role. C) An IAM policy grant. Grants give access. They do not cap it. D) A guardrail. Guardrails filter content. They do not touch IAM. Reveal answer Why B is correct: Three layers: SCPs bound the org, permissions boundaries bound the role, IAM grants the access. "No role may ever" is the permissions-boundary sentence.

### D3 decision rules: if the stem says X, pick Y

- If the content is bad, notify. Do not block it. Use a guardrail in monitor mode with intervene metrics. Block mode does not meet the constraint.
- Block specific topics. Block them in inputs and outputs. Use a denied topics policy on inputs and outputs.
- Jailbreaks and prompt injections are content filter category prompt attacks. Evaluate them on inputs.
- The most mathematically certain outputs obey policy. Automated Reasoning checks this. It uses detect mode and formal logic.
- Is the source supporting the answer? Use a contextual grounding check. Does the answer obey the policy? Use Automated Reasoning. There are two verifiers. There are two lies.
- You use an external model. You need AWS guardrails. Use ApplyGuardrail standalone. guardrailConfig cannot reach outside Bedrock.
- Who called which model and when? Use CloudTrail. Which prompts did the system block and why? Use CloudWatch. For full prompt and response text, use model invocation logging. This is an opt-in feature.
- Alert before limits. Count tokens before the call. Metrics fire after the call.
- Restrict models across the whole organization. Use SCPs. Use StackSets for deployment. Cap one role. Use permissions boundaries.
- Use Bedrock with no public internet. Use interface VPC endpoints. A NAT gateway does not meet the constraint.
- Critical flagged interactions need human review. Use A2I. Not all traffic needs review.
- Customer data must not train models. Follow the service terms and opt-out. Invocation log data stays in your account.

### D3 trap patterns

- Guardrails are not for IAM, tenant isolation, token quotas, or billing. This is a fabricated capability.
- CloudTrail is not for prompt content. It provides metadata only. Content needs invocation logging.
- CloudWatch is not for "who called what." CloudTrail does this job.
- Built-in safety does not mean you skip evaluation. The deployer owns evaluation and governance. Always evaluate.
- Do not attach a guardrail to a CloudWatch log group to mask PHI. Guardrails attach to Bedrock model flows. They do not attach to log groups.
- A NAT gateway is not for "no public internet." It provides public internet access.
- Check the region first for cross-region data residency issues.

- You have full invocation logging in a healthcare scenario. There is no retention policy. Logs can contain PII. Question the logging itself.
- Automated Reasoning does not block inline. It runs in detect mode only.

### D3 rapid-fire questions

#### ***D3 · EASY · ONE ANSWER***

**CC27. You must block bullying topics in prompts and answers. You must notify on bullying but never block. What changes?**

A) The filter type is different.

The denied-topics filter is the same. The constraint is about the mode. It is not about the feature.

**B) Guardrail mode: block or monitor/detect with intervene metrics**

**Why B is correct:** "notify but do not block" is the monitor-mode sentence. Block mode does not meet the stated constraint.

#### ***D3 · MEDIUM · ONE ANSWER***

**CC28. Compliance needs the full prompt and response text for blocked calls. Which log stores this?**

A) CloudTrail

CloudTrail records API metadata only. It records who, when, and which action. It never records token content.

**B) Model invocation logging (opt-in) to S3**

C) CloudWatch metrics

Metrics collect counts. They do not store text.

**Why B is correct:** There are three depths. CloudTrail shows who, what, and when. CloudWatch shows what happens now. Invocation logging shows the content. Only B stores the actual text.

**D3 · MEDIUM · ONE ANSWER**

**CC29. Restrict foundation model access across 40 AWS accounts. Prevent drift. Which control do you use?**

**A) SCPs via Organizations. Deploy them with StackSets.**

B) Edit per-account IAM roles by hand.

40 hand-edited roles have 40 chances to drift. Org-wide maximum permissions need SCPs.

C) Guardrails

Guardrails filter content. They are not access control.

**Why A is correct:** SCPs set the org-wide maximum. StackSets deploy them at scale. Permissions boundaries cap a single role. They do not cap the organization.

**D3 · EASY · ONE ANSWER**

**CC30. Bedrock must run in a VPC. It must have no public internet. Which networking do you use?**

A) NAT gateway

A NAT gateway provides public internet access. This does not meet the explicit constraint.

**B) VPC interface endpoints (PrivateLink) for the Bedrock APIs**

**Why B is correct:** Interface endpoints keep traffic on the AWS network. They have no internet path. "No public internet" always means endpoints. It never means NAT.

**D3 · MEDIUM · ONE ANSWER**

**CC31. The team calls an external model. This model is not a Bedrock model. Compliance needs AWS guardrails on its outputs. How do you do this?**

A) `guardrailConfig` on Converse

Converse calls Bedrock models only. The config cannot reach an external model.

**B) Use ApplyGuardrail as a standalone API call on the text**

**Why B is correct:** ApplyGuardrail applies guardrail policies to any text. This includes non-Bedrock output. This is the exam's favorite guardrail scope trap.

**D3 · MEDIUM · ONE ANSWER**

**CC32. A financial advice bot must have outputs that obey policy. This must be mathematically certain. Which check do you use?**

A) Content filters

Classifier-based filters are probabilistic. "Mathematically certain" means the formal-logic check.

B) Automated Reasoning checks

**Why B is correct:** Formal logic verification is the strongest guarantee in the guardrail set. Note its limit: detect mode only. It returns findings. It does not block inline.

**D3 · MEDIUM · ONE ANSWER**

**CC33. Protect PII in customer-service GenAI conversations. Use minimal custom work. Select the set.**

A) Macie + Comprehend + Guardrails

B) Macie + Translate + Rekognition

Translate keeps meaning across languages. Rekognition is vision. Neither detects PII in text.

C) A hand-rolled regex Lambda

This is custom work. Three managed services cover the pipeline. "Minimal custom work" makes hand-rolled wrong.

**Why A is correct:** This is the pipeline order. Macie finds PII in the lake before ingestion. Comprehend detects PII in live text. Guardrails filter PII at the model boundary.

**D3 · EASY · RECALL**

**CC34. Name the six guardrail policies.**

**Answer:** content filters, denied topics, word filters, sensitive information filters, contextual grounding checks, Automated Reasoning checks. Content filters alone have 24 settings. This is 6 categories times 4 strengths.

**D3 · EASY · RECALL**

**CC35. CloudTrail event history keeps how many days? What does CloudTrail never record?**

**Answer:** 90 days. Deliver the trail to S3 for longer storage. It never records prompt or response content. It only records API metadata.

**D3 · MEDIUM · ONE ANSWER**

**CC36.** An option says "use Guardrails to cap each tenant at 1,000 tokens per day." What is the verdict?

- A) Reject: fabricated feature. Guardrails cannot do token quotas or billing.**
- B) Accept: guardrails control usage  
Guardrails filter content. Quotas are API Gateway usage plans. Isolation is IAM. Read every guardrail option for scope creep.

**Why A is correct:** This is the scope trap. It is a real service plus an invented capability. Guardrails never do IAM, tenant isolation, token quotas, or billing.

**D3 · MEDIUM · ONE ANSWER**

**CC37.** "Proactively alert before token limits are hit." Alarm on the InputTokenCount metric or count tokens before the call?

- A) Alarm on InputTokenCount  
The metric fires after the call runs. "Proactively" makes every reactive option wrong.
- B) Count tokens client-side before the call. Alert on the estimate.**

**Why B is correct:** Proactive means before the call. Token math happens client-side, ahead of time. This includes input plus output tokens times price.

**D3 · EASY · RECALL**

**CC38.** At rest is KMS. In transit is TLS. A healthcare scenario enables full invocation logging. There is no retention policy. What is wrong?

**Answer:** Logs can contain PII/PHI. The design needs retention policies and redaction. Or, it needs a justification to disable logging. The exam expects you to question the logging itself in regulated scenarios.

**Next:** D4 makes the guarded application cheap and fast. It uses token math, caching, batch, Provisioned Throughput, and timeout math. This kills synchronous designs.

**D3 · EASY · ONE ANSWER**

**QH09. Compliance requires an audit trail of every Bedrock API call. A separate requirement asks what the user actually typed. Which logging pair is correct?**

- A) CloudTrail for both.  
CloudTrail logs API calls, never prompt text.
- B) CloudTrail for API calls, CloudWatch Logs invocation logging for prompts.**
- C) CloudWatch Logs for both.  
Logs show prompts, not the API audit trail.
- D) Guardrails logs for both.  
Guardrails log policy decisions, not full audit or prompt history.

**Why B is correct:** CloudTrail records every API call for compliance. Model invocation logging in CloudWatch Logs records prompts and answers. Each answers a different question.

**The trap:** "use CloudTrail to find the bad prompt". CloudTrail never holds prompt text.

**D3 · MEDIUM · ONE ANSWER**

**QH10. An agent calls the wrong tool. Which trace type shows the reasoning and tool selection?**

- A) Preprocessing trace.  
Covers input handling, not the agent loop.
- B) Orchestration trace.**
- C) Postprocessing trace.  
Covers output handling after the loop.
- D) Guardrail trace.  
Shows policy decisions, not agent reasoning.

**Why B is correct:** The orchestration trace records the agent loop: reasoning, tool selection, tool results. It is the first place to look when the agent misbehaves.

**The trap:** any trace type. Each type owns a different phase.

## **D4: Operational Efficiency and Optimization for GenAI Applications (12%)**

D3 made the application safe. D4 makes the application cheap and fast. Safety is not important if the cost or the latency kills the project.

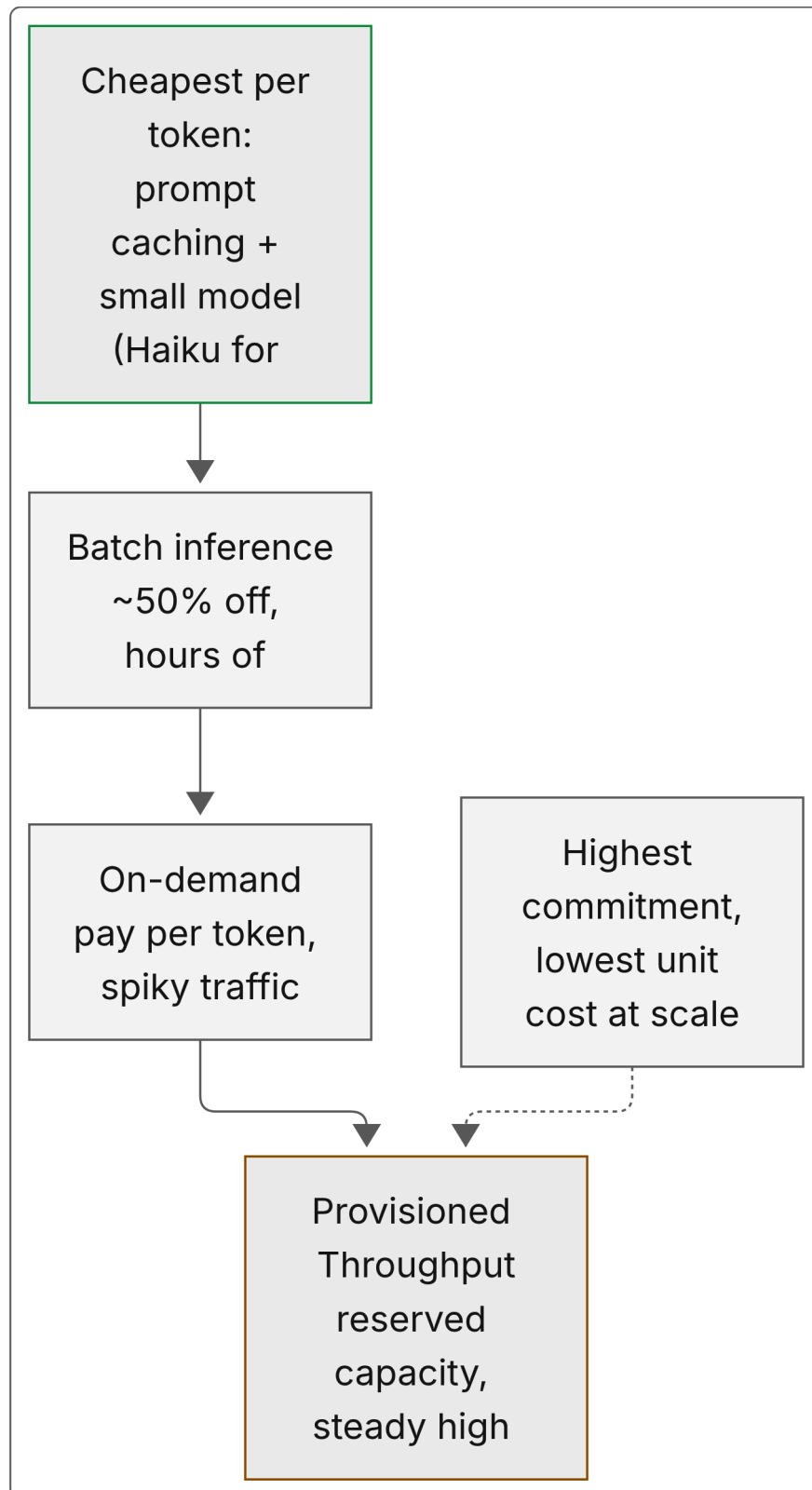
D4 is the smallest scored domain. It has the most decision rules. Each question is a cost, latency, or capacity scenario. The adjective in the question names the correct answer before you read the options. Learn the three adjectives and the three-way routing confusion set. Then D4 gives you free points.

### The three adjectives, three different winners

"**Least operational overhead**" means the fewest self-managed components. Use Aurora Serverless instead of a self-managed OpenSearch cluster. Use Converse instead of hand-rolled provider bodies. Use managed routers instead of your Lambda code. "**Least custom development effort**" means managed features instead of your own code. Use Knowledge Bases instead of a hand-built ingestion pipeline.

Use prompt routers instead of your classifier Lambda. Use Guardrails instead of hand-rolled filters. "**Most cost-effective**" means token math. Use the smallest model that meets the quality standard. Use caching for repeated prefixes. Use batch for offline bulk.

Use cascading simple queries to cheap models. The exam changes these adjectives for the same scenarios. The scenario does not change. The correct answer changes.



The cost ladder per token: cache and right-size first, batch the offline work, reserve capacity only for steady baselines.

Source: original figure for the AWS AIP-C01 Study Guide.

## The levers in cost order

### The levers in cost order: cheapest first

1. Right-size + cascade

2. Prompt caching

3. Batch inference

4. Provisioned Throughput

TOKEN MATH first: cost = (input + output tokens) x price. Proactive means math before the call, not after.

The cost levers in order: right-size and cascade first, then prompt caching, then batch, Provisioned Throughput last

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Bars shrink as cost per call drops. Read top to bottom for the order to try. Takeaway: PT is last in cost order and first in capacity order; token math comes before any of them.

**Right-sizing and cascading** are first. They reduce every token. Select the smallest model that meets the quality standard on your golden dataset. Then automate it. A router or cascade sends simple requests to a cheap, lightweight model. These requests are for classification, extraction, and short answers.

It sends only complex requests to the flagship model. **Prompt caching** is next for repeated long prefixes. Mark the shared block with `cachePoint`. Pay the reduced input-token rate on hits. Prove this with the cache read hit rate.

**Batch inference** handles offline bulk.

Put JSONL into S3. Put results to S3. This has hours of latency. It is about 50 percent off on-demand. **Provisioned Throughput** is last in cost order. It is first in capacity order.

It uses model units on 1 or 6 month terms. It has hourly billing, whether you use it or not. It routes invocations to the provisioned ARN. It guarantees latency. It stops throttling on steady baselines. On spiky traffic, it bills you for sleeping hours.

So spiky traffic stays on-demand.

```
TOKEN MATH (do this before the call, not after)
cost = (input_tokens + output_tokens) x price_per_token
cache hit: input_tokens x cached_rate    (reduced rate on the repeated prefix)
batch:    cost x ~0.5                    (offline only, hours of latency)
PT:       model_units x hourly_rate x hours in term  (used or not)
Estimate per request, multiply by volume, compare options.
"Proactive" cost control means this math happens client-side.
```

– **Worked example: token math on one million requests**

A chatbot handles 1,000,000 requests each month. Each request has 2,000 input tokens and 500 output tokens on average. Use these illustrative prices: \$3 per million input tokens, \$15 per million output tokens.

On-demand cost per request: input part =  $2,000 / 1,000,000 \times \$3 = \$0.006$ . Output part =  $500 / 1,000,000 \times \$15 = \$0.0075$ . Total per request =  $\$0.0135$ . Monthly =  $\$0.0135 \times 1,000,000 = \$13,500$ .

Batch inference is about 50% off:  $\$13,500 \times 0.5 = \$6,750$  each month. This is only if no one waits for the answers. That "roughly half" is the whole batch argument.

Prompt caching: 1,800 of the 2,000 input tokens are a repeated system prompt. Cache hits bill at one-tenth the rate (\$0.30 per million). The input part drops to  $(200 / 1,000,000 \times \$3) + (1,800 / 1,000,000 \times \$0.30) = \$0.0006 + \$0.00054 = \$0.00114$  per request. The answers are the same. The input cost is about one-sixth. The precondition is real.

The prefix must repeat.

**Streaming** is the latency lever. It is not a cost lever. `ConverseStream` reduces perceived latency to the first token. This is often less than one second. Total generation time does not change. "Users report slow responses" on interactive traffic is always streaming.

PT does not speed the first token. Caching needs repeated prefixes. For throttling, the fix ladder is: retry with backoff for transient throttles. Use PT for sustained throttles on steady traffic. Use routers to shed load to cheaper models. Use profiles to spread across regions.

"Bought PT but still throttles" means the code passes the base model ID. It does not pass the provisioned ARN.

The confusion set one more time. The exam tests it directly. **Inference profiles** route by region for availability. **Prompt routers** route by complexity for cost. **Provisioned Throughput** reserves capacity by the hour. Regional failover needs a profile.

Simple queries on an expensive flagship need a router. Throttled at peak on steady traffic needs PT. If you mix them up, the distractor wins.

**Token math: worked, not waved at**

Every cost question is token math. Input tokens plus output tokens, times the price. The exam gives example prices. You do the arithmetic.

### Token economics: the one equation

#### One chat call: 2,000 input tokens + 500 output tokens

price: \$3.00 per 1M input tokens, \$15.00 per 1M output tokens (example)

input cost =  $2,000 / 1,000,000 \times \$3.00 = \$0.0060$

output cost =  $500 / 1,000,000 \times \$15.00 = \$0.0075$

**total per call =  $\$0.0135 \times 1\text{M calls} = \$13,500 / \text{month}$**

Two levers only: fewer tokens per call, or fewer calls. Everything in this chapter pulls one of them.

One call with 2,000 input tokens and 500 output tokens costs \$0.0135 at example prices. Input tokens and output tokens price separately.

Source: original figure for the AWS AIP-C01 Study Guide.

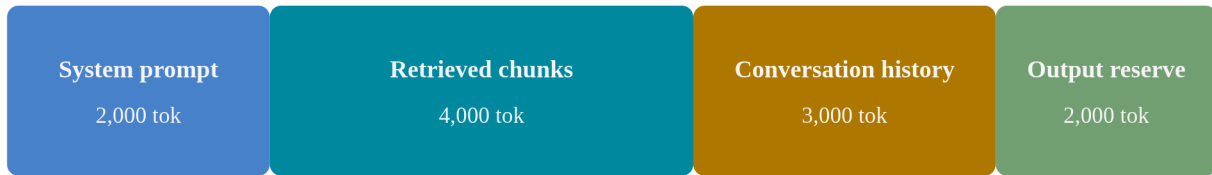
#### How to read this diagram

1. Top: count the input tokens. Multiply by the input price.
2. Middle: count the output tokens. Multiply by the output price.
3. Bottom: add the two. That is the call cost.

Token math decides model choice. A flagship model at 10x the price must earn its place. If a small model scores the same on the task, the small model wins. This is "most cost-effective" in one line.

## The token budget: every input must fit the window

System plus retrieved chunks plus history plus reserved output.



**Total in + reserved out must fit the context window.**

200K window, 11K used here: room to spare. Small window with long docs: chunk, summarize, or upsize.

**The classic design bug: retrieved chunks grow until the prompt no longer fits.**

Budget the window like money: system + chunks + history + output.

Four tenants share one context window. System prompt, retrieved chunks, history, and output reserve must all fit. If they do not fit, something must shrink.

Source: original figure for the AWS AIP-C01 Study Guide.

How to read this diagram

1. Four blocks: system prompt, retrieved chunks, conversation history, output reserve.
2. The window is fixed. The blocks compete for it.
3. Overflow means cut history, cut chunks, or pick a bigger window.

**Exam relevance.** "Most cost-effective" means do the token math. Compare input plus output cost across the options. Never pick the flagship by default.

## Caching: three kinds, three preconditions

Three caches cut the bill. Each has one precondition. The exam tests the precondition.

| Cache            | Precondition                           | What it saves                                     | Proof it works                                                             |
|------------------|----------------------------------------|---------------------------------------------------|----------------------------------------------------------------------------|
| Prompt caching   | A long prefix repeats across calls     | Repeated input tokens bill at a reduced rate      | Cache read hit rate. Mark the prefix with cachePoint.                      |
| Semantic caching | Users ask similar questions many times | Skips the model call completely                   | Cache hit rate on similar queries.                                         |
| Edge caching     | Deterministic assets serve many users  | Skips the origin and the model for static content | Cache hit ratio at the edge. Set an aggressive TTL if the content changes. |

**PROMPT CACHING: PAY ONCE FOR THE STATIC PREFIX****Cache HIT**

same static prefix + cachePoint  
skip recompute: lower cost, lower latency

**Cache MISS**

prefix changed  
full prompt processed again

**How it works:**

1. Put the long unchanging text first: system prompt, few-shot examples, document dumps.
2. Mark the boundary with cachePoint.
3. Bedrock caches the processed prefix. Later calls with the same prefix reuse it.
4. Keep the changing part (the user question) after the cachePoint.

Exam verb: repeated prefix means caching. Order matters: static first, changing last.

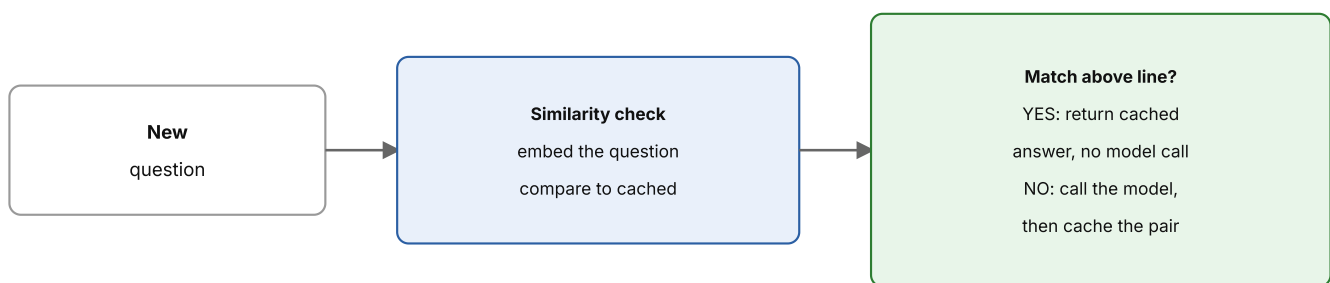
If the prefix changes every call, caching buys nothing. The prefix must repeat to pay off.

A cache hit skips the repeated prefix. The prefix must repeat for the cache to pay off. Unique traffic gets nothing.

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this diagram**

1. Left: the long prefix repeats across calls. Mark it with cachePoint.
2. Middle: the hit. Bedrock serves the prefix from cache.
3. Right: the bill. Cached input tokens cost less.

**Semantic caching**

Different words, same meaning still hits. Set the similarity line with real traffic.

Semantic caching matches meaning, not exact text. A similar question gets the cached answer. The model never runs.

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this diagram**

1. Left: a new question arrives. It resembles a cached one.

2. Middle: the semantic match. Meaning matches, not characters.
3. Right: the cached answer returns. Zero model cost.

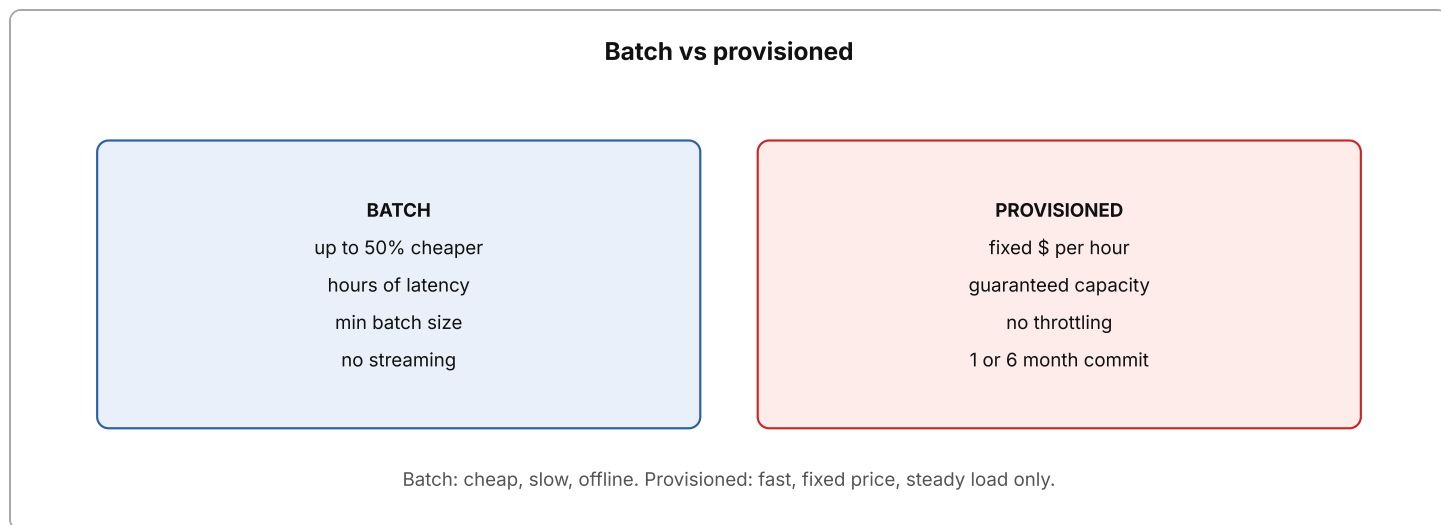
**Never confuse:** prompt caching with semantic caching. Prompt caching needs a repeated prefix and still calls the model. Semantic caching needs similar questions and skips the call. Edge caching is different again: it caches deterministic assets (generated images, static reports, reference files) on a CDN like CloudFront, close to the user. The model never runs for a cache hit. GenAI is dynamic, but AI systems still ship static content. The preconditions differ.

**Exam relevance.** "Same long system prompt on every call" means prompt caching. "Users ask similar questions many times" means semantic caching. "Static generated content served to many users" means edge caching with an aggressive TTL. Read the precondition.

## Batch versus Provisioned Throughput

Two levers handle non-interactive and steady traffic. They are not interchangeable. The exam pairs each with its symptom.

| Lever                  | Symptom                                                     | Price shape                                                |
|------------------------|-------------------------------------------------------------|------------------------------------------------------------|
| Batch inference        | Offline bulk work, no SLA, hours of latency OK              | About 50% off on-demand for the same model and tokens.     |
| Provisioned Throughput | Steady predictable traffic, throttling at peak, latency SLA | Hourly per model unit. It bills whether you use it or not. |



Batch trades latency for price. Provisioned Throughput trades hourly cost for guaranteed capacity. The symptom picks the lever.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left: batch. Hours of latency, half the price.
2. Right: Provisioned Throughput. Reserved capacity, hourly bill.
3. The middle line: never use one for the other's symptom.

The cost levers in order: right-size and cascade first, then prompt caching, then batch, Provisioned Throughput last. PT is last in cost order and first in capacity order. Token math comes before any of them.

## One scenario, all three ladders

Grounding picks the knowledge strategy. Pricing picks the cost shape. Safety picks the controls.

**Scenario: "Support answers from 10,000 product docs, cheap, safe, auditable."**

Run it through all three ladders. Each ladder answers one question.

### Grounding ladder

Docs change: RAG with Knowledge Bases. Not fine-tuning.

### Inference pricing ladder

Simple FAQ traffic: prompt router to cheap models.

### Safety control ladder

Customer-facing: guardrails plus invocation logging.

One scenario, three independent decisions. The exam composes them like this.

One scenario, three ladders. Grounding picks RAG. Pricing picks batch. Safety picks guardrails. Each ladder answers one constraint.

Source: original figure for the AWS AIP-C01 Study Guide.

## How to read this diagram

1. Read the scenario constraints first: facts, budget, safety.
2. Run each constraint through its own ladder.
3. The answers compose. One scenario can need three levers.

**Exam relevance.** "Throttled at peak, steady traffic" means Provisioned Throughput for the baseline plus on-demand for peaks. "Nightly bulk, no SLA" means batch. The wrong answers are real levers for the wrong symptom.

## Resource utilization: stop paying for idle GPUs

Overprovisioning wastes money. Four techniques raise utilization.

1. **Batching.** Group many requests into one GPU pass. Throughput rises. Latency per request rises too. Use it when no user waits: bulk scoring, offline evals, dataset generation.
2. **Tensor parallelism.** Split the model weights across several GPUs. Each GPU holds one shard. This fits models that exceed one GPU and cuts latency for large models. It needs fast links between GPUs.
3. **Capacity planning.** Match provisioned capacity to real traffic, not to peak guesses. Review the utilization numbers each month. Shrink what sits idle.

4. **Monitor model utilization.** Track GPU use, queue depth, and throttle counts in CloudWatch. Low GPU use with long queues means the batch size is wrong, not the capacity.

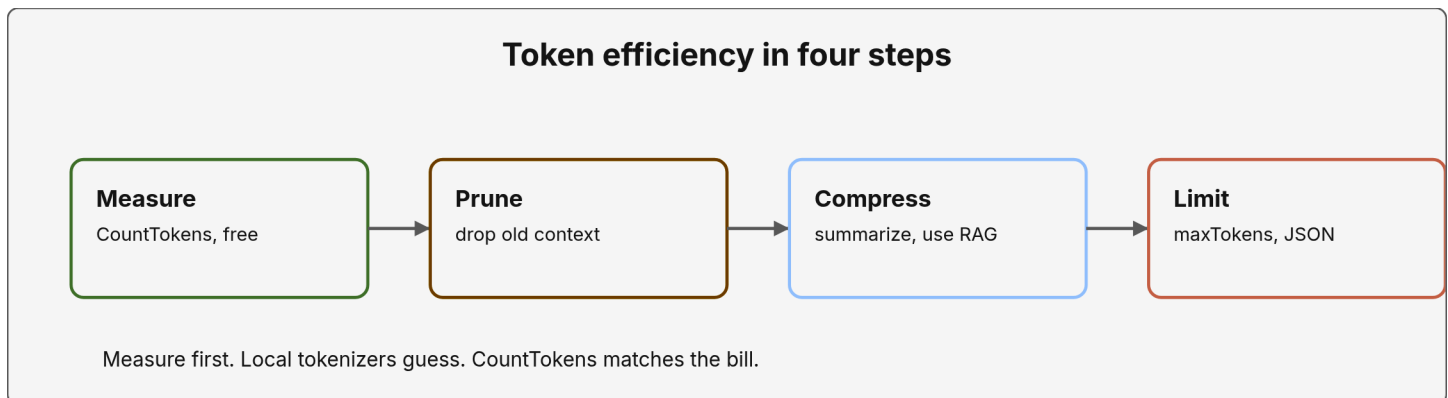
**Exam relevance.** "GPUs idle while requests queue" points at batching or tensor parallelism, not at more GPUs. "One model too large for one GPU" points at tensor parallelism. For the largest distributed training and inference jobs, AWS points at UltraServers (Trn2, P6e-GB200): purpose-built servers with high-bandwidth links between accelerators.

### Token efficiency: measure, prune, compress, limit

More tokens mean more processing time and more money. Four technique groups cut tokens.

1. **Count tokens first.** The Bedrock CountTokens API returns the token count for an input without running inference. It accepts the same body as InvokeModel or Converse. It is free. Use it to check context-window fit and to estimate batch cost before any billed call.
2. **Prune context.** Drop old chat history. Chunk big documents and send only relevant chunks. Retrieve fewer chunks. Filter chunks by metadata.
3. **Compress prompts.** Summarize old chat history or large documents before sending. A small model can do the summarizing. Prefer RAG over stuffing full documents into the prompt.
4. **Limit responses.** Set maxTokens. Force a short format through JSON output settings. State the length in the prompt. Use few-shot examples to set verbosity.

**CloudWatch metrics.** Track InputTokenCount and OutputTokenCount per model. Watch throttles and invocation errors. For streaming, watch TimeToFirstToken.



Measure first with CountTokens. Then prune context, compress prompts, and limit responses. Each step cuts the bill.

*Measure first with CountTokens. Then prune context, compress prompts, and limit responses. Each step cuts the bill.* Source: original figure for the AWS AIP-C01 Study Guide.

**Under the hood.** CountTokens runs the real tokenizer without running the model. Local tokenizers guess because each model family tokenizes differently. The exam twist: CountTokens can reject inference-profile model IDs that InvokeModel accepts. Test with the base model ID when unsure. The API is free either way.

**Exam relevance.** "Estimate cost before inference" means CountTokens. "Free" is the fact the exam checks: local tokenizers guess, CountTokens measures.

## Intelligent Prompt Routing: route by complexity

Model size trades cost against capability. Small models handle preprocessing well: summarization, classification, chunking. A small model can even be the backbone when RAG and tool use do the heavy lifting.

**Dynamic routing.** Route simple queries to cheap models and hard queries to flagships, without custom code. Bedrock Intelligent Prompt Routing scores each prompt and picks the tier. Agents can also score complexity with a tool and route through conditional flows, Agent Squad, or Lambda.

**Exam relevance.** "Simple questions cheap, hard questions flagship, no custom routing code" names Intelligent Prompt Routing. "Cheapest model that meets the constraint" is the standing cost rule.

## TTFT: the latency users feel

Time to first token is the delay between sending the request and receiving the first token. It only exists for streaming calls. Non-streaming calls have one end-to-end latency number.

**What drives it.** Long prompts drive TTFT up because the model processes the whole prompt before emitting anything. Large models, cold Provisioned Throughput, and throttling queues push it up. Shorter prompts, prompt caching, smaller models, and streaming push it down.

**Measure it.** Bedrock publishes TimeToFirstToken to the AWS/Bedrock CloudWatch namespace. InvocationLatency covers all calls. Set alarms on p95 TTFT so regressions page someone instead of silently annoying users.

**Trap: "TTFT for a batch job."** Wrong. Batch inference is not streaming. TTFT has no meaning there. The exam also confuses TTFT with end-to-end latency and with tokens per second. Name the metric the stem describes.

## Retry: backoff with jitter

Throttling is a signal, not a bug. The client must retry with exponential backoff and jitter. Wait, then wait longer, and add a random offset so a fleet of clients does not retry in lockstep.

| Setting    | Value                          |
|------------|--------------------------------|
| First wait | About 1 second                 |
| Growth     | Double each time: 1s, 2s, 4s   |
| Jitter     | Small random offset per retry  |
| Attempts   | 3 to 5, then surface the error |

AWS SDK clients offer custom retry policies with these knobs. Provisioned Throughput solves sustained throttles. More retries do not.

**Connection pooling.** Keep a pool of open connections to the backend. It saves the cost of new handshakes. For HTTP clients in AWS, 10 to 20 connections per instance with a 60 to 300 second TTL is the working range. Balance utilization against reuse.

**Exam relevance.** "Throttled, retry" always means exponential backoff with jitter. "Sustained throttles at steady load" means Provisioned Throughput, not more retries.

## A/B tests need statistics: Evidently

Comparing two CloudWatch dashboards by eye is not an experiment. It has no randomization and no significance test. CloudWatch Evidently runs real experiments: feature flags plus A/B tests with built-in statistics.

**The two APIs.** EvaluateFeature assigns the requesting user to variant A or B and returns the variant configuration. PutProjectEvents reports the outcome metric, like a thumbs-up or task success. Evidently computes whether B beats A with statistical significance. Define the feature with variations (model A versus model B, topK 10 versus 50), set traffic fractions, and pick the winning metric. A 90/10 split limits blast radius while the test runs.

**Under the hood.** EvaluateFeature assigns the variant deterministically per user so one user sees one variant. PutProjectEvents reports the outcome. Evidently runs the significance math. The exam twist: "we compared two dashboards" is not an experiment. No randomization and no significance test means no conclusion.

**Exam relevance.** "Statistically significant winner" or "which variant wins" points at Evidently. "Two Lambda versions, compare dashboards" is the distractor. Prompt Management variants cover prompt text only. Real experiments vary models, retrieval depth, temperature, or whole pipelines.

## More D4 decision rules

- If the stem says "most cost-effective," do the token math. Compare input plus output cost. Never pick the flagship by default.
- If the stem says "same long system prompt on every call," pick prompt caching. Mark the prefix with cachePoint.
- If the stem says "users ask similar questions many times," pick semantic caching. It skips the call completely.
- If the stem says "nightly bulk work, no SLA," pick batch inference. JSONL in S3. About 50% off.
- If the stem says "steady traffic, throttling at peak," pick Provisioned Throughput for the baseline. Keep on-demand for peaks.
- If the stem says "slow first token," pick streaming. Provisioned Throughput fixes capacity. It does not fix first-token speed.
- If the stem says "simple queries on an expensive model," pick a prompt router or cascade. Never pick Provisioned Throughput for a cost problem.
- If the stem says "latency SLA with steady volume," pick Provisioned Throughput. It buys guaranteed capacity.
- If the stem says "spiky or low traffic," pick on-demand. Provisioned Throughput bills hourly whether you use it or not.
- If the stem says "cut retrieval cost," tune top-k and chunking first. Smaller context means fewer input tokens.

## More D4 traps, with why the wrong answer fails

- Provisioned Throughput for a cost problem. PT is capacity. It does not make simple queries cheaper. Cost problems need routers, caching, or smaller models.
- Provisioned Throughput for bursty traffic. PT bills hourly whether you use it or not. Bursty traffic needs on-demand.
- Prompt caching on unique traffic. No repeated prefix means no hits. The cache never pays off.

- Semantic caching for exact-only answers. Semantic caching returns similar answers. Exact needs need exact retrieval.
- Batch inference for interactive requests. Batch has hours of latency. Waiting users need sync or streaming.
- Fine-tuning to cut cost. Fine-tuning changes behavior. It does not cut per-call cost. Distillation or smaller models cut cost.
- Bigger model for "most cost-effective." The flagship costs more per token. Do the math. The small model often wins.
- Streaming to fix throttling. Streaming fixes perceived latency. Throttling is a capacity problem. It needs Provisioned Throughput.
- Cross-region profiles for cost. Profiles route for availability. They do not cut cost. Prompt routers cut cost.
- Ignoring the output reserve. Output tokens cost more than input tokens. A missing reserve breaks the budget.

#### D4 self-test: more questions with answers

D4 · MEDIUM · ONE ANSWER CC67. Traffic is steady at 500 requests per minute. Throttling hits at peak. A latency SLA applies. What do you buy? A) More on-demand quota. On-demand still throttles at peak. B) Provisioned Throughput for the baseline, on-demand for peaks. C) Batch inference. Batch has hours of latency. The SLA forbids it. D) A prompt router. Routers cut cost. They do not add capacity. Reveal answer Why B is correct: "Steady traffic, throttling at peak, latency SLA" is the Provisioned Throughput sentence. PT buys guaranteed capacity for the baseline. On-demand covers the peaks.

D4 · EASY · ONE ANSWER CC68. Every call sends the same 8,000-token system prompt. The prompt never changes. What cuts the bill? A) Semantic caching. Similar questions are not the pattern here. The prompt repeats. B) Prompt caching with cachePoint on the system prompt. C) Batch inference. The calls are interactive. Batch has hours of latency. D) Fine-tuning. Fine-tuning changes behavior. The need is cost on a repeated prefix. Reveal answer Why B is correct: "Same long system prompt on every call" is the prompt-caching sentence. Mark the prefix with cachePoint. Cached input tokens bill at a reduced rate.

D4 · MEDIUM · ONE ANSWER CC69. A support bot gets the same 200 questions daily, phrased differently each time. What cuts the bill most? A) Prompt caching. The questions differ. The prefix does not repeat. B) Semantic caching. C) Provisioned Throughput. PT is capacity. The need is cost on repeated questions. D) A bigger model. This raises the bill. Reveal answer Why B is correct: "Similar questions many times" is the semantic-caching sentence. The cache matches meaning, not exact text. Hits skip the model call completely.

D4 · EASY · ONE ANSWER CC70. Users complain the chatbot feels slow. The answers are long. What fixes perceived latency? A) Provisioned Throughput. PT fixes capacity. It does not fix first-token speed. B) Streaming with ConverseStream. C) Batch inference. Batch is slower, not faster. D) A bigger model. Bigger models are slower per token. Reveal answer Why B is correct: Streaming sends tokens as they generate. The user reads while the model writes. Perceived latency drops. PT and model size do not fix first-token speed.

D4 · MEDIUM · ONE ANSWER CC71. A team runs 2,000 input tokens and 500 output tokens per call at example prices. Which change cuts the bill most? A) Cut input tokens by half. Input tokens are cheaper than output tokens. B) Cut output tokens by half. C) Switch to batch. The calls are interactive. Batch does not fit. D) Add Provisioned Throughput. PT is capacity, not cost. Reveal answer Why B is correct: Output tokens price higher than input tokens. Cutting the expensive tokens cuts the bill most. Do the token math: price times count, per side.

**D4 decision rules: if the question says X, pick Y**

- "Least operational overhead" means the fewest self-managed components. "Least custom development effort" means managed features. "Most cost-effective" means token math.
- "Throttled at peak, steady traffic" means Provisioned Throughput (provisioned ARN as modelId). "Spiky or low traffic" means on-demand.
- "80% simple queries on the flagship" means a prompt router or cascade. Never use PT for a cost problem.
- "Repeated long prefix" means prompt caching. Proof is cache read hit rate. "Unique traffic" means caching is dead weight.
- "Nightly bulk, no SLA" means batch inference (about 50% off). Never use it for interactive SLAs.
- "Slow first token, users complain" means streaming. Not PT, not caching.
- Use inference profiles for regional failover. Use prompt routers for cost by complexity. Do not swap them.
- Choose the cheapest option that meets all requirements. Do not choose premium extras with no stated need.

**D4 trap patterns**

- Use Provisioned Throughput (PT) to fix cost for bursty traffic. PT bills hourly. It bills even if you do not use it.
- Use PT to fix latency for slow first token. Capacity is not perceived latency.
- Use prompt routers for availability. Use inference profiles for cost. The options swap the pair.
- Cache unique traffic. If there is no repeated prefix, there is no saving.
- Batch for interactive Service Level Agreements (SLAs). Hours of latency breaks the SLA.
- Total token spend shows if caching works. The metric is cache read hit rate.
- Use a bigger model as the quality answer. Do not do this without a golden-dataset check. Right-size first.
- Use reactive metrics for proactive requirements. Count before the call.

## D4 rapid-fire questions

### D4 · MEDIUM · ONE ANSWER

**CC39. Token costs increase. 80% of requests are simple FAQ lookups on the flagship model. Quality must not decrease. What is the best fix?**

A) Fine-tune the flagship.

This is the wrong lever. It needs much effort. The cost problem is model size per request. It is not model behavior.

**B) Use a prompt router or cascade. Send simple queries to a lightweight model. Send complex queries to the flagship.**

C) Use Provisioned Throughput.

PT is capacity. It is not cost. It does not make simple queries cheaper.

**Why B is correct:** The problem is "simple queries on an expensive model." Routers route by complexity for cost. A lightweight model scores the same on simple extraction. This model is the correct size. It does not have lower quality.

### D4 · MEDIUM · ONE ANSWER

**CC40. Steady production traffic throttles at peak on on-demand. What is the fix? What is the routing problem?**

**A) Use Provisioned Throughput. Pass the provisioned ARN as modelId.**

B) Use Provisioned Throughput. Keep passing the base model ID.

The base model ID keeps hitting shared on-demand quotas. Idle PT capacity means the code never routes to it.

C) Use batch inference.

Batch is offline bulk. It is not steady interactive traffic.

**Why A is correct:** Steady plus throttled is the PT shape. (model units, 1 to 6 month terms, hourly billing). The ARN is important. Reserved capacity only serves traffic addressed to it.

**D4 · EASY · ONE ANSWER****CC41. "Cheapest for nightly 500k-row classification, no SLA." Which inference?****A) Realtime Converse.**

Realtime pricing for offline bulk is the expensive default.

**B) Do batch inference. Use JSONL to S3. Get results from S3. Save about 50% of the cost.**

**Why B is correct:** There is no SLA. Bulk processing is batch processing. Hours of latency is acceptable. The 50% cost reduction is important.

**D4 · MEDIUM · ONE ANSWER****CC42. Users report slow responses on unique interactive queries. Should you use streaming, PT, or caching?****A) Streaming (ConverseStream)****B) Provisioned Throughput**

PT fixes capacity. It does not fix first-token speed.

**C) Prompt caching**

Unique queries do not have a repeated prefix. Caching is not useful.

**Why A is correct:** "Slow responses" on interactive traffic is perceived latency. Streaming is the only way to change first-token time.

**D4 · MEDIUM · ONE ANSWER****CC43. A scenario needs regional failover and cost-based routing. Which combination is correct?****A) A prompt router selects the model by complexity. Each model uses its inference profile for failover.****B) Prompt router alone**

Routers route by complexity. They do not route by region. Failover needs the profile layer.

**C) Inference profile alone**

Profiles route by region. They do not route by complexity. Cost routing needs the router layer.

**Why A is correct:** The confusion set is complete. Use the router for cost. Use the profile for availability. Stack them together. PT would answer a capacity question that no one asked.

**D4 · EASY · RECALL****CC44. Which metric shows that prompt caching works?**

**Answer:** The cache read hit rate. Total token spend changes for many reasons. The hit rate shows only the cache performance.

**D4 · MEDIUM · ONE ANSWER****CC45. Traffic is spiky. There are long idle nights. The team proposes Provisioned Throughput "to save money." What is the verdict?**

- A) Reject: PT bills hourly. It bills whether you use it or not. Idle nights still cost money.
- B) Accept: reserved capacity is always cheaper  
PT gives a latency and capacity guarantee for steady baselines. It is not a discount for unpredictability. On-demand is better for spiky traffic.

**Why A is correct:** Match the pricing model to the traffic shape. Spiky or low traffic stays on-demand. PT serves the predictable baseline.

**D4 · EASY · RECALL****CC46. State the three adjectives and their winners.**

**Answer:** "Least operational overhead" wins with the fewest self-managed components. "Least custom development effort" wins with managed features over your code. "Most cost-effective" wins with token math. Use a smaller model, caching, batch, or cascade.

**D4 · EASY · RECALL****CC47. State the prompt caching preconditions: minimum prefix and TTL.**

**Answer:** A repeated long prefix. The minimum is about one thousand tokens. This depends on the model. The default TTL is 5 minutes. It can be up to 1 hour on supported models. Mark the boundary with cachePoint.

**D4 · MEDIUM · ONE ANSWER**

**CC48. The prompt router's default fallback model fires on most traffic. What is wrong?**

- A) The tier boundaries are not correct. The router cannot classify. All items fall through.**
- B) The flagship model is too expensive.  
Cost is not the problem here. A constant fallback means wrong classification. You fix this with evaluation and boundary tuning.

**Why A is correct:** The fallback means the router cannot make a decision. You fix this with evaluation-driven boundary tuning. Do not use a bigger model.

**Next:** D5 shows the application works. D5 fixes the application if it does not work. Use the evaluation ladder. Then, use change-point debugging.

**D4 · EASY · ONE ANSWER**

**QH11. Before a 10,000-prompt batch job, the team wants the exact input token count with zero inference cost. Which API do they call?**

- A) InvokeModel and read the usage fields.  
That runs inference and bills for it.
- B) CountTokens.**
- C) A local tokenizer estimate.  
Local tokenizers do not match the billed tokenizer.
- D) Converse with maxTokens set to 1.  
Still a billed inference call.

**Why B is correct:** CountTokens returns the billed token count for an input without running inference. It is free. It accepts the same body as InvokeModel or Converse.

**The trap:** local tokenizer estimates. Only CountTokens matches the bill.

**D4 · MEDIUM · ONE ANSWER**

**QH12. Users complain the chat feels slow: a blank screen for seconds before the first word. Which metric and fix apply?**

- A) InvocationLatency, add Provisioned Throughput.  
Latency covers the whole call, not the first-token wait.
- B) TimeToFirstToken, stream the response and shorten the prompt.**
- C) OutputTokenCount, reduce maxTokens.  
Token counts do not measure the first-token wait.
- D) Throttles, add retries.  
No throttling in the stem. The problem is prefill time.

**Why B is correct:** TTFT measures request-to-first-token for streaming calls. Long prompts drive it up. Streaming plus shorter prompts plus prompt caching bring it down. Bedrock emits it to CloudWatch natively.

**The trap:** TTFT reasoning on non-streaming calls. The metric only exists for streaming.

## D5: Testing, Validation, and Troubleshooting (11%)

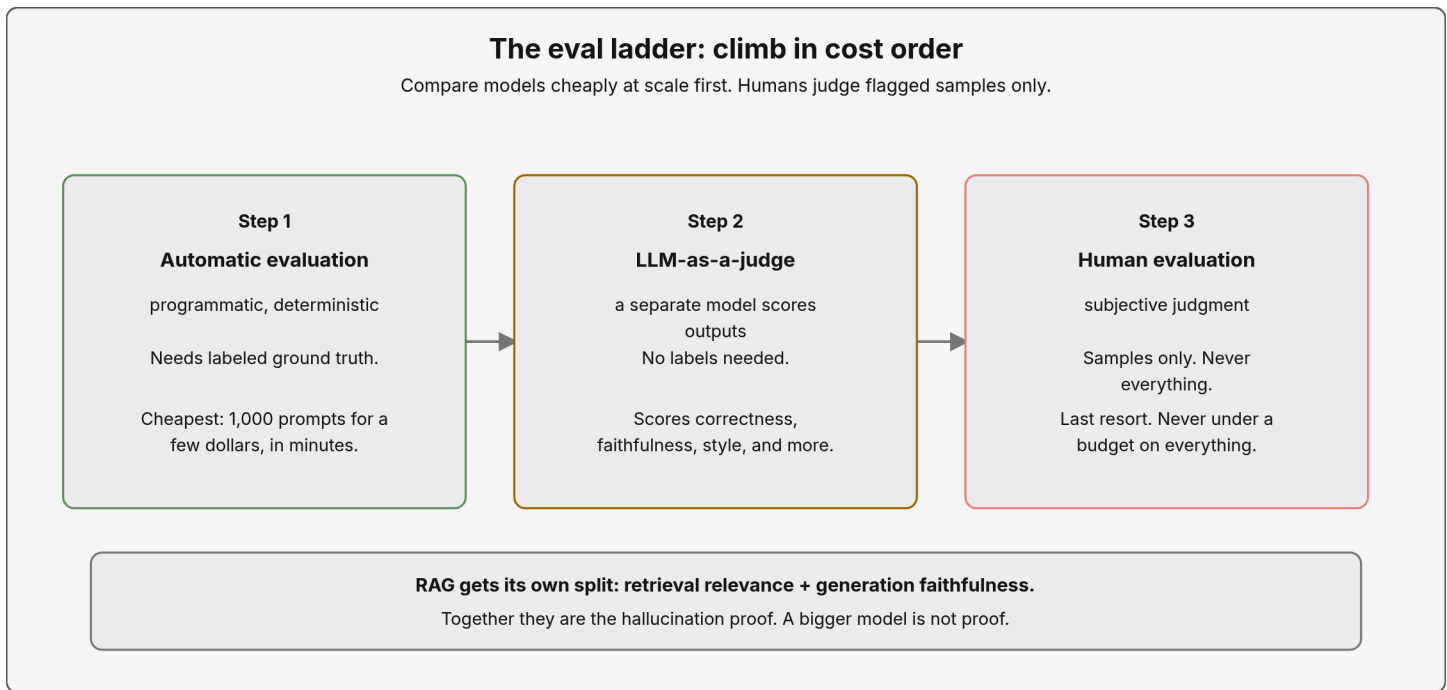
All information until now assumed the application works. D5 proves the application works. D5 fixes the application if it does not work.

D5 asks one question in many ways. How do you prove a GenAI application works? How do you fix it if it does not work? The proof part is evaluation. Climb the evaluation ladder. Start with the cheapest option.

The fix part is change-point debugging. The broken item is the item that changed most recently.

### The evaluation ladder: automatic, then judge, then human

**Automatic evaluation** (programmatic) compares model outputs. It compares them with known-correct answers. It uses deterministic code. It needs a labeled dataset. Every prompt has a **ground truth** label. Built-in metrics are **accuracy**.



Climb the ladder in order. Automatic evaluation first. Then LLM as a judge. Then human evaluation on samples only.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

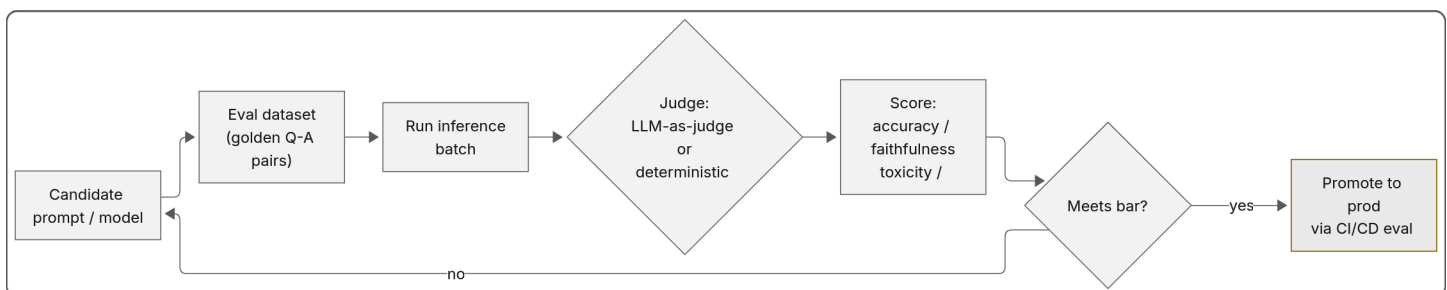
1. Bottom rung: automatic. Code compares outputs to known answers.
2. Middle rung: LLM as a judge. A separate model scores samples with a rubric.
3. Top rung: human. People review a sample. It costs the most.

This shows if it matched. Another metric is **stability**. This shows if the answer stays correct with small prompt changes, like typos. The console calls this robustness. Another metric is **toxicity**. This checks for harmful content.

A 1,000-prompt automatic evaluation on a small model costs a few dollars. It finishes in minutes. This is why it is in every CI pipeline. **LLM-as-a-judge** uses a separate model. It scores outputs for **correctness, completeness, faithfulness, harmfulness, refusal, and style/tone**. You can define custom metrics.

It needs no labels. So, it works with unlabeled data. But a judge call costs a judge call. **Human evaluation** is the last option. It uses subjective judgment on samples only. Do not use it for everything.

Do not use it under a budget.



The eval loop: candidate versus golden dataset, scored by metric or judge, promoted only through the gate.

Source: original figure for the AWS AIP-C01 Study Guide.

Use the ladder in practice. Compare models cheaply at scale. Use automatic metrics first. Run LLM-as-a-judge on the model that remains. Put humans on flagged samples only. "Two days to compare two models" means this ladder in cost order.

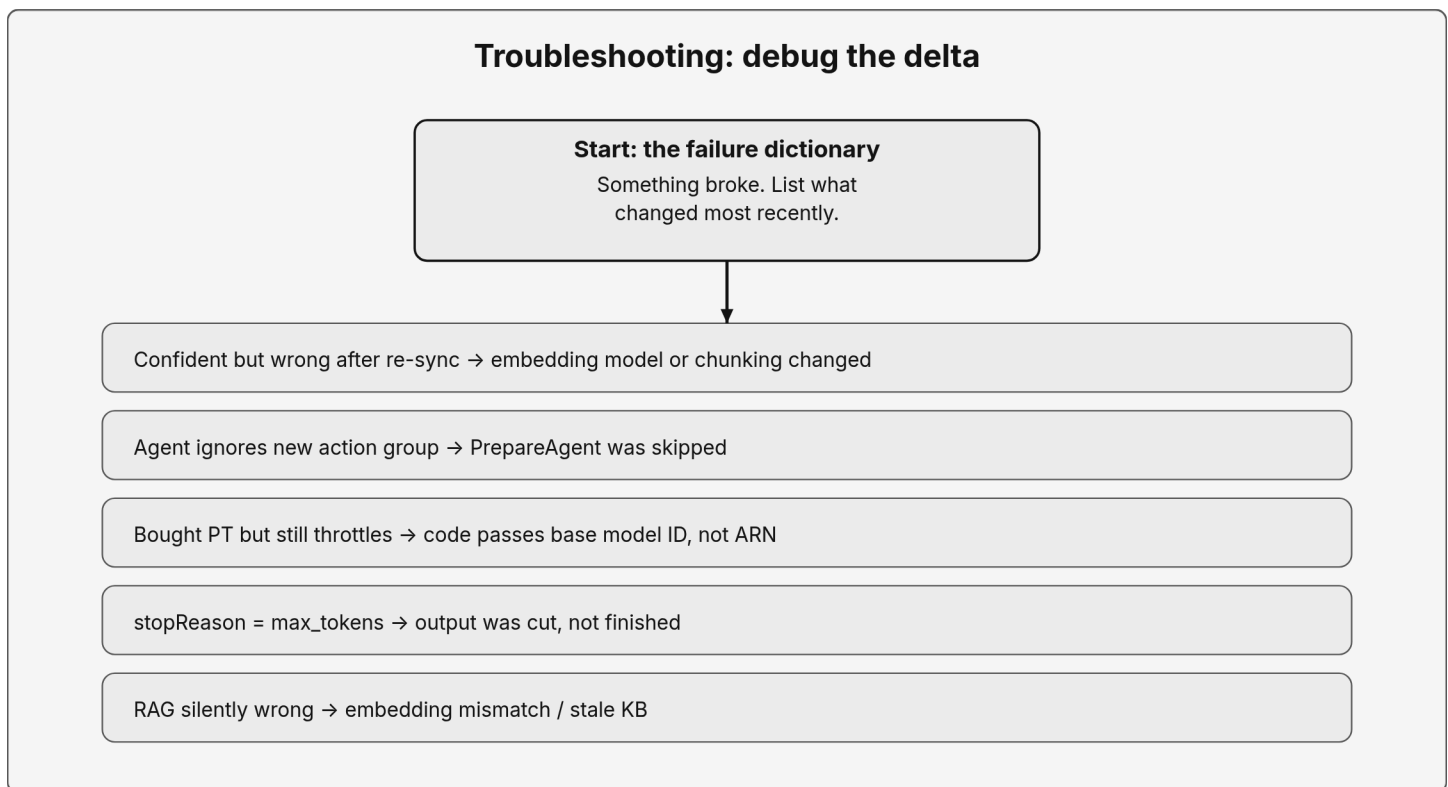
RAG has its own evaluation split. This is **retrieval relevance**. Did we get the correct chunks? This is also **generation faithfulness**. Is the answer supported by the chunks? LLM-as-judge powers this.

"Prove the RAG system does not hallucinate" means retrieval plus faithfulness. Do not use a bigger model. Provider benchmarks never decide your model choice. They measure the provider's prompts, not your prompts. Your own evaluation job on your golden dataset is the only score that matters. New prompt versions, new model versions, and new guardrail versions all pass through the same **CI/CD eval gate**.

This is an automated quality check inside the continuous integration and deployment pipeline. Score above the threshold or do not deploy. A/B testing uses two aliases. It uses shadow traffic. Shadow traffic copies real user requests. It routes them to the new version.

Users do not see them. This happens through inference profiles. This catches problems the gate misses.

### Troubleshoot: debug the delta



Troubleshooting: start from what changed most recently, then match the symptom in the failure dictionary

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Top box: the method. Below: symptom-to-cause pairs. Takeaway: debug the delta first; the five pairs cover the exam's favorite breakages.

Change-point debugging is the whole method. List what changed most recently. Debug that first. "Answers are confident but wrong after a re-sync" means the re-sync changed the embedding model version. Or it changed the chunking configuration. "Model ignores the new prompt template" means the Prompt Management version was never published.

Or AppConfig serves a cached value. "Agent ignores the new action group" means you skipped prepare-agent. "Bought PT but still throttles" means the code passes the base model ID. It does not pass the provisioned ARN.

The failure dictionary: **AccessDenied on InvokeModel** means check model access. Check regional availability. Do this before you change IAM. **Throttling** means retry with backoff for short spikes. Use PT for continuous load. **stopReason** `guardrail_intervened` means the guardrail blocked the call. Check which policy fired.

Enable trace. **stopReason** `max_tokens` on a good answer means the output was cut. **ValidationException** on **InvokeModel** means the provider body is not correct. Claude needs `anthropic_version` and `max_tokens`. This is not an IAM problem. **RAG retrieval failing silently** means embedding mismatch. Or it means split chunks.

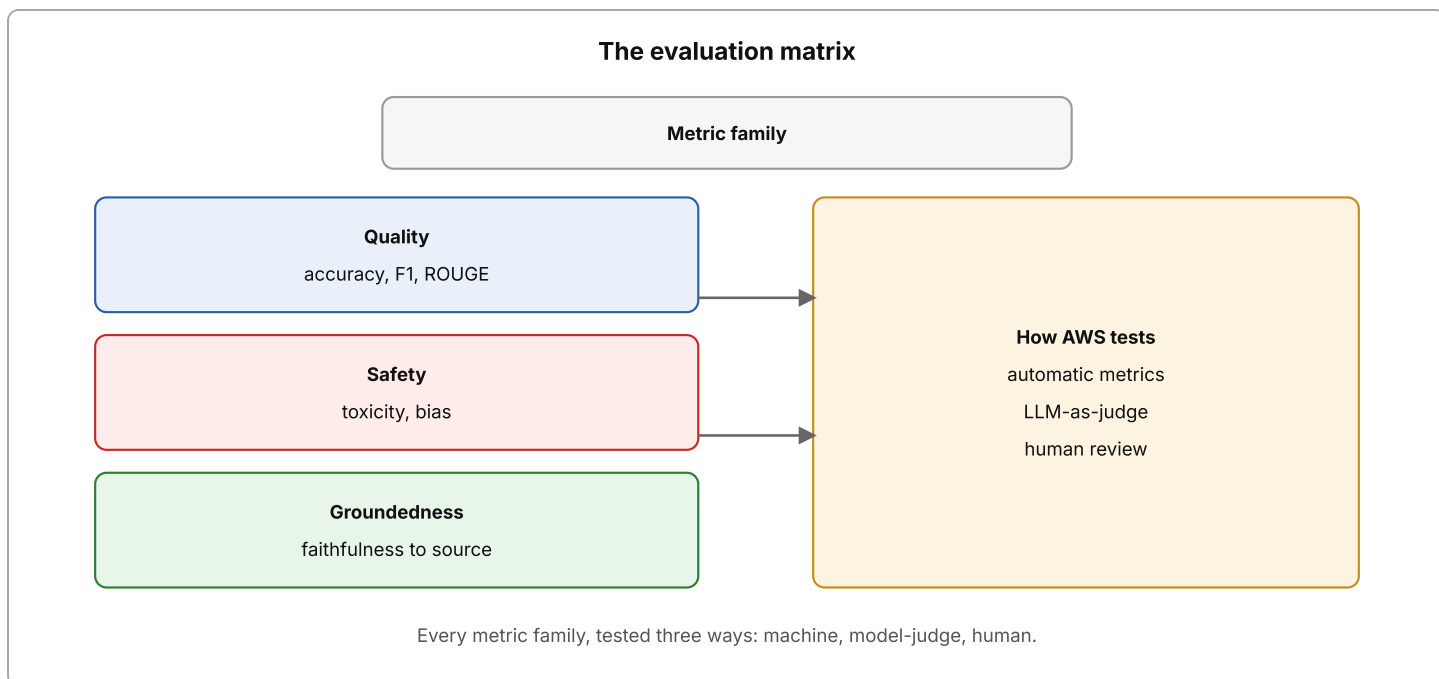
Or it means an out-of-sync knowledge base. **Quality drift in production** means scheduled evaluation jobs against the golden dataset. It also means CloudWatch anomaly detection on token metrics. Do not use a hand-built pipeline.

## The metrics that matter

Bedrock evaluations score outputs on named metrics. The exam names the metric. You name what it checks.

| Metric                 | What it checks                                                          |
|------------------------|-------------------------------------------------------------------------|
| Correctness            | The answer is right.                                                    |
| Completeness           | The answer covers all requests.                                         |
| Faithfulness           | The source supports every claim. This is the anti-hallucination metric. |
| Harmfulness            | The content is unsafe.                                                  |
| Refusal                | The model correctly refused what it must refuse.                        |
| Style and tone         | The answer matches the required voice.                                  |
| Stability (robustness) | The answer stays correct with small prompt changes, like typos.         |
| Toxicity               | The output holds harmful content.                                       |

Three evaluation methods climb one ladder. Automatic evaluation is code against known answers. It needs labels. It is cheapest. LLM-as-a-judge uses a separate model with a rubric. It needs no labels. Human evaluation uses people on samples only. It costs the most.

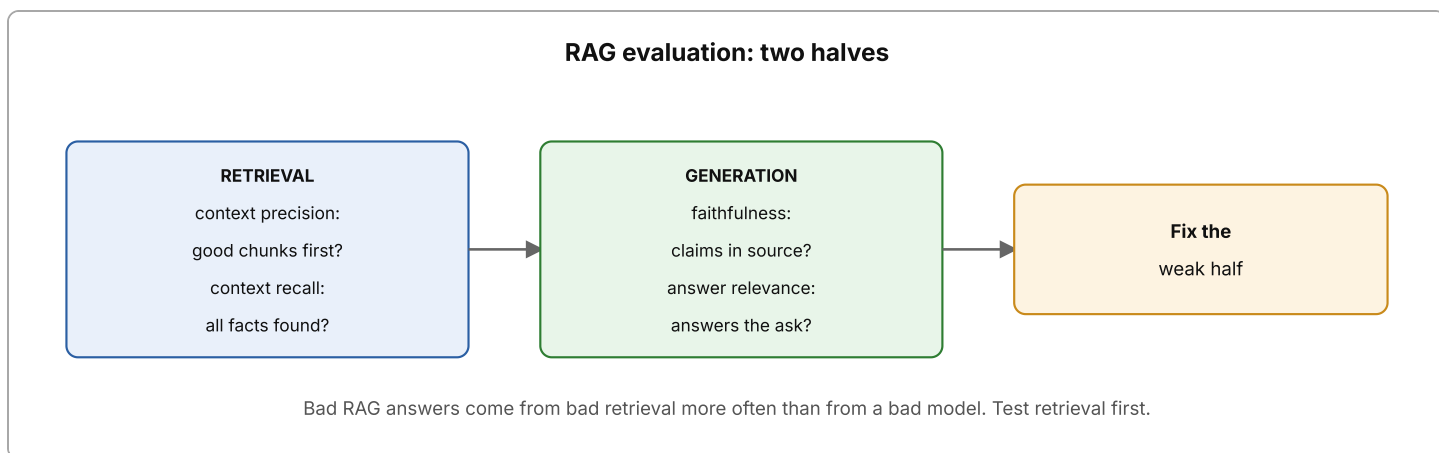


The eval matrix crosses what you measure with how you measure it. Quality, safety, and groundedness each work with automatic, judge, or human methods.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Rows: what you measure. Quality, safety, groundedness.
2. Columns: how you measure. Automatic, judge, human.
3. Pick the cell that fits the budget and the question.



RAG evaluation splits in two. Retrieval quality asks if the right chunks came back. Faithfulness asks if the answer stayed inside them.

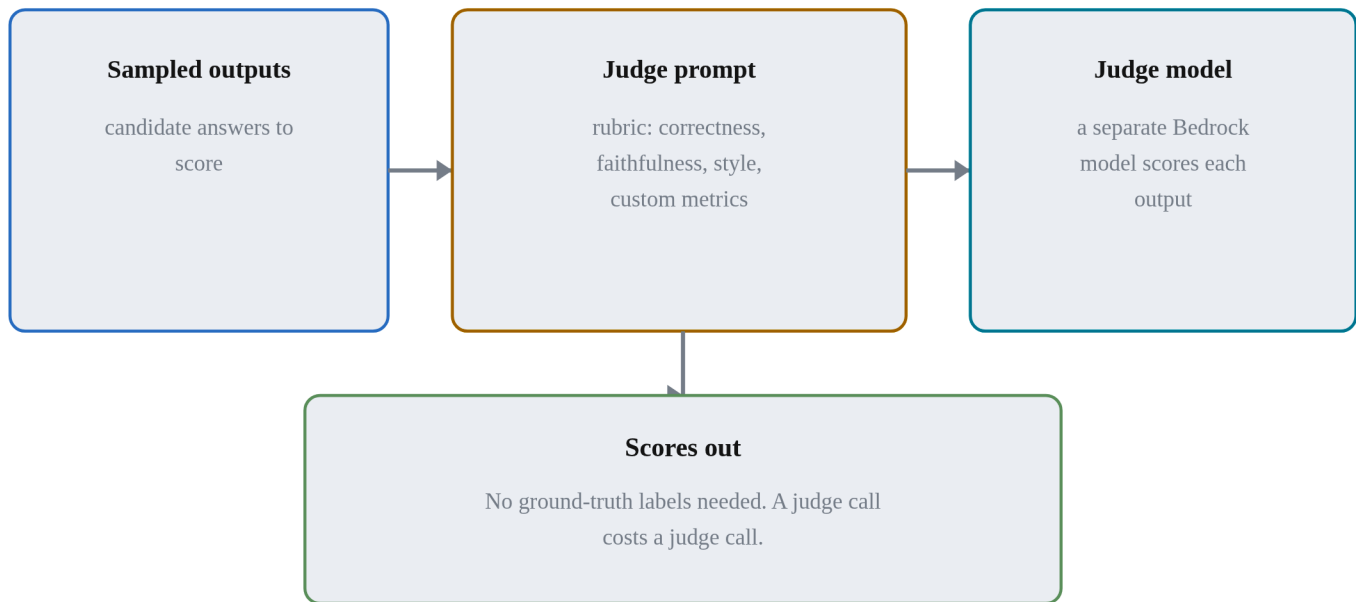
Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left: retrieval. Right chunks or wrong chunks.
2. Right: generation. Every claim needs a source chunk.
3. A failure on the left needs chunking fixes. A failure on the right needs grounding fixes.

## LLM-as-a-judge on Bedrock: rubric in, scores out

A separate model scores outputs against your rubric. No labels required.



The judge setup: sampled outputs plus a rubric prompt go to a separate judge model. The judge returns scores. You can define custom metrics.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left: sampled outputs from the candidate. Not all outputs.
2. Middle: the rubric prompt. It defines the metric.
3. Right: the judge model. It scores. It is separate from the candidate.

**Exam relevance.** D5 is 11% of the exam. "Which metric checks hallucinations" is always faithfulness. "No labels, human-like quality at scale" is always LLM-as-a-judge.

### Safe rollout: shadow, A/B, canary

New prompts and models reach production through gates. Three rollout patterns matter.

| Pattern  | What happens                                            | Use when                                      |
|----------|---------------------------------------------------------|-----------------------------------------------|
| Shadow   | Copies of real requests go to the new version invisibly | You need production data with zero user risk. |
| A/B test | Real users split across two versions                    | You need a fair comparison on live traffic.   |
| Canary   | A small slice of traffic goes to the new version first  | You need early warning before full rollout.   |

## Shadow traffic versus A/B: test without hurting users

Shadow is invisible. A/B is a controlled slice of real traffic.

### Shadow traffic

Copies of real requests go to the new version. Users never see its answers.

Compare shadow scores against production. Zero user risk.

### A/B with two aliases

Real users are split: alias A serves most, alias B serves a slice.

Measures real user impact. Small blast radius.

Both catch regressions the CI/CD eval gate misses. Gate first, then shadow or A/B.

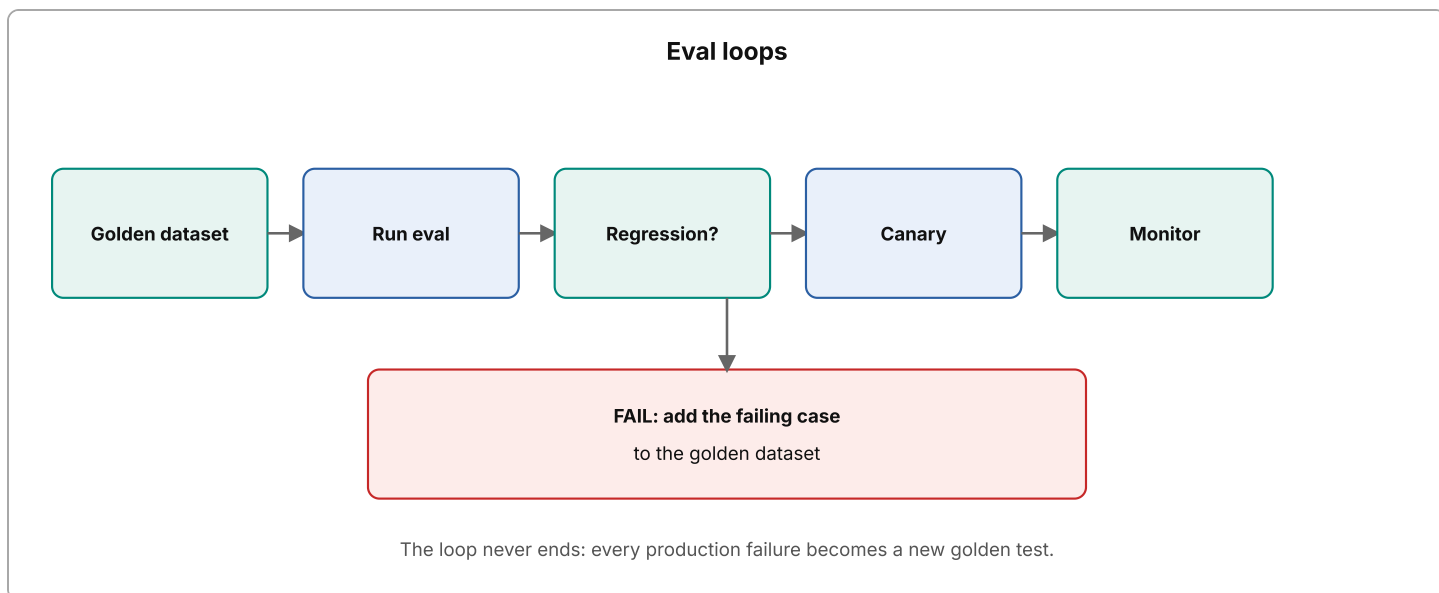
Shadow sends invisible copies. A/B splits real users. Shadow proves safety. A/B proves the winner.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left: shadow. Real requests copy to the new version. Users never see it.
2. Right: A/B. Users split across two aliases. Metrics decide.
3. Order: shadow first for safety, then A/B for the winner.

The eval loop never stops. A candidate prompt or model runs against a golden dataset. A judge scores accuracy, faithfulness, toxicity, and latency. The candidate promotes to production only through a CI/CD eval gate. Production failures feed back into the dataset.



The loop: golden dataset, regression check, canary, monitor, feed failures back. Every production failure becomes a future test.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Start: the golden dataset. Known prompts, known answers.
2. Middle: regression check, then canary. Gates before production.
3. End: monitor. Failures return to the dataset.

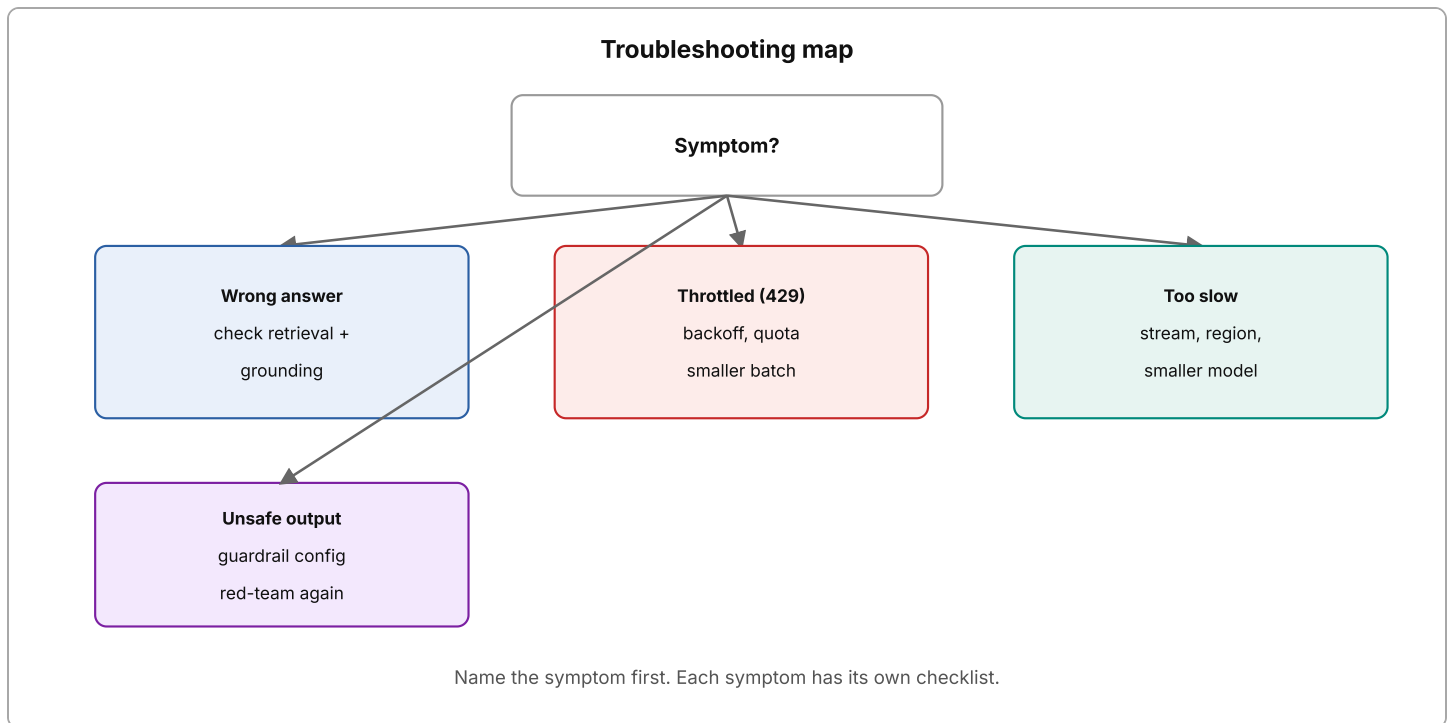
**Never confuse:** shadow with A/B. Shadow touches no users. A/B splits users. Canary with A/B. Canary is a small slice for safety. A/B is a fair split for comparison.

**Exam relevance.** "Zero user risk on production data" means shadow. "Fair comparison on live traffic" means A/B. "Promote only through the gate" means the CI/CD eval gate.

### The failure dictionary: symptom to cause

Troubleshooting starts from what changed most recently. Then match the symptom to the cause.

| Symptom                              | Check first                                   | Why                                                                 |
|--------------------------------------|-----------------------------------------------|---------------------------------------------------------------------|
| Confident but wrong after a re-sync  | Embedding model version and chunking settings | The ingest pipeline changed. The vector space broke.                |
| Answers cut off mid-sentence         | maxTokens and stopReason                      | stopReason max_tokens means the output was cut. Raise maxTokens.    |
| AccessDenied on InvokeModel          | IAM: model access and region                  | The error is permissions or availability. Check model access first. |
| Claude calls fail with a shape error | anthropic_version and max_tokens in the body  | The error is the request shape. Do not debug IAM for one hour.      |
| Throttling at peak                   | Provisioned Throughput for the baseline       | Capacity problem. PT buys guaranteed throughput.                    |
| Slow first token on long answers     | Streaming                                     | Perceived latency problem. PT does not fix first-token speed.       |
| Guardrail blocks a legit request     | Which policy fired, then the strength dial    | stopReason guardrail_intervened names the block. Tune the dial.     |
| Costs spike with no traffic change   | Token counts and cache hit rate               | Longer prompts or dead cache raise the bill. Check the delta.       |



Start from the delta: what changed most recently. Then match the symptom to the checklist. The five pairs cover the exam's favorite breakages.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Top: the method. Debug the delta first.
2. Middle: symptom rows. Find the match.

3. Bottom: the fix. Each symptom has one first check.

**Exam relevance.** Troubleshooting questions give a symptom and a recent change. The answer is almost always the change. Read the stem for the delta.

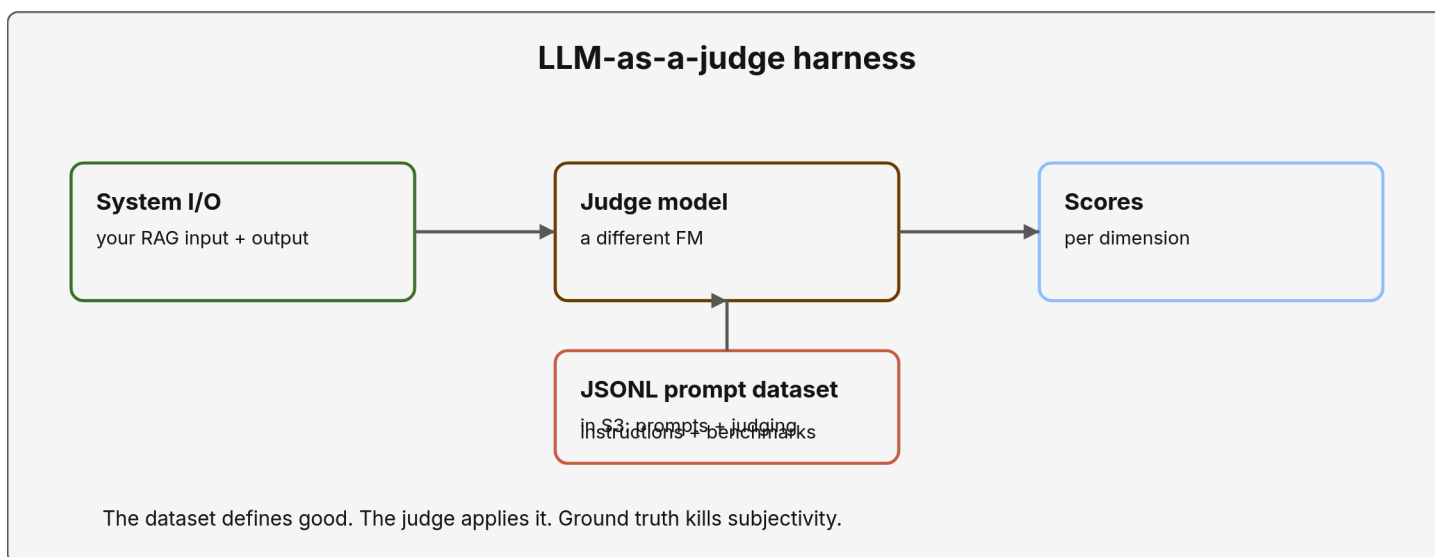
## FM evaluation: four ways to judge

**Human evaluation.** People judge helpfulness and feel. Slow and subjective, but it catches what metrics miss.

**Benchmark datasets.** Subject-matter experts build prompt-plus-response sets. Score accuracy, speed, efficiency, scalability, and context retrieval against them. Repeatable and fast once built.

**LLM-as-a-judge.** A different model grades your system output. In Bedrock Evaluations you must supply a prompt dataset in JSONL format in S3. The dataset holds the prompts plus the judging instructions. The judge reads your input and output and returns scores.

**Hybrid.** Combine human and benchmark approaches. Comparing the two reveals the limits of each.



Your system input and output meet the judge model. The JSONL dataset in S3 defines good. The judge returns scores per dimension.

*Your system input and output meet the judge model. The JSONL dataset in S3 defines good. The judge returns scores per dimension.* Source: original figure for the AWS AIP-C01 Study Guide.

How to read this diagram

1. Left box: your system input and output.
2. Middle box: the judge model, a different FM.
3. Bottom box: the JSONL prompt dataset in S3. It feeds the judge.
4. Right box: scores per dimension.

**Exam relevance.** "LLM-as-a-judge in Bedrock" always needs the JSONL prompt dataset in S3. "Least subjective at scale" points at benchmarks. "Catches nuance" points at humans.

## ROUGE, BLEU, BERTScore: recall, precision, meaning

| Metric    | Measures            | How                                             |
|-----------|---------------------|-------------------------------------------------|
| ROUGE     | Recall              | How much of the reference appears in the output |
| BLEU      | Precision           | How much of the output appears in the reference |
| BERTScore | Semantic similarity | Compares embeddings, so synonyms do not fool it |

**Exam relevance.** "Did the summary miss reference content" is ROUGE (recall). "Did the output add unsupported words" is BLEU (precision). "Same meaning, different words" is BERTScore.

### More D5 decision rules

- If the stem says "objective check with known answers," pick automatic evaluation. It needs labels. It is cheapest.
- If the stem says "no labels, human-like quality at scale," pick LLM-as-a-judge. Define the metric in the rubric.
- If the stem says "subjective judgment," pick human evaluation on samples only. Never use humans for everything.
- If the stem says "hallucination metric," pick faithfulness. It checks that the source supports every claim.
- If the stem says "answer stays right with typos," pick stability (robustness).
- If the stem says "RAG answers are wrong," split the eval: retrieval quality first, faithfulness second.
- If the stem says "compare two models fast and cheap," climb the ladder: automatic, then judge, then humans on flagged samples.
- If the stem says "zero user risk on production data," pick shadow deployment.
- If the stem says "fair comparison on live traffic," pick A/B testing.
- If the stem says "promote to production," require the CI/CD eval gate. No gate, no promotion.
- If the stem says "production failure," feed it back into the golden dataset. The loop never stops.

### More D5 traps, with why the wrong answer fails

- Human evaluation for everything. Humans cost the most. Use them on samples only. Climb the ladder.
- LLM-as-a-judge with no rubric. The judge needs a rubric prompt. Without it the scores mean nothing.
- Same model as candidate and judge. The judge must be separate. A model grading itself is biased.
- Provider benchmarks for your model choice. Benchmarks do not measure your data. Only your golden dataset counts.
- Automatic metrics with no labels. Automatic evaluation compares against known answers. No labels means no automatic eval.
- A/B testing for zero-risk validation. A/B splits real users. Zero risk means shadow.
- Skipping the eval gate. The gate is the promotion rule. No gate, no production.
- Exact-match accuracy for summarization. Summaries have many right answers. Exact match is the wrong metric. Use judge-scored faithfulness.

- One eval, then done. Prompts and models change. The loop runs forever. Feed failures back.
- stopReason end\_turn means "check the answer." Wrong. end\_turn means the run finished. Trust the answer.

### D5 self-test: more questions with answers

D5 · MEDIUM · ONE ANSWER CC72. A team must score 10,000 summaries for faithfulness. They have no labeled data and a small budget. Which method do they use? A) Human reviewers on all 10,000. Humans on everything is the most expensive choice. B) LLM-as-a-judge with a faithfulness rubric. C) Automatic exact-match metrics. Summaries have many right answers. Exact match fails. It also needs labels. D) Provider benchmark scores. Benchmarks do not measure your summaries. Reveal answer Why B is correct: "No labels, human-like quality at scale" is the judge sentence. The judge needs no labels. The rubric defines faithfulness. Humans would cost too much.

D5 · EASY · ONE ANSWER CC73. A RAG system gives fluent answers that cite the wrong chunks. Where is the failure? A) Retrieval. The chunks are wrong, so retrieval failed first. B) Generation faithfulness. The answer left its chunks. But the chunks were wrong first. C) The chunking strategy. This is the fix, not the failure location. D) The prompt template. The prompt did not pick the chunks. Reveal answer Why A is correct: Split the RAG eval. Retrieval quality asks if the right chunks came back. Wrong chunks mean retrieval failed. Fix chunking and embeddings first.

D5 · MEDIUM · ONE ANSWER CC74. A new prompt must face production traffic with zero user risk. Which rollout do you use? A) A/B test. A/B splits real users. Users face risk. B) Shadow deployment. C) Canary to 50% of traffic. Half the users face the new prompt. D) Direct rollout with monitoring. Monitoring watches the damage. It does not prevent it. Reveal answer Why B is correct: Shadow sends invisible copies of real requests to the new version. Users never see it. Zero user risk means shadow.

D5 · EASY · ONE ANSWER CC75. A model update must not ship until it beats the current prompt on the golden dataset. What enforces this? A) Manual review. Manual review does not scale and has no gate. B) The CI/CD eval gate. C) A dashboard. Dashboards show numbers. They do not block promotion. D) Developer judgment. Judgment is not a gate. Reveal answer Why B is correct: The candidate promotes to production only through the eval gate. The gate compares against the golden dataset. No gate, no promotion.

D5 · MEDIUM · ONE ANSWER CC76. After a model swap, users report "the answers feel different but not wrong." Which check catches this first? A) Automatic exact-match. Exact match misses style shifts. B) LLM-as-a-judge on style and tone. C) Human review of all outputs. Too expensive for a first check. D) Toxicity metrics. The answers are not toxic. Reveal answer Why B is correct: Style and tone is a named judge metric. The judge scores samples against the rubric. It catches drift that exact match cannot see.

### D5 decision rules: if the stem says X, pick Y

- "Compare models cheaply at scale" means automatic metrics. Then, use LLM-as-judge on the model that remains. Use humans on flagged samples. Never use humans on everything under a budget.
- "Needs ground truth" means automatic evaluation. "No labels available" means LLM-as-judge.
- "Prove RAG does not hallucinate" means retrieval relevance plus generation faithfulness.
- "Critical flagged interactions" means A2I human review loop. Not all traffic.
- "Something broke" means debug the most recent change first. Use change-point debugging.
- "AccessDenied on InvokeModel" means model availability in the region before IAM.

- "Test without production traffic" means evaluation jobs with a golden dataset. Use A/B with two aliases. Use shadow traffic through inference profiles.
- "Deploy only if quality holds" means CI/CD evaluation gate with a score threshold.

## D5 trap patterns

- Humans read all data. This costs money. The cost increases with each step.
- Use provider benchmarks to choose a model. Market the model until your own evaluation runs.
- Debug IAM for a bad provider body. ValidationException shows the request shape.
- Debug generation when retrieval changed. Debug the difference.
- `max_tokens` stopReason means the process is complete. It means to cut the text.
- Deploy a new prompt or model version. Do not use an evaluation gate. Use a score threshold or do not deploy.
- Use A2I on all traffic. Flag only some samples.

## D5 rapid-fire questions

### D5 · MEDIUM · ONE ANSWER

CC49. You have two days to compare two models. You have a small budget. Which evaluation order do you use?

A) Humans read all outputs from both models

Humans reading all data is the most expensive choice. It uses too much money and time.

**B) First, use automatic metrics. Then, use LLM-as-judge on the remaining model. Last, use humans on flagged samples.**

C) Provider benchmark scores

Benchmarks do not measure your data. They do not measure your prompts. They do not measure your quality level.

**Why B is correct:** The cost increases with each step. Automatic evaluation is the cheapest. It needs labels. The judge does not need labels. Humans are the last choice for samples.

**D5 · MEDIUM · ONE ANSWER**

**CC50. "Show that the RAG system does not make errors." Which evaluation do you use?**

**A) Measure retrieval relevance. Then, measure generation faithfulness. Use LLM-as-judge.**

**B) A bigger generation model**

A bigger model does not show anything. Proof is measurement. The system must retrieve the correct parts. The answer must use these parts.

**C) Automatic accuracy on a labeled set**

Accuracy needs correct labels for each prompt. Faithfulness directly measures support from the retrieved parts.

**Why A is correct:** This is the RAG evaluation split. Retrieval relevance asks, "Are the parts correct?" Faithfulness asks, "Does the answer use the parts?" Together, they show no errors.

**D5 · EASY · ONE ANSWER**

**CC51. An InvokeModel call returns ValidationException. Is it IAM or the request shape?**

**A) IAM: check the role policy**

Validation errors are about the request body. They are not about permissions. Permission problems return access errors.

**B) Request shape: the provider body is bad. For Claude, it lacks anthropic\_version or max\_tokens.**

**Why B is correct:** Read the exception name. ValidationException means the body did not pass validation. The common cause is a Claude body that does not have its required fields.

**D5 · EASY · ONE ANSWER**

**CC52. The evaluator asks which model is better for YOUR prompts. Do you use a provider benchmark or your own evaluation job?**

**A) Provider benchmark**

Benchmarks do not use your data. Benchmarks do not use your prompts. Benchmarks do not use your cost. Benchmarks do not use your latency. Benchmarks do not use your regional availability. This is not a complete answer.

**B) Your own evaluation job on your golden dataset**

**Why B is correct:** This is the capability-floor filter. Only your prompts against your quality bar decide your model.

**D5 · MEDIUM · ONE ANSWER**

**CC53. The model does not use a new prompt template at runtime. What are the two possible problems?**

**A) The Prompt Management version was not published. Or, AppConfig serves a cached value.**

**B) The embedding model drifted.**

Embeddings are for retrieval. A prompt template problem is a version problem. Or, it is a configuration problem.

**Why A is correct:** Use change-point debugging for configuration. A new template goes missing in two places. It can be a draft version. Or, it can be a published version. It can also be a cached runtime configuration.

**D5 · MEDIUM · ONE ANSWER**

**CC54. "Detect quality drift in production with minimal custom work." Which design is best?**

**A) A hand-built Glue and Athena pipeline over raw logs**

This is custom work. Managed pieces exist. "Minimal custom work" means you do not build it yourself.

**B) Schedule Bedrock evaluation jobs against the golden dataset. Add CloudWatch anomaly detection on token metrics.**

**Why B is correct:** Periodic evaluation jobs check quality against the golden set. CloudWatch checks the quantitative signals. AWS manages it. It runs on a schedule. It needs minimal code.

**D5 · EASY · RECALL**

**CC55. Which evaluation needs ground-truth labels? Which evaluation works without labels?**

**Answer:** Automatic (programmatic) evaluation needs labels. It needs labels for accuracy. It needs labels for stability. It needs labels for toxicity. LLM-as-a-judge works without labels. It scores correctness.

It scores faithfulness. It scores fluency. It scores more.

**D5 · EASY · RECALL**

**CC56. A new model version is ready. What stops it from going to production?**

**Answer:** The CI/CD evaluation gate stops it. The candidate runs against the golden dataset. It deploys only if it is above the score threshold. A/B testing with two aliases or shadow traffic finds what the gate misses.

**D5 · MEDIUM · ONE ANSWER**

**CC57. Throttling errors increase with sustained traffic at peak. Should you retry with backoff? Or, should you use Provisioned Throughput?**

A) Retry with backoff only

Backoff handles temporary increases. Sustained throttling at peak means there is a capacity problem.

B) Use Provisioned Throughput for the steady baseline. Backoff stays for temporary increases.

**Why B is correct:** Sustained throttling on steady traffic is a Provisioned Throughput problem. It gives dedicated capacity. It has no throttling from shared quotas. Temporary increases still use backoff.

**D5 · EASY · RECALL**

**CC58. State the change-point debugging rule in one sentence.**

**Answer:** The broken thing is what changed most recently. So, debug the difference first.

**Next:** This is the numbers sheet. It has every hard number from all five domains on one table.

**Production GenAI: the Application Builder**

Production systems are much more than Bedrock. The Generative AI Application Builder on AWS is the reference solution. It ships example architectures for production GenAI: ingestion pipelines, knowledge bases, agents, guardrails, observability, and frontends wired together.

**Not in the repo volumes.** The Application Builder architectures are not covered in the repo volumes. The study hub points at the AWS Solutions Library page for the architecture diagrams. Review one end-to-end diagram before the exam so "which component is missing" questions feel familiar.

**Exam relevance.** "Production RAG system, name the missing piece" expects the full architecture vocabulary: S3 eventing, ingestion, vector store, guardrails, CloudWatch, API Gateway. No single service is the whole answer.

**The GenAI Lens review**

The AWS Well-Architected Framework has a Generative AI Lens. It is a published checklist for reviewing GenAI workloads, built on the six Well-Architected pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability. The Lens adds GenAI concerns like responsible AI, data governance, model selection, and evaluation gates.

**The lifecycle.** Build a small proof of concept on Bedrock first. Review it against the GenAI Lens. Extract reusable components: prompt templates, model routing layers, guardrail policies, evaluation harnesses. Then go to production. A PoC on on-demand inference costs almost nothing and answers three questions: does it work, is it fast enough, is it worth it.

**Trap: "deploy the GenAI Lens."** Wrong. The Lens is a review checklist, not a product. The exam uses "deploy the Lens" as a nonsense distractor.

**Exam relevance.** "Design review standard for GenAI" names the GenAI Lens. "Validate before committing to Provisioned Throughput" names the PoC-first lifecycle.

## Appendix services: the five the volumes skip

---

The main volumes teach Bedrock in depth. Five services live at the edges of a GenAI pipeline. Two move data in: AppFlow and Transfer Family. Two analyze it: QuickSight and Neptune Analytics. One turns a good conversation into a reusable tool: Q Apps.

The exam tests each with one signature scenario. Learn the one-line trigger for each service. This section gives you the decision rules, the traps, the numbers, and a worked curveball per service.

### Amazon AppFlow: SaaS data in, no code

Amazon AppFlow is a fully managed integration service. It moves data between SaaS applications and AWS services. It moves data in both directions. You write no code. You build no custom connectors.

SaaS means software you use over the internet. Examples are Salesforce, ServiceNow, Zendesk, and Workday.

GenAI needs enterprise data. That data lives in SaaS tools. Support tickets live in Zendesk. Cases live in Salesforce. Employee records live in Workday.

Without AppFlow, engineers write API glue scripts. Those scripts break when the vendor changes pagination or rate limits. AppFlow replaces the glue with a managed flow. Credentials live in one central place. They never sit in code.

A flow runs in four steps. First, a connector profile authenticates once to the SaaS app. It uses OAuth or stored credentials. AppFlow keeps the credentials in Secrets Manager.

Second, a flow definition names the source, the destination, and the field mappings. It names transforms: masking, truncation, concatenation. It names filters: only records that match a condition sync.

Third, a trigger runs the flow. Triggers are on demand, scheduled, or event-driven. A SaaS change event can start a near-real-time sync. Fourth, incremental pulls fetch only records changed since the last run. There are no full snapshots.

Common destinations are S3, Redshift, and Salesforce itself. The data lands ready for Glue ETL or a Knowledge Base ingestion job. Flows integrate with the Glue Data Catalog. The synced data is queryable.

### Decision rules: if the stem says X, pick AppFlow.

- If the stem says SaaS data into AWS (Salesforce, Zendesk, Workday, ServiceNow), pick AppFlow.
- If the stem says field mapping or masking PII during the sync, pick AppFlow. EventBridge cannot map fields.
- If the stem says only changed records, or asks for the cheapest sync of 10 million daily records, pick incremental pulls.
- If the stem says hourly or daily sync with no streaming need, pick a scheduled AppFlow flow. Kinesis is overkill.
- If the stem says least operational overhead for a SaaS sync, pick AppFlow over a Lambda poller. The poller re-implements pagination, retry, and credential handling.
- If the stem says SaaS traffic must avoid the public internet, pick AppFlow with PrivateLink.

**Traps.**

- EventBridge routes events. It does not map fields. It does not mask data. It does not do incremental record pulls.
- A Lambda poller works, but it is not the least overhead. It re-implements pagination, retry, and credentials.
- DMS replicates databases. It does not call SaaS APIs. It has no masking transform.
- Glue alone has no SaaS connector. It processes data after AppFlow lands it.
- Kinesis is a streaming service. An hourly sync does not need it.

**Numbers to memorize.**

- Pricing counts flow runs plus data processed: about one dollar per flow run (illustrative). It is not per seat. The exam does not test exact pricing.
- A scheduled flow that syncs 10 million records daily costs about the same whether the records changed or not. Incremental pulls are the cost answer.

**FIGURE 1.1. APPFLOW MOVES SAAS DATA INTO THE LAKE**

A flow is the recipe: source, destination, schedule, preparation steps

Credentials for the SaaS connection live in Secrets Manager, not in the flow.

Exam angle: 'Salesforce data into S3 for a Knowledge Base, no code' → AppFlow.

Trap: it is not streaming (that is Kinesis) and not file transfer (that is Transfer Family).

Why here: the main volumes mention it once; exam scenarios name the SaaS source.

AppFlow syncs SaaS data into S3, where it becomes RAG fuel. The flow handles auth, mapping, masking, and incremental sync. Downstream, Glue or a Knowledge Base treats the S3 prefix as a normal data source.

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this diagram**

1. **Left boxes are SaaS sources.** Data starts in systems AWS does not own. AppFlow owns the arrow from left to middle.
2. **The middle box does four jobs:** authenticate with Secrets Manager, map fields, transform or mask, and pull incrementally. Each is a separate exam concept.
3. **The S3 box is the handoff.** After this point, the normal GenAI pipeline (Glue, BDA, Knowledge Bases) takes over. AppFlow is only the ingestion story.
4. **What breaks:** a rotated SaaS credential fails the flow at step one. A wrong field mapping shows up downstream as nulls, not errors. Debug the flow first, not the RAG answers.

**CB15. The flow that billed for unchanged records.**

**Scenario.** A team syncs 10 million Salesforce records to S3 daily with a scheduled AppFlow flow. Only a few thousand records change each day. The bill stays high. A teammate blames the data volume. What is wrong, and what is the fix?

**Worked answer.** The flow pulls full snapshots. A scheduled flow costs about the same compute whether records changed or not. The fix is incremental pulls: sync only records changed since the last run. The bill then tracks changed records, not total records.

The teammate blamed volume. The real problem was the pull mode.

**The trap.** "Scheduled" sounds cheap. Cost follows the pull mode, not the schedule. Incremental is the cost answer.

**APPENDIX · EASY · RECALL**

**CC59. A company wants Zendesk support tickets in S3 each hour. It wants only new tickets. It wants customer emails masked. The tickets feed a RAG knowledge base. Which service meets the need with the least operational overhead?**

- A) EventBridge receives Zendesk change events and routes them to S3.  
EventBridge routes events. It does not map fields or mask data.
- B) AppFlow with a scheduled flow, incremental pull, and a masking transformation.**
- C) A Lambda function polls the Zendesk API hourly and writes to S3.  
This re-implements pagination, retry, and credential handling. It is not the least overhead.
- D) Kinesis Data Streams ingests the tickets in real time.  
Streaming infrastructure is overkill for an hourly sync.

**Why B is correct:** AppFlow supports SaaS sources, scheduled triggers, incremental pulls, and field-level masking. One managed flow meets all four requirements.

**The trap:** EventBridge. It sounds event-driven and modern, but it cannot map or mask.

**APPENDIX · EASY · RECALL****CC60. Where does an AppFlow flow keep the SaaS credentials?**

- A) In the flow definition JSON.  
The flow never embeds secrets.
- B) In Secrets Manager, through a connector profile.**
- C) In Lambda environment variables.  
There is no Lambda in an AppFlow flow.
- D) In the S3 destination bucket.  
Credentials never land in the data bucket.

**Why B is correct:** The connector profile authenticates once. AppFlow stores the credentials in Secrets Manager. The flow definition references the profile, never the secret.

**The trap:** any option that puts secrets in code or config. Central storage is the point.

**APPENDIX · MEDIUM · RECALL****CC61. A flow syncs Salesforce cases to Redshift. The SaaS traffic must not cross the public internet. Which option meets the requirement?**

- A) Use a NAT gateway for the flow.  
A NAT gateway still uses the public internet path.
- B) Use AppFlow with a PrivateLink connection.**
- C) Use DMS with a VPC endpoint.  
DMS replicates databases, not SaaS APIs.
- D) Encrypt the fields with KMS.  
Encryption protects data at rest. It does not change the network path.

**Why B is correct:** AppFlow supports private connections through PrivateLink. SaaS traffic never crosses the public internet.

**The trap:** encryption as a network answer. KMS and PrivateLink solve different problems.

**AWS Transfer Family: partner file drops, no servers**

AWS Transfer Family is a fully managed file-transfer service. It runs SFTP, FTPS, plain FTP, and AS2 servers. S3 or EFS backs the servers. Enterprises exchange data as file drops. A partner uploads a nightly CSV over SFTP. A vendor sends invoices over AS2.

Before Transfer Family, teams ran their own SFTP servers on EC2. They handled patching, scaling, and key management. Then they copied files into S3. Transfer Family removes the server fleet. AWS manages the protocol endpoint. S3 is the storage.

A file drop is one hop from a GenAI ingestion pipeline. Partners keep using their existing clients.

Setup runs in three steps. First, create a Transfer Family server for a protocol, for example SFTP. Second, point it at an S3 bucket or an EFS file system. Third, add users. Users authenticate with SSH keys, passwords, or an external identity provider. Each user maps to a home directory inside the bucket.

Uploads and downloads go straight to S3 objects. An S3 event notification then triggers the next step: Lambda validation, Bedrock Data Automation, or a Knowledge Base ingestion job.

One rule matters most. Plain FTP works only on VPC-internal endpoints. It must not cross the public internet. SFTP, FTPS, and AS2 work on public and VPC-internal endpoints.

**Decision rules: if the stem says X, pick Transfer Family.**

- If the stem says partners upload files with their existing SFTP clients, pick Transfer Family. Presigned URLs do not work here.
- If the stem says plain FTP and the client cannot change, pick a Transfer Family FTP server on a VPC-internal endpoint.
- If the stem says nightly CSV over SFTP or vendor invoices over AS2, pick Transfer Family with an S3 backend.
- If the stem says each upload must start a Lambda or an ingestion job, pick Transfer Family plus S3 event notifications.
- If the stem says no server management for file exchange, pick Transfer Family over an EC2-hosted server.

**Traps.**

- S3 presigned URLs work for your own applications. They do not work for partners. Partners only speak SFTP. They will not rewrite their tooling.
- DataSync moves data with installed agents. It is not a protocol server.
- AppFlow connects to SaaS APIs. It does not speak file-transfer protocols.
- An EC2-hosted FTP server adds patching and scaling work. Transfer Family removes it.
- Plain FTP on a public endpoint is an invalid combination. FTP stays VPC-internal.

**Numbers to memorize.**

- Pricing is per protocol-enabled endpoint per hour, plus data transferred (illustrative). The exam-relevant number is not the price. It is the protocol matrix below.
- The protocol matrix: SFTP yes on public, yes on VPC. FTPS yes on public, yes on VPC. Plain FTP no on public, yes on VPC-internal. AS2 yes on public, yes on VPC.

**PROTOCOL MATRIX: WHICH ENDPOINT SPEAKS WHICH PROTOCOL**

One rule matters: plain FTP never crosses the public internet.

|                                                                                                                                                                                 |                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <p><b>SFTP</b></p> <p>SSH File Transfer Protocol</p> <p>Public endpoint: yes. VPC-internal endpoint: yes.</p>                                                                   | <p><b>FTPS</b></p> <p>FTP over TLS/SSL</p> <p>Public endpoint: yes. VPC-internal endpoint: yes.</p>      |
| <p><b>FTP (plain)</b></p> <p>Unencrypted file transfer</p> <p>Public endpoint: NO. VPC-internal endpoint: yes.</p> <p>Unencrypted FTP must never cross the public internet.</p> | <p><b>AS2</b></p> <p>B2B messaging protocol</p> <p>Public endpoint: yes. VPC-internal endpoint: yes.</p> |

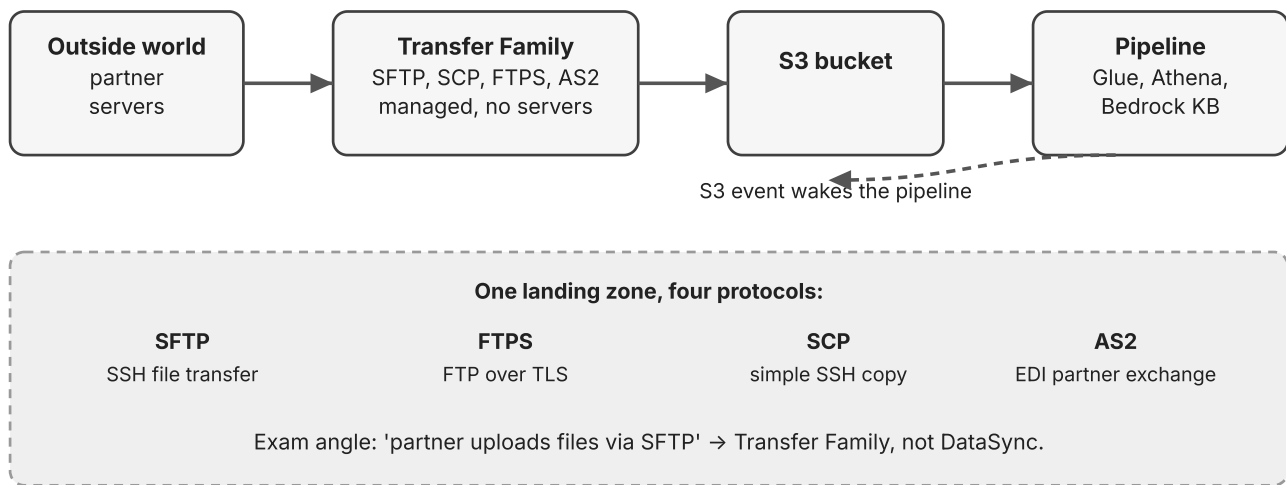
The exam asks exactly one question here: which protocol is VPC-internal only? Plain FTP.

Transfer Family protocols by endpoint. SFTP, FTPS, and AS2 work on public and VPC-internal endpoints. Plain FTP works only on VPC-internal endpoints.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Read each row left to right: the protocol, then the public endpoint, then the VPC-internal endpoint.
2. SFTP, FTPS, and AS2: both endpoints work.
3. Plain FTP: only the VPC-internal endpoint works.

**FIGURE 1.2. TRANSFER FAMILY CATCHES PARTNER FILE DROPS**

Transfer Family terminates the legacy protocol and stores objects in S3. From there the standard GenAI ingestion pipeline takes over. The partner never learns anything changed.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

- Nothing changes for the partner.** There is no client migration. There is no retraining. This is the whole value proposition.
- AWS manages the server.** There is no EC2 fleet. There is no patching. There is no key server to operate. The protocol ends here.
- S3 is the real destination.** The file becomes an object. Everything downstream (events, Lambda, BDA) uses normal S3 mechanics.
- What breaks:** a user mapped to the wrong home directory writes to the wrong prefix. Ingestion then processes stale or empty data. When drops "stop arriving," check the user-to-directory mapping first.

### CB16. The FTP server the public internet could reach.

**Scenario.** A supplier sends inventory CSVs over plain FTP. The supplier cannot change the client. An architect deploys a Transfer Family FTP server on a public endpoint with an S3 backend. The deployment fails validation. What is wrong, and what is the fix?

**Worked answer.** Plain FTP works only on VPC-internal endpoints. A public endpoint cannot serve plain FTP. The fix is a VPC-internal endpoint. The FTP server lives inside the private VPC. Files still land in S3, and S3 event notifications still start the Lambda validation.

The protocol did not change. The endpoint type did.

**The trap.** "FTP server" sounds public by default. The exam tests the one protocol with an endpoint restriction. FTP stays internal.

**APPENDIX · EASY · RECALL**

**CC62. A healthcare partner must upload claim files each night. The partner uses an existing SFTP client. Files must go to S3 and start document processing. Which solution works?**

- A) Transfer Family SFTP server with an S3 backend and S3 event notifications.**
- B) S3 presigned URLs for the partner uploads.**  
Presigned URLs work for your own applications, not for partners who only speak SFTP.
- C) AWS DataSync between the partner and S3.**  
DataSync moves data with installed agents. It is not a protocol server.
- D) Amazon AppFlow with an SFTP connector.**  
AppFlow connects to SaaS APIs, not file-transfer protocols.

**Why A is correct:** Transfer Family terminates SFTP and stores objects in S3. S3 event notifications chain into document processing. The partner changes nothing.

**The trap:** "existing SFTP client" is the trigger phrase. It names Transfer Family every time.

**APPENDIX · MEDIUM · RECALL**

**CC63. A supplier sends inventory CSVs over plain FTP and cannot change the client. Files must land in S3 inside a private VPC. Each upload must start a Lambda function for validation. Which architecture works?**

- A) Transfer Family FTP server on a VPC-internal endpoint, S3 backend, S3 event notifications to Lambda.**
- B) Transfer Family FTP server on a public endpoint with an S3 backend.**  
Public endpoints do not support plain FTP.
- C) An EC2-hosted FTP server with a cron job that copies files to S3.**  
This adds server management. Transfer Family removes it.
- D) Amazon AppFlow with an FTP connector writing to S3.**  
AppFlow connects to SaaS APIs. It does not speak file-transfer protocols.

**Why A is correct:** Plain FTP works only on VPC-internal endpoints. The S3 backend stores the files. S3 events start Lambda. All three constraints hold.

**The trap:** option B looks identical except the endpoint. The endpoint is the whole question.

**APPENDIX · EASY · RECALL****CC64. Which protocols work on a Transfer Family public endpoint?**

- A) SFTP only.  
FTPS and AS2 also work on public endpoints.
- B) SFTP, FTPS, and AS2.**
- C) SFTP, FTPS, FTP, and AS2.  
Plain FTP works only on VPC-internal endpoints.
- D) FTP and AS2 only.  
FTP is the one protocol excluded from public endpoints.

**Why B is correct:** The protocol matrix allows SFTP, FTPS, and AS2 on public endpoints. Plain FTP is VPC-internal only.

**The trap:** memorizing "SFTP only" for public. Three protocols work. Only FTP has this restriction.

**Amazon QuickSight: dashboards over GenAI operations**

Amazon QuickSight is a serverless business-intelligence service. It provides interactive dashboards, paginated reports, and embedded analytics. You manage no servers. Its in-memory engine is SPICE. SPICE imports data and answers dashboard queries from memory. It does not hit the source database for each click.

GenAI applications make data worth watching: token spend per team, evaluation scores over time, RAG retrieval hit rates, user feedback. Engineers query that data in Athena or OpenSearch. Product owners and finance need dashboards. QuickSight turns S3, Athena, Redshift, and RDS data into shareable dashboards.

The flow has five steps. Connect a data source: S3, Athena, Redshift, RDS, or a SaaS app through AppFlow. Query it directly, or import it into SPICE for fast repeated queries. Build analyses with drag-and-drop visuals. Publish them as dashboards. Add ML insights: anomaly detection flags unusual spend spikes, forecasting projects next month's token usage, and natural-language Q answers typed questions with charts.

QuickSight controls who sees what. Row-level security shows each viewer only their rows. Column-level security hides columns.

Editions matter. Standard is the basic tier. Enterprise adds ML insights, row-level and column-level security, and encryption at rest. A scenario that mentions anomaly detection or per-team data isolation points to Enterprise.

**Decision rules: if the stem says X, pick QuickSight.**

- If the stem says a dashboard of monthly Bedrock spend per team from CUR data in S3, pick QuickSight.
- If the stem says each team must see only its own rows, pick QuickSight Enterprise for row-level security.
- If the stem says automatic alerts for unusual spikes, pick QuickSight Enterprise for anomaly detection.
- If the stem says near-real-time dashboard, pick direct query mode. SPICE import refreshes on a schedule.
- If the stem says log analytics over data already in OpenSearch, pick OpenSearch Dashboards, not QuickSight.

- If the stem says governed sharing of business data from many sources, pick QuickSight over CloudWatch dashboards.

### Traps.

- OpenSearch Dashboards shows data already in OpenSearch: logs and traces. QuickSight combines business data from many sources with governed sharing.
- CloudWatch dashboards have no row-level security and no governed multi-team sharing.
- A stale SPICE refresh schedule makes the dashboard confidently wrong. When numbers look wrong, check the dataset refresh time before you question the pipeline.
- Athena saved queries shared by email are not interactive dashboards. They have no access control.

### Numbers to memorize.

- SPICE datasets refresh on a schedule you set: as frequent as every 15 minutes on Enterprise.
- SPICE capacity sells in GB blocks (illustrative, tens of cents per GB-month). On Enterprise, per-reader pricing counts sessions. The exam does not test exact figures.

#### SPICE IMPORT VERSUS DIRECT QUERY

Speed versus freshness. The scenario names the winner.

##### SPICE import

In-memory engine

Import once, answer dashboard clicks from memory. Fast repeated queries.

Refresh on a schedule: as frequent as every 15 minutes on Enterprise.

##### Direct query

Hit the source live

Every dashboard click queries the source. Slower, but minutes fresh.

"Near real-time dashboard" means direct query, not SPICE import.

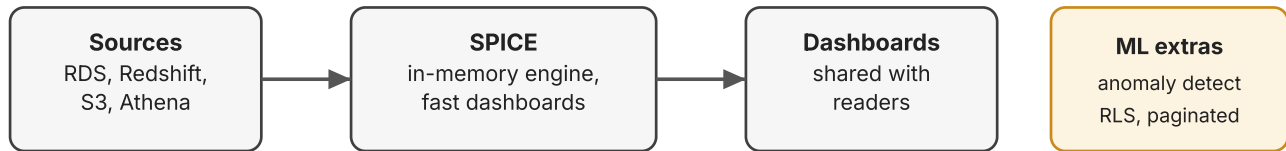
Stale refresh schedule is the classic bug: the dashboard looks confidently wrong.

SPICE imports data and answers from memory. It refreshes on a schedule. Direct query is slower but always fresh.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Left path: SPICE. It imports data. Queries run from memory.
2. Right path: direct query. Each query reads the source. It is slower but fresh.
3. Bottom: speed trades against freshness.

**FIGURE 2.1. QUICKSIGHT SERVES DASHBOARDS FROM SPICE**

SPICE is the in-memory cache: dashboards read SPICE, not the source database.

Exam angle: 'executives need dashboards over the data lake' → QuickSight + SPICE.

Trap: anomaly detection is built-in ML, not a SageMaker project you build.

RLS (row-level security) limits which rows each reader sees.

Why here: BI-adjacent; the main volumes cover analytics storage, not the dashboard layer.

QuickSight aggregates GenAI operational data from several sources into one governed dashboard. SPICE keeps repeated views fast. ML insights watch for anomalies automatically.

Source: original figure for the AWS AIP-C01 Study Guide.

#### How to read this diagram

- Left: the raw sources.** Billing data, evaluation logs, usage tables. None of them looks like a dashboard.
- Middle: QuickSight plus SPICE.** Import once, query from memory many times. Direct query is the alternative when data must be minutes fresh.
- Right: the outputs.** Dashboards for humans. Anomaly alerts for operations. Row-level security for governance.
- What breaks:** a stale SPICE refresh schedule makes the dashboard confidently wrong. If numbers look wrong, check the dataset refresh time first.

#### CB17. The dashboard that was confidently wrong.

**Scenario.** Finance watches a QuickSight dashboard of Bedrock token spend per department. The numbers disagree with the CUR data in S3. The pipeline team re-checks the ingestion jobs. Everything looks correct. What do you check next?

**Worked answer.** Check the SPICE dataset refresh schedule. SPICE answers from imported memory, not from live S3. A stale refresh makes the dashboard confidently wrong while every upstream job is healthy. The fix is a more frequent refresh, or direct query mode if the dashboard must be near real time. The pipeline was never broken.

The dashboard showed yesterday's data.

**The trap.** Debugging the pipeline first. When a SPICE dashboard disagrees with its source, the refresh schedule is suspect number one.

**APPENDIX · EASY · RECALL**

**CC65. Finance needs a monthly dashboard of Bedrock token spend per department. Each department must see only its own data. The system must alert on unusual spikes. Which solution is correct?**

- A) OpenSearch Dashboards over the S3 billing data.  
OpenSearch Dashboards shows OpenSearch indices. It does not do governed sharing of S3 billing data.
- B) QuickSight Enterprise with SPICE, row-level security, and ML insights.**
- C) CloudWatch dashboards with metric math.  
CloudWatch dashboards have no row-level security and no business-user exploration.
- D) Athena saved queries shared by email.  
Saved queries are not interactive dashboards. They have no access control.

**Why B is correct:** QuickSight Enterprise provides interactive dashboards over S3 data, row-level security for per-team isolation, and anomaly detection for spike alerts.

**The trap:** two Enterprise features in one stem. Row-level security plus anomaly detection names the edition.

**APPENDIX · MEDIUM · RECALL**

**CC66. A team needs a near-real-time dashboard over Athena data. Which QuickSight mode do you choose?**

- A) SPICE import with the default schedule.  
SPICE refreshes on a schedule. The default is not near real time.
- B) Direct query mode.**
- C) SPICE import with Enterprise edition.  
Enterprise allows 15-minute refreshes. Near real time still needs direct query.
- D) Paginated reports on a schedule.  
Reports are snapshots, not a live dashboard.

**Why B is correct:** Direct query reads the source on each query. It is slower than SPICE but always fresh. "Near real time" rules out any import schedule.

**The trap:** "Enterprise refreshes every 15 minutes" sounds fresh enough. Near real time means direct query.

**APPENDIX · EASY · RECALL****CC67. Which two features require QuickSight Enterprise rather than Standard?**

A) Interactive dashboards and paginated reports.

Both editions provide these.

**B) ML insights (anomaly detection, forecasting) and row-level security.**

C) SPICE import and direct query.

Both editions support both query modes.

D) Embedded analytics and S3 data sources.

These are not the edition differentiators.

**Why B is correct:** Enterprise adds ML insights, natural-language Q, row-level and column-level security, and encryption at rest. Standard is the basic tier.

**The trap:** any scenario with anomaly detection or per-team isolation. Both point to Enterprise.

**Neptune Analytics: graph analytics plus vector search**

Neptune Analytics is a serverless graph-analytics engine. It is part of the Amazon Neptune family. It loads graph data into memory and runs analytical queries on it, including vector similarity search. Do not confuse it with Neptune Database. Neptune Database is a transactional graph database. Neptune Analytics is an analytics engine.

Pick by workload, not by name. "Find the 50 most similar incident reports to this one" is analytical. "Detect communities of related entities" is analytical. These need bulk graph computation and vector search over millions of nodes. That is Neptune Analytics. "Transactional graph with ACID writes" is Neptune Database.

To use it, create a graph and load data from S3 or from a Neptune Database snapshot. The graph holds nodes, edges, and vector properties. Vector search uses HNSW, an approximate nearest-neighbor index. HNSW trades a tiny accuracy loss for large speedups on big vector sets. You query vectors with the `vectors.topK.byEmbedding` API: pass an embedding, ask for the top K most similar vectors.

Treat the older name `vectors.topKByEmbedding` as deprecated. The exam may show it as a distractor. The engine is serverless and in-memory. You pay for the graph's memory footprint while it runs, not for provisioned instances.

**Decision rules: if the stem says X, pick Neptune Analytics.**

- If the stem says in-memory analytics and vector similarity over a loaded graph, pick Neptune Analytics.
- If the stem says transactional graph with ACID writes, pick Neptune Database.
- If an option shows `vectors.topK.byEmbedding`, it is current. If it shows `vectors.topKByEmbedding`, it is the deprecated trap.
- If the stem says the embedding model changes dimension, plan to recreate the graph. Dimensions are immutable.
- If the stem says a vector store for RAG that also needs graph structure, consider Neptune Analytics.

**Traps.**

- The deprecated API name `topKByEmbedding` will appear as an answer choice. The current name is `topK.byEmbedding`.
- Neptune Database is the transactional engine. It is not the analytics vector API. Pick by workload shape, not by brand.
- OpenSearch Service does vector search but no graph analytics.
- HNSW tuning changes recall. It does not change dimension mutability.
- Switching embedding models (for example 768 dimensions to 1536) silently breaks vector search until you recreate the graph with the new dimension.

**Numbers to memorize.**

- Set vector dimensions at graph creation: maximum 65,000 dimensions. You cannot change them afterward.
- HNSW search is approximate: expect recall in the high 90s percent range, with millisecond queries over millions of vectors (illustrative; tune HNSW parameters for your recall target).

**NEPTUNE DATABASE VERSUS NEPTUNE ANALYTICS**

Same family, different jobs: transactions versus analytics.

**Neptune Database**

Transactional graph database

Fast single-record reads and writes. The GraphRAG home from D1.

Ask: "fetch this entity and its neighbors."

**Neptune Analytics**

Serverless graph-analytics engine

Loads the graph into memory for bulk computation and vector similarity search.

Ask: "find the 50 most similar incident reports."

**Do not confuse them.**

Transactional questions go to the database. Analytical and similarity questions go to Analytics.

Neptune Database serves transactional graph queries. Neptune Analytics serves bulk graph computation and vector search.

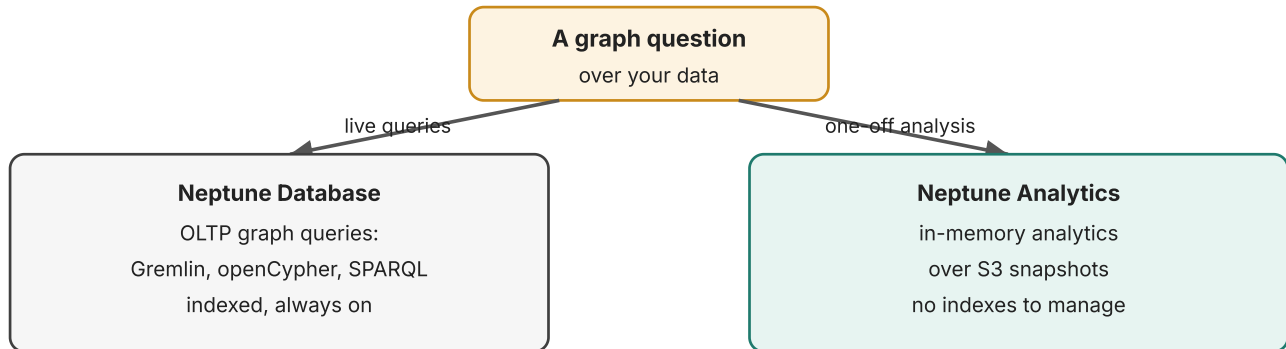
Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this diagram**

1. Left: Neptune Database. It answers live graph queries.

2. Right: Neptune Analytics. It runs bulk computation and vector search.
3. Bottom: pick by workload, not by name.

FIGURE 2.2. TWO GRAPH ENGINES FOR TWO JOBS



Exam angle: 'ad-hoc graph analytics over data in S3' → Neptune Analytics, not the database.

Trap: 'Analytics' in the name does not mean dashboards; it means in-memory graph compute.

Neptune Database serves transactional graph reads and writes. Neptune Analytics loads the graph into memory for analytical vector and traversal queries. Pick by workload shape, not by brand.

Source: original figure for the AWS AIP-C01 Study Guide.

How to read this diagram

1. **Left: the database.** Good for writes and point reads. This is the GraphRAG home.
2. **Arrow: load, do not replicate.** Analytics loads a graph snapshot into memory. It is not a live transactional replica.
3. **Right: the analytics engine.** HNSW gives fast approximate vector search. The API name and the immutable-dimension rule are the exam facts.
4. **What breaks:** you switch embedding models, for example from 768 dimensions to 1536. Vector search silently breaks. Recreate the graph with the new dimension. Check dimension mismatch first.

#### CB18. The embedding upgrade that broke search.

**Scenario.** A team runs nightly analytics on a 40-million-node knowledge graph with Neptune Analytics. They upgrade the embedding model from 768 to 1536 dimensions. Vector search returns garbage. No errors appear.

A teammate proposes tuning the HNSW recall parameter. What is wrong, and what is the fix?

**Worked answer.** Set vector dimensions at graph creation. They stay immutable afterward. The graph still expects 768-dimension vectors; the new model emits 1536.

HNSW tuning changes recall, not dimension mutability. The fix is to recreate the graph with 1536 dimensions and reload the data. Silent garbage with no errors is the signature of a dimension mismatch.

**The trap.** Tuning the index when the schema changed. HNSW parameters cannot fix a dimension mismatch. Only a new graph can.

#### APPENDIX · EASY · RECALL

CC68. An application needs approximate nearest-neighbor vector search over a large knowledge graph, queried with `vectors.topK.byEmbedding`. Which service is correct?

A) Neptune Analytics.

B) Neptune Database.

Neptune Database is the transactional engine. It is not the analytics vector API.

C) OpenSearch Service.

OpenSearch does vector search but no graph analytics.

D) Neptune Database with `vectors.topKByEmbedding`.

Two errors: wrong engine and the deprecated API name.

**Why A is correct:** "In-memory analytics and vector similarity over a loaded graph" is the Neptune Analytics workload. The API name confirms it.

**The trap:** the brand. "Neptune" appears in both options. The workload decides.

#### APPENDIX · MEDIUM · RECALL

CC69. A team changes its embedding model dimension next quarter and does not want to recreate the graph. They use `vectors.topKByEmbedding`. What must they change?

A) Nothing. The API call and the dimension plan are valid.

The API name counts as deprecated, and dimensions stay immutable.

B) Use `vectors.topK.byEmbedding`, and recreate the graph when the dimension changes.

C) Move to Neptune Database and keep the API name.

Neptune Database is transactional. It does not offer this vector API.

D) Increase the HNSW recall parameter to allow dimension changes.

HNSW tuning changes recall. It does not change dimension mutability.

**Why B is correct:** treat `topKByEmbedding` as deprecated and use `topK.byEmbedding`. Set vector dimensions at graph creation; they stay immutable, so a dimension change requires a new graph. There is no way around it.

**The trap:** the "do not want to recreate" wish. The exam tests whether you know the immutable rule beats the wish.

**APPENDIX · EASY · RECALL****CC70. What is the maximum vector dimension at Neptune Analytics graph creation?**

A) 1,536.

That is a common embedding size, not the graph limit.

B) 8,192.

Not in source.

C) 65,000.

D) There is no limit.

Set dimensions at creation. The maximum is 65,000.

**Why C is correct:** The maximum vector dimension at graph creation is 65,000. Set it at creation; it stays immutable afterward.

**The trap:** confusing a model output size (1,536) with the graph limit.

**Amazon Q Apps: no-code GenAI tools inside Q Business**

Amazon Q Apps are generative AI applications you build without code. You build them inside Amazon Q Business. A user describes what they want in natural language. Or the user starts from an existing Q Business conversation. Q makes an app from the description or the conversation.

An app is a chain of cards. Input cards take text or file uploads. Output cards show generated text. Action cards call plugins, for example creating a Jira ticket.

Cards refer to other cards with @ mentions. The output of one card becomes the input of the next card. You publish finished apps to the organization's app library. Anyone can use them.

Colleagues do not see or change the prompts.

Q Business gives every employee a chat assistant. But chat is temporary. You retype the same good prompt every week. Q Apps turn a useful conversation into a lasting tool. The marketing team builds a campaign-brief generator once. The whole team reuses it.

IT keeps control: Q Apps use Q Business governance. Admin guardrails apply automatically. Identity Center access control applies automatically. Connector permissions apply automatically.

Q Apps are part of the Q Business Pro tier. The license is per user per month. Do not memorize an infrastructure number. The exam tests the card model, the @ wiring, library sharing, and governance inheritance.

**Decision rules: if the stem says X, pick Q Apps.**

- If the stem says a team builds the tool themselves with no code and no developers, pick Q Apps.
- If the stem says a reusable leave-policy Q&A tool the HR team builds itself, pick Q Apps, not an agent.
- If the stem says coding assistant for developers, pick Q Developer.

- If the stem says developer-built autonomous agents with action groups and Lambda, pick Bedrock Agents.
- If the stem says the app must respect existing data permissions, pick Q Apps: connector permissions follow the user, not the app author.

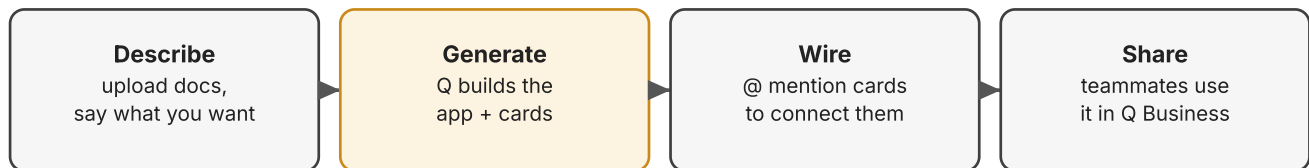
### Traps.

- Q Developer is the coding assistant for developers. Q Apps are for non-developers. Different audience, different product.
- Bedrock Agents are developer-built with code and Lambda actions. Q Apps are built in natural language with no code and no Lambda.
- A Knowledge Base with a custom frontend needs developers to build the frontend.
- A card that refers to data the runner cannot reach fails at that card with a permissions error. Connector permissions follow the user, not the app author.

### Numbers to memorize.

- Q Apps are part of the Q Business Pro tier. The license is per user per month. There is no infrastructure number to memorize.
- The card model has four card types: text input, file upload, text output, action. Cards wire together with @ mentions.

FIGURE 2.3. Q APPS TURNS A DESCRIPTION INTO A WORKING APP



Cards are the building blocks: each card does one small job (summarize, extract, answer).

Exam angle: 'business users build their own GenAI apps, no code' → Q Apps.

Trap: it lives inside Q Business; it is not Bedrock Agents and not a custom build.

Why here: end-user GenAI tooling; the main volumes cover the builder's path.

A Q App is a chain of cards wired with @ mentions. The user fills the input card. Each downstream card runs in turn. Teams share the finished app like a template, not like code.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. **Top: the start point.** A sentence, not a codebase. This is the audience difference.
2. **Middle: the card chain.** Each card does one task. @ mentions are the wiring. Data moves from left to right. One card feeds the next.

3. **Bottom: the sharing and safety story.** The library makes the app reusable. Q Business governance makes it safe. Both are automatic. The app author builds neither.
4. **What breaks:** a card refers to data the runner cannot reach. The app fails at that card with a permissions error. Connector permissions follow the user.

#### CB19. The card that failed on someone else's data.

**Scenario.** An HR analyst builds a Q App from her best leave-policy conversation. She publishes it to the organization library. It works for her.

A colleague in another department runs it and gets a permissions error at the policy-summary card. The colleague blames the app. What is wrong, and what is the fix?

**Worked answer.** Nothing is wrong with the app. Connector permissions follow the user, not the app author. The analyst can read the policy documents; the colleague cannot.

The app fails at the card that touches the restricted data. The fix is access: grant the colleague read permission on the underlying documents, or scope the app to users who have it. Rebuilding the app changes nothing.

**The trap.** Blaming the tool for a permissions boundary. Q Apps inherit Q Business governance. The user, not the app, carries the permissions.

#### APPENDIX · EASY · RECALL

**CC71.** An HR staff member wants a reusable tool. It must take a policy excerpt and make a plain-language summary. She must build it without developer help. The whole HR department must use it with existing access controls. Which service is correct?

- A) Bedrock Agents with an action group.  
Developers build agents with code and Lambda actions.
- B) Q Apps published to the organization library.**
- C) Q Developer with custom rules.  
Q Developer is the coding assistant for developers, not an HR workflow tool.
- D) A Knowledge Base with a custom frontend.  
The custom frontend needs developers.

**Why B is correct:** Q Apps need no code. You build them from a conversation, share them through the library, and they inherit Q Business governance.

**The trap:** "without developer help" rules out agents, Q Developer, and custom frontends in one phrase.

**APPENDIX · EASY · RECALL****CC72. How do cards in a Q App pass data to each other?**

- A) Through Lambda functions between cards.  
Q Apps use no code and no Lambda.
- B) Through @ mentions. The output of one card becomes the input of the next.**
- C) Through an S3 staging bucket.  
Not in source. Cards wire directly.
- D) Through EventBridge events.  
EventBridge routes events. It does not wire Q App cards.

**Why B is correct:** Cards refer to other cards with @ mentions. The wiring is the exam fact.

**The trap:** Lambda between cards. That is the Bedrock Agents pattern, not Q Apps.

**APPENDIX · MEDIUM · RECALL****CC73. The support team wants a reusable tool from its best troubleshooting conversation. No code. It must respect existing data permissions. Which is correct, and why not Bedrock Agents?**

- A) Q Apps. Bedrock Agents are developer-built with code and Lambda, which violates the no-code requirement.**
- B) Bedrock Agents, because agents are more powerful.  
Power is not the constraint. No-code is.
- C) Q Developer, because it writes the tool for the team.  
Q Developer assists developers with code. It does not make no-code team tools.
- D) A standalone chatbot with no governance.  
Q Apps inherit Q Business guardrails and Identity Center controls. The exam expects governance.

**Why A is correct:** Q Apps turn a conversation into a reusable no-code tool. They inherit Q Business governance. Connector permissions follow the user.

**The trap:** "more powerful" as an answer. The exam tests the constraint, not the power ranking.

**Curveballs: the exam twists back**

These are the questions that punish shallow reading. Each one changes a constraint, breaks an assumption, or pairs two confusing services. Read the twist. Work the answer. Name the trap.

### CB01. The reranker made answers worse

**Scenario.** A RAG system adds a reranker on the top-20 vector hits. Answer quality drops. The team blames the reranker model. What do you check first?

**Worked answer.** Check what the vector search retrieves before the reranker runs. A reranker can only reorder the candidates it receives. If the top-20 contains no relevant chunk, the reranker promotes the least-bad wrong answer. The fix is upstream: chunking strategy, embedding model, or hybrid search. The reranker was never the problem. The retrieval stage starved it.

**The trap.** Blaming the last stage. Debug the pipeline in order: pre-retrieval, retrieval, post-retrieval.

### CB02. Provisioned Throughput versus batch inference

**Scenario.** A nightly job scores 2 million support tickets with no SLA. A teammate proposes Provisioned Throughput for steady performance. What is wrong, and what wins?

**Worked answer.** Provisioned Throughput reserves capacity for consistent low-latency inference at high volume. A nightly job with no SLA needs the opposite: cheapest cost, latency irrelevant. Batch inference takes JSONL from S3, runs offline, and costs about half of on-demand. PT would burn money reserving capacity nobody needs during the day.

**The trap.** "Steady" sounds like PT. The exam means steady *latency need*, not steady schedule. No SLA plus bulk equals batch.

### CB03. Guardrails versus IAM

**Scenario.** A multi-tenant app must block one tenant from calling an expensive flagship model, and must block PII in all model outputs. One control for both?

**Worked answer.** No. Two different controls. IAM (or the API gateway policy layer) decides *who may call what*: block the tenant at the model-access level. Guardrails decide *what content may pass*: the PII filter runs on inputs and outputs. IAM never inspects prompt text. Guardrails never manage tenant permissions. The exam pairs them to test whether you know the boundary.

**The trap.** One bouncer for two doors. Identity controls access. Guardrails control content.

### CB04. AgentCore versus Strands versus Agent Squad

**Scenario.** A team builds a Python agent with custom tools, then needs to route customer requests across five specialist agents, then needs serverless deployment with memory. They ask for one service. What do you tell them?

**Worked answer.** Three different jobs, three different answers. Strands Agents builds the code-first tool-use loop. Agent Squad routes across the five specialists with an intent classifier. AgentCore Runtime hosts the whole thing serverless with Gateway, Memory, Identity, and Observability. No single service does all three. The exam wants the three names mapped to the three jobs.

**The trap.** "One service" framing. Split the scenario into build, route, and host.

### CB05. The KB deletion that kept billing

**Scenario.** A team deletes a Bedrock Knowledge Base. The OpenSearch Serverless bill continues. What did they miss?

**Worked answer.** Deleting the KB does not delete the vector store. Deleting the vector store data does not delete the serverless collection. The collection keeps running with zero data and keeps billing. Delete three things: the KB in Bedrock, the data in the store, and the collection in OpenSearch Service. The exam tests this chain because it burns real money.

**The trap.** "Delete the KB" sounds complete. It is one of three deletions.

## CB06. Cross-region inference with Provisioned Throughput

**Scenario.** A design uses a global inference profile for resilience plus Provisioned Throughput for steady latency. Will it work?

**Worked answer.** No. Cross-region inference does not work with Provisioned Throughput. PT pins a model to reserved capacity in specific regions. Cross-region routing needs on-demand capacity it can shift. Pick one: PT for guaranteed throughput in-region, or cross-region profiles for resilience and cost. The exam offers both together as a contradiction distractor.

**The trap.** Two good features that cancel each other. Check compatibility, not just desirability.

## CB07. The prompt that outgrew the model

**Scenario.** A RAG prompt works on one model but fails on a cheaper model with a context error. The team wants to keep the cheap model. What is the fix order?

**Worked answer.** First measure with the CountTokens API (free, matches the bill). Then cut: retrieve fewer chunks, filter chunks by metadata before vector search, prune old chat history, compress with a small summarizer model. Only then consider prompt caching for the repeated prefix. Do not start by switching models again. The failure is input size, not model quality.

**The trap.** "The cheap model is broken." The model is fine. The prompt is too big.

## CB08. Semantic caching returns stale answers

**Scenario.** A semantic cache serves RAG answers fast, but users get yesterday answers to today questions. The cache hit rate is high. What broke?

**Worked answer.** The similarity threshold is too loose, or the cached entries never expire. Semantic caching matches on embedding similarity, not exact text. Near-duplicate questions with different time context ("sales today" versus "sales yesterday") can score above the threshold and return the stale answer. Fixes: tighten the threshold, add TTLs, and include time-sensitive metadata in the cache key. High hit rate with wrong answers means the cache is confident and wrong.

**The trap.** "High hit rate means the cache works." Hit rate measures speed, not correctness.

## CB09. The agent that forgot the customer

**Scenario.** A support agent remembers context within a chat but forgets the customer tier between sessions. Sessions end after 600 seconds idle. Where does the memory go?

**Worked answer.** Session memory is short-term by design. Cross-session facts need long-term storage optimized for persistence: DynamoDB keyed by customer ID with server-side encryption, read at session start and written at session end. ElastiCache would also forget. The exam checks the latency-versus-persistence split: in-session goes to fast stores, cross-session goes to durable stores.

**The trap.** "Memory" as one thing. Short-term and long-term memory are different stores with different goals.

### CB10. The evaluation that graded itself

**Scenario.** A team uses the same flagship model as both the RAG generator and the LLM judge. Scores look great. Should you trust them?

**Worked answer.** No. A judge that shares the generator blind spots will miss the same failures. Use a different model as the judge for a second opinion. Also confirm the JSONL prompt dataset in S3 actually defines the judging criteria. Great scores from a self-judging loop measure agreement, not quality. The exam tests this as an independence requirement.

**The trap.** "The judge is a strong model." Strength does not fix shared bias. Independence does.

### CB11. BDA chokes on the daily file dump

**Scenario.** 10,000 mixed files land in S3 each morning. The BDA pipeline processes them one by one from a Lambda. At peak, jobs fail and files pile up. What is the correct shape?

**Worked answer.** S3 events fire per file. An EventBridge rule routes by file type. InvokeDataAutomationAsync queues the jobs. BDA async jobs queue behind service quotas, so buffer the burst with SQS and let poison files land in a dead-letter queue after retries. One-by-one synchronous Lambda calls collapse under burst load. The async plus queue shape absorbs the peak.

**The trap.** "Lambda per file" sounds serverless and fine. Burst plus quotas needs buffering.

### CB12. The fine-tune that should have been RAG

**Scenario.** A team fine-tunes a model on the product catalog so it can answer pricing questions. Prices change weekly. Answers go stale within days. What was the error?

**Worked answer.** Knowledge versus behavior. Fine-tuning bakes behavior into weights. It is the wrong tool for changing facts. RAG serves the live catalog at query time with citations. The team paid training cost for a system that decays weekly. The approach ladder says: changing knowledge means RAG, always.

**The trap.** "The model knows our data" as a goal. Models should reason. Stores should remember.

### CB13. The cheaper model that cost more

**Scenario.** A team switches to a cheaper per-token model. The monthly bill rises. How?

**Worked answer.** Per-token price is not per-answer price. The cheaper model needs longer prompts, more retries, or more output tokens to reach the same quality. Total tokens times price is the bill. Measure with CountTokens and CloudWatch InputTokenCount and OutputTokenCount before and after the switch. Intelligent Prompt Routing exists for this reason: route simple queries to cheap models and hard queries to flagships instead of switching wholesale.

**The trap.** Price per token as the whole cost story. Tokens per task completes it.

### CB14. The MCP server that could not scale

**Scenario.** A stateless MCP tool server runs on a single container. Traffic spikes and tool calls time out. The tools are simple JSON transforms. What is the fix?

**Worked answer.** Stateless and lightweight means Lambda, not a single container. Lambda scales each tool call independently and you pay per call. Containers fit complex tools with long runs or local state. The exam maps server shape to host: stateless goes serverless, complex goes containers, managed at scale goes AgentCore.

**The trap.** "Containers" as the default for everything. Match the host to the workload shape.

**D5 · EASY · ONE ANSWER**

**QH13.** A summary omits key facts from the reference. Which metric catches this?

**B) ROUGE.**

**A) BLEU.**

BLEU measures precision: unsupported words in the output.

**C) BERTScore.**

Measures semantic similarity, not reference coverage.

**D) Exact match.**

Too strict for summaries with valid rephrasing.

**Why B is correct:** ROUGE measures recall: how much of the reference appears in the output. Missing reference content is a recall failure.

**The trap:** BLEU for missing content. BLEU is precision. Missing content is recall.

**D5 · MEDIUM · ONE ANSWER**

**QH14.** The team wants LLM-as-a-judge evaluation in Bedrock. What must they provide?

**A) A CSV of prompts in DynamoDB.**

Wrong format and wrong store.

**B) A prompt dataset in JSONL format in S3.**

**C) A fine-tuned judge model.**

Bedrock supplies the judge. You supply the dataset.

**D) Human labels for every output.**

That is human evaluation, not LLM-as-a-judge.

**Why B is correct:** Bedrock Evaluations needs a JSONL prompt dataset in S3. It holds the prompts plus the judging instructions and benchmarks. The judge model scores your system output against it.

**The trap:** any dataset in any store. The exam checks JSONL in S3 specifically.

## Must-memorize numbers

| Must-memorize numbers                                     |                                                            |                                                                           |                                                              |
|-----------------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>75 / 65 / 180 min</b><br>questions / scored / time     | <b>750 of 1000</b><br>passing score                        | <b>D1 31% D2 26%</b><br>D3 20% D4 12% D5 11%                              | <b>Lambda 15 min</b><br>900-second ceiling                   |
| <b>API Gateway REST 29 s</b><br>integration timeout       | <b>SFN Express 5 min</b><br>no callbacks; Std up to 1 year | <b>Batch ~50% off</b><br>JSONL in S3, hours of latency                    | <b>Caching 1000+ tokens</b><br>TTL 5 min, up to 1 hour       |
| <b>Guardrails 6 × 4</b><br>6 policies, 4 filter strengths | <b>CloudTrail 90 days</b><br>metadata only, never content  | <b>stopReason trio</b><br>end_turn / max_tokens /<br>guardrail_intervened | <b>Titan Embed 1024/512/256</b><br>256 = quarter the storage |

The numbers to memorize: exam shape, timeouts, discounts, caching rules, guardrail settings, logging retention

Source: original figure for the AWS AIP-C01 Study Guide.

**How to read this.** Twelve cards, one number each. Scan for the number the question is testing: timeouts, discounts, or policy counts. Takeaway: when a stem quotes a number, find its card.

| Item                                              | Number                                                                                                                                                       |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Questions / score / time / pass                   | 75 / 65 / 180 minutes / 750 of 1000                                                                                                                          |
| Domain weights (scored questions are approximate) | D1 31% (~20), D2 26% (~17), D3 20% (~13), D4 12% (~8), D5 11% (~7)                                                                                           |
| Lambda max duration                               | 15 minutes (900 seconds)                                                                                                                                     |
| API Gateway REST integration timeout              | 29 seconds                                                                                                                                                   |
| Step Functions Express maximum duration           | 5 minutes. There are no callbacks.                                                                                                                           |
| Step Functions Standard maximum duration          | The duration is up to 1 year. The cost is about \$0.025 for 1,000 state transitions. Waits are free.                                                         |
| Step Functions Distributed Map parallel children  | Up to 10,000 over S3 objects                                                                                                                                 |
| Batch inference discount                          | The cost is about 50% less than on-demand. Put JSONL into S3. Put results to S3. Latency is hours.                                                           |
| Provisioned Throughput terms                      | Model units have 1 or 6 month terms. Billing is hourly. Use the provisioned ARN.                                                                             |
| Prompt caching                                    | The repeated prefix is about 1,000+ tokens (this depends on the model). TTL default is 5 minutes. TTL is up to 1 hour. The proof is the cache read hit rate. |
| Titan Text Embeddings v2 dimensions               | 1024, 512, or 256. 256 cuts storage to one quarter of 1024.                                                                                                  |
| Cohere Embed languages; input_type                | 100+ languages; search_query versus search_document                                                                                                          |
| Guardrail content filters                         | There are 6 categories. These are hate, insults, sexual, violence, misconduct, and prompt attack. There are 4 strengths. This makes 24 settings.             |
| Guardrail policies                                | 6: content filters, denied topics, word filters, sensitive info filters, contextual grounding, Automated Reasoning                                           |
| stopReason values                                 | end_turn (done), max_tokens (cut, not finished), guardrail_intervened (blocked)                                                                              |
| CloudTrail event history                          | The system keeps metadata for 90 days. It does not keep prompt or response content.                                                                          |
| Understand Comprehend PII entity types.           | There are more than 30 entity types. These include names, SSNs, and account numbers.                                                                         |
| Claude InvokeModel needs specific fields.         | The required fields are anthropic_version "bedrock-2023-05-31" and max_tokens.                                                                               |
| This is the threshold for a small vector dataset. | Use Aurora Serverless with pgvector for less than 1 million records. Use OpenSearch Serverless for many records.                                             |
| Fine-tuning data scale                            | There are hundreds to a few thousand labeled examples in S3. Jobs run for many hours.                                                                        |
| Fixed-size chunking default                       | About 300 tokens per chunk, with overlap.                                                                                                                    |
| Titan embedding dimensions                        | 1024 default. Titan Multimodal G1 offers 256, 384, or 1024.                                                                                                  |
| CountTokens API                                   | Free. Same body as InvokeModel or Converse. Matches the bill.                                                                                                |
| Retry backoff shape                               | Start near 1 second. Double each time. Add jitter. Cap at 3 to 5 attempts.                                                                                   |

| Item                           | Number                                                                                               |
|--------------------------------|------------------------------------------------------------------------------------------------------|
| OpenSearch semantic shard size | 30 to 50 GB per shard. Hybrid search uses 10 to 30 GB.                                               |
| S3 Vectors scale               | 10,000 indices per bucket. 2 billion vectors per index. Strongly consistent.                         |
| AgentCore runtime contract     | Port 8080. POST /invocations runs the agent. GET /ping is the health check.                          |
| Cross-region global profile    | Up to 10 percent cheaper than standard. Geographic profiles keep data in-region at standard pricing. |

## These are the top 50 traps. Each trap is one line long.

1. Use fine-tuning for knowledge. This includes quarterly documents, citations, and new facts. Always use RAG.
2. Converse offers embeddings. Converse only makes text. Embeddings need InvokeModel.
3. Guardrails are for Converse only. ApplyGuardrail works for any text. This includes external models.
4. Provider benchmarks decide the model. Only your golden-dataset evaluation is important.
5. AccessDenied on InvokeModel is an IAM issue. First, check model access and regional availability.
6. If a Claude body does not have anthropic\_version or max\_tokens, you will debug IAM for one hour. The error is the request shape.
7. Cross-region inference profile is under data residency. Profiles move data across regions.
8. Provisioned Throughput fixes bursty traffic costs. It bills hourly. This is true even if you do not use it.
9. Use prompt caching on unique traffic. If there is no repeated prefix, there is no saving.
10. Use ElastiCache as a vector store. It is ephemeral caching. Durable vectors live in a vector store.
11. Kendra offers RAG. Kendra is enterprise search. It is not the RAG path.
12. stopReason max\_tokens means the task finished. It means the system cut the output.
13. Set both Temperature and topP. They work together. Choose one.
14. Lambda waits, sleeps, or polls for more than 15 minutes. It stops.
15. The agent ID is alone in production. It routes to DRAFT, not the PROD alias.
16. One session ID is for many users. This leaks conversation memory.
17. Express is for human waits that last many hours. It has a 5-minute limit. It has no callbacks.
18. You raise the Lambda timeout for gateway timeouts. The 29-second limit is at API Gateway.
19. You make Flow nodes (S3 copy node). Check the node and feature lists.
20. CloudFormation is for runtime switching. Deployments are not runtime. AppConfig is for runtime switching.
21. Agents are for fixed sequences. There is reasoning overhead with no benefit.
22. Multi-tenant agents do not have per-KB IAM filtering. Tenants must not reach other tenants' data.
23. Guardrails are for IAM, tenant isolation, token quotas, or billing. This is a made-up capability.
24. CloudTrail is for prompt content. It has metadata only. Content needs invocation logging.
25. CloudWatch is for who called what. CloudTrail does this job.

26. Built-in safety means we do not do evaluation. The deployer owns evaluation and governance. Always do evaluation.
27. A guardrail is on a CloudWatch log group to hide PHI. Guardrails are on Bedrock model flows. They are not on log groups.
28. NAT gateway is for no public internet. It gives public internet access.
29. Full invocation logging is in healthcare with no retention policy. Logs can have PII. Ask about the logging.
30. Automated Reasoning must block inline. It runs in detect mode only.
31. PT is a latency fix for slow first tokens. Capacity is not perceived latency.
32. Routers are for availability. Profiles are for cost. The options swap the pair.
33. Batch is for interactive SLAs. Hours of latency does not meet the SLA.
34. Total token spend shows that caching works. The metric is cache read hit rate.
35. A bigger model is the quality answer. Do not check the golden dataset. Right-size first.
36. Use reactive metrics for proactive requirements. Count before the call.
37. Humans read everything under a budget. The cost increases in order.
38. Debug IAM for a bad provider body. ValidationException shows the request shape.
39. Debug generation when retrieval changed. Debug the difference.
40. Deploy a new prompt or model version with no evaluation gate. Use a score threshold or do not deploy.
41. Use A2I on all traffic. Use only flagged samples.
42. Put tables into a vector store. Relational data becomes text-to-SQL.
43. Use a different embedding model at query time than at ingestion. Retrieval does not work.
44. Serve a custom model with the base model ID. Fine-tuned models need Provisioned Throughput.
45. Use direct SDK calls when the scenario needs authorization and throttling. Use API Gateway.
46. The answer for "least operational overhead" is self-managed components. The adjective picks managed.
47. The cheapest option that meets every requirement wins. A premium extra with no stated need does not win.
48. Do not skip the PoC. Do not buy PT before validation. Use on-demand first. Prove it. Then commit.
49. Treat action-group parameters as a dictionary. They arrive as a list of name/value dictionaries.
50. Remember to enableTrace when you explain agent behavior. It is the transparency answer.

You finished the crash course. Did you miss more than five rapid-fire questions? Re-read the decision rules for the domains you missed. Then drill the full question bank. Did you miss five or fewer? You are ready to schedule.

#### KEY TAKEAWAY

**Where to go next.** This volume was the sprint. If anything felt thin, go deep in the matching domain volume. Use D1 through D5 in exam-weight order. Then take the 75-question mock exam in the [question bank](#). Drill new angles in [How AWS Asks](#).

## The one-glance cheat sheet

Every row below earns its place. Service, when to use it, the number that matters, and the trap that catches the careless. Read this last. Read it twice.

### D1 cheat sheet: models, data, RAG

| Service or method        | Use when                                                                                | Key number or fact                     | Trap                           |
|--------------------------|-----------------------------------------------------------------------------------------|----------------------------------------|--------------------------------|
| RAG with Knowledge Bases | Facts change. Answers must cite sources                                                 | Chunk about 300 tokens with overlap    | Never fine-tune for knowledge  |
| Fine-tuning (SFT)        | New tone, format, or behavior from labeled pairs                                        | Bakes behavior into weights            | Cannot cite sources            |
| Continued pre-training   | Domain vocabulary from unlabeled text                                                   | Highest effort of the training options | Not for tone changes           |
| Distillation             | One narrow task, cost is the pain                                                       | Near-big quality at small-model cost   | Not for new knowledge          |
| Titan Text Embeddings v2 | Default English embeddings                                                              | Configurable dimensions                | Must match at ingest and query |
| Cohere Embed             | Multilingual content                                                                    | 100+ languages                         | Not the default for English    |
| RetrieveAndGenerate      | Cited answers, least effort                                                             | One managed call                       | No custom reranking            |
| Converse                 | Unified text API across providers                                                       | Text only                              | Cannot do embeddings           |
| InvokeModel              | Native bodies, embeddings                                                               | Only embeddings path                   | Provider-specific shape        |
| Reranker                 | Top-k vector hits, then score true relevance. Amazon and Cohere models. Rerank API.     |                                        |                                |
| Chunking types           | Fixed 300 tokens, none, hierarchical parent/child, semantic with buffer and breakpoint. |                                        |                                |
| Multimodal embeddings    | Titan G1: inputText plus base64 inputImage. Lengths 256, 384, 1024.                     |                                        |                                |
| Reasoning models         | Chain of thought on: higher cost, slower, more accurate.                                |                                        |                                |
| Divider strings          | Preprocessing markers at section boundaries. Split on them before size limits.          |                                        |                                |

**D2 cheat sheet: build and integrate**

| Service or method       | Use when                                                                              | Key number or fact            | Trap                           |
|-------------------------|---------------------------------------------------------------------------------------|-------------------------------|--------------------------------|
| Bedrock Agents          | Steps change each time. Tools needed                                                  | Action groups map to Lambda   | Overhead for fixed sequences   |
| Bedrock Flows           | Fixed sequence, same every time                                                       | No dynamic step choice        | Cannot handle changing steps   |
| ApplyGuardrail          | Safety check on any text, any model                                                   | Standalone API                | Not tied to Converse           |
| Batch inference         | Bulk offline work, no SLA                                                             | About 50% off. JSONL in S3    | Hours of latency               |
| Async invocation        | One slow request, nobody waits                                                        | S3 in, S3 out                 | Not for bulk work              |
| Step Functions Standard | Human approval takes hours                                                            | 1 year max. Wait-for-callback | Not Express                    |
| AppConfig               | Swap models without redeploying                                                       | Model ID read at runtime      | CloudFormation is a deployment |
| Strands Agents          | Open-source Python SDK. Agent loop: reason, select tool, execute. @tool functions.    |                               |                                |
| Agent Squad             | Intent classifier routes to specialist agents. Routing, not tool loops.               |                               |                                |
| AgentCore               | Runtime, Gateway, Memory, Identity, Observability. Any framework, serverless.         |                               |                                |
| MCP                     | Open standard. JSON-RPC 2.0 data layer. Tools, resources, prompts.                    |                               |                                |
| BDA                     | Standard output guesses. Custom blueprints enforce fields. InvokeDataAutomationAsync. |                               |                                |
| SageMaker in AIP        | Usually the distractor. Train there, import to Bedrock for serverless inference.      |                               |                                |

**D3 cheat sheet: safety and security**

| Service or method               | Use when                                                                 | Key number or fact               | Trap                                   |
|---------------------------------|--------------------------------------------------------------------------|----------------------------------|----------------------------------------|
| Denied topics                   | Block a subject in any wording                                           | Rephrasing does not escape       | Not word filters                       |
| Word filters                    | Block exact words                                                        | Rephrasing escapes               | Not denied topics                      |
| Contextual grounding            | Check answers against sources                                            | Anti-hallucination policy        | Not attack detection                   |
| Prompt attack detection         | Jailbreak and injection patterns                                         | Probabilistic. One layer of many | Does not stop all attacks              |
| Model invocation logging        | Full prompt and response text for audit                                  | Opt-in. Encrypt with KMS         | CloudTrail never holds content         |
| VPC interface endpoints         | No public internet                                                       | PrivateLink to Bedrock           | NAT gateway still uses public internet |
| Macie / Comprehend / Guardrails | PII at rest / in text / at inference                                     | Match the place to the service   | One service per place                  |
| Agent tracing                   | Preprocessing, orchestration, postprocessing, routing classifier.        |                                  |                                        |
| Token redaction                 | Lambda handlers filter text pre and post model. Regex or Comprehend NER. |                                  |                                        |
| CloudTrail vs CloudWatch        | CloudTrail logs API calls. CloudWatch Logs holds the prompts.            |                                  |                                        |

**D4 cheat sheet: cost and capacity**

| Service or method       | Use when                                                                             | Key number or fact          | Trap                      |
|-------------------------|--------------------------------------------------------------------------------------|-----------------------------|---------------------------|
| Prompt caching          | Long prefix repeats across calls                                                     | cachePoint. Watch hit rate  | Useless on unique traffic |
| Semantic caching        | Similar questions asked many times                                                   | Skips the call completely   | Needs similar questions   |
| Provisioned Throughput  | Steady traffic, throttling at peak, SLA                                              | Hourly billing, used or not | Not a cost fix            |
| Prompt router / cascade | Simple queries on an expensive model                                                 | Routes by complexity        | Not for capacity          |
| Streaming               | Waiting user, long answers                                                           | Fixes perceived latency     | Does not fix throttling   |
| CountTokens             | Free token count before inference. Check fit, estimate cost.                         |                             |                           |
| TTFT                    | Request to first token, streaming only. CloudWatch TimeToFirstToken.                 |                             |                           |
| Prompt routing          | Simple queries to cheap models, hard ones to flagships. No custom code.              |                             |                           |
| Evidently               | EvaluateFeature assigns variants. PutProjectEvents reports metrics. Real statistics. |                             |                           |

**D5 cheat sheet: test and fix**

| Service or method        | Use when                                                           | Key number or fact               | Trap                              |
|--------------------------|--------------------------------------------------------------------|----------------------------------|-----------------------------------|
| Automatic evaluation     | Objective checks, known answers                                    | Needs labels. Cheapest           | No labels means no automatic eval |
| LLM-as-a-judge           | No labels, human-like quality at scale                             | Separate judge model plus rubric | Judge must differ from candidate  |
| Human evaluation         | Subjective judgment                                                | Samples only. Costs most         | Never for everything              |
| Faithfulness             | Hallucination check                                                | Source supports every claim      | Not correctness                   |
| Shadow deployment        | Zero user risk on production data                                  | Invisible copies                 | Not A/B                           |
| CI/CD eval gate          | Promote to production                                              | No gate, no promotion            | Dashboards do not block           |
| LLM-as-a-judge           | Needs a JSONL prompt dataset in S3. Judge scores input and output. |                                  |                                   |
| ROUGE / BLEU / BERTScore | Recall, precision, semantic similarity.                            |                                  |                                   |
| RAG eval trio            | Context relevance, groundedness, answer relevance.                 |                                  |                                   |

## Memory aids

Mnemonics, never-confuse pairs, and if-this-then-that rules. These are the last things you read before the exam.

### Mnemonics

- **PRAG-F:** the approach ladder. Prompt, RAG, Agents, Fine-tuning. Climb in order. Stop at the cheapest rung that works.
- **SIX-P:** the six guardrail policies. Content filters, denied topics, word filters, sensitive information filters, contextual grounding, automated reasoning. "Six-P stops bad output."
- **C3PO for logs:** CloudTrail for audit, CloudWatch for operations, invocation logging for content. Three logs, three depths.
- **SIP for identity:** SCPs bound the org, permissions boundaries bound the role, IAM grants the access. Read top to bottom.
- **AJH for eval:** Automatic, Judge, Human. Climb in order. Cost rises with each step.
- **CRAB for cost:** Cache, Right-size, Async-batch, Baseline-PT. The cost levers in the order you try them.

### Never confuse X with Y

| X                       | Y                       | The line                                                               |
|-------------------------|-------------------------|------------------------------------------------------------------------|
| RAG                     | Fine-tuning             | New knowledge means RAG. New behavior means fine-tuning.               |
| Converse                | InvokeModel             | Unified text API versus native body API. Only InvokeModel embeds.      |
| Agents                  | Flows                   | Dynamic step choice versus fixed sequence.                             |
| Retrieve                | RetrieveAndGenerate     | Chunks only versus one managed call with citations.                    |
| Denied topics           | Word filters            | Block meaning versus block exact strings.                              |
| Contextual grounding    | Prompt attack detection | Answers against sources versus inputs against jailbreak patterns.      |
| Prompt caching          | Semantic caching        | Repeated prefix versus similar questions.                              |
| Batch inference         | Provisioned Throughput  | Half price for offline bulk versus hourly capacity for steady traffic. |
| Async invocation        | Batch inference         | One slow request versus many requests.                                 |
| Inference profiles      | Prompt routers          | Route for availability versus route for cost.                          |
| Neptune                 | Vector stores           | Graph relationships versus document chunks.                            |
| Kendra                  | Knowledge Bases         | Enterprise search versus the RAG path.                                 |
| CloudTrail              | Invocation logging      | API metadata versus full prompt and response text.                     |
| Step Functions Standard | Step Functions Express  | One year with callbacks versus five minutes, no pause.                 |
| Shadow                  | A/B                     | Invisible copies versus split users.                                   |

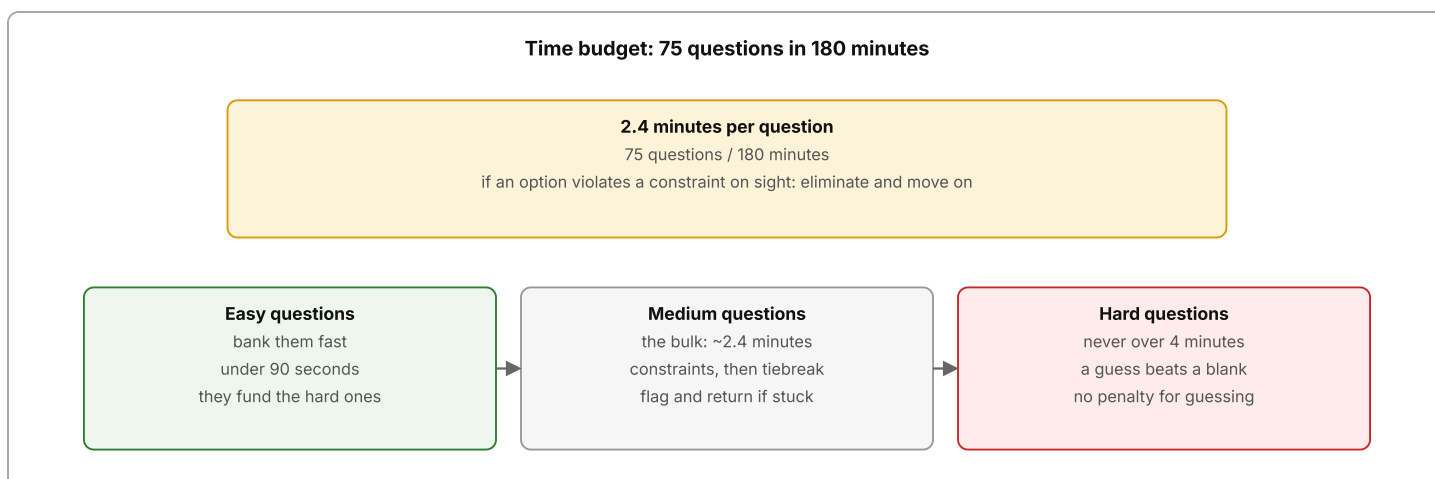
### If this, then that: the master list

- If facts change, then RAG. If behavior must change, then fine-tune.

- If steps change, then agents. If steps are fixed, then Flows.
- If embeddings, then InvokeModel. If portable text calls, then Converse.
- If any wording must be blocked, then denied topics. If exact words, then word filters.
- If the source must support the answer, then contextual grounding.
- If no public internet, then VPC interface endpoints.
- If full text for audit, then invocation logging. If who-called-what, then CloudTrail.
- If the prefix repeats, then prompt caching. If questions repeat, then semantic caching.
- If steady plus throttling, then Provisioned Throughput. If offline bulk, then batch.
- If simple queries on a big model, then a router. If slow first token, then streaming.
- If no labels, then LLM-as-a-judge. If known answers, then automatic eval.
- If zero user risk, then shadow. If fair live comparison, then A/B.
- If approval takes hours, then Step Functions Standard with wait-for-callback.
- If the model ID must change without a deploy, then AppConfig.

## Exam day: time, method, and trap radar

75 questions. 180 minutes. That is 2.4 minutes per question. The method below keeps you on pace.



Budget 2.4 minutes per question. Easy questions take under 90 seconds. Hard questions cap at 4 minutes. Bank the saved time.

Source: original figure for the AWS AIP-C01 Study Guide.

### How to read this diagram

1. Easy: answer in under 90 seconds. Bank the time.
2. Medium: stay on the 2.4-minute pace.
3. Hard: cap at 4 minutes. Flag and move on.

### The question method

1. **Extract the constraints.** Read the stem. List every constraint: facts change, no internet, must cite, notify only. If an option violates one constraint, eliminate it.

2. **Name the verb.** The stem has one operative verb: block, cite, ground, act, cut cost, add capacity. Match the verb to the service.
3. **Climb the ladder.** Prefer the cheapest rung that works. Managed over self-built. Prompt over RAG over agents over fine-tuning.
4. **Check the trap.** Every wrong option is a real service doing the wrong job, an absolute word, or a reversed cause. Name the trap before you commit.
5. **Trust the first answer.** Change it only when you find a violated constraint. Gut changes lose points.

#### The 6 question patterns AWS reuses

##### 1. Constraint extraction

find 2-4 hard constraints  
eliminate violators

##### 2. Most correct

two options work:  
tiebreakers decide

##### 3. Ordering

anchor first and last  
lifecycle dependencies

##### 4. Vocabulary precision

one precise term decides  
5 terms to know cold

##### 5. Managed-first ladder

climb until the need is met  
stop at the cheapest rung

##### 6. Wrong-plane distractors

right service, wrong job  
ask what the job really is

Six patterns cover the exam. Constraint extraction and wrong-plane distractors appear most. Vocabulary precision punishes loose terms.

Source: original figure for the AWS AIP-C01 Study Guide.

#### How to read this diagram

1. Learn the six patterns by name.
2. On each question, name the pattern first.
3. The pattern tells you where the trap hides.

#### The 30 traps in 5 families: name each one on sight

##### Distractor services

Lambda 15-min timeout  
KB scope, guardrails plane  
fabricated features

##### Absolute wording

always/never in options  
adjective precision  
built-in safety is enough

##### Associate-level answers

custom code vs managed  
ladder jumping  
proactive vs reactive

##### Reversed causality

PII pipeline order  
missing prepare step  
sequential vs parallel

##### Missed constraints

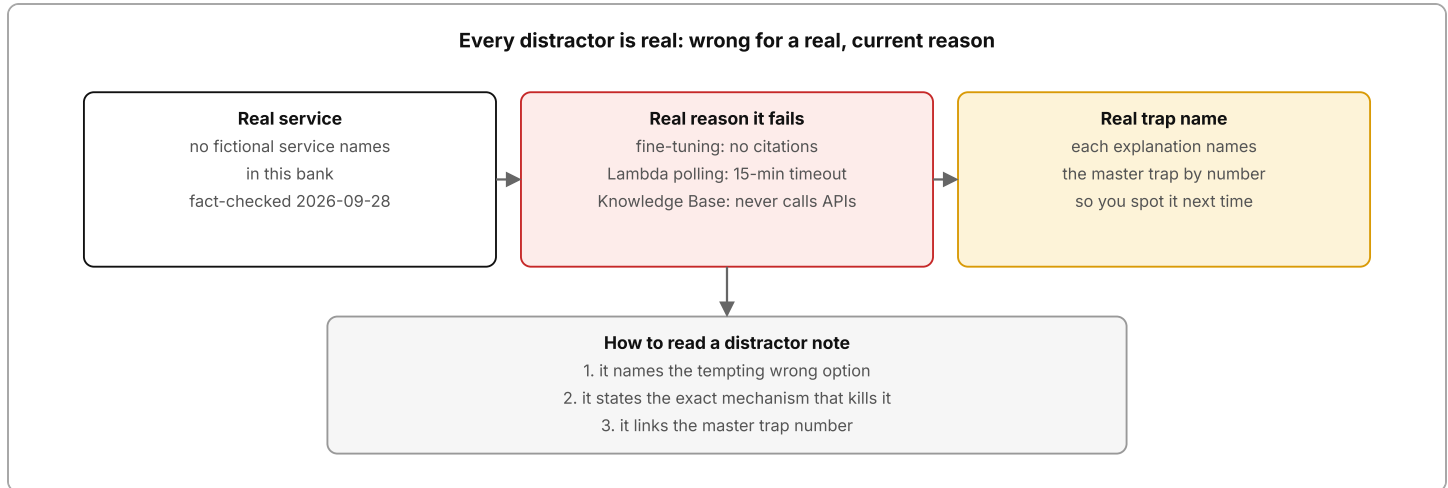
residency blindness  
no public internet  
capacity confusion set

Traps come in five families. Distractor services name a real service for the wrong job. Absolute wording uses "always" and "never" to lie.

Source: original figure for the AWS AIP-C01 Study Guide.

## How to read this diagram

1. Five families. Learn each family's tell.
2. Distractor services: ask if the service's real job matches the stem.
3. Absolute wording: "always" and "never" are usually wrong.



Every distractor has three parts. A real service. A real reason it fails here. The trap it belongs to. Dissect each option this way.

Source: original figure for the AWS AIP-C01 Study Guide.

## How to read this diagram

1. Read the option. Name the service.
2. Ask: does its real job match the stem's verb?
3. If not, name the trap family and eliminate it.

**PREVIOUS**

**System Design Guide**

**NEXT**

**Appendix · Coverage Gaps**